

Python 3 — Biblioteca Estándar

Tomo 4 de 6

Contenido de este tomo:

1. `xml.etree.ElementTree`— The ElementTree XML API
2. `xml.dom`— The Document Object Model API
3. `xml.dom.minidom`— Minimal DOM implementation
4. `xml.dom.pulldom`— Support for building partial DOM trees
5. `xml.sax`— Support for SAX2 parsers
6. `xml.sax.handler`— Base classes for SAX handlers
7. `xml.sax.saxutils`— SAX Utilities
8. `xml.sax.xmlreader`— Interface for XML parsers
9. `xml.parsers.expat`— Fast XML parsing using Expat
10. Internet Protocols and Support
11. `webbrowser`— Convenient web-browser controller
12. `wsgiref`— WSGI Utilities and Reference Implementation
13. `urllib`— URL handling modules
14. `urllib.request`— Extensible library for opening URLs
15. `urllib.parse`— Parse URLs into components
16. `urllib.error`— Exception classes raised by `urllib.request`
17. `urllib.robotparser`— Parser for robots.txt
18. `http`— HTTP modules
19. `http.client`— HTTP protocol client
20. `ftplib`— FTP protocol client
21. `poplib`— POP3 protocol client
22. `imaplib`— IMAP4 protocol client
23. `smtplib`— SMTP protocol client
24. `uuid`— UUID objects according to RFC 9562
25. `socketserver`— A framework for network servers

- 26. `http.server`— HTTP servers
- 27. `http.cookies`— HTTP state management
- 28. `http.cookiejar`— Cookie handling for HTTP clients
- 29. `xmlrpc`— XMLRPC server and client modules
- 30. `xmlrpc.client`— XML-RPC client access
- 31. `xmlrpc.server`— Basic XML-RPC servers
- 32. `ipaddress`— IPv4/IPv6 manipulation library
- 33. Multimedia Services
- 34. `wave`— Read and write WAV files
- 35. `colorsys`— Conversions between color systems
- 36. Internationalization
- 37. `gettext`— Multilingual internationalization services
- 38. `locale`— Internationalization services
- 39. Graphical user interfaces with Tk
- 40. `tkinter`— Python interface to Tcl/Tk
- 41. `tkinter.colorchooser`— Color choosing dialog
- 42. `tkinter.font`— Tkinter font wrapper
- 43. Tkinter Dialogs
- 44. `tkinter.messagebox`— Tkinter message prompts
- 45. `tkinter.scrolledtext`— Scrolled Text Widget

xml.etree.ElementTree— The ElementTree XML API

Source code: `Lib/xml/etree/ElementTree.py`

The `xml.etree.ElementTree` module implements a simple and efficient API for parsing and creating XML data.

Changed in version 3.3: This module will use a fast implementation whenever available.

Deprecated since version 3.3: The `xml.etree.cElementTree` module is deprecated.

■ **Nota:** Note if you need to parse untrusted or unauthenticated data, see [XML security](#).

Tutorial

This is a short tutorial for using `xml.etree.ElementTree` (ET in short). The goal is to demonstrate some of the building blocks and basic concepts of the module.

XML tree and elements

XML is an inherently hierarchical data format, and the most natural way to represent it is with a tree. ET has two classes for this purpose - `ElementTree` represents the whole XML document as a tree, and `Element` represents a single node in this tree. Interactions with the whole document (reading and writing to/from files) are usually done on the `ElementTree` level. Interactions with a single XML element and its sub-elements are done on the `Element` level.

Parsing XML

We'll be using the `fictivecountry_data.xml` XML document as the sample data for this section:

```
<?xml version="1.0"?>
<data>
  <country name="Liechtenstein">
    <rank>1</rank>
    <year>2008</year>
    <gdppc>141100</gdppc>
    <neighbor name="Austria" direction="E"/>
    <neighbor name="Switzerland" direction="W"/>
  </country>
  <country name="Singapore">
    <rank>4</rank>
    <year>2011</year>
    <gdppc>59900</gdppc>
    <neighbor name="Malaysia" direction="N"/>
  </country>
  <country name="Panama">
    <rank>68</rank>
    <year>2011</year>
    <gdppc>13600</gdppc>
    <neighbor name="Costa Rica" direction="W"/>
    <neighbor name="Colombia" direction="E"/>
  </country>
</data>
```

We can import this data by reading from a file:

```
import xml.etree.ElementTree as ET
tree = ET.parse('country_data.xml')
root = tree.getroot()
```

Or directly from a string:

```
root = ET.fromstring(country_data_as_string)
```

fromstring() parses XML from a string directly into anElement, which is the root element of the parsed tree. Other parsing functions may create anElementTree. Check the documentation to be sure.

As anElement, root has a tag and a dictionary of attributes:

```
>>> root.tag
'data'
>>> root.attrib
{}
```

It also has children nodes over which we can iterate:

```
>>> for child in root:
...     print(child.tag, child.attrib)
...
country {'name': 'Liechtenstein'}
country {'name': 'Singapore'}
country {'name': 'Panama'}
```

Children are nested, and we can access specific child nodes by index:

```
>>> root[0][1].text
'2008'
```

■ **Nota:** Not all elements of the XML input will end up as elements of the parsed tree. Currently, this module skips over any XML comments, processing instructions, and document type declarations in the input. Nevertheless, trees built using this module's API rather than parsing from XML text can have comments and processing instructions in them; they will be included when generating XML output. A document type declaration may be accessed by passing a customTreeBuilder instance to theXMLParser constructor.

Pull API for non-blocking parsing

Most parsing functions provided by this module require the whole document to be read at once before returning any result. It is possible to use anXMLParser and feed data into it incrementally, but it is a push API that calls methods on a callback target, which is too low-level and inconvenient for most needs. Sometimes what the user really wants is to be able to parse XML incrementally, without blocking operations, while enjoying the convenience of fully constructedElement objects.

The most powerful tool for doing this isXMLPullParser. It does not require a blocking read to obtain the XML data, and is instead fed with data incrementally withXMLPullParser.feed() calls. To get the parsed XML elements, callXMLPullParser.read_events(). Here is an example:

```
>>> parser = ET.XMLPullParser(['start', 'end'])
>>> parser.feed('<mytag>sometext')
>>> list(parser.read_events())
[('start', <Element 'mytag' at 0x7fa66db2be58>)]
>>> parser.feed(' more text</mytag>')
>>> for event, elem in parser.read_events():
...     print(event)
...     print(elem.tag, 'text=', elem.text)
...
end
mytag text= sometext more text
```

The obvious use case is applications that operate in a non-blocking fashion where the XML data is being received from a socket or read incrementally from some storage device. In such cases, blocking reads are unacceptable.

Because it's so flexible,XMLPullParser can be inconvenient to use for simpler use-cases. If you don't mind your application blocking on reading XML data but would still like to have incremental parsing capabilities, take a look atiterparse(). It can be useful when you're reading a large XML document and don't want to hold it wholly in memory.

Where immediate feedback through events is wanted, calling method `XMLPullParser.flush()` can help reduce delay; please make sure to study the related security notes.

Finding interesting elements

`Element` has some useful methods that help iterate recursively over all the sub-tree below it (its children, their children, and so on). For example, `Element.iter()`:

```
>>> for neighbor in root.iter('neighbor'):
...     print(neighbor.attrib)
...
{'name': 'Austria', 'direction': 'E'}
{'name': 'Switzerland', 'direction': 'W'}
{'name': 'Malaysia', 'direction': 'N'}
{'name': 'Costa Rica', 'direction': 'W'}
{'name': 'Colombia', 'direction': 'E'}
```

`Element.findall()` finds only elements with a tag which are direct children of the current element. `Element.find()` finds the first child with a particular tag, and `Element.text` accesses the element's text content. `Element.get()` accesses the element's attributes:

```
>>> for country in root.findall('country'):
...     rank = country.find('rank').text
...     name = country.get('name')
...     print(name, rank)
...
Liechtenstein 1
Singapore 4
Panama 68
```

More sophisticated specification of which elements to look for is possible by using XPath.

Modifying an XML File

`ElementTree` provides a simple way to build XML documents and write them to files. The `ElementTree.write()` method serves this purpose.

Once created, an `Element` object may be manipulated by directly changing its fields (such as `Element.text`), adding and modifying attributes (`Element.set()` method), as well as adding new children (for example with `Element.append()`).

Let's say we want to add one to each country's rank, and add an updated attribute to the rank element:

```
>>> for rank in root.iter('rank'):
...     new_rank = int(rank.text) + 1
...     rank.text = str(new_rank)
...     rank.set('updated', 'yes')
...
>>> tree.write('output.xml')
```

Our XML now looks like this:

```
<?xml version="1.0"?>
<data>
  <country name="Liechtenstein">
    <rank updated="yes">2</rank>
    <year>2008</year>
    <gdppc>141100</gdppc>
    <neighbor name="Austria" direction="E"/>
    <neighbor name="Switzerland" direction="W"/>
  </country>
  <country name="Singapore">
    <rank updated="yes">5</rank>
    <year>2011</year>
    <gdppc>59900</gdppc>
```

```

        <neighbor name="Malaysia" direction="N"/>
    </country>
    <country name="Panama">
        <rank updated="yes">69</rank>
        <year>2011</year>
        <gdppc>13600</gdppc>
        <neighbor name="Costa Rica" direction="W"/>
        <neighbor name="Colombia" direction="E"/>
    </country>
</data>

```

We can remove elements using `Element.remove()`. Let's say we want to remove all countries with a rank higher than 50:

```

>>> for country in root.findall('country'):
...     # using root.findall() to avoid removal during traversal
...     rank = int(country.find('rank').text)
...     if rank > 50:
...         root.remove(country)
...
>>> tree.write('output.xml')

```

Note that concurrent modification while iterating can lead to problems, just like when iterating and modifying Python lists or dicts. Therefore, the example first collects all matching elements with `root.findall()`, and only then iterates over the list of matches.

Our XML now looks like this:

```

<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank updated="yes">2</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank updated="yes">5</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
</data>

```

Building XML documents

The `SubElement()` function also provides a convenient way to create new sub-elements for a given element:

```

>>> a = ET.Element('a')
>>> b = ET.SubElement(a, 'b')
>>> c = ET.SubElement(a, 'c')
>>> d = ET.SubElement(c, 'd')
>>> ET.dump(a)
<a><b /><c><d /></c></a>

```

Parsing XML with Namespaces

If the XML input has namespaces, tags and attributes with prefixes in the form `prefix:sometag` get expanded to `{uri}sometag` where the prefix is replaced by the full URI. Also, if there is a default namespace, that full URI gets prepended to all of the non-prefixed tags.

Here is an XML example that incorporates two namespaces, one with the prefix “fictional” and the other serving as the default namespace:

```
<?xml version="1.0"?>
<actors xmlns:fictional="http://characters.example.com"
        xmlns="http://people.example.com">
  <actor>
    <name>John Cleese</name>
    <fictional:character>Lancelot</fictional:character>
    <fictional:character>Archie Leach</fictional:character>
  </actor>
  <actor>
    <name>Eric Idle</name>
    <fictional:character>Sir Robin</fictional:character>
    <fictional:character>Gunther</fictional:character>
    <fictional:character>Commander Clement</fictional:character>
  </actor>
</actors>
```

One way to search and explore this XML example is to manually add the URI to every tag or attribute in the xpath of `afind()` or `findall()`:

```
root = fromstring(xml_text)
for actor in root.findall('{http://people.example.com}actor'):
    name = actor.find('{http://people.example.com}name')
    print(name.text)
    for char in actor.findall('{http://characters.example.com}character'):
        print(' |-->', char.text)
```

A better way to search the namespaced XML example is to create a dictionary with your own prefixes and use those in the search functions:

```
ns = {'real_person': 'http://people.example.com',
      'role': 'http://characters.example.com'}

for actor in root.findall('real_person:actor', ns):
    name = actor.find('real_person:name', ns)
    print(name.text)
    for char in actor.findall('role:character', ns):
        print(' |-->', char.text)
```

These two approaches both output:

```
John Cleese
|--> Lancelot
|--> Archie Leach
Eric Idle
|--> Sir Robin
|--> Gunther
|--> Commander Clement
```

XPath support

This module provides limited support for XPath expressions for locating elements in a tree. The goal is to support a small subset of the abbreviated syntax; a full XPath engine is outside the scope of the module.

Example

Here's an example that demonstrates some of the XPath capabilities of the module. We'll be using the `countrydataXML` document from the `Parsing XML` section:

```
import xml.etree.ElementTree as ET

root = ET.fromstring(countrydata)

# Top-level elements
```

```

root.findall(".")

# All 'neighbor' grand-children of 'country' children of the top-level
# elements
root.findall("./country/neighbor")

# Nodes with name='Singapore' that have a 'year' child
root.findall("./year/..[@name='Singapore']")

# 'year' nodes that are children of nodes with name='Singapore'
root.findall("./**[@name='Singapore']/year")

# All 'neighbor' nodes that are the second child of their parent
root.findall("./neighbor[2]")

```

For XML with namespaces, use the usual qualified{namespace}tag notation:

```

# All dublin-core "title" tags in the document
root.findall("./{http://purl.org/dc/elements/1.1/}title")

```

Supported XPath syntax

Syntax | Meaning | tag | Selects all child elements with the given tag. For example, | spam | selects all child elements named | spam | , and | spam/egg | selects all grandchildren named | egg | in all children named | spam | . | {namespace}* | selects all tags in the given namespace, | {*}spam | selects tags named | spam | in any (or no) namespace, and | {*}* | only selects tags that are not in a namespace. | Changed in version 3.8: | Support for star-wildcards was added. | * | Selects all child elements, including comments and processing instructions. For example, | */egg | selects all grandchildren named | egg | . | . | Selects the current node. This is mostly useful at the beginning of the path, to indicate that it's a relative path. | // | Selects all subelements, on all levels beneath

Predicates (expressions within square brackets) must be preceded by a tag name, an asterisk, or another predicate. position predicates must be preceded by a tag name.

Reference

Functions

xml.etree.ElementTree.canonicalize(xml_data=None,*,out=None,from_file=None,options)**

strip_text: set to true to strip whitespace before and after text content

rewrite_prefixes: set to true to replace namespace prefixes by “n{number}”

qname_aware_tags: a set of qname aware tag names in which prefixes

qname_aware_attrs: a set of qname aware attribute names in which prefixes

C14N 2.0 transformation function. Canonicalization is a way to normalise XML output in a way that allows byte-by-byte comparisons and digital signatures. It reduces the freedom that XML serializers have and instead generates a more constrained XML representation. The main restrictions regard the placement of namespace declarations, the ordering of attributes, and ignorable whitespace. This function takes an XML data string (xml_data) or a file path or file-like object (from_file) as input, conver

(default: false)

(default: false)

should be replaced in text content (default: empty)

should be replaced in text content (default: empty)

xml.etree.ElementTree.Comment(text=None)

Comment element factory. This factory function creates a special element that will be serialized as an XML comment by the standard serializer. The comment string can be either a bytestring or a Unicode string. `text` is a string containing the comment string. Returns an element instance representing a comment. Note that XMLParser skips over comments in the input instead of creating comment objects for them. An `ElementTree` will only contain comment nodes if they have been inserted into the tree using

xml.etree.ElementTree.dump(elem)

Writes an element tree or element structure to `sys.stdout`. This function should be used for debugging only. The exact output format is implementation dependent. In this version, it's written as an ordinary XML file. `elem` is an element tree or an individual element. Changed in version 3.8: The `dump()` function now preserves the attribute order specified by the user.

xml.etree.ElementTree.fromstring(text, parser=None)

Parses an XML section from a string constant. Same as `XML()`. `text` is a string containing XML data. `parser` is an optional parser instance. If not given, the standard `XMLParser` parser is used. Returns an `Element` instance.

xml.etree.ElementTree.fromstringlist(sequence, parser=None)

Parses an XML document from a sequence of string fragments. `sequence` is a list or other sequence containing XML data fragments. `parser` is an optional parser instance. If not given, the standard `XMLParser` parser is used. Returns an `Element` instance. Added in version 3.2.

xml.etree.ElementTree.indent(tree, space=" ", level=0)

Appends whitespace to the subtree to indent the tree visually. This can be used to generate pretty-printed XML output. `tree` can be an `Element` or `ElementTree`. `space` is the whitespace string that will be inserted for each indentation level, two space characters by default. For indenting partial subtrees inside of an already indented tree, pass the initial indentation level as `level`. Added in version 3.9.

xml.etree.ElementTree.iselement(element)

Check if an object appears to be a valid element object. `element` is an element instance. Return `True` if this is an element object.

xml.etree.ElementTree.iterparse(source, events=None, parser=None)

Parses an XML section into an element tree incrementally, and reports what's going on to the user. `source` is a filename or file object containing XML data. `events` is a sequence of events to report back. The supported events are the strings "start", "end", "comment", "pi", "start-ns" and "end-ns" (the "ns" events are used to get detailed namespace information). If `events` is omitted, only "end" events are reported. `parser` is an optional parser instance. If not given, the standard `XMLParser` parser is used. `parser` must

xml.etree.ElementTree.parse(source, parser=None)

Parses an XML section into an element tree. `source` is a filename or file object containing XML data. `parser` is an optional parser instance. If not given, the standard `XMLParser` parser is used. Returns an `ElementTree` instance.

xml.etree.ElementTree.ProcessingInstruction(target, text=None)

PI element factory. This factory function creates a special element that will be serialized as an XML processing instruction. `target` is a string containing the PI target. `text` is a string containing the PI contents, if given. Returns an element instance, representing a processing instruction. Note that XMLParser skips over processing instructions in the input instead of creating PI objects for them. An `ElementTree` will only contain processing instruction nodes if they have been inserted into the tree

xml.etree.ElementTree.register_namespace(prefix, uri)

Registers a namespace prefix. The registry is global, and any existing mapping for either the given prefix or the namespace URI will be removed. `prefix` is a namespace prefix. `uri` is a namespace uri. Tags and attributes in

this namespace will be serialized with the given prefix, if at all possible. Added in version 3.2.

xml.etree.ElementTree.SubElement(parent,tag,attrib={},extra)**

Subelement factory. This function creates an element instance, and appends it to an existing element. The element name, attribute names, and attribute values can be either bytestrings or Unicode strings. `parent` is the parent element. `tag` is the subelement name. `attrib` is an optional dictionary, containing element attributes. `extra` contains additional attributes, given as keyword arguments. Returns an element instance.

xml.etree.ElementTree.tostring(element,encoding='us-ascii',method='xml',*,xml_declaration=None,default_namespace=None,short_empty_elements=True)

Generates a string representation of an XML element, including all subelements. `element` is an `Element` instance. `encoding[1]` is the output encoding (default is US-ASCII). Use `encoding="unicode"` to generate a Unicode string (otherwise, a bytestring is generated). `method` is either "xml", "html" or "text" (default is "xml"). `xml_declaration`, `default_namespace` and `short_empty_elements` has the same meaning as in `ElementTree.write()`. Returns an (optionally) encoded string containing the XML data. Changed in version 3.4: Added

xml.etree.ElementTree.tostringlist(element,encoding='us-ascii',method='xml',*,xml_declaration=None,default_namespace=None,short_empty_elements=True)

Generates a string representation of an XML element, including all subelements. `element` is an `Element` instance. `encoding[1]` is the output encoding (default is US-ASCII). Use `encoding="unicode"` to generate a Unicode string (otherwise, a bytestring is generated). `method` is either "xml", "html" or "text" (default is "xml"). `xml_declaration`, `default_namespace` and `short_empty_elements` has the same meaning as in `ElementTree.write()`. Returns a list of (optionally) encoded strings containing the XML data. It does not guarantee

xml.etree.ElementTree.XML(text,parser=None)

Parses an XML section from a string constant. This function can be used to embed "XML literals" in Python code. `text` is a string containing XML data. `parser` is an optional parser instance. If not given, the standard `XMLParser` parser is used. Returns an `Element` instance.

xml.etree.ElementTree.XMLID(text,parser=None)

Parses an XML section from a string constant, and also returns a dictionary which maps from element id:s to elements. `text` is a string containing XML data. `parser` is an optional parser instance. If not given, the standard `XMLParser` parser is used. Returns a tuple containing an `Element` instance and a dictionary.

XInclude support

This module provides limited support for XInclude directives, via the `xml.etree.ElementInclude` helper module. This module can be used to insert subtrees and text strings into element trees, based on information in the tree.

Example

Here's an example that demonstrates use of the XInclude module. To include an XML document in the current document, use the `{http://www.w3.org/2001/XInclude}include` element and set the `parse` attribute to "xml", and use the `href` attribute to specify the document to include.

```
<?xml version="1.0"?>
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include href="source.xml" parse="xml" />
</document>
```

By default, the `href` attribute is treated as a file name. You can use custom loaders to override this behaviour. Also note that the standard helper does not support XPointer syntax.

To process this file, load it as usual, and pass the root element to the `xml.etree.ElementTree` module:

```
from xml.etree import ElementTree, ElementInclude

tree = ElementTree.parse("document.xml")
root = tree.getroot()

ElementInclude.include(root)
```

The `ElementInclude` module replaces the `{http://www.w3.org/2001/XInclude}includeelement` with the root element from the source XML document. The result might look something like this:

```
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  <para>This is a paragraph.</para>
</document>
```

If the `parseattribute` is omitted, it defaults to "xml". The `href` attribute is required.

To include a text document, use the `{http://www.w3.org/2001/XInclude}includeelement`, and set the `parseattribute` to "text":

```
<?xml version="1.0"?>
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  Copyright (c) <xi:include href="year.txt" parse="text" />.
</document>
```

The result might look something like:

```
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  Copyright (c) 2003.
</document>
```

Reference

Functions

`xml.etree.ElementInclude.default_loader(href,parse,encoding=None)`

Default loader. This default loader reads an included resource from disk. `href` is a URL. `parse` is for parse mode either "xml" or "text". `encoding` is an optional text encoding. If not given, encoding is utf-8. Returns the expanded resource. If the parse mode is "xml", this is an `Element` instance. If the parse mode is "text", this is a string. If the loader fails, it can return `None` or raise an exception.

`xml.etree.ElementInclude.include(elem,loader=None,base_url=None,max_depth=6)`

This function expands XInclude directives in-place in tree pointed by `elem`. `elem` is either the root `Element` or an `ElementTree` instance to find such element. `loader` is an optional resource loader. If omitted, it defaults to `default_loader()`. If given, it should be a callable that implements the same interface as `default_loader()`. `base_url` is base URL of the original file, to resolve relative include file references. `max_depth` is the maximum number of recursive inclusions. Limited to reduce the risk of malicious

Element Objects

`class xml.etree.ElementTree.Element(tag,attrib={},**extra)`

tag

text

tail

attrib

clear()
get(key,default=None)
items()
keys()
set(key,value)
append(subelement)
extend(subelements)
find(match,namespace=None)
findall(match,namespace=None)
findtext(match,default=None,namespace=None)
insert(index,subelement)
iter(tag=None)
iterfind(match,namespace=None)
itertext()
makeelement(tag,attrib)
remove(subelement)

Element class. This class defines the Element interface, and provides a reference implementation of this interface. The element name, attribute names, and attribute values can be either bytestrings or Unicode strings. `tag` is the element name. `attrib` is an optional dictionary, containing element attributes. `extra` contains additional attributes, given as keyword arguments. `tagA` string identifying what kind of data this element represents (the element type, in other words). `text` and `tail` These attributes can be

A string identifying what kind of data this element represents (the element type, in other words).

These attributes can be used to hold additional data associated with the element. Their values are usually strings but may be any application-specific object. If the element is created from an XML file, the `text` attribute holds either the text between the element's start tag and its first child or end tag, or `None`, and the `tail` attribute holds either the text between the element's end tag and the next tag, or `None`. For the XML data `<a><b>1<c>2<d/>3</c></b>4</a>` the element has `None` for both `text` and `tail`

A dictionary containing the element's attributes. Note that while the `attrib` value is always a real mutable Python dictionary, an `ElementTree` implementation may choose to use another internal representation, and create the dictionary only if someone asks for it. To take advantage of such implementations, use the dictionary methods below whenever possible.

Resets an element. This function removes all subelements, clears all attributes, and sets the text and tail attributes to `None`.

Gets the element attribute named `key`. Returns the attribute value, or `default` if the attribute was not found.

Returns the element attributes as a sequence of (name, value) pairs. The attributes are returned in an arbitrary order.

Returns the element's attribute names as a list. The names are returned in an arbitrary order.

Set the attribute `key` on the element to `value`.

Adds the element `subelement` to the end of this element's internal list of subelements. Raises `TypeError` if `subelement` is not an `Element`.

Appendssubelementsfrom an iterable of elements. RaisesTypeErrorif a subelement is not anElement.Added in version 3.2.

Finds the first subelement matchingmatch.matchmay be a tag name or apath. Returns an element instance orNone.namespacesis an optional mapping from namespace prefix to full name. Pass"as prefix to move all unprefix tag names in the expression into the given namespace.

Finds all matching subelements, by tag name orpath. Returns a list containing all matching elements in document order.namespacesis an optional mapping from namespace prefix to full name. Pass"as prefix to move all unprefix tag names in the expression into the given namespace.

Finds text for the first subelement matchingmatch.matchmay be a tag name or apath. Returns the text content of the first matching element, ordefaultif no element was found. Note that if the matching element has no text content an empty string is returned.namespacesis an optional mapping from namespace prefix to full name. Pass"as prefix to move all unprefix tag names in the expression into the given namespace.

Insertssubelementat the given position in this element. RaisesTypeErrorifsubelementis not anElement.

Creates a treeiteratorwith the current element as the root. The iterator iterates over this element and all elements below it, in document (depth first) order. Iftagis notNoneor'', only elements whose tag equalstagare returned from the iterator. If the tree structure is modified during iteration, the result is undefined.Added in version 3.2.

Finds all matching subelements, by tag name orpath. Returns an iterable yielding all matching elements in document order.namespacesis an optional mapping from namespace prefix to full name.Added in version 3.2.

Creates a text iterator. The iterator loops over this element and all subelements, in document order, and returns all inner text.Added in version 3.2.

Creates a new element object of the same type as this element. Do not call this method, use theSubElement()factory function instead.

Removessubelementfrom the element. Unlike the find* methods this method compares elements based on the instance identity, not on tag value or contents.

ElementTree Objects

classxml.etree.ElementTree.ElementTree(element=None,file=None)

_setroot(element)

find(match,namespace=None)

findall(match,namespace=None)

findtext(match,default=None,namespace=None)

getroot()

iter(tag=None)

iterfind(match,namespace=None)

parse(source,parser=None)

write(file,encoding='us-ascii',xml_declaration=None,default_namespace=None,method='xml',*,short_empty_elements=True)

ElementTree wrapper class. This class represents an entire element hierarchy, and adds some extra support for serialization to and from standard XML.elementis the root element. The tree is initialized with the contents of the XMLfileif given._setroot(element)Replaces the root element for this tree. This discards the current contents of the tree, and replaces it with the given element. Use with care.elementis an element

instance.find(match, namespaces=None) Same as Element.find(), starting at the

Replaces the root element for this tree. This discards the current contents of the tree, and replaces it with the given element. Use with care. element is an element instance.

Same as Element.find(), starting at the root of the tree.

Same as Element.findall(), starting at the root of the tree.

Same as Element.findtext(), starting at the root of the tree.

Returns the root element for this tree.

Creates and returns a tree iterator for the root element. The iterator loops over all elements in this tree, in section order. tag is the tag to look for (default is to return all elements).

Same as Element.iterfind(), starting at the root of the tree. Added in version 3.2.

Loads an external XML section into this element tree. source is a file name or file object. parser is an optional parser instance. If not given, the standard XMLParser parser is used. Returns the section root element.

Writes the element tree to a file, as XML. file is a file name, or a file object opened for writing. encoding[1] is the output encoding (default is US-ASCII). xml_declaration controls if an XML declaration should be added to the file. Use False for never, True for always, None for only if not US-ASCII or UTF-8 or Unicode (default is None). default_namespaces sets the default XML namespace (for "xmlns"). method is either "xml", "html" or "text" (default is "xml"). The keyword-only short_empty_elements parameter controls th

This is the XML file that is going to be manipulated:

```
<html>
  <head>
    <title>Example page</title>
  </head>
  <body>
    <p>Moved to <a href="http://example.org/">example.org</a>
    or <a href="http://example.com/">example.com</a>.</p>
  </body>
</html>
```

Example of changing the attribute "target" of every link in first paragraph:

```
>>> from xml.etree.ElementTree import ElementTree
>>> tree = ElementTree()
>>> tree.parse("index.xhtml")
<Element 'html' at 0xb77e6fac>
>>> p = tree.find("body/p")      # Finds first occurrence of tag p in body
>>> p
<Element 'p' at 0xb77ec26c>
>>> links = list(p.iter("a"))    # Returns list of all links
>>> links
[<Element 'a' at 0xb77ec2ac>, <Element 'a' at 0xb77ec1cc>]
>>> for i in links:              # Iterates through all found links
...     i.attrib["target"] = "blank"
...
>>> tree.write("output.xhtml")
```

QName Objects

class xml.etree.ElementTree.QName(text_or_uri, tag=None)

QName wrapper. This can be used to wrap a QName attribute value, in order to get proper namespace handling on output. text_or_uri is a string containing the QName value, in the form {uri}local, or, if the tag argument is given, the URI part of a QName. If tag is given, the first argument is interpreted as a URI, and this argument is interpreted as a local name. QName instances are opaque.

TreeBuilder Objects

classxml.etree.ElementTree.TreeBuilder(element_factory=None,*,comment_factory=None,pi_factory=None,insert_comments=False,insert_pis=False)

close()

data(data)

end(tag)

start(tag,attrs)

comment(text)

pi(target,text)

doctype(name,publicid,system)

start_ns(prefix,uri)

end_ns(prefix)

Generic element structure builder. This builder converts a sequence of start, data, end, comment and pi method calls to a well-formed element structure. You can use this class to build an element structure using a custom XML parser, or a parser for some other XML-like format.`element_factory`, when given, must be a callable accepting two positional arguments: a tag and a dict of attributes. It is expected to return a new element instance. The `comment_factory` and `pi_factory` functions, when given, sho

Flushes the builder buffers, and returns the toplevel document element. Returns an `Element` instance.

Adds text to the current element. `data` is a string. This should be either a bytestring, or a Unicode string.

Closes the current element. `tag` is the element name. Returns the closed element.

Opens a new element. `tag` is the element name. `attrs` is a dictionary containing element attributes. Returns the opened element.

Creates a comment with the given `text`. If `insert_comments` is true, this will also add it to the tree. Added in version 3.8.

Creates a process instruction with the given `target` `name` and `text`. If `insert_pis` is true, this will also add it to the tree. Added in version 3.8.

Handles a doctype declaration. `name` is the doctype name. `publicid` is the public identifier. `system` is the system identifier. This method does not exist on the default `TreeBuilder` class. Added in version 3.2.

Is called whenever the parser encounters a new namespace declaration, before the `start()` callback for the opening element that defines it. `prefix` is "for the default namespace and the declared namespace prefix name otherwise. `uri` is the namespace URI. Added in version 3.8.

Is called after the `end()` callback of an element that declared a namespace prefix mapping, with the name of the prefix that went out of scope. Added in version 3.8.

classxml.etree.ElementTree.C14NWriterTarget(write,*,with_comments=False,strip_text=False,rewrite_prefixes=False,qname_aware_tags=None,qname_aware_attrs=None,exclude_attrs=None,exclude_tags=None)

AC14N 2.0 writer. Arguments are the same as for the `canonicalize()` function. This class does not build a tree but translates the callback events directly into a serialised form using the `write` function. Added in version 3.8.

XMLParser Objects

classxml.etree.ElementTree.XMLParser(*,target=None,encoding=None)

close()

feed(data)

flush()

This class is the low-level building block of the module. It uses `xml.parsers.expat` for efficient, event-based parsing of XML. It can be fed XML data incrementally with the `feed()` method, and parsing events are translated to a push API - by invoking callbacks on the `target` object. If `target` is omitted, the standard `TreeBuilder` is used. If `encoding[1]` is given, the value overrides the encoding specified in the XML file. Changed in version 3.8: Parameters are now keyword-only. The `html` argument is no longer supported.

Finishes feeding data to the parser. Returns the result of calling the `close()` method of the `target` passed during construction; by default, this is the top-level document element.

Feeds data to the parser. `data` is encoded data.

Triggers parsing of any previously fed unparsed data, which can be used to ensure more immediate feedback, in particular with Expat $\geq 2.6.0$. The implementation of `flush()` temporarily disables reparsing deferral with Expat (if currently enabled) and triggers a reparsing. Disabling reparsing deferral has security consequences; please see `xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()` for details. Note that `flush()` has been backported to some prior releases of CPython as a security fix. Check for availability.

XMLPullParser Objects

`class xml.etree.ElementTree.XMLPullParser(events=None)`

`feed(data)`

`flush()`

`close()`

`read_events()`

A pull parser suitable for non-blocking applications. Its input-side API is similar to that of `XMLParser`, but instead of pushing calls to a callback target, `XMLPullParser` collects an internal list of parsing events and lets the user read from it. `events` is a sequence of events to report back. The supported events are the strings "start", "end", "comment", "pi", "start-ns" and "end-ns" (the "ns" events are used to get detailed namespace information). If `events` is omitted, only "end" events are reported. `feed(data)`

Feed the given bytes data to the parser.

Triggers parsing of any previously fed unparsed data, which can be used to ensure more immediate feedback, in particular with Expat $\geq 2.6.0$. The implementation of `flush()` temporarily disables reparsing deferral with Expat (if currently enabled) and triggers a reparsing. Disabling reparsing deferral has security consequences; please see `xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()` for details. Note that `flush()` has been backported to some prior releases of CPython as a security fix. Check for availability.

Signal the parser that the data stream is terminated. Unlike `XMLParser.close()`, this method always returns `None`. Any events not yet retrieved when the parser is closed can still be read with `read_events()`.

Return an iterator over the events which have been encountered in the data fed to the parser. The iterator yields `(event, elem)` pairs, where `event` is a string representing the type of event (e.g. "end") and `elem` is the encountered `Element` object, or other context value as follows. `start`, `end`: the current `Element`. `comment`, `pi`: the current comment / processing instruction. `start-ns`: a tuple `(prefix, uri)` naming the declared namespace mapping. `end-ns`: `None` (this may change in a future version). Events provided in a previous

Exceptions

`class xml.etree.ElementTree.ParseError`

code

position

XML parse error, raised by the various parsing methods in this module when parsing fails. The string representation of an instance of this exception will contain a user-friendly error message. In addition, it will have the following attributes available: `code` A numeric error code from the expat parser. See the documentation of `xml.parsers.expat` for the list of error codes and their meanings. `position` A tuple of `line`, `column` numbers, specifying where the error occurred.

A numeric error code from the expat parser. See the documentation of `xml.parsers.expat` for the list of error codes and their meanings.

A tuple of `line`, `column` numbers, specifying where the error occurred.

Footnotes

The encoding string included in XML output should conform to the appropriate standards. For example, "UTF-8" is valid, but "UTF8" is not. See <https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl> and <https://www.iana.org/assignments/character-sets/character-sets.xhtml>.

xml.dom— The Document Object Model API

Source code: `Lib/xml/dom/__init__.py`

The Document Object Model, or “DOM,” is a cross-language API from the World Wide Web Consortium (W3C) for accessing and modifying XML documents. A DOM implementation presents an XML document as a tree structure, or allows client code to build such a structure from scratch. It then gives access to the structure through a set of objects which provided well-known interfaces.

The DOM is extremely useful for random-access applications. SAX only allows you a view of one bit of the document at a time. If you are looking at one SAX element, you have no access to another. If you are looking at a text node, you have no access to a containing element. When you write a SAX application, you need to keep track of your program's position in the document somewhere in your own code. SAX does not do it for you. Also, if you need to look ahead in the XML document, you are just out of luck.

Some applications are simply impossible in an event driven model with no access to a tree. Of course you could build some sort of tree yourself in SAX events, but the DOM allows you to avoid writing that code. The DOM is a standard tree representation for XML data.

The Document Object Model is being defined by the W3C in stages, or “levels” in their terminology. The Python mapping of the API is substantially based on the DOM Level 2 recommendation.

DOM applications typically start by parsing some XML into a DOM. How this is accomplished is not covered at all by DOM Level 1, and Level 2 provides only limited improvements: There is a `DOMImplementation` object class which provides access to document creation methods, but no way to access an XML reader/parser/document builder in an implementation-independent way. There is also no well-defined way to access these methods without an existing document object. In Python, each DOM implementation will provide a function `getDOMImplementation()`. DOM Level 3 adds a Load/Store specification, which defines an interface to the reader, but this is not yet available in the Python standard library.

Once you have a DOM document object, you can access the parts of your XML document through its properties and methods. These properties are defined in the DOM specification; this portion of the reference manual describes the interpretation of the specification in Python.

The specification provided by the W3C defines the DOM API for Java, ECMAScript, and OMG IDL. The Python mapping defined here is based in large part on the IDL version of the specification, but strict compliance is not required (though implementations are free to support the strict mapping from IDL). See section [Conformance](#) for a detailed discussion of mapping requirements.

See also

Document Object Model (DOM) Level 2 Specification

Document Object Model (DOM) Level 1 Specification

Python Language Mapping Specification

The W3C recommendation upon which the Python DOM API is based.

The W3C recommendation for the DOM supported by `xml.dom.minidom`.

This specifies the mapping from OMG IDL to Python.

Module Contents

The `xml.dom` contains the following functions:

`xml.dom.registerDOMImplementation(name, factory)`

Register the factory function with the name `name`. The factory function should return an object which implements the `DOMImplementation` interface. The factory function can return the same object every time, or a new one for each call, as appropriate for the specific implementation (e.g. if that implementation supports some customization).

`xml.dom.getDOMImplementation(name=None, features=())`

Return a suitable DOM implementation. The `name` is either well-known, the module name of a DOM implementation, or `None`. If it is not `None`, imports the corresponding module and returns a `DOMImplementation` object if the import succeeds. If no name is given, and if the environment variable `PYTHON_DOM` is set, this variable is used to find the implementation. If `name` is not given, this examines the available implementations to find one with the required feature set. If no implementation can be found, raise a

Some convenience constants are also provided:

`xml.dom.EMPTY_NAMESPACE`

The value used to indicate that no namespace is associated with a node in the DOM. This is typically found as the `namespaceURI` of a node, or used as the `namespaceURI` parameter to a namespaces-specific method.

`xml.dom.XML_NAMESPACE`

The namespace URI associated with the reserved prefix `xml`, as defined by `Namespaces` in XML (section 4).

`xml.dom.XMLNS_NAMESPACE`

The namespace URI for namespace declarations, as defined by Document Object Model (DOM) Level 2 Core Specification (section 1.1.8).

`xml.dom.XHTML_NAMESPACE`

The URI of the XHTML namespace as defined by XHTML 1.0: The Extensible HyperText Markup Language (section 3.1.1).

In addition, `xml.dom` contains a `baseNode` class and the DOM exception classes. The `Node` class provided by this module does not implement any of the methods or attributes defined by the DOM specification; concrete DOM implementations must provide those. The `Node` class provided as part of this module does provide the constants used for the `nodeType` attribute on concrete `Node` objects; they are located within the class rather than at the module level to conform with the DOM specifications.

Objects in the DOM

The definitive documentation for the DOM is the DOM specification from the W3C.

Note that DOM attributes may also be manipulated as nodes instead of as simple strings. It is fairly rare that you must do this, however, so this usage is not yet documented.

Interface		Section		Purpose		DOMImplementation		DOMImplementation Objects		Interface to the underlying implementation.
Node		Node Objects		Base interface for most objects in a document.		NodeList		NodeList Objects		Interface for a sequence of nodes.
DocumentType		DocumentType Objects		Information about the declarations needed to process a document.		Document		Document Objects		Object which represents an entire document.
Element		Element Objects		Element nodes in the document hierarchy.		Attr		Attr Objects		Attribute value nodes on element nodes.
Comment		Comment Objects		Representation of comments in the source document.		Text		Text and CDATASection Objects		Nodes containing textual content from the document.
ProcessingInstruction		ProcessingInstruction Objects		Representation of processing instructions in the source document.						

An additional section describes the exceptions defined for working with the DOM in Python.

DOMImplementation Objects

The `DOMImplementation` interface provides a way for applications to determine the availability of particular features in the DOM they are using. DOM Level 2 added the ability to create `newDocument` and `DocumentType` objects using the `DOMImplementation` as well.

`DOMImplementation.hasFeature(feature,version)`

Return `True` if the feature identified by the pair of strings `feature` and `version` is implemented.

`DOMImplementation.createDocument(namespaceUri,qualifiedName,doctype)`

Return a new `Document` object (the root of the DOM), with a `childElement` object having the given `namespaceUri` and `qualifiedName`. The `doctype` must be a `DocumentType` object created by `createDocumentType()`, or `None`. In the Python DOM API, the first two arguments can also be `None` in order to indicate that no `Element` child is to be created.

`DOMImplementation.createDocumentType(qualifiedName,publicId,systemId)`

Return a new `DocumentType` object that encapsulates the given `qualifiedName`, `publicId`, and `systemId` strings, representing the information contained in an XML document type declaration.

Node Objects

All of the components of an XML document are subclasses of `Node`.

`Node.nodeType`

An integer representing the node type. Symbolic constants for the types are on the `Node` object: `ELEMENT_NODE`, `ATTRIBUTE_NODE`, `TEXT_NODE`, `CDATA_SECTION_NODE`, `ENTITY_NODE`, `PROCESSING_INSTRUCTION_NODE`, `COMMENT_NODE`, `DOCUMENT_NODE`, `DOCUMENT_TYPE_NODE`, `NOTATION_NODE`. This is a read-only attribute.

`Node.parentNode`

The parent of the current node, or `None` for the document node. The value is always a `Node` object or `None`. For `Element` nodes, this will be the parent element, except for the root element, in which case it will be the `Document` object. For `Attr` nodes, this is always `None`. This is a read-only attribute.

`Node.attributes`

A `NamedNodeMap` of attribute objects. Only elements have actual values for this; others provide `None` for this attribute. This is a read-only attribute.

`Node.previousSibling`

The node that immediately precedes this one with the same parent. For instance the element with an end-tag that comes just before the self element's start-tag. Of course, XML documents are made up of more than just elements so the previous sibling could be text, a comment, or something else. If this node is the first child of the parent, this attribute will be `None`. This is a read-only attribute.

`Node.nextSibling`

The node that immediately follows this one with the same parent. See also `previousSibling`. If this is the last child of the parent, this attribute will be `None`. This is a read-only attribute.

`Node.childNodes`

A list of nodes contained within this node. This is a read-only attribute.

`Node.firstChild`

The first child of the node, if there are any, or `None`. This is a read-only attribute.

`Node.lastChild`

The last child of the node, if there are any, orNone. This is a read-only attribute.

Node.localName

The part of thetagNamefollowing the colon if there is one, else the entiretagName. The value is a string.

Node.prefix

The part of thetagNamepreceding the colon if there is one, else the empty string. The value is a string, orNone.

Node.namespaceURI

The namespace associated with the element name. This will be a string orNone. This is a read-only attribute.

Node.nodeName

This has a different meaning for each node type; see the DOM specification for details. You can always get the information you would get here from another property such as thetagNameproperty for elements or thenameproperty for attributes. For all node types, the value of this attribute will be either a string orNone. This is a read-only attribute.

Node.nodeValue

This has a different meaning for each node type; see the DOM specification for details. The situation is similar to that withnodeName. The value is a string orNone.

Node.hasAttributes()

ReturnTrueif the node has any attributes.

Node.hasChildNodes()

ReturnTrueif the node has any child nodes.

Node.isSameNode(other)

ReturnTrueifotherrefers to the same node as this node. This is especially useful for DOM implementations which use any sort of proxy architecture (because more than one object can refer to the same node).NoteThis is based on a proposed DOM Level 3 API which is still in the “working draft” stage, but this particular interface appears uncontroversial. Changes from the W3C will not necessarily affect this method in the Python DOM interface (though any new W3C API for this would also be supported).

Node.appendChild(newChild)

Add a new child node to this node at the end of the list of children, returningnewChild. If the node was already in the tree, it is removed first.

Node.insertBefore(newChild,refChild)

Insert a new child node before an existing child. It must be the case thatrefChildis a child of this node; if not,ValueErroris raised.newChildis returned. IfrefChildisNone, it insertsnewChildat the end of the children’s list.

Node.removeChild(oldChild)

Remove a child node.oldChildmust be a child of this node; if not,ValueErroris raised.oldChildis returned on success. IfoldChildwill not be used further, itsunlink()method should be called.

Node.replaceChild(newChild,oldChild)

Replace an existing node with a new node. It must be the case thatoldChildis a child of this node; if not,ValueErroris raised.

Node.normalize()

Join adjacent text nodes so that all stretches of text are stored as single `Text` instances. This simplifies processing text from a DOM tree for many applications.

Node.cloneNode(deep)

Clone this node. Setting `deep` means to clone all child nodes as well. This returns the clone.

NodeList Objects

A `NodeList` represents a sequence of nodes. These objects are used in two ways in the DOM Core recommendation: an `Element` object provides one as its list of child nodes, and the `getElementsByTagName()` and `getElementsByTagNameNS()` methods of `Node` return objects with this interface to represent query results.

The DOM Level 2 recommendation defines one method and one attribute for these objects:

NodeList.item(i)

Return the *i*th item from the sequence, if there is one, or `None`. The index *i* is not allowed to be less than zero or greater than or equal to the length of the sequence.

NodeList.length

The number of nodes in the sequence.

In addition, the Python DOM interface requires that some additional support is provided to allow `NodeList` objects to be used as Python sequences. All `NodeList` implementations must include support for `__len__()` and `__getitem__()`; this allows iteration over the `NodeList` in for statements and proper support for the `len()` built-in function.

If a DOM implementation supports modification of the document, the `NodeList` implementation must also support the `__setitem__()` and `__delitem__()` methods.

DocumentType Objects

Information about the notations and entities declared by a document (including the external subset if the parser uses it and can provide the information) is available from a `DocumentType` object. The `DocumentType` for a document is available from the `Document` object's `doctype` attribute; if there is no `DOCTYPE` declaration for the document, the document's `doctype` attribute will be set to `None` instead of an instance of this interface.

`DocumentType` is a specialization of `Node`, and adds the following attributes:

DocumentType.publicId

The public identifier for the external subset of the document type definition. This will be a string or `None`.

DocumentType.systemId

The system identifier for the external subset of the document type definition. This will be a URI as a string, or `None`.

DocumentType.internalSubset

A string giving the complete internal subset from the document. This does not include the brackets which enclose the subset. If the document has no internal subset, this should be `None`.

DocumentType.name

The name of the root element as given in the `DOCTYPE` declaration, if present.

DocumentType.entities

This is a `NamedNodeMap` giving the definitions of external entities. For entity names defined more than once, only the first definition is provided (others are ignored as required by the XML recommendation). This may

beNone if the information is not provided by the parser, or if no entities are defined.

DocumentType.notations

This is a `NamedNodeMap` giving the definitions of notations. For notation names defined more than once, only the first definition is provided (others are ignored as required by the XML recommendation). This may be `None` if the information is not provided by the parser, or if no notations are defined.

Document Objects

A `Document` represents an entire XML document, including its constituent elements, attributes, processing instructions, comments etc. Remember that it inherits properties from `Node`.

Document.documentElement

The one and only root element of the document.

Document.createElement(tagName)

Create and return a new element node. The element is not inserted into the document when it is created. You need to explicitly insert it with one of the other methods such as `insertBefore()` or `appendChild()`.

Document.createElementNS(namespaceURI, tagName)

Create and return a new element with a namespace. The `tagName` may have a prefix. The element is not inserted into the document when it is created. You need to explicitly insert it with one of the other methods such as `insertBefore()` or `appendChild()`.

Document.createTextNode(data)

Create and return a text node containing the data passed as a parameter. As with the other creation methods, this one does not insert the node into the tree.

Document.createComment(data)

Create and return a comment node containing the data passed as a parameter. As with the other creation methods, this one does not insert the node into the tree.

Document.createProcessingInstruction(target, data)

Create and return a processing instruction node containing the `target` and `data` passed as parameters. As with the other creation methods, this one does not insert the node into the tree.

Document.createAttribute(name)

Create and return an attribute node. This method does not associate the attribute node with any particular element. You must use `setAttributeNode()` on the appropriate `Element` object to use the newly created attribute instance.

Document.createAttributeNS(namespaceURI, qualifiedName)

Create and return an attribute node with a namespace. The `tagName` may have a prefix. This method does not associate the attribute node with any particular element. You must use `setAttributeNode()` on the appropriate `Element` object to use the newly created attribute instance.

Document.getElementsByTagName(tagName)

Search for all descendants (direct children, children's children, etc.) with a particular element type name.

Document.getElementsByTagNameNS(namespaceURI, localName)

Search for all descendants (direct children, children's children, etc.) with a particular namespace URI and `localName`. The `localName` is the part of the namespace after the prefix.

Element Objects

Element is a subclass of Node, so inherits all the attributes of that class.

Element.tagName

The element type name. In a namespace-using document it may have colons in it. The value is a string.

Element.getElementsByTagName(tagName)

Same as equivalent method in the Document class.

Element.getElementsByTagNameNS(namespaceURI, localName)

Same as equivalent method in the Document class.

Element.hasAttribute(name)

Return True if the element has an attribute named by name.

Element.hasAttributeNS(namespaceURI, localName)

Return True if the element has an attribute named by namespaceURI and localName.

Element.getAttribute(name)

Return the value of the attribute named by name as a string. If no such attribute exists, an empty string is returned, as if the attribute had no value.

Element.getAttributeNode(attrname)

Return the Attr node for the attribute named by attrname.

Element.getAttributeNS(namespaceURI, localName)

Return the value of the attribute named by namespaceURI and localName as a string. If no such attribute exists, an empty string is returned, as if the attribute had no value.

Element.getAttributeNodeNS(namespaceURI, localName)

Return an attribute value as a node, given a namespaceURI and localName.

Element.removeAttribute(name)

Remove an attribute by name. If there is no matching attribute, a NotFoundErr is raised.

Element.removeAttributeNode(oldAttr)

Remove and return oldAttr from the attribute list, if present. If oldAttr is not present, NotFoundErr is raised.

Element.removeAttributeNS(namespaceURI, localName)

Remove an attribute by name. Note that it uses a localName, not a qname. No exception is raised if there is no matching attribute.

Element.setAttribute(name, value)

Set an attribute value from a string.

Element.setAttributeNode(newAttr)

Add a new attribute node to the element, replacing an existing attribute if necessary if the name attribute matches. If a replacement occurs, the old attribute node will be returned. If newAttr is already in use, InuseAttributeErr will be raised.

Element.setAttributeNodeNS(newAttr)

Add a new attribute node to the element, replacing an existing attribute if necessary if the namespaceURI and localName attributes match. If a replacement occurs, the old attribute node will be

returned. If `newAttr` is already in use, `InuseAttributeErr` will be raised.

Element.setAttributeNS(namespaceURI,qname,value)

Set an attribute value from a string, given a `namespaceURI` and a `qname`. Note that a `qname` is the whole attribute name. This is different than above.

Attr Objects

`Attr` inherits from `Node`, so inherits all its attributes.

Attr.name

The attribute name. In a namespace-using document it may include a colon.

Attr.localName

The part of the name following the colon if there is one, else the entire name. This is a read-only attribute.

Attr.prefix

The part of the name preceding the colon if there is one, else the empty string.

Attr.value

The text value of the attribute. This is a synonym for the `nodeValue` attribute.

NamedNodeMap Objects

`NamedNodeMap` does not inherit from `Node`.

NamedNodeMap.length

The length of the attribute list.

NamedNodeMap.item(index)

Return an attribute with a particular index. The order you get the attributes in is arbitrary but will be consistent for the life of a DOM. Each item is an attribute node. Get its value with the `value` attribute.

There are also experimental methods that give this class more mapping behavior. You can use them or you can use the `standardizedGetAttribute*()` family of methods on the `Element` objects.

Comment Objects

`Comment` represents a comment in the XML document. It is a subclass of `Node`, but cannot have child nodes.

Comment.data

The content of the comment as a string. The attribute contains all characters between the leading `<!--` and trailing `-->`, but does not include them.

Text and CDATASection Objects

The `Text` interface represents text in the XML document. If the parser and DOM implementation support the DOM's XML extension, portions of the text enclosed in CDATA marked sections are stored in `CDATASection` objects. These two interfaces are identical, but provide different values for the `nodeType` attribute.

These interfaces extend the `Node` interface. They cannot have child nodes.

Text.data

The content of the text node as a string.

■ Nota: NoteThe use of aCDATASectionnode does not indicate that the node represents a complete CDATA marked section, only that the content of the node was part of a CDATA section. A single CDATA section may be represented by more than one node in the document tree. There is no way to determine whether two adjacentCDATASectionnodes represent different CDATA marked sections.

ProcessingInstruction Objects

Represents a processing instruction in the XML document; this inherits from theNodeinterface and cannot have child nodes.

ProcessingInstruction.target

The content of the processing instruction up to the first whitespace character. This is a read-only attribute.

ProcessingInstruction.data

The content of the processing instruction following the first whitespace character.

Exceptions

The DOM Level 2 recommendation defines a single exception,DOMException, and a number of constants that allow applications to determine what sort of error occurred.DOMExceptioninstances carry acodeattribute that provides the appropriate value for the specific exception.

The Python DOM interface provides the constants, but also expands the set of exceptions so that a specific exception exists for each of the exception codes defined by the DOM. The implementations must raise the appropriate specific exception, each of which carries the appropriate value for thecodeattribute.

exceptionxml.dom.DOMException

Base exception class used for all specific DOM exceptions. This exception class cannot be directly instantiated.

exceptionxml.dom.DomstringSizeErr

Raised when a specified range of text does not fit into a string. This is not known to be used in the Python DOM implementations, but may be received from DOM implementations not written in Python.

exceptionxml.dom.HierarchyRequestErr

Raised when an attempt is made to insert a node where the node type is not allowed.

exceptionxml.dom.IndexSizeErr

Raised when an index or size parameter to a method is negative or exceeds the allowed values.

exceptionxml.dom.InuseAttributeErr

Raised when an attempt is made to insert anAttrnode that is already present elsewhere in the document.

exceptionxml.dom.InvalidAccessErr

Raised if a parameter or an operation is not supported on the underlying object.

exceptionxml.dom.InvalidCharacterErr

This exception is raised when a string parameter contains a character that is not permitted in the context it's being used in by the XML 1.0 recommendation. For example, attempting to create anElementnode with a space in the element type name will cause this error to be raised.

exceptionxml.dom.InvalidModificationErr

Raised when an attempt is made to modify the type of a node.

exceptionxml.dom.InvalidStateErr

Raised when an attempt is made to use an object that is not defined or is no longer usable.

exceptionxml.dom.NamespaceErr

If an attempt is made to change any object in a way that is not permitted with regard to theNamespaces in XMLrecommendation, this exception is raised.

exceptionxml.dom.NotFoundErr

Exception when a node does not exist in the referenced context. For example,NamedNodeMap.removeNamedItem()will raise this if the node passed in does not exist in the map.

exceptionxml.dom.NotSupportedErr

Raised when the implementation does not support the requested type of object or operation.

exceptionxml.dom.NoDataAllowedErr

This is raised if data is specified for a node which does not support data.

exceptionxml.dom.NoModificationAllowedErr

Raised on attempts to modify an object where modifications are not allowed (such as for read-only nodes).

exceptionxml.dom.SyntaxErr

Raised when an invalid or illegal string is specified.

exceptionxml.dom.WrongDocumentErr

Raised when a node is inserted in a different document than it currently belongs to, and the implementation does not support migrating the node from one document to the other.

The exception codes defined in the DOM recommendation map to the exceptions described above according to this table:

Constant	Exception	DOMSTRING_SIZE_ERR	DomstringSizeErr	HIERARCHY_REQUEST_ERR	HierarchyRequestErr	INDEX_SIZE_ERR	IndexSizeErr	INUSE_ATTRIBUTE_ERR	InuseAttributeErr	INVALID_ACCESS_ERR	InvalidAccessErr	INVALID_CHARACTER_ERR	InvalidCharacterErr	INVALID_MODIFICATION_ERR	InvalidModificationErr	INVALID_STATE_ERR	InvalidStateErr	NAMESPACE_ERR	NamespaceErr	NOT_FOUND_ERR	NotFoundErr	NOT_SUPPORTED_ERR	NotSupportedErr	NO_DATA_ALLOWED_ERR	NoDataAllowedErr	NO_MODIFICATION_ALLOWED_ERR	NoModificationAllowedErr	SYNTAX_ERR	SyntaxErr	WRONG_DOCUMENT_ERR	WrongDocumentErr
----------	-----------	--------------------	------------------	-----------------------	---------------------	----------------	--------------	---------------------	-------------------	--------------------	------------------	-----------------------	---------------------	--------------------------	------------------------	-------------------	-----------------	---------------	--------------	---------------	-------------	-------------------	-----------------	---------------------	------------------	-----------------------------	--------------------------	------------	-----------	--------------------	------------------

Conformance

This section describes the conformance requirements and relationships between the Python DOM API, the W3C DOM recommendations, and the OMG IDL mapping for Python.

Type Mapping

The IDL types used in the DOM specification are mapped to Python types according to the following table.

IDL Type	Python Type	boolean	bool	or	int	int	int	long	int	int	unsigned	int	int	DOMString	str	or	bytes	null	None
----------	-------------	---------	------	----	-----	-----	-----	------	-----	-----	----------	-----	-----	-----------	-----	----	-------	------	------

Accessor Methods

The mapping from OMG IDL to Python defines accessor functions for IDLattributedeclarations in much the way the Java mapping does. Mapping the IDL declarations

```
readonly attribute string someValue;  
    attribute string anotherValue;
```

yields three accessor functions: a “get” method for someValue(_get_someValue()), and “get” and “set” methods for anotherValue(_get_anotherValue() and _set_anotherValue()). The mapping, in particular, does not require that the IDL attributes are accessible as normal Python attributes: object.someValue is not required to work, and may raise an AttributeError.

The Python DOM API, however, does require that normal attribute access work. This means that the typical surrogates generated by Python IDL compilers are not likely to work, and wrapper objects may be needed on the client if the DOM objects are accessed via CORBA. While this does require some additional consideration for CORBA DOM clients, the implementers with experience using DOM over CORBA from Python do not consider this a problem. Attributes that are declared readonly may not restrict write access in all DOM implementations.

In the Python DOM API, accessor functions are not required. If provided, they should take the form defined by the Python IDL mapping, but these methods are considered unnecessary since the attributes are accessible directly from Python. “Set” accessors should never be provided for readonly attributes.

The IDL definitions do not fully embody the requirements of the W3C DOM API, such as the notion of certain objects, such as the return value of getElementByTagName(), being “live”. The Python DOM API does not require implementations to enforce such requirements.

xml.dom.minidom— Minimal DOM implementation

Source code: `Lib/xml/dom/minidom.py`

`xml.dom.minidom` is a minimal implementation of the Document Object Model interface, with an API similar to that in other languages. It is intended to be simpler than the full DOM and also significantly smaller. Users who are not already proficient with the DOM should consider using the `xml.etree.ElementTree` module for their XML processing instead.

■ **Nota:** Not if you need to parse untrusted or unauthenticated data, see XML security.

DOM applications typically start by parsing some XML into a DOM. With `xml.dom.minidom`, this is done through the parse functions:

```
from xml.dom.minidom import parse, parseString

dom1 = parse('c:\\temp\\mydata.xml') # parse an XML file by name

datasource = open('c:\\temp\\mydata.xml')
dom2 = parse(datasource) # parse an open file

dom3 = parseString('<myxml>Some data<empty/> some more data</myxml>')
```

The `parse()` function can take either a filename or an open file object.

`xml.dom.minidom.parse(filename_or_file, parser=None, bufsize=None)`

Return a `Document` from the given input. `filename_or_file` may be either a file name, or a file-like object. `parser`, if given, must be a SAX2 parser object. This function will change the document handler of the parser and activate namespace support; other parser configuration (like setting an entity resolver) must have been done in advance.

If you have XML in a string, you can use the `parseString()` function instead:

`xml.dom.minidom.parseString(string, parser=None)`

Return a `Document` that represents the string. This method creates an `io.StringIO` object for the string and passes that on to `parse()`.

Both functions return a `Document` object representing the content of the document.

What the `parse()` and `parseString()` functions do is connect an XML parser with a “DOM builder” that can accept parse events from any SAX parser and convert them into a DOM tree. The names of the functions are perhaps misleading, but are easy to grasp when learning the interfaces. The parsing of the document will be completed before these functions return; it's simply that these functions do not provide a parser implementation themselves.

You can also create a `Document` by calling a method on a “DOM Implementation” object. You can get this object either by calling the `getDOMImplementation()` function in the `xml.dom` package or the `xml.dom.minidom` module. Once you have a `Document`, you can add child nodes to it to populate the DOM:

```
from xml.dom.minidom import getDOMImplementation

impl = getDOMImplementation()

newdoc = impl.createDocument(None, "some_tag", None)
top_element = newdoc.documentElement
text = newdoc.createTextNode('Some textual content.')
top_element.appendChild(text)
```

Once you have a DOM document object, you can access the parts of your XML document through its properties and methods. These properties are defined in the DOM specification. The main property of the document object is the `documentElement` property. It gives you the main element in the XML document: the one that holds all

others. Here is an example program:

```
dom3 = parseString("<myxml>Some data</myxml>")
assert dom3.documentElement.tagName == "myxml"
```

When you are finished with a DOM tree, you may optionally call the `unlink()` method to encourage early cleanup of the now-unneeded objects. `unlink()` is an `xml.dom.minidom`-specific extension to the DOM API that renders the node and its descendants essentially useless. Otherwise, Python's garbage collector will eventually take care of the objects in the tree.

See also

Document Object Model (DOM) Level 1 Specification

The W3C recommendation for the DOM supported by `xml.dom.minidom`.

DOM Objects

The definition of the DOM API for Python is given as part of the `xml.dom` module documentation. This section lists the differences between the API and `xml.dom.minidom`.

Node.unlink()

Break internal references within the DOM so that it will be garbage collected on versions of Python without cyclic GC. Even when cyclic GC is available, using this can make large amounts of memory available sooner, so calling this on DOM objects as soon as they are no longer needed is good practice. This only needs to be called on the `Document` object, but may be called on child nodes to discard children of that node. You can avoid calling this method explicitly by using the `with` statement. The following

Node.writexml(writer, indent="", addindent="", newl="", encoding=None, standalone=None)

Write XML to the writer object. The writer receives texts but not bytes as input, it should have a `write()` method which matches that of the file object interface. The `indent` parameter is the indentation of the current node. The `addindent` parameter is the incremental indentation to use for subnodes of the current one. The `newl` parameter specifies the string to use to terminate newlines. For the `Document` node, an additional keyword argument `encoding` can be used to specify the encoding field of the XML header.

Node.toxml(encoding=None, standalone=None)

Return a string or byte string containing the XML represented by the DOM node. With an explicit `encoding` argument, the result is a byte string in the specified encoding. With no `encoding` argument, the result is a Unicode string, and the XML declaration in the resulting string does not specify an encoding. Encoding this string in an encoding other than UTF-8 is likely incorrect, since UTF-8 is the default encoding of XML. The `standalone` argument behaves exactly as in `writexml()`. Changed in version 3.8: The

Node.toprettyxml(indent='t', newl='n', encoding=None, standalone=None)

Return a pretty-printed version of the document. `indent` specifies the indentation string and defaults to a tabulator; `newl` specifies the string emitted at the end of each line and defaults to `\n`. The `encoding` argument behaves like the corresponding argument of `toxml()`. The `standalone` argument behaves exactly as in `writexml()`. Changed in version 3.8: The `toprettyxml()` method now preserves the attribute order specified by the user. Changed in version 3.9: The `standalone` parameter was added.

DOM Example

This example program is a fairly realistic example of a simple program. In this particular case, we do not take much advantage of the flexibility of the DOM.

```

import xml.dom.minidom

document = """\
<slideshow>
<title>Demo slideshow</title>
<slide><title>Slide title</title>
<point>This is a demo</point>
<point>Of a program for processing slides</point>
</slide>

<slide><title>Another demo slide</title>
<point>It is important</point>
<point>To have more than</point>
<point>one slide</point>
</slide>
</slideshow>
"""

dom = xml.dom.minidom.parseString(document)

def getText(nodelist):
    rc = []
    for node in nodelist:
        if node.nodeType == node.TEXT_NODE:
            rc.append(node.data)
    return ''.join(rc)

def handleSlideshow(slideshow):
    print("<html>")
    handleSlideshowTitle(slideshow.getElementsByTagName("title")[0])
    slides = slideshow.getElementsByTagName("slide")
    handleToc(slides)
    handleSlides(slides)
    print("</html>")

def handleSlides(slides):
    for slide in slides:
        handleSlide(slide)

def handleSlide(slide):
    handleSlideTitle(slide.getElementsByTagName("title")[0])
    handlePoints(slide.getElementsByTagName("point"))

def handleSlideshowTitle(title):
    print(f"<title>{getText(title.childNodes)}</title>")

def handleSlideTitle(title):
    print(f"<h2>{getText(title.childNodes)}</h2>")

def handlePoints(points):
    print("<ul>")
    for point in points:
        handlePoint(point)
    print("</ul>")

def handlePoint(point):
    print(f"<li>{getText(point.childNodes)}</li>")

def handleToc(slides):
    for slide in slides:
        title = slide.getElementsByTagName("title")[0]
        print(f"<p>{getText(title.childNodes)}</p>")

handleSlideshow(dom)

```

minidom and the DOM standard

The `xml.dom.minidom` module is essentially a DOM 1.0-compatible DOM with some DOM 2 features (primarily namespace features).

Usage of the DOM interface in Python is straight-forward. The following mapping rules apply:

- Interfaces are accessed through instance objects. Applications should not instantiate the classes themselves; they should use the creator functions available on the `Document` object. Derived interfaces support all operations (and attributes) from the base interfaces, plus any new operations.
- Operations are used as methods. Since the DOM uses only inparameters, the arguments are passed in normal order (from left to right). There are no optional arguments. void operations return `None`.
- IDL attributes map to instance attributes. For compatibility with the OMG IDL language mapping for Python, an attribute `foo` can also be accessed through accessor methods `_get_foo()` and `_set_foo()`. readonly attributes must not be changed; this is not enforced at runtime.
- The types `short`, `int`, `unsigned int`, `unsigned long long`, and `boolean` all map to Python integer objects.
- The type `DOMString` maps to Python strings. `xml.dom.minidom` supports either bytes or strings, but will normally produce strings. Values of type `DOMString` may also be `None` where allowed to have the IDL `null` value by the DOM specification from the W3C.
- `const` declarations map to variables in their respective scope (e.g. `xml.dom.minidom.Node.PROCESSING_INSTRUCTION_NODE`); they must not be changed.
- `DOMException` is currently not supported in `xml.dom.minidom`. Instead, `xml.dom.minidom` uses standard Python exceptions such as `TypeError` and `AttributeError`.
- `NodeList` objects are implemented using Python's built-in list type. These objects provide the interface defined in the DOM specification, but with earlier versions of Python they do not support the official API. They are, however, much more "Pythonic" than the interface defined in the W3C recommendations.

The following interfaces have no implementation in `xml.dom.minidom`:

- `DOMTimeStamp`
- `EntityReference`

Most of these reflect information in the XML document that is not of general utility to most DOM users.

Footnotes

The encoding name included in the XML output should conform to the appropriate standards. For example, "UTF-8" is valid, but "UTF8" is not valid in an XML document's declaration, even though Python accepts it as an encoding name. See <https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl> and <https://www.iana.org/assignments/character-sets/character-sets.xhtml>.

xml.dom.pulldom— Support for building partial DOM trees

Source code: `Lib/xml/dom/pulldom.py`

The `xml.dom.pulldom` module provides a “pull parser” which can also be asked to produce DOM-accessible fragments of the document where necessary. The basic concept involves pulling “events” from a stream of incoming XML and processing them. In contrast to SAX which also employs an event-driven processing model together with callbacks, the user of a pull parser is responsible for explicitly pulling events from the stream, looping over those events until either processing is finished or an error condition occurs.

■ **Nota:** Note if you need to parse untrusted or unauthenticated data, see [XML security](#).

Changed in version 3.7.1: The SAX parser no longer processes general external entities by default to increase security by default. To enable processing of external entities, pass a custom parser instance in:

```
from xml.dom.pulldom import parse
from xml.sax import make_parser
from xml.sax.handler import feature_external_ges
```

```
parser = make_parser()
parser.setFeature(feature_external_ges, True)
parse(filename, parser=parser)
```

Example:

```
from xml.dom import pulldom

doc = pulldom.parse('sales_items.xml')
for event, node in doc:
    if event == pulldom.START_ELEMENT and node.tagName == 'item':
        if int(node.getAttribute('price')) > 50:
            doc.expandNode(node)
            print(node.toxml())
```

`event` is a constant and can be one of:

- `START_ELEMENT`
- `END_ELEMENT`
- `COMMENT`
- `START_DOCUMENT`
- `END_DOCUMENT`
- `CHARACTERS`
- `PROCESSING_INSTRUCTION`
- `IGNORABLE_WHITESPACE`

`node` is an object of type `xml.dom.minidom.Document`, `xml.dom.minidom.Element` or `xml.dom.minidom.Text`.

Since the document is treated as a “flat” stream of events, the document “tree” is implicitly traversed and the desired elements are found regardless of their depth in the tree. In other words, one does not need to consider hierarchical issues such as recursive searching of the document nodes, although if the context of elements were important, one would either need to maintain some context-related state (i.e. remembering where one is in the document at any given point) or to make use of the `DOMEventStream.expandNode()` method and switch to DOM-related processing.

classxml.dom.pulldom.PullDOM(documentFactory=None)

Subclass ofxml.sax.handler.ContentHandler.

classxml.dom.pulldom.SAX2DOM(documentFactory=None)

Subclass ofxml.sax.handler.ContentHandler.

xml.dom.pulldom.parse(stream_or_string,parser=None,bufsize=None)

Return aDOMEventStreamfrom the given input.stream_or_stringmay be either a file name, or a file-like object.parser, if given, must be anXMLReaderobject. This function will change the document handler of the parser and activate namespace support; other parser configuration (like setting an entity resolver) must have been done in advance.

If you have XML in a string, you can use theparseString()function instead:

xml.dom.pulldom.parseString(string,parser=None)

Return aDOMEventStreamthat represents the (Unicode)string.

xml.dom.pulldom.default_bufsize

Default value for thebufsizeparameter toparse().The value of this variable can be changed before callingparse()and the new value will take effect.

DOMEventStream Objects

classxml.dom.pulldom.DOMEventStream(stream,parser,bufsize)

getEvent()

expandNode(node)

reset()

Changed in version 3.11:Support for__getitem__()method has been removed.getEvent()Return a tuple containingeventand the currentnodeasxml.dom.minidom.Documentif event equalsSTART_DOCUMENT,xml.dom.minidom.Elementif event equalsSTART_ELEMENTorEND_ELEMENTorxml.dom.minidom.Textif event equalsCHARACTERS. The current node does not contain information about its children, unlessexpandNode()is called.expandNode(node)Expands all children ofnodeintonode.
Example:fromxml.domimportpulldomxml='<html><title>Foo

Return a tuple containingeventand the currentnodeasxml.dom.minidom.Documentif event equalsSTART_DOCUMENT,xml.dom.minidom.Elementif event equalsSTART_ELEMENTorEND_ELEMENTorxml.dom.minidom.Textif event equalsCHARACTERS. The current node does not contain information about its children, unlessexpandNode()is called.

Expands all children ofnodeintonode. Example:fromxml.domimportpulldomxml='<html><title>Foo</title><p>Some text <div>and more</div></p> </html>'doc=pulldom.parseString(xml)forevent,nodeindoc:ifevent==pulldom.START_ELEMENTandnode.tagName=='p':# Following statement only prints '<p>'print(node.toxml())doc.expandNode(node)# Following statement prints node with all its children '<p>Some text <div>and more</div></p>'print(node.toxml())

xml.sax— Support for SAX2 parsers

Source code: `Lib/xml/sax/__init__.py`

The `xml.sax` package provides a number of modules which implement the Simple API for XML (SAX) interface for Python. The package itself provides the SAX exceptions and the convenience functions which will be most used by users of the SAX API.

■ **Nota:** Not if you need to parse untrusted or unauthenticated data, see XML security.

Changed in version 3.7.1: The SAX parser no longer processes general external entities by default to increase security. Before, the parser created network connections to fetch remote files or loaded local files from the file system for DTD and entities. The feature can be enabled again with `methodsetFeature()` on the parser object and argument `feature_external_ges`.

The convenience functions are:

`xml.sax.make_parser(parser_list=[])`

Create and return a `SAXXMLReader` object. The first parser found will be used. If `parser_list` is provided, it must be an iterable of strings which name modules that have a function named `create_parser()`. Modules listed in `parser_list` will be used before modules in the default list of parsers. Changed in version 3.8: The `parser_list` argument can be any iterable, not just a list.

`xml.sax.parse(filename_or_stream, handler, error_handler=handler.ErrorHandler())`

Create a SAX parser and use it to parse a document. The document, passed in as `filename_or_stream`, can be a filename or a file object. The `handler` parameter needs to be a `SAXContentHandler` instance. If `error_handler` is given, it must be a `SAXErrorHandler` instance; if omitted, `SAXParseException` will be raised on all errors. There is no return value; all work must be done by the handler passed in.

`xml.sax.parseString(string, handler, error_handler=handler.ErrorHandler())`

Similar to `parse()`, but parses from a buffer `string` received as a parameter. `string` must be a `str` instance or a bytes-like object. Changed in version 3.5: Added support of `str` instances.

A typical SAX application uses three kinds of objects: readers, handlers and input sources. “Reader” in this context is another term for parser, i.e. some piece of code that reads the bytes or characters from the input source, and produces a sequence of events. The events then get distributed to the handler objects, i.e. the reader invokes a method on the handler. A SAX application must therefore obtain a reader object, create or open the input sources, create the handlers, and connect these objects all together. As the final step of preparation, the reader is called to parse the input. During parsing, methods on the handler objects are called based on structural and syntactic events from the input data.

For these objects, only the interfaces are relevant; they are normally not instantiated by the application itself. Since Python does not have an explicit notion of interface, they are formally introduced as classes, but applications may use implementations which do not inherit from the provided classes. The `InputSource`, `Locator`, `Attributes`, `AttributesNS`, and `XMLReader` interfaces are defined in the module `xml.sax.xmlreader`. The handler interfaces are defined in `xml.sax.handler`. For convenience, `InputSource` (which is often instantiated directly) and the handler classes are also available from `xml.sax`. These interfaces are described below.

In addition to these classes, `xml.sax` provides the following exception classes.

`exception xml.sax.SAXException(msg, exception=None)`

Encapsulate an XML error or warning. This class can contain basic error or warning information from either the XML parser or the application: it can be subclassed to provide additional functionality or to add localization. Note

that although the handlers defined in the `ErrorHandler` interface receive instances of this exception, it is not required to actually raise the exception — it is also useful as a container for information. When instantiated, `msg` should be a human-readable description of the error.

`exception.xml.sax.SAXParseException(msg, exception, locator)`

Subclass of `SAXException` raised on parse errors. Instances of this class are passed to the methods of the `SAXErrorHandler` interface to provide information about the parse error. This class supports the `SAXLocator` interface as well as the `SAXException` interface.

`exception.xml.sax.SAXNotRecognizedException(msg, exception=None)`

Subclass of `SAXException` raised when a `SAXXMLReader` is confronted with an unrecognized feature or property. SAX applications and extensions may use this class for similar purposes.

`exception.xml.sax.SAXNotSupportedException(msg, exception=None)`

Subclass of `SAXException` raised when a `SAXXMLReader` is asked to enable a feature that is not supported, or to set a property to a value that the implementation does not support. SAX applications and extensions may use this class for similar purposes.

See also

SAX: The Simple API for XML

`Module xml.sax.handler`

`Module xml.sax.saxutils`

`Module xml.sax.xmlreader`

This site is the focal point for the definition of the SAX API. It provides a Java implementation and online documentation. Links to implementations and historical information are also available.

Definitions of the interfaces for application-provided objects.

Convenience functions for use in SAX applications.

Definitions of the interfaces for parser-provided objects.

SAXException Objects

The `SAXException` class supports the following methods:

`SAXException.getMessage()`

Return a human-readable message describing the error condition.

`SAXException.getException()`

Return an encapsulated exception object, or `None`.

xml.sax.handler— Base classes for SAX handlers

Source code: `Lib/xml/sax/handler.py`

The SAX API defines five kinds of handlers: content handlers, DTD handlers, error handlers, entity resolvers and lexical handlers. Applications normally only need to implement those interfaces whose events they are interested in; they can implement the interfaces in a single object or in multiple objects. Handler implementations should inherit from the base classes provided in the module `xml.sax.handler`, so that all methods get default implementations.

class xml.sax.handler.ContentHandler

This is the main callback interface in SAX, and the one most important to applications. The order of events in this interface mirrors the order of the information in the document.

class xml.sax.handler.DTDHandler

Handle DTD events. This interface specifies only those DTD events required for basic parsing (unparsed entities and attributes).

class xml.sax.handler.EntityResolver

Basic interface for resolving entities. If you create an object implementing this interface, then register the object with your Parser, the parser will call the method in your object to resolve all external entities.

class xml.sax.handler.ErrorHandler

Interface used by the parser to present error and warning messages to the application. The methods of this object control whether errors are immediately converted to exceptions or are handled in some other way.

class xml.sax.handler.LexicalHandler

Interface used by the parser to represent low frequency events which may not be of interest to many applications.

In addition to these classes, `xml.sax.handler` provides symbolic constants for the feature and property names.

xml.sax.handler.feature_namespaces

value: "http://xml.org/sax/features/namespaces" true: Perform Namespace processing. false: Optionally do not perform Namespace processing (implies namespace-prefixes; default). access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_namespace_prefixes

value: "http://xml.org/sax/features/namespace-prefixes" true: Report the original prefixed names and attributes used for Namespace declarations. false: Do not report attributes used for Namespace declarations, and optionally do not report original prefixed names (default). access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_string_interning

value: "http://xml.org/sax/features/string-interning" true: All element names, prefixes, attribute names, Namespace URIs, and local names are interned using the built-in intern function. false: Names are not necessarily interned, although they may be (default). access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_validation

value: "http://xml.org/sax/features/validation" true: Report all validation errors (implies external-general-entities and external-parameter-entities). false: Do not report validation errors. access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_external_ges

WarningEnabling opens a vulnerability toexternal entity attacksif the parser is used with user-provided XML content. Please reflect on yourthreat modelbefore enabling this

feature.value:"http://xml.org/sax/features/external-general-entities"true: Include all external general (text) entities.false: Do not include external general entities.access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_external_pes

value:"http://xml.org/sax/features/external-parameter-entities"true: Include all external parameter entities, including the external DTD subset.false: Do not include any external parameter entities, even the external DTD subset.access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.all_features

List of all features.

xml.sax.handler.property_lexical_handler

value:"http://xml.org/sax/properties/lexical-handler"data type: xml.sax.handler.LexicalHandler (not supported in Python 2)description: An optional extension handler for lexical events like comments.access: read/write

xml.sax.handler.property_declaration_handler

value:"http://xml.org/sax/properties/declaration-handler"data type: xml.sax.sax2lib.DeclHandler (not supported in Python 2)description: An optional extension handler for DTD-related events other than notations and unparsed entities.access: read/write

xml.sax.handler.property_dom_node

value:"http://xml.org/sax/properties/dom-node"data type: org.w3c.dom.Node (not supported in Python 2)description: When parsing, the current DOM node being visited if this is a DOM iterator; when not parsing, the root DOM node for iteration.access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.property_xml_string

value:"http://xml.org/sax/properties/xml-string"data type: Bytesdescription: The literal string of characters that was the source for the current event.access: read-only

xml.sax.handler.all_properties

List of all known property names.

ContentHandler Objects

Users are expected to subclassContentHandlerto support their application. The following methods are called by the parser on the appropriate events in the input document:

ContentHandler.setDocumentLocator(locator)

Called by the parser to give the application a locator for locating the origin of document events.SAX parsers are strongly encouraged (though not absolutely required) to supply a locator: if it does so, it must supply the locator to the application by invoking this method before invoking any of the other methods in the DocumentHandler interface.The locator allows the application to determine the end position of any document-related event, even if the parser is not reporting an error. Typically,

ContentHandler.startDocument()

Receive notification of the beginning of a document.The SAX parser will invoke this method only once, before any other methods in this interface or in DTDHandler (except forsetDocumentLocator()).

ContentHandler.endDocument()

Receive notification of the end of a document. The SAX parser will invoke this method only once, and it will be the last method invoked during the parse. The parser shall not invoke this method until it has either abandoned parsing (because of an unrecoverable error) or reached the end of input.

ContentHandler.startPrefixMapping(prefix,uri)

Begin the scope of a prefix-URI Namespace mapping. The information from this event is not necessary for normal Namespace processing: the SAX XML reader will automatically replace prefixes for element and attribute names when the `feature_namespaces` feature is enabled (the default). There are cases, however, when applications need to use prefixes in character data or in attribute values, where they cannot safely be expanded automatically; the `startPrefixMapping()` and `endPrefixMapping()` events supply the i

ContentHandler.endPrefixMapping(prefix)

End the scope of a prefix-URI mapping. See `startPrefixMapping()` for details. This event will always occur after the corresponding `endElement()` event, but the order of `endPrefixMapping()` events is not otherwise guaranteed.

ContentHandler.startElement(name,attrs)

Signals the start of an element in non-namespace mode. The `name` parameter contains the raw XML 1.0 name of the element type as a string and the `attrs` parameter holds an object of the `Attributes` interface containing the attributes of the element. The object passed as `attrs` may be re-used by the parser; holding on to a reference to it is not a reliable way to keep a copy of the attributes. To keep a copy of the attributes, use the `copy()` method of the `attrs` object.

ContentHandler.endElement(name)

Signals the end of an element in non-namespace mode. The `name` parameter contains the name of the element type, just as with the `startElement()` event.

ContentHandler.startElementNS(name,qname,attrs)

Signals the start of an element in namespace mode. The `name` parameter contains the name of the element type as a `(uri,localname)` tuple, the `qname` parameter contains the raw XML 1.0 name used in the source document, and the `attrs` parameter holds an instance of the `AttributesNS` interface containing the attributes of the element. If no namespace is associated with the element, the `uri` component of `name` will be `None`. The object passed as `attrs` may be re-used by the parser; holding on to a reference to it is not a r

ContentHandler.endElementNS(name,qname)

Signals the end of an element in namespace mode. The `name` parameter contains the name of the element type, just as with the `startElementNS()` method, likewise the `qname` parameter.

ContentHandler.characters(content)

Receive notification of character data. The Parser will call this method to report each chunk of character data. SAX parsers may return all contiguous character data in a single chunk, or they may split it into several chunks; however, all of the characters in any single event must come from the same external entity so that the Locator provides useful information. `content` may be a string or bytes instance; the `expatreader` module always produces strings. Note The earlier SAX 1 interface provided by the

ContentHandler.ignorableWhitespace(whitespace)

Receive notification of ignorable whitespace in element content. Validating Parsers must use this method to report each chunk of ignorable whitespace (see the W3C XML 1.0 recommendation, section 2.10): non-validating parsers may also use this method if they are capable of parsing and using content models. SAX parsers may return all contiguous whitespace in a single chunk, or they may split it into several chunks; however, all of the characters in any single event must come from the same external e

ContentHandler.processingInstruction(target,data)

Receive notification of a processing instruction. The Parser will invoke this method once for each processing instruction found: note that processing instructions may occur before or after the main document element. A SAX parser should never report an XML declaration (XML 1.0, section 2.8) or a text declaration (XML 1.0, section 4.3.1) using this method.

ContentHandler.skippedEntity(name)

Receive notification of a skipped entity. The Parser will invoke this method once for each entity skipped. Non-validating processors may skip entities if they have not seen the declarations (because, for example, the entity was declared in an external DTD subset). All processors may skip external entities, depending on the values of the `feature_external_ges` and `feature_external_pes` properties.

DTDHandler Objects

DTDHandler instances provide the following methods:

DTDHandler.notationDecl(name, publicId, systemId)

Handle a notation declaration event.

DTDHandler.unparsedEntityDecl(name, publicId, systemId, ndata)

Handle an unparsed entity declaration event.

EntityResolver Objects

EntityResolver.resolveEntity(publicId, systemId)

Resolve the system identifier of an entity and return either the system identifier to read from as a string, or an `InputStream` to read from. The default implementation returns `systemId`.

ErrorHandler Objects

Objects with this interface are used to receive error and warning information from the `XMLReader`. If you create an object that implements this interface, then register the object with your `XMLReader`, the parser will call the methods in your object to report all warnings and errors. There are three levels of errors available: warnings, (possibly) recoverable errors, and unrecoverable errors. All methods take a `SAXParseException` as the only parameter. Errors and warnings may be converted to an exception by raising the passed-in exception object.

ErrorHandler.error(exception)

Called when the parser encounters a recoverable error. If this method does not raise an exception, parsing may continue, but further document information should not be expected by the application. Allowing the parser to continue may allow additional errors to be discovered in the input document.

ErrorHandler.fatalError(exception)

Called when the parser encounters an error it cannot recover from; parsing is expected to terminate when this method returns.

ErrorHandler.warning(exception)

Called when the parser presents minor warning information to the application. Parsing is expected to continue when this method returns, and document information will continue to be passed to the application. Raising an exception in this method will cause parsing to end.

LexicalHandler Objects

Optional SAX2 handler for lexical events.

This handler is used to obtain lexical information about an XML document. Lexical information includes information describing the document encoding used and XML comments embedded in the document, as well as section boundaries for the DTD and for any CDATA sections. The lexical handlers are used in the same manner as content handlers.

Set the LexicalHandler of an XMLReader by using the setProperty method with the property identifier 'http://xml.org/sax/properties/lexical-handler'.

LexicalHandler.comment(content)

Reports a comment anywhere in the document (including the DTD and outside the document element).

LexicalHandler.startDTD(name,public_id,system_id)

Reports the start of the DTD declarations if the document has an associated DTD.

LexicalHandler.endDTD()

Reports the end of DTD declaration.

LexicalHandler.startCDATA()

Reports the start of a CDATA marked section. The contents of the CDATA marked section will be reported through the characters handler.

LexicalHandler.endCDATA()

Reports the end of a CDATA marked section.

xml.sax.saxutils— SAX Utilities

Source code: `Lib/xml/sax/saxutils.py`

The module `xml.sax.saxutils` contains a number of classes and functions that are commonly useful when creating SAX applications, either in direct use, or as base classes.

`xml.sax.saxutils.escape(data, entities={})`

Escape '&', '<', and '>' in a string of data. You can escape other strings of data by passing a dictionary as the optional `entities` parameter. The keys and values must all be strings; each key will be replaced with its corresponding value. The characters '&', '<' and '>' are always escaped, even if `entities` is provided. **Note** This function should only be used to escape characters that can't be used directly in XML. Do not use this function as a general string translation function.

`xml.sax.saxutils.unescape(data, entities={})`

Unescape '&', '<', and '>' in a string of data. You can unescape other strings of data by passing a dictionary as the optional `entities` parameter. The keys and values must all be strings; each key will be replaced with its corresponding value. '&', '<', and '>' are always unescaped, even if `entities` is provided.

`xml.sax.saxutils.quoteattr(data, entities={})`

Similar to `escape()`, but also prepares `data` to be used as an attribute value. The return value is a quoted version of `data` with any additional required replacements. `quoteattr()` will select a quote character based on the content of `data`, attempting to avoid encoding any quote characters in the string. If both single- and double-quote characters are already in `data`, the double-quote characters will be encoded and `data` will be wrapped in double-quotes. The resulting string can be used directly as an attri

`class xml.sax.saxutils.XMLGenerator(out=None, encoding='iso-8859-1', short_empty_elements=False)`

This class implements the `ContentHandler` interface by writing SAX events back into an XML document. In other words, using an `XMLGenerator` as the content handler will reproduce the original document being parsed. `out` should be a file-like object which will default to `sys.stdout`. `encoding` is the encoding of the output stream which defaults to 'iso-8859-1'. `short_empty_elements` controls the formatting of elements that contain no content: if `False` (the default) they are emitted as a pair of start/end tags, if se

`class xml.sax.saxutils.XMLFilterBase(base)`

This class is designed to sit between an `XMLReader` and the client application's event handlers. By default, it does nothing but pass requests up to the reader and events on to the handlers unmodified, but subclasses can override specific methods to modify the event stream or the configuration requests as they pass through.

`xml.sax.saxutils.prepare_input_source(source, base="")`

This function takes an input source and an optional base URL and returns a fully resolved `InputSource` object ready for reading. The input source can be given as a string, a file-like object, or an `InputSource` object; parsers will use this function to implement the polymorphic `source` argument to their `parse()` method.

xml.sax.xmlreader— Interface for XML parsers

Source code: `Lib/xml/sax/xmlreader.py`

SAX parsers implement the `XMLReader` interface. They are implemented in a Python module, which must provide a function `create_parser()`. This function is invoked by `xml.sax.make_parser()` with no arguments to create a new parser object.

class xml.sax.xmlreader.XMLReader

Base class which can be inherited by SAX parsers.

class xml.sax.xmlreader.IncrementalParser

In some cases, it is desirable not to parse an input source at once, but to feed chunks of the document as they get available. Note that the reader will normally not read the entire file, but read it in chunks as well; still `parse()` won't return until the entire document is processed. So these interfaces should be used if the blocking behaviour of `parse()` is not desirable. When the parser is instantiated it is ready to begin accepting data from the feed method immediately. After parsing has been finished

class xml.sax.xmlreader.Locator

Interface for associating a SAX event with a document location. A locator object will return valid results only during calls to `DocumentHandler` methods; at any other time, the results are unpredictable. If information is not available, methods may return `None`.

class xml.sax.xmlreader.InputSource(system_id=None)

Encapsulation of the information needed by the `XMLReader` to read entities. This class may include information about the public identifier, system identifier, byte stream (possibly with character encoding information) and/or the character stream of an entity. Applications will create objects of this class for use in the `XMLReader.parse()` method and for returning from `EntityResolver.resolveEntity`. An `InputSource` belongs to the application, the `XMLReader` is not allowed to modify `InputSource` objects passed to it

class xml.sax.xmlreader.AttributesImpl(attrs)

This is an implementation of the `Attributes` interface (see section [The Attributes Interface](#)). This is a dictionary-like object which represents the element attributes in a `startElement()` call. In addition to the most useful dictionary operations, it supports a number of other methods as described by the interface. Objects of this class should be instantiated by readers; `attrs` must be a dictionary-like object containing a mapping from attribute names to attribute values.

class xml.sax.xmlreader.AttributesNSImpl(attrs, qnames)

Namespace-aware variant of `AttributesImpl`, which will be passed to `startElementNS()`. It is derived from `AttributesImpl`, but understands attribute names as two-tuples of `namespaceURI` and `localName`. In addition, it provides a number of methods expecting qualified names as they appear in the original document. This class implements the `AttributesNS` interface (see section [The AttributesNS Interface](#)).

XMLReader Objects

The `XMLReader` interface supports the following methods:

XMLReader.parse(source)

Process an input source, producing SAX events. The `source` object can be a system identifier (a string identifying the input source – typically a file name or a URL), a `pathlib.Path` or path-like object, or an `InputSource` object. When `parse()` returns, the input is completely processed, and the parser object can be

discarded or reset.Changed in version 3.5:Added support of character streams.Changed in version 3.8:Added support of path-like objects.

XMLReader.getContentHandler()

Return the currentContentHandler.

XMLReader.setContentHandler(handler)

Set the currentContentHandler. If noContentHandleris set, content events will be discarded.

XMLReader.getDTDHandler()

Return the currentDTDHandler.

XMLReader.setDTDHandler(handler)

Set the currentDTDHandler. If noDTDHandleris set, DTD events will be discarded.

XMLReader.getEntityResolver()

Return the currentEntityResolver.

XMLReader.setEntityResolver(handler)

Set the currentEntityResolver. If noEntityResolveris set, attempts to resolve an external entity will result in opening the system identifier for the entity, and fail if it is not available.

XMLReader.getErrorHandler()

Return the currentErrorHandler.

XMLReader.setErrorHandler(handler)

Set the current error handler. If noErrorHandleris set, errors will be raised as exceptions, and warnings will be printed.

XMLReader.setLocale(locale)

Allow an application to set the locale for errors and warnings.SAX parsers are not required to provide localization for errors and warnings; if they cannot support the requested locale, however, they must raise a SAX exception. Applications may request a locale change in the middle of a parse.

XMLReader.getFeature(featurename)

Return the current setting for featurefeaturename. If the feature is not recognized,SAXNotRecognizedExceptionis raised. The well-known featurenames are listed in the modulexml.sax.handler.

XMLReader.setFeature(featurename,value)

Set thefeaturenametovalue. If the feature is not recognized,SAXNotRecognizedExceptionis raised. If the feature or its setting is not supported by the parser,SAXNotSupportedExceptionis raised.

XMLReader.getProperty(propertyname)

Return the current setting for propertypropertyname. If the property is not recognized, aSAXNotRecognizedExceptionis raised. The well-known propertynames are listed in the modulexml.sax.handler.

XMLReader.setProperty(propertyname,value)

Set thepropertynametovalue. If the property is not recognized,SAXNotRecognizedExceptionis raised. If the property or its setting is not supported by the parser,SAXNotSupportedExceptionis raised.

IncrementalParser Objects

Instances of `IncrementalParser` offer the following additional methods:

`IncrementalParser.feed(data)`

Process a chunk of data.

`IncrementalParser.close()`

Assume the end of the document. That will check well-formedness conditions that can be checked only at the end, invoke handlers, and may clean up resources allocated during parsing.

`IncrementalParser.reset()`

This method is called after `close` has been called to reset the parser so that it is ready to parse new documents. The results of calling `parse` or `feed` after `close` without calling `reset` are undefined.

Locator Objects

Instances of `Locator` provide these methods:

`Locator.getColumnNumber()`

Return the column number where the current event begins.

`Locator.getLineNumber()`

Return the line number where the current event begins.

`Locator.getPublicId()`

Return the public identifier for the current event.

`Locator.getSystemId()`

Return the system identifier for the current event.

InputSource Objects

`InputSource.setPublicId(id)`

Sets the public identifier of this `InputSource`.

`InputSource.getPublicId()`

Returns the public identifier of this `InputSource`.

`InputSource.setSystemId(id)`

Sets the system identifier of this `InputSource`.

`InputSource.getSystemId()`

Returns the system identifier of this `InputSource`.

`InputSource.setEncoding(encoding)`

Sets the character encoding of this `InputSource`. The encoding must be a string acceptable for an XML encoding declaration (see section 4.3.3 of the XML recommendation). The `encoding` attribute of the `InputSource` is ignored if the `InputSource` also contains a character stream.

`InputSource.getEncoding()`

Get the character encoding of this `InputSource`.

`InputSource.setByteStream(bytefile)`

Set the byte stream (abinary file) for this input source.The SAX parser will ignore this if there is also a character stream specified, but it will use a byte stream in preference to opening a URI connection itself.If the application knows the character encoding of the byte stream, it should set it with the `setEncoding` method.

`InputSource.getByteStream()`

Get the byte stream for this input source.The `getEncoding` method will return the character encoding for this byte stream, or `None` if unknown.

`InputSource.setCharacterStream(charfile)`

Set the character stream (atext file) for this input source.If there is a character stream specified, the SAX parser will ignore any byte stream and will not attempt to open a URI connection to the system identifier.

`InputSource.getCharacterStream()`

Get the character stream for this input source.

TheAttributesInterface

Attributesobjects implement a portion of the mapping protocol, including the methods `copy()`, `get()`, `__contains__()`, `items()`, `keys()`, and `values()`. The following methods are also provided:

`Attributes.getLength()`

Return the number of attributes.

`Attributes.getNames()`

Return the names of the attributes.

`Attributes.getType(name)`

Returns the type of the attribute name, which is normally 'CDATA'.

`Attributes.getValue(name)`

Return the value of attribute name.

TheAttributesNSInterface

This interface is a subtype of the `Attributes` interface (see section [The Attributes Interface](#)). All methods supported by that interface are also available on `AttributesNS` objects.

The following methods are also available:

`AttributesNS.getValueByQName(name)`

Return the value for a qualified name.

`AttributesNS.getNameByQName(name)`

Return the (namespace,localname) pair for a qualified name.

`AttributesNS.getQNameByName(name)`

Return the qualified name for a (namespace,localname) pair.

`AttributesNS.getQNames()`

Return the qualified names of all attributes.

xml.parsers.expat— Fast XML parsing using Expat

■ Nota: Notelf you need to parse untrusted or unauthenticated data, seeXML security.

The`xml.parsers.expat`module is a Python interface to the Expat non-validating XML parser. The module provides a single extension type,`xmlparser`, that represents the current state of an XML parser. After an`xmlparser`object has been created, various attributes of the object can be set to handler functions. When an XML document is then fed to the parser, the handler functions are called for the character data and markup in the XML document.

This module uses the`pyexpat`module to provide access to the Expat parser. Direct use of the`pyexpat`module is deprecated.

This module provides one exception and one type object:

exceptionxml.parsers.expat.ExpatError

The exception raised when Expat reports an error. See sectionExpatError Exceptionsfor more information on interpreting Expat errors.

exceptionxml.parsers.expat.error

Alias forExpatError.

xml.parsers.expat.XMLParserType

The type of the return values from theParserCreate()function.

The`xml.parsers.expat`module contains two functions:

xml.parsers.expat.ErrorString(errno)

Returns an explanatory string for a given error number`errno`.

xml.parsers.expat.ParserCreate(encoding=None,namespace_separator=None)

Creates and returns a new`xmlparser`object.`encoding`, if specified, must be a string naming the encoding used by the XML data. Expat doesn't support as many encodings as Python does, and its repertoire of encodings can't be extended; it supports UTF-8, UTF-16, ISO-8859-1 (Latin1), and ASCII. If`encoding[1]`is given it will override the implicit or explicit encoding of the document.Parsers created throughParserCreate()are called "root" parsers, in the sense that they do not have any parent parser a

See also

The Expat XML Parser

Home page of the Expat project.

XMLParser Objects

`xmlparser`objects have the following methods:

xmlparser.Parse(data[,isfinal])

Parses the contents of the string`data`, calling the appropriate handler functions to process the parsed data.`isfinal`must be true on the final call to this method; it allows the parsing of a single file in fragments, not the submission of multiple files.`data`can be the empty string at any time.

xmlparser.ParseFile(file)

Parse XML data reading from the objectfile.fileonly needs to provide theread(nbytes)method, returning the empty string when there's no more data.

xmlparser.SetBase(base)

Sets the base to be used for resolving relative URIs in system identifiers in declarations. Resolving relative identifiers is left to the application: this value will be passed through as thebaseargument to theExternalEntityRefHandler(),NotationDeclHandler(), andUnparsedEntityDeclHandler()functions.

xmlparser.GetBase()

Returns a string containing the base set by a previous call toSetBase(), orNoneifSetBase()hasn't been called.

xmlparser.GetInputContext()

Returns the input data that generated the current event as a string. The data is in the encoding of the entity which contains the text. When called while an event handler is not active, the return value isNone.

xmlparser.ExternalEntityParserCreate(context[,encoding])

Create a "child" parser which can be used to parse an external parsed entity referred to by content parsed by the parent parser. Thecontextparameter should be the string passed to theExternalEntityRefHandler()handler function, described below. The child parser is created with theordered_attributesandspecified_attributesset to the values of this parser.

xmlparser.SetParamEntityParsing(flag)

Control parsing of parameter entities (including the external DTD subset). Possibleflagvalues areXML_PARAM_ENTITY_PARSING_NEVER,XML_PARAM_ENTITY_PARSING_UNLESS_STANDALONEandXML_PARAM_ENTITY_PARSING_ALWAYS. Return true if setting the flag was successful.

xmlparser.UseForeignDTD([flag])

Calling this with a true value forflag(the default) will cause Expat to call theExternalEntityRefHandlerwithNonefor all arguments to allow an alternate DTD to be loaded. If the document does not contain a document type declaration, theExternalEntityRefHandlerwill still be called, but theStartDoctypeDeclHandlerandEndDoctypeDeclHandlerwill not be called.Passing a false value forflagwill cancel a previous call that passed a true value, but otherwise has no effect.This method can only be called before

xmlparser.SetReparseDeferralEnabled(enabled)

WarningCallingSetReparseDeferralEnabled(False)has security implications, as detailed below; please make sure to understand these consequences prior to using theSetReparseDeferralEnabledmethod.Expat 2.6.0 introduced a security mechanism called "reparse deferral" where instead of causing denial of service through quadratic runtime from reparsing large tokens, reparsing of unfinished tokens is now delayed by default until a sufficient amount of input is reached. Due to this delay, registered handle

xmlparser.GetReparseDeferralEnabled()

Returns whether reparse deferral is currently enabled for the given Expat parser instance.Added in version 3.13.

xmlparserobjects have the following methods to mitigate some common XML vulnerabilities.

xmlparser.SetAllocTrackerActivationThreshold(threshold,/)

Sets the number of allocated bytes of dynamic memory needed to activate protection against disproportionate use of RAM.By default, parser objects have an allocation activation threshold of 64 MiB, or equivalently 67,108,864 bytes.AnExpatErroris raised if this method is called on anon-rootparser. The correspondinglinenoandoffsetshould not be used as they may have no special meaning.Added in version 3.14.1.

xmlparser.SetAllocTrackerMaximumAmplification(max_factor,/)

Sets the maximum amplification factor between direct input and bytes of dynamic memory allocated. The amplification factor is calculated as allocated/direct while parsing, where direct is the number of bytes read from the primary document in parsing and allocated is the number of bytes of dynamic memory allocated in the parser hierarchy. The max_factor value must be a non-NaN float value greater than or equal to 1.0. Amplification factors greater than 100.0 can be observed near the start of parsing even with

xmlparser objects have the following attributes:

xmlparser.buffer_size

The size of the buffer used when buffer_text is true. A new buffer size can be set by assigning a new integer value to this attribute. When the size is changed, the buffer will be flushed.

xmlparser.buffer_text

Setting this to true causes the xmlparser object to buffer textual content returned by Expat to avoid multiple calls to the `CharacterDataHandler()` callback whenever possible. This can improve performance substantially since Expat normally breaks character data into chunks at every line ending. This attribute is false by default, and may be changed at any time. Note that when it is false, data that does not contain newlines may be chunked too.

xmlparser.buffer_used

If buffer_text is enabled, the number of bytes stored in the buffer. These bytes represent UTF-8 encoded text. This attribute has no meaningful interpretation when buffer_text is false.

xmlparser.ordered_attributes

Setting this attribute to a non-zero integer causes the attributes to be reported as a list rather than a dictionary. The attributes are presented in the order found in the document text. For each attribute, two list entries are presented: the attribute name and the attribute value. (Older versions of this module also used this format.) By default, this attribute is false; it may be changed at any time.

xmlparser.specified_attributes

If set to a non-zero integer, the parser will report only those attributes which were specified in the document instance and not those which were derived from attribute declarations. Applications which set this need to be especially careful to use what additional information is available from the declarations as needed to comply with the standards for the behavior of XML processors. By default, this attribute is false; it may be changed at any time.

The following attributes contain values relating to the most recent error encountered by an xmlparser object, and will only have correct values once a call to `Parse()` or `ParseFile()` has raised an `xml.parsers.expat.ExpatError` exception.

xmlparser.ErrorByteIndex

Byte index at which an error occurred.

xmlparser.ErrorCode

Numeric code specifying the problem. This value can be passed to the `ErrorString()` function, or compared to one of the constants defined in the `errors` object.

xmlparser.ErrorColumnNumber

Column number at which an error occurred.

xmlparser.ErrorLineNumber

Line number at which an error occurred.

The following attributes contain values relating to the current parse location in an `xmlparser` object. During a callback reporting a parse event they indicate the location of the first of the sequence of characters that generated the event. When called outside of a callback, the position indicated will be just past the last parse event (regardless of whether there was an associated callback).

`xmlparser.CurrentByteIndex`

Current byte index in the parser input.

`xmlparser.CurrentColumnNumber`

Current column number in the parser input.

`xmlparser.CurrentLineNumber`

Current line number in the parser input.

Here is the list of handlers that can be set. To set a handler on an `xmlparser` object, use `parser.handlername=func`. `handlername` must be taken from the following list, and `func` must be a callable object accepting the correct number of arguments. The arguments are all strings, unless otherwise stated.

`xmlparser.XmlDeclHandler(version,encoding,standalone)`

Called when the XML declaration is parsed. The XML declaration is the (optional) declaration of the applicable version of the XML recommendation, the encoding of the document text, and an optional “standalone” declaration. `version` and `encoding` will be strings, and `standalone` will be `1` if the document is declared standalone, `0` if it is declared not to be standalone, or `-1` if the standalone clause was omitted. This is only available with Expat version 1.95.0 or newer.

`xmlparser.StartDoctypeDeclHandler(doctypeName,systemId,publicId,has_internal_subset)`

Called when Expat begins parsing the document type declaration (`<!DOCTYPE...`). The `doctypeName` is provided exactly as presented. The `systemId` and `publicId` parameters give the system and public identifiers if specified, or `None` if omitted. `has_internal_subset` will be `true` if the document contains an internal document declaration subset. This requires Expat version 1.2 or newer.

`xmlparser.EndDoctypeDeclHandler()`

Called when Expat is done parsing the document type declaration. This requires Expat version 1.2 or newer.

`xmlparser.ElementDeclHandler(name,model)`

Called once for each element type declaration. `name` is the name of the element type, and `model` is a representation of the content model.

`xmlparser.AttnlistDeclHandler(ename,attname,type,default,required)`

Called for each declared attribute for an element type. If an attribute list declaration declares three attributes, this handler is called three times, once for each attribute. `ename` is the name of the element to which the declaration applies and `attname` is the name of the attribute declared. The attribute type is a string passed as `type`; the possible values are `'CDATA'`, `'ID'`, `'IDREF'`, ... `default` gives the default value for the attribute used when the attribute is not specified by the document instance,

`xmlparser.StartElementHandler(name,attributes)`

Called for the start of every element. `name` is a string containing the element name, and `attributes` is the element attributes. If `ordered_attributes` is `true`, this is a list (see `ordered_attributes` for a full description). Otherwise it's a dictionary mapping names to values.

`xmlparser.EndElementHandler(name)`

Called for the end of every element.

xmlparser.ProcessingInstructionHandler(target,data)

Called for every processing instruction.

xmlparser.CharacterDataHandler(data)

Called for character data. This will be called for normal character data, CDATA marked content, and ignorable whitespace. Applications which must distinguish these cases can use theStartCdataSectionHandler,EndCdataSectionHandler, andElementDeclHandlercallbacks to collect the required information. Note that the character data may be chunked even if it is short and so you may receive more than one call toCharacterDataHandler(). Set thebuffer_textinstance attribute toTrue to avoid that.

xmlparser.UnparsedEntityDeclHandler(entityName,base,systemId,publicId,notationName)

Called for unparsed (NDATA) entity declarations. This is only present for version 1.2 of the Expat library; for more recent versions, useEntityDeclHandlerinstead. (The underlying function in the Expat library has been declared obsolete.)

xmlparser.EntityDeclHandler(entityName,is_parameter_entity,value,base,systemId,publicId,notationName)

Called for all entity declarations. For parameter and internal entities,value will be a string giving the declared contents of the entity; this will beNonefor external entities. ThenotationNameparameter will beNonefor parsed entities, and the name of the notation for unparsed entities.is_parameter_entitywill be true if the entity is a parameter entity or false for general entities (most applications only need to be concerned with general entities). This is only available starting with version 1

xmlparser.NotationDeclHandler(notationName,base,systemId,publicId)

Called for notation declarations.notationName,base, andsystemId, andpublicIdare strings if given. If the public identifier is omitted,publicIdwill beNone.

xmlparser.StartNamespaceDeclHandler(prefix,uri)

Called when an element contains a namespace declaration. Namespace declarations are processed before theStartElementHandleris called for the element on which declarations are placed.

xmlparser.EndNamespaceDeclHandler(prefix)

Called when the closing tag is reached for an element that contained a namespace declaration. This is called once for each namespace declaration on the element in the reverse of the order for which theStartNamespaceDeclHandlerwas called to indicate the start of each namespace declaration's scope. Calls to this handler are made after the correspondingEndElementHandlerfor the end of the element.

xmlparser.CommentHandler(data)

Called for comments.datais the text of the comment, excluding the leading'<!--'and trailing'-->'.

xmlparser.StartCdataSectionHandler()

Called at the start of a CDATA section. This andEndCdataSectionHandlerare needed to be able to identify the syntactical start and end for CDATA sections.

xmlparser.EndCdataSectionHandler()

Called at the end of a CDATA section.

xmlparser.DefaultHandler(data)

Called for any characters in the XML document for which no applicable handler has been specified. This means characters that are part of a construct which could be reported, but for which no handler has been supplied.

xmlparser.DefaultHandlerExpand(data)

This is the same as the `DefaultHandler()`, but doesn't inhibit expansion of internal entities. The entity reference will not be passed to the default handler.

xmlparser.NotStandaloneHandler()

Called if the XML document hasn't been declared as being a standalone document. This happens when there is an external subset or a reference to a parameter entity, but the XML declaration does not set `standalone` to `yes` in an XML declaration. If this handler returns `0`, then the parser will raise an `XML_ERROR_NOT_STANDALONE` error. If this handler is not set, no exception is raised by the parser for this condition.

xmlparser.ExternalEntityRefHandler(context,base,systemId,publicId)

Warning: Implementing a handler that accesses local files and/or the network may create a vulnerability to external entity attacks if `xmlparser` is used with user-provided XML content. Please reflect on your threat model before implementing this handler. Called for references to external entities. `base` is the current base, as set by a previous call to `SetBase()`. The public and system identifiers, `systemId` and `publicId`, are strings if given; if the public identifier is not given, `publicId` will be `None`. The `context`

ExpatError Exceptions

`ExpatError` exceptions have a number of interesting attributes:

ExpatError.code

Expat's internal error number for the specific error. The `errors.messages` dictionary maps these error numbers to Expat's error messages. For example: `from xml.parsers.expat import ParserCreate, ExpatError, errors; p = ParserCreate(); try: p.Parse(some_xml_document) except ExpatError as err: print("Error:", errors.messages[err.code])` The `errors` module also provides error message constants and a dictionary `codes` mapping these messages back to the error codes, see below.

ExpatError.lineno

Line number on which the error was detected. The first line is numbered 1.

ExpatError.offset

Character offset into the line where the error occurred. The first column is numbered 0.

Example

The following program defines three handlers that just print out their arguments.

```
import xml.parsers.expat

# 3 handler functions
def start_element(name, attrs):
    print('Start element:', name, attrs)
def end_element(name):
    print('End element:', name)
def char_data(data):
    print('Character data:', repr(data))

p = xml.parsers.expat.ParserCreate()

p.StartElementHandler = start_element
p.EndElementHandler = end_element
p.CharacterDataHandler = char_data
```

```
p.Parse("""<?xml version="1.0"?>
<parent id="top"><child1 name="paul">Text goes here</child1>
<child2 name="fred">More text</child2>
</parent>""", 1)
```

The output from this program is:

```
Start element: parent {'id': 'top'}
Start element: child1 {'name': 'paul'}
Character data: 'Text goes here'
End element: child1
Character data: '\n'
Start element: child2 {'name': 'fred'}
Character data: 'More text'
End element: child2
Character data: '\n'
End element: parent
```

Content Model Descriptions

Content models are described using nested tuples. Each tuple contains four values: the type, the quantifier, the name, and a tuple of children. Children are simply additional content model descriptions.

The values of the first two fields are constants defined in `xml.parsers.expat.modelmodule`. These constants can be collected in two groups: the model type group and the quantifier group.

The constants in the model type group are:

`xml.parsers.expat.model.XML_CTYPE_ANY`

The element named by the model name was declared to have a content model of ANY.

`xml.parsers.expat.model.XML_CTYPE_CHOICE`

The named element allows a choice from a number of options; this is used for content models such as `(A|B|C)`.

`xml.parsers.expat.model.XML_CTYPE_EMPTY`

Elements which are declared to be EMPTY have this model type.

`xml.parsers.expat.model.XML_CTYPE_MIXED`

`xml.parsers.expat.model.XML_CTYPE_NAME`

`xml.parsers.expat.model.XML_CTYPE_SEQ`

Models which represent a series of models which follow one after the other are indicated with this model type. This is used for models such as `(A,B,C)`.

The constants in the quantifier group are:

`xml.parsers.expat.model.XML_CQUANT_NONE`

No modifier is given, so it can appear exactly once, as for `A`.

`xml.parsers.expat.model.XML_CQUANT_OPT`

The model is optional: it can appear once or not at all, as for `A?`.

`xml.parsers.expat.model.XML_CQUANT_PLUS`

The model must occur one or more times (like `A+`).

`xml.parsers.expat.model.XML_CQUANT_REP`

The model must occur zero or more times, as for `A*`.

Expat error constants

The following constants are provided in the `xml.parsers.expat.errors` module. These constants are useful in interpreting some of the attributes of the `ExpatError` exception objects raised when an error has occurred. Since for backwards compatibility reasons, the constants' value is the error message and not the numeric error code, you do this by comparing its `code` attribute with `errors.codes[errors.XML_ERROR_CONSTANT_NAME]`.

The `errors` module has the following attributes:

`xml.parsers.expat.errors.codes`

A dictionary mapping string descriptions to their error codes. Added in version 3.2.

`xml.parsers.expat.errors.messages`

A dictionary mapping numeric error codes to their string descriptions. Added in version 3.2.

`xml.parsers.expat.errors.XML_ERROR_ASYNC_ENTITY`

`xml.parsers.expat.errors.XML_ERROR_ATTRIBUTE_EXTERNAL_ENTITY_REF`

An entity reference in an attribute value referred to an external entity instead of an internal entity.

`xml.parsers.expat.errors.XML_ERROR_BAD_CHAR_REF`

A character reference referred to a character which is illegal in XML (for example, `character0`, or `'�'`).

`xml.parsers.expat.errors.XML_ERROR_BINARY_ENTITY_REF`

An entity reference referred to an entity which was declared with a notation, so cannot be parsed.

`xml.parsers.expat.errors.XML_ERROR_DUPLICATE_ATTRIBUTE`

An attribute was used more than once in a start tag.

`xml.parsers.expat.errors.XML_ERROR_INCORRECT_ENCODING`

`xml.parsers.expat.errors.XML_ERROR_INVALID_TOKEN`

Raised when an input byte could not properly be assigned to a character; for example, a NUL byte (value 0) in a UTF-8 input stream.

`xml.parsers.expat.errors.XML_ERROR_JUNK_AFTER_DOC_ELEMENT`

Something other than whitespace occurred after the document element.

`xml.parsers.expat.errors.XML_ERROR_MISPLACED_XML_PI`

An XML declaration was found somewhere other than the start of the input data.

`xml.parsers.expat.errors.XML_ERROR_NO_ELEMENTS`

The document contains no elements (XML requires all documents to contain exactly one top-level element)..

`xml.parsers.expat.errors.XML_ERROR_NO_MEMORY`

Expat was not able to allocate memory internally.

`xml.parsers.expat.errors.XML_ERROR_PARAM_ENTITY_REF`

A parameter entity reference was found where it was not allowed.

`xml.parsers.expat.errors.XML_ERROR_PARTIAL_CHAR`

An incomplete character was found in the input.

`xml.parsers.expat.errors.XML_ERROR_RECURSIVE_ENTITY_REF`

An entity reference contained another reference to the same entity; possibly via a different name, and possibly indirectly.

xml.parsers.expat.errors.XML_ERROR_SYNTAX

Some unspecified syntax error was encountered.

xml.parsers.expat.errors.XML_ERROR_TAG_MISMATCH

An end tag did not match the innermost open start tag.

xml.parsers.expat.errors.XML_ERROR_UNCLOSED_TOKEN

Some token (such as a start tag) was not closed before the end of the stream or the next token was encountered.

xml.parsers.expat.errors.XML_ERROR_UNDEFINED_ENTITY

A reference was made to an entity which was not defined.

xml.parsers.expat.errors.XML_ERROR_UNKNOWN_ENCODING

The document encoding is not supported by Expat.

xml.parsers.expat.errors.XML_ERROR_UNCLOSED_CDATA_SECTION

A CDATA marked section was not closed.

xml.parsers.expat.errors.XML_ERROR_EXTERNAL_ENTITY_HANDLING

xml.parsers.expat.errors.XML_ERROR_NOT_STANDALONE

The parser determined that the document was not “standalone” though it declared itself to be in the XML declaration, and theNotStandaloneHandler was set and returned 0.

xml.parsers.expat.errors.XML_ERROR_UNEXPECTED_STATE

xml.parsers.expat.errors.XML_ERROR_ENTITY_DECLARED_IN_PE

xml.parsers.expat.errors.XML_ERROR_FEATURE_REQUIRES_XML_DTD

An operation was requested that requires DTD support to be compiled in, but Expat was configured without DTD support. This should never be reported by a standard build of the xml.parsers.expat module.

xml.parsers.expat.errors.XML_ERROR_CANT_CHANGE_FEATURE_ONCE_PARSING

A behavioral change was requested after parsing started that can only be changed before parsing has started. This is (currently) only raised by UseForeignDTD().

xml.parsers.expat.errors.XML_ERROR_UNBOUND_PREFIX

An undeclared prefix was found when namespace processing was enabled.

xml.parsers.expat.errors.XML_ERROR_UNDECLARING_PREFIX

The document attempted to remove the namespace declaration associated with a prefix.

xml.parsers.expat.errors.XML_ERROR_INCOMPLETE_PE

A parameter entity contained incomplete markup.

xml.parsers.expat.errors.XML_ERROR_XML_DECL

The document contained no document element at all.

xml.parsers.expat.errors.XML_ERROR_TEXT_DECL

There was an error parsing a text declaration in an external entity.

xml.parsers.expat.errors.XML_ERROR_PUBLICID

Characters were found in the public id that are not allowed.

xml.parsers.expat.errors.XML_ERROR_SUSPENDED

The requested operation was made on a suspended parser, but isn't allowed. This includes attempts to provide additional input or to stop the parser.

xml.parsers.expat.errors.XML_ERROR_NOT_SUSPENDED

An attempt to resume the parser was made when the parser had not been suspended.

xml.parsers.expat.errors.XML_ERROR_ABORTED

This should not be reported to Python applications.

xml.parsers.expat.errors.XML_ERROR_FINISHED

The requested operation was made on a parser which was finished parsing input, but isn't allowed. This includes attempts to provide additional input or to stop the parser.

xml.parsers.expat.errors.XML_ERROR_SUSPEND_PE**xml.parsers.expat.errors.XML_ERROR_RESERVED_PREFIX_XML**

An attempt was made to undeclare reserved namespace prefixxmlor to bind it to another namespace URI.

xml.parsers.expat.errors.XML_ERROR_RESERVED_PREFIX_XMLNS

An attempt was made to declare or undeclare reserved namespace prefixxmlns.

xml.parsers.expat.errors.XML_ERROR_RESERVED_NAMESPACE_URI

An attempt was made to bind the URI of one the reserved namespace prefixesxml andxmlns to another namespace prefix.

xml.parsers.expat.errors.XML_ERROR_INVALID_ARGUMENT

This should not be reported to Python applications.

xml.parsers.expat.errors.XML_ERROR_NO_BUFFER

This should not be reported to Python applications.

xml.parsers.expat.errors.XML_ERROR_AMPLIFICATION_LIMIT_BREACH

The limit on input amplification factor (from DTD and entities) has been breached.

xml.parsers.expat.errors.XML_ERROR_NOT_STARTED

The parser was tried to be stopped or suspended before it started. Added in version 3.14.

Footnotes

The encoding string included in XML output should conform to the appropriate standards. For example, "UTF-8" is valid, but "UTF8" is not. See <https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl> and <https://www.iana.org/assignments/character-sets/character-sets.xhtml>.

Internet Protocols and Support

The modules described in this chapter implement internet protocols and support for related technology. They are all implemented in Python. Most of these modules require the presence of the system-dependent `modulesocket`, which is currently supported on most popular platforms. Here is an overview:

- `webbrowser`— Convenient web-browser controller
Command-line interface
Browser controller objects
- `wsgiref`— WSGI Utilities and Reference Implementation
`wsgiref.util`— WSGI environment utilities
`wsgiref.headers`— WSGI response header tools
`wsgiref.simple_server`— a simple WSGI HTTP server
`wsgiref.validate`— WSGI conformance checker
`wsgiref.handlers`— server/gateway base classes
`wsgiref.types`— WSGI types for static type checking
Examples
- `urllib`— URL handling modules
- `urllib.request`— Extensible library for opening URLs
Request Objects
OpenerDirector Objects
BaseHandler Objects
HTTPRedirectHandler Objects
HTTPCookieProcessor Objects
ProxyHandler Objects
HTTPPasswordMgr Objects
HTTPPasswordMgrWithPriorAuth Objects
AbstractBasicAuthHandler Objects
HTTPBasicAuthHandler Objects
ProxyBasicAuthHandler Objects
AbstractDigestAuthHandler Objects
HTTPDigestAuthHandler Objects
ProxyDige
- `urllib.response`— Response classes used by `urllib`
- `urllib.parse`— Parse URLs into components
URL Parsing
URL parsing security
Parsing ASCII Encoded Bytes
Structured Parse Results
URL Quoting
- `urllib.error`— Exception classes raised by `urllib.request`
- `urllib.robotparser`— Parser for robots.txt
- `http`— HTTP modules
HTTP status codes
HTTP status category
HTTP methods
- `http.client`— HTTP protocol client
HTTPConnection Objects
HTTPResponse Objects
Examples
HTTPMessage Objects
- `ftplib`— FTP protocol client
Reference
FTP objects
FTP_TLS objects
Module variables
- `poplib`— POP3 protocol client
POP3 Objects
POP3 Example
- `imaplib`— IMAP4 protocol client
IMAP4 Objects
IMAP4 Example
- `smtplib`— SMTP protocol client
SMTP Objects
SMTP Example
- `uuid`— UUID objects according to RFC 9562
Command-Line Usage
Example
Command-Line Example
- `socketserver`— A framework for network servers
Server Creation Notes
Server Objects
Request Handler Objects
Examples
`socketserver.TCPServer`
Examples
`socketserver.UDPServer`
Example
Asynchronous Mixins
- `http.server`— HTTP servers
Command-line interface
Security considerations
- `http.cookies`— HTTP state management
Cookie Objects
Morsel Objects
Example
- `http.cookiejar`— Cookie handling for HTTP clients
CookieJar and FileCookieJar Objects
FileCookieJar subclasses and co-operation with web browsers
CookiePolicy Objects
DefaultCookiePolicy Objects
Cookie Objects
Examples
- `xmlrpc`— XMLRPC server and client modules
- `xmlrpc.client`— XML-RPC client access
ServerProxy Objects
DateTime Objects
Binary Objects
Fault Objects
ProtocolError Objects
MultiCall Objects
Convenience Functions
Example of Client Usage
Example of Client and Server Usage

- xmlrpc.server— Basic XML-RPC serversSimpleXMLRPCServer objectsSimpleXMLRPCServer exampleCGIXMLRPCRequestHandlerDocumenting XMLRPC serverDocXMLRPCServer objectsDocCGIXMLRPCRequestHandler
- ipaddress— IPv4/IPv6 manipulation libraryConvenience factory functionsIP AddressesAddress objectsConversion to Strings and IntegersOperatorsComparison operatorsArithmetic operatorsIP Network definitionsPrefix, net mask and host maskNetwork objectsOperatorsLogical operatorsIterationNetworks as containers of addressesInterface objectsOperatorsLogical operatorsOther Module Level FunctionsCustom Excep

webbrowser— Convenient web-browser controller

Source code: `Lib/webbrowser.py`

The `webbrowser` module provides a high-level interface to allow displaying web-based documents to users. Under most circumstances, simply calling the `open()` function from this module will do the right thing.

Under Unix, graphical browsers are preferred under X11, but text-mode browsers will be used if graphical browsers are not available or an X11 display isn't available. If text-mode browsers are used, the calling process will block until the user exits the browser.

If the environment variable `BROWSER` exists, it is interpreted as the `os.pathsep`-separated list of browsers to try ahead of the platform defaults. When the value of a list part contains the string `%s`, then it is interpreted as a literal browser command line to be used with the argument URL substituted for `%s`; if the value is a single word that refers to one of the already registered browsers this browser is added to the front of the search list; if the part does not contain `%s`, it is simply interpreted as the name of the browser to launch.^[1]

Changed in version 3.14: The `BROWSER` variable can now also be used to reorder the list of platform defaults. This is particularly useful on macOS where the platform defaults do not refer to command-line tools on `PATH`.

For non-Unix platforms, or when a remote browser is available on Unix, the controlling process will not wait for the user to finish with the browser, but allow the remote browser to maintain its own windows on the display. If remote browsers are not available on Unix, the controlling process will launch a new browser and wait.

On iOS, the `BROWSER` environment variable, as well as any arguments controlling `autoraise`, browser preference, and new tab/window creation will be ignored. Web pages will always be opened in the user's preferred browser, in a new tab, with the browser being brought to the foreground. The use of the `webbrowser` module on iOS requires the `ctypes` module. If `ctypes` isn't available, calls to `open()` will fail.

Command-line interface

The script `webbrowser` can be used as a command-line interface for the module. It accepts a URL as the argument. It accepts the following optional parameters:

-n, --new-window

Opens the URL in a new browser window, if possible.

-t, --new-tab

Opens the URL in a new browser tab.

The options are, naturally, mutually exclusive. Usage example:

```
python -m webbrowser -t "https://www.python.org"
```

Availability: not WASI, not Android.

The following exception is defined:

exception webbrowser.Error

Exception raised when a browser control error occurs.

The following functions are defined:

webbrowser.open(url, new=0, autoraise=True)

Display `url` using the default browser. If `new` is 0, the `url` is opened in the same browser window if possible. If `new` is 1, a new browser window is opened if possible. If `new` is 2, a new browser page ("tab") is opened if possible.

If `autoRaise` is `True`, the window is raised if possible (note that under many window managers this will occur regardless of the setting of this variable). Returns `True` if a browser was successfully launched, `False` otherwise. Note that on some platforms, trying to open a filename using `t`

`webbrowser.open_new(url)`

Open `url` in a new window of the default browser, if possible, otherwise, open `url` in the only browser window. Returns `True` if a browser was successfully launched, `False` otherwise.

`webbrowser.open_new_tab(url)`

Open `url` in a new page (“tab”) of the default browser, if possible, otherwise equivalent to `open_new()`. Returns `True` if a browser was successfully launched, `False` otherwise.

`webbrowser.get(using=None)`

Return a controller object for the browser type `using`. If `using` is `None`, return a controller for a default browser appropriate to the caller’s environment.

`webbrowser.register(name, constructor, instance=None, *, preferred=False)`

Register the browser type `name`. Once a browser type is registered, the `get()` function can return a controller for that browser type. If `instance` is not provided, or is `None`, `constructor` will be called without parameters to create an instance when needed. If `instance` is provided, `constructor` will never be called, and may be `None`. Setting `preferred` to `True` makes this browser a preferred result for a `get()` call with no argument. Otherwise, this entry point is only useful if you plan to either set the `BROWSER` variable

A number of browser types are predefined. This table gives the type names that may be passed to the `get()` function and the corresponding instantiations for the controller classes, all defined in this module.

Type Name		Class Name		Notes		'mozilla'		Mozilla('mozilla')		'firefox'		Mozilla('mozilla')			
	'epiphany'		Epiphany('epiphany')		'kfmclient'		Konqueror()		(1)		'konqueror'				
Konqueror()		(1)		'kfm'		Konqueror()		(1)		'opera'		Opera()		'links'	
GenericBrowser('links')		'elinks'		Elinks('elinks')		'lynx'		GenericBrowser('lynx')		'w3m'		GenericBrowser('w3m')		'windows-default'	
WindowsDefault		(2)		'macosx'		MacOSXOSAScript('default')		(3)		'safari'		MacOSXOSAScript('safari')		(3)	
'google-chrome'		Chrome('google-chrome')		'chrome'		Chrome('chrome')		'chromium'		Chromium('chromium')		'chromium-browser'		Chromium('chromium-browser')	
'iosbrowser'		IOSBrowser		(4)											

Notes:

- “Konqueror” is the file manager for the KDE desktop environment for Unix, and only makes sense to use if KDE is running. Some way of reliably detecting KDE would be nice; the `KDEDIR` variable is not sufficient. Note also that the name “kfm” is used even when using the `konqueror` command with KDE 2 — the implementation selects the best strategy for running Konqueror.
- Only on Windows platforms.
- Only on macOS.
- Only on iOS.

Added in version 3.2: A new `MacOSXOSAScript` class has been added and is used on Mac instead of the previous `MacOSX` class. This adds support for opening browsers not currently set as the OS default.

Added in version 3.3: Support for Chrome/Chromium has been added.

Changed in version 3.12: Support for several obsolete browsers has been removed. Removed browsers include Grail, Mosaic, Netscape, Galeon, Skipstone, Iceape, and Firefox versions 35 and below.

Changed in version 3.13: Support for iOS has been added.

Here are some simple examples:

```
url = 'https://docs.python.org/'

# Open URL in a new tab, if a browser window is already open.
webbrowser.open_new_tab(url)

# Open URL in new window, raising the window if possible.
webbrowser.open_new(url)
```

Browser controller objects

Browser controllers provide the `name` attribute, and the following three methods which parallel module-level convenience functions:

controller.name

System-dependent name for the browser.

controller.open(url,new=0,autoraise=True)

Display `url` using the browser handled by this controller. If `new` is 1, a new browser window is opened if possible. If `new` is 2, a new browser page ("tab") is opened if possible.

controller.open_new(url)

Open `url` in a new window of the browser handled by this controller, if possible, otherwise, open `url` in the only browser window. Alias `open_new()`.

controller.open_new_tab(url)

Open `url` in a new page ("tab") of the browser handled by this controller, if possible, otherwise equivalent to `open_new()`.

Footnotes

Executables named here without a full path will be searched in the directories given in the `PATHEXT` environment variable.

wsgiref— WSGI Utilities and Reference Implementation

Source code: [Lib/wsgiref](#)

■■ Advertencia: `Warningwsgiref` is a reference implementation and is not recommended for production. The module only implements basic security checks.

The Web Server Gateway Interface (WSGI) is a standard interface between web server software and web applications written in Python. Having a standard interface makes it easy to use an application that supports WSGI with a number of different web servers.

Only authors of web servers and programming frameworks need to know every detail and corner case of the WSGI design. You don't need to understand every detail of WSGI just to install a WSGI application or to write a web application using an existing framework.

`wsgiref` is a reference implementation of the WSGI specification that can be used to add WSGI support to a web server or framework. It provides utilities for manipulating WSGI environment variables and response headers, base classes for implementing WSGI servers, a demo HTTP server that serves WSGI applications, types for static type checking, and a validation tool that checks WSGI servers and applications for conformance to the WSGI specification (PEP 3333).

See [wsgi.readthedocs.io](#) for more information about WSGI, and links to tutorials and other resources.

wsgiref.util— WSGI environment utilities

This module provides a variety of utility functions for working with WSGI environments. A WSGI environment is a dictionary containing HTTP request variables as described in PEP 3333. All of the functions taking an `environ` parameter expect a WSGI-compliant dictionary to be supplied; please see PEP 3333 for a detailed specification and `WSGIEnvironment` for a type alias that can be used in type annotations.

wsgiref.util.guess_scheme(environ)

Return a guess for whether `wsgi.url_scheme` should be “http” or “https”, by checking for a `HTTPSEnvironment` variable in the `environ` dictionary. The return value is a string. This function is useful when creating a gateway that wraps CGI or a CGI-like protocol such as FastCGI. Typically, servers providing such protocols will include a `HTTPS` variable with a value of “1”, “yes”, or “on” when a request is received via SSL. So, this function returns “https” if such a value is found, and “http” otherwise.

wsgiref.util.request_uri(environ, include_query=True)

Return the full request URI, optionally including the query string, using the algorithm found in the “URL Reconstruction” section of PEP 3333. If `include_query` is false, the query string is not included in the resulting URI.

wsgiref.util.application_uri(environ)

Similar to `request_uri()`, except that the `PATH_INFO` and `QUERY_STRING` variables are ignored. The result is the base URI of the application object addressed by the request.

wsgiref.util.shift_path_info(environ)

Shift a single name from `PATH_INFO` to `SCRIPT_NAME` and return the name. The `environ` dictionary is modified in-place; use a copy if you need to keep the original `PATH_INFO` or `SCRIPT_NAME` intact. If there are no remaining path segments in `PATH_INFO`, `None` is returned. Typically, this routine is used to process each portion of a request URI path, for example to treat the path as a series of dictionary keys. This routine modifies the passed-in environment to make it suitable for invoking another WSGI application that

wsgiref.util.setup_testing_defaults(envIRON)

Update `envIRON` with trivial defaults for testing purposes. This routine adds various parameters required for WSGI, including `HTTP_HOST`, `SERVER_NAME`, `SERVER_PORT`, `REQUEST_METHOD`, `SCRIPT_NAME`, `PATH_INFO`, and all of the PEP 3333-defined `wsgi.*` variables. It only supplies default values, and does not replace any existing settings for these variables. This routine is intended to make it easier for unit tests of WSGI servers and applications to set up dummy environments. It should NOT be used by actual WSGI serv

In addition to the environment functions above, the `wsgiref.util` module also provides these miscellaneous utilities:

wsgiref.util.is_hop_by_hop(header_name)

Return `True` if `'header_name'` is an HTTP/1.1 "Hop-by-Hop" header, as defined by RFC 2616.

class wsgiref.util.FileWrapper(filelike, blksize=8192)

A concrete implementation of the `wsgiref.types.FileWrapper` protocol used to convert a file-like object to an iterator. The resulting objects are iterables. As the object is iterated over, the optional `blksize` parameter will be repeatedly passed to the `filelikeobject.read()` method to obtain bytestrings to yield. When `read()` returns an empty bytestring, iteration is ended and is not resumable. If `filelike` has a `close()` method, the returned object will also have a `close()` method, and it will invoke the `filelikeobj`

wsgiref.headers– WSGI response header tools

This module provides a single class, `Headers`, for convenient manipulation of WSGI response headers using a mapping-like interface.

class wsgiref.headers.Headers([headers])

get_all(name)

add_header(name, value, **_params)

Create a mapping-like object wrapping `headers`, which must be a list of header name/value tuples as described in PEP 3333. The default value of `headers` is an empty list. `Headers` objects support typical mapping operations including `__getitem__()`, `get()`, `__setitem__()`, `setdefault()`, `__delitem__()` and `__contains__()`. For each of these methods, the key is the header name (treated case-insensitively), and the value is the first value associated with that header name. Setting a header deletes any existing values

Return a list of all the values for the named header. The returned list will be sorted in the order they appeared in the original header list or were added to this instance, and may contain duplicates. Any fields deleted and re-inserted are always appended to the header list. If no fields exist with the given name, returns an empty list.

Add a (possibly multi-valued) header, with optional MIME parameters specified via keyword arguments. `name` is the header field to add. Keyword arguments can be used to set MIME parameters for the header field. Each parameter must be a string or `None`. Underscores in parameter names are converted to dashes, since dashes are illegal in Python identifiers, but many MIME parameter names include dashes. If the parameter value is a string, it is added to the header value parameters in the form `name="valu`

wsgiref.simple_server– a simple WSGI HTTP server

This module implements a simple HTTP server (based on `http.server`) that serves WSGI applications. Each server instance serves a single WSGI application on a given host and port. If you want to serve multiple applications on a single host and port, you should create a WSGI application that parses `PATH_INFO` to select which application to invoke for each request. (E.g., using the `shift_path_info()` function from `wsgiref.util`.)

wsgiref.simple_server.make_server(host,port,app,server_class=WSGIServer,handler_class=WSGIRequestHandler)

Create a new WSGI server listening on `host` and `port`, accepting connections for `app`. The return value is an instance of the supplied `server_class`, and will process requests using the specified `handler_class`. `app` must be a WSGI application object, as defined by PEP 3333. Example usage: `from wsgiref.simple_server import make_server, demo_app` `with make_server("", 8000, demo_app) as httpd: print("Serving HTTP on port 8000...") # Respond to requests until process is killed` `httpd.serve_forever()` # Alternative: serve one request

wsgiref.simple_server.demo_app(environ,start_response)

This function is a small but complete WSGI application that returns a text page containing the message "Hello world!" and a list of the key/value pairs provided in the `environ` parameter. It's useful for verifying that a WSGI server (such as `wsgiref.simple_server`) is able to run a simple WSGI application correctly. The `start_response` callable should follow the `StartResponse` protocol.

class wsgiref.simple_server.WSGIServer(server_address,RequestHandlerClass)

set_app(application)

get_app()

Create a `WSGIServer` instance. `server_address` should be a `(host,port)` tuple, and `RequestHandlerClass` should be the subclass of `http.server.BaseHTTPRequestHandler` that will be used to process requests. You do not normally need to call this constructor, as the `make_server()` function can handle all the details for you. `WSGIServer` is a subclass of `http.server.HTTPServer`, so all of its methods (such as `serve_forever()` and `handle_request()`) are available. `WSGIServer` also provides these WSGI-specific methods: `set_app(application)`

Sets the callable `application` as the WSGI application that will receive requests.

Returns the currently set application callable.

class wsgiref.simple_server.WSGIRequestHandler(request,client_address,server)

get_environ()

get_stderr()

handle()

Create an HTTP handler for the given `request` (i.e. a socket), `client_address` (a `(host,port)` tuple), and `server` (`WSGIServer` instance). You do not need to create instances of this class directly; they are automatically created as needed by `WSGIServer` objects. You can, however, subclass this class and supply it as a `handler_class` to the `make_server()` function. Some possibly relevant methods for overriding in subclasses: `get_environ()` Return a `WSGIEnvironment` dictionary for a request. The default implementation copies

Return a `WSGIEnvironment` dictionary for a request. The default implementation copies the contents of the `WSGIServer` object's `base_environ` dictionary attribute and then adds various headers derived from the HTTP request. Each call to this method should return a new dictionary containing all of the relevant CGI environment variables as specified in PEP 3333.

Return the object that should be used as the `wsgi.errors` stream. The default implementation just returns `sys.stderr`.

Process the HTTP request. The default implementation creates a handler instance using `awsgiref.handlers` class to implement the actual WSGI application interface.

wsgiref.validate— WSGI conformance checker

When creating new WSGI application objects, frameworks, servers, or middleware, it can be useful to validate the new code's conformance using `wsgiref.validate`. This module provides a function that creates WSGI application objects that validate communications between a WSGI server or gateway and a WSGI application object, to check both sides for protocol conformance.

Note that this utility does not guarantee complete PEP 3333 compliance; an absence of errors from this module does not necessarily mean that errors do not exist. However, if this module does produce an error, then it is virtually certain that either the server or application is not 100% compliant.

This module is based on the `paste.lint` module from Ian Bicking's "Python Paste" library.

`wsgiref.validate.validator(application)`

Wrap `application` and return a new WSGI application object. The returned application will forward all requests to the original `application`, and will check that both the `application` and the server invoking it are conforming to the WSGI specification and to RFC 2616. Any detected nonconformance results in an `AssertionError` being raised; note, however, that how these errors are handled is server-dependent. For example, `wsgiref.simple_server` and other servers based on `wsgiref.handlers` (that don't override the `e`

`wsgiref.handlers`— server/gateway base classes

This module provides base handler classes for implementing WSGI servers and gateways. These base classes handle most of the work of communicating with a WSGI application, as long as they are given a CGI-like environment, along with input, output, and error streams.

`class wsgiref.handlers.CGIHandler`

CGI-based invocation via `sys.stdin`, `sys.stdout`, `sys.stderr` and `os.environ`. This is useful when you have a WSGI application and want to run it as a CGI script. Simply invoke `CGIHandler().run(app)`, where `app` is the WSGI application object you wish to invoke. This class is a subclass of `BaseCGIHandler` that sets `wsgi.run_on_ceto` to `true`, `wsgi.multithread` to `false`, and `wsgi.multiprocess` to `true`, and always uses `sys` and `os` to obtain the necessary CGI streams and environment.

`class wsgiref.handlers.IISCGIHandler`

A specialized alternative to `CGIHandler`, for use when deploying on Microsoft's IIS web server, without having set the config `allowPathInfo` option (IIS >= 7) or metabase `allowPathInfoForScriptMappings` (IIS < 7). By default, IIS gives a `PATH_INFO` that duplicates the `SCRIPT_NAME` at the front, causing problems for WSGI applications that wish to implement routing. This handler strips any such duplicated path. IIS can be configured to pass the correct `PATH_INFO`, but this causes another bug where `PATH_TRANSLATED` is

`class wsgiref.handlers.BaseCGIHandler(stdin, stdout, stderr, environ, multithread=True, multiprocess=False)`

Similar to `CGIHandler`, but instead of using the `sys` and `os` modules, the CGI environment and I/O streams are specified explicitly. The `multithread` and `multiprocess` values are used to set the `wsgi.multithread` and `wsgi.multiprocess` flags for any applications run by the handler instance. This class is a subclass of `SimpleHandler` intended for use with software other than HTTP "origin servers". If you are writing a gateway protocol implementation (such as CGI, FastCGI, SCGI, etc.) that uses a `Status:header` to send an

`class wsgiref.handlers.SimpleHandler(stdin, stdout, stderr, environ, multithread=True, multiprocess=False)`

Similar to `BaseCGIHandler`, but designed for use with HTTP origin servers. If you are writing an HTTP server implementation, you will probably want to subclass this instead of `BaseCGIHandler`. This class is a subclass of `BaseHandler`. It overrides the `__init__()`, `get_stdin()`, `get_stderr()`, `add_cgi_vars()`, `_write()`, and `_flush()` methods to support explicitly setting the environment and streams via the constructor. The supplied environment and streams are stored in the `stdin`, `stdout`, `stderr`, and `environ` attribute

classwsgiref.handlers.BaseHandler

run(app)

_write(data)

_flush()

get_stdin()

get_stderr()

add_cgi_vars()

wsgi_multithread

wsgi_multiprocess

wsgi_run_once

os_environ

server_software

get_scheme()

setup_environ()

log_exception(exc_info)

traceback_limit

error_output(environ,start_response)

error_status

error_headers

error_body

wsgi_file_wrapper

sendfile()

origin_server

http_version

This is an abstract base class for running WSGI applications. Each instance will handle a single HTTP request, although in principle you could create a subclass that was reusable for multiple requests. `BaseHandler` instances have only one method intended for external use: `run(app)`. Run the specified WSGI application, `app`. All of the other `BaseHandler` methods are invoked by this method in the process of running the application, and thus exist primarily to allow customizing the process. The following method

Run the specified WSGI application, `app`.

Buffer the bytes `data` for transmission to the client. It's okay if this method actually transmits the data; `BaseHandler` just separates write and flush operations for greater efficiency when the underlying system actually has such a distinction.

Force buffered data to be transmitted to the client. It's okay if this method is a no-op (i.e., if `_write()` actually sends the data).

Return an object compatible with `InputStream` suitable for use as `thewsgi.input` of the request currently being processed.

Return an object compatible with `ErrorStream` suitable for use as `thewsgi.errors` of the request currently being processed.

Insert CGI variables for the current request into `theenviron` attribute.

The value to be used for `thewsgi.multithreadenvironment` variable. It defaults to `true` in `BaseHandler`, but may have a different default (or be set by the constructor) in the other subclasses.

The value to be used for `thewsgi.multiprocessenvironment` variable. It defaults to `true` in `BaseHandler`, but may have a different default (or be set by the constructor) in the other subclasses.

The value to be used for `thewsgi.run_onceenvironment` variable. It defaults to `false` in `BaseHandler`, but `CGIHandler` sets it to `true` by default.

The default environment variables to be included in every request's WSGI environment. By default, this is a copy of `os.environ` at the time that `wsgiref.handlers` was imported, but subclasses can either create their own at the class or instance level. Note that the dictionary should be considered read-only, since the default value is shared between multiple classes and instances.

If `theorigin_serverattribute` is set, this attribute's value is used to set the default `SERVER_SOFTWARE` WSGI environment variable, and also to set a default `Server:header` in HTTP responses. It is ignored for handlers (such as `BaseCGIHandler` and `CGIHandler`) that are not HTTP origin servers. Changed in version 3.3: The term "Python" is replaced with implementation specific term like "CPython", "Jython" etc.

Return the URL scheme being used for the current request. The default implementation uses `theguess_scheme()` function from `wsgiref.util` to guess whether the scheme should be "http" or "https", based on the current request's `environ` variables.

Set `theenviron` attribute to a fully populated WSGI environment. The default implementation uses all of the above methods and attributes, plus `theget_stdin().get_stderr()`, and `add_cgi_vars()` methods and `thewsgi_file_wrapperattribute`. It also inserts a `SERVER_SOFTWARE` key if not present, as long as `theorigin_serverattribute` is a `true` value and `theserver_softwareattribute` is set.

Log the `exc_info` tuple in the server log. `exc_info` is a `(type,value,traceback)` tuple. The default implementation simply writes the traceback to the request's `swsgi.errors` stream and flushes it. Subclasses can override this method to change the format or retarget the output, mail the traceback to an administrator, or whatever other action may be deemed suitable.

The maximum number of frames to include in tracebacks output by the default `log_exception()` method. If `None`, all frames are included.

This method is a WSGI application to generate an error page for the user. It is only invoked if an error occurs before headers are sent to the client. This method can access the current error using `sys.exception()`, and should pass that information to `start_response` when calling it (as described in the "Error Handling" section of PEP 3333). In particular, the `start_response` callable should follow the `StartResponse` protocol. The default implementation just uses `theerror_status`, `error_headers`, and `error_body`.

The HTTP status used for error responses. This should be a status string as defined in PEP 3333; it defaults to a 500 code and message.

The HTTP headers used for error responses. This should be a list of WSGI response headers `((name,value)tuples)`, as described in PEP 3333. The default list just sets the content type to `text/plain`.

The error response body. This should be an HTTP response body bytestring. It defaults to the plain text, "A server error occurred. Please contact the administrator."

`Awsgi.file_wrapperfactory`, compatible with `wsgiref.types.FileWrapper`, or `None`. The default value of this attribute is `thewsgiref.util.FileWrapperclass`.

Override to implement platform-specific file transmission. This method is called only if the application's return value is an instance of the class specified by the `wsgi_file_wrapper` attribute. It should return a true value if it was able to successfully transmit the file, so that the default transmission code will not be executed. The default implementation of this method just returns a false value.

This attribute should be set to a true value if the handler's `_write()` and `_flush()` are being used to communicate directly to the client, rather than via a CGI-like gateway protocol that wants the HTTP status in a special `Status:header`. This attribute's default value is true in `BaseHandler`, but false in `BaseCGIHandler` and `CGIHandler`.

If `origin_server` is true, this string attribute is used to set the HTTP version of the response set to the client. It defaults to "1.0".

wsgiref.handlers.read_environ()

Transcode CGI variables from `os.environ` to PEP 3333 "bytes in unicode" strings, returning a new dictionary. This function is used by `CGIHandler` and `ISCGIHandler` in place of directly using `os.environ`, which is not necessarily WSGI-compliant on all platforms and web servers using Python 3 – specifically, ones where the OS's actual environment is Unicode (i.e. Windows), or ones where the environment is bytes, but the system encoding used by Python to decode it is anything other than ISO-8859-1 (e.g. Unix

wsgiref.types – WSGI types for static type checking

This module provides various types for static type checking as described in PEP 3333.

Added in version 3.11.

class wsgiref.types.StartResponse

A typing.Protocol describing `start_response()` callables (PEP 3333).

wsgiref.types.WSGIEnvironment

A type alias describing a WSGI environment dictionary.

wsgiref.types.WSGIApplication

A type alias describing a WSGI application callable.

class wsgiref.types.InputStream

A typing.Protocol describing a WSGI Input Stream.

class wsgiref.types.ErrorStream

A typing.Protocol describing a WSGI Error Stream.

class wsgiref.types.FileWrapper

A typing.Protocol describing a file wrapper. See `wsgiref.util.FileWrapper` for a concrete implementation of this protocol.

Examples

This is a working "Hello World" WSGI application, where the `start_response` callable should follow the `StartResponse` protocol:

```
"""
```

```
Every WSGI application must have an application object - a callable object that accepts two arguments. For that purpose, we're going to use a function (note that you're not limited to a function, you can
```

use a class for example). The first argument passed to the function is a dictionary containing CGI-style environment variables and the second variable is the callable object.

```
"""
```

```
from wsgiref.simple_server import make_server
```

```
def hello_world_app(environ, start_response):
```

```
    status = "200 OK" # HTTP Status
```

```
    headers = [("Content-type", "text/plain; charset=utf-8")] # HTTP Headers
```

```
    start_response(status, headers)
```

```
    # The returned object is going to be printed
```

```
    return [b"Hello World"]
```

```
with make_server("", 8000, hello_world_app) as httpd:
```

```
    print("Serving on port 8000...")
```

```
    # Serve until process is killed
```

```
    httpd.serve_forever()
```

Example of a WSGI application serving the current directory, accept optional directory and port number (default: 8000) on the command line:

```
"""
```

```
Small wsgiref based web server. Takes a path to serve from and an optional port number (defaults to 8000), then tries to serve files. MIME types are guessed from the file names, 404 errors are raised if the file is not found.
```

```
"""
```

```
import mimetypes
```

```
import os
```

```
import sys
```

```
from wsgiref import simple_server, util
```

```
def app(environ, respond):
```

```
    # Get the file name and MIME type
```

```
    fn = os.path.join(path, environ["PATH_INFO"][1:])
```

```
    if "." not in fn.split(os.path.sep)[-1]:
```

```
        fn = os.path.join(fn, "index.html")
```

```
    mime_type = mimetypes.guess_file_type(fn)[0]
```

```
    # Return 200 OK if file exists, otherwise 404 Not Found
```

```
    if os.path.exists(fn):
```

```
        respond("200 OK", [("Content-Type", mime_type)])
```

```
        return util.FileWrapper(open(fn, "rb"))
```

```
    else:
```

```
        respond("404 Not Found", [("Content-Type", "text/plain")])
```

```
        return [b"not found"]
```

```
if __name__ == "__main__":
```

```
    # Get the path and port from command-line arguments
```

```
    path = sys.argv[1] if len(sys.argv) > 1 else os.getcwd()
```

```
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 8000
```

```
    # Make and start the server until control-c
```

```
    httpd = simple_server.make_server("", port, app)
```

```
    print(f"Serving {path} on port {port}, control-C to stop")
```

```
    try:
```

```
        httpd.serve_forever()
```

```
    except KeyboardInterrupt:
```

```
        print("Shutting down.")
```

```
        httpd.server_close()
```

urllib— URL handling modules

Source code: [Lib/urllib/](#)

`urllib` is a package that collects several modules for working with URLs:

- `urllib.request` for opening and reading URLs
- `urllib.error` containing the exceptions raised by `urllib.request`
- `urllib.parse` for parsing URLs
- `urllib.robotparser` for parsing `robots.txt` files

urllib.request— Extensible library for opening URLs

Source code: `Lib/urllib/request.py`

The `urllib.request` module defines functions and classes which help in opening URLs (mostly HTTP) in a complex world — basic and digest authentication, redirections, cookies and more.

See also

The `Requests` package is recommended for a higher-level HTTP client interface.

■■■ Advertencia: Warning On macOS it is unsafe to use this module in programs using `os.fork()` because the `getproxies()` implementation for macOS uses a higher-level system API. Set the environment variable `no_proxy` to avoid this problem (e.g. `os.environ["no_proxy"]="*"`).

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

The `urllib.request` module defines the following functions:

`urllib.request.urlopen(url, data=None, [timeout,]*, context=None)`

Open `url`, which can be either a string containing a valid, properly encoded URL, or a `Request` object. `data` must be an object specifying additional data to be sent to the server, or `None` if no such data is needed. See [Request](#) for details. The `urllib.request` module uses HTTP/1.1 and includes `Connection: close` header in its HTTP requests. The optional `timeout` parameter specifies a timeout in seconds for blocking operations like the connection attempt (if not specified, the global default timeout setting will be used)

`urllib.request.install_opener(opener)`

Install an `OpenerDirector` instance as the default global opener. Installing an opener is only necessary if you want `urlopen` to use that opener; otherwise, simply call `OpenerDirector.open()` instead of `urlopen()`. The code does not check for a real `OpenerDirector`, and any class with the appropriate interface will work.

`urllib.request.build_opener([handler,...])`

Return an `OpenerDirector` instance, which chains the handlers in the order given. `handlers` can be either instances of `BaseHandler`, or subclasses of `BaseHandler` (in which case it must be possible to call the constructor without any parameters). Instances of the following classes will be in front of the handlers, unless the handlers contain them, instances of them or subclasses of them: `ProxyHandler` (if proxy settings are detected), `UnknownHandler`, `HTTPHandler`, `HTTPDefaultErrorHandler`, `HTTPRedirectHandler`, `FTPHandler`

`urllib.request.pathname2url(path, *, add_scheme=False)`

Convert the given local path to a file:URL. This function uses `quote()` function to encode the path. If `add_scheme` is false (the default), the return value omits the file:scheme prefix. Set `add_scheme` to true to return a complete URL. This example shows the function being used on Windows:

```
>>> from urllib.request import pathname2url
>>> path = 'C:\\Program Files'
>>> pathname2url(path, add_scheme=True)
file:///C:/Program%20Files'
Changed in version 3.14: Windows drive letters are no longer converted to uppercase, and:cha
```

`urllib.request.url2pathname(url, *, require_scheme=False, resolve_host=False)`

Convert the given file:URL to a local path. This function uses `unquote()` to decode the URL. If `require_scheme` is false (the default), the given value should omit a file:scheme prefix. If `require_scheme` is set to true, the given value should include the prefix; a `URLError` is raised if it doesn't. The URL authority is discarded if it is empty, `localhost`, or the local hostname. Otherwise, if `resolve_host` is set to true, the authority is resolved using `socket.gethostbyname()` and discarded if it matches a local IP address

urllib.request.getproxies()

This helper function returns a dictionary of scheme to proxy server URL mappings. It scans the environment for variables named `<scheme>_proxy`, in a case insensitive approach, for all operating systems first, and when it cannot find it, looks for proxy information from System Configuration for macOS and Windows Systems Registry for Windows. If both lowercase and uppercase environment variables exist (and disagree), lowercase is preferred. Note if the environment variable `REQUEST_METHOD` is set, which u

The following classes are provided:

class urllib.request.Request(url, data=None, headers={}, origin_req_host=None, unverifiable=False, method=None)

This class is an abstraction of a URL request. `url` should be a string containing a valid, properly encoded URL. `data` must be an object specifying additional data to send to the server, or `None` if no such data is needed. Currently HTTP requests are the only ones that use `data`. The supported object types include bytes, file-like objects, and iterables of bytes-like objects. If no `Content-Length` nor `Transfer-Encoding` header field has been provided, `HTTPHandler` will set these headers according to the type of `data`.

class urllib.request.OpenerDirector

The `OpenerDirector` class opens URLs via `BaseHandlers` chained together. It manages the chaining of handlers, and recovery from errors.

class urllib.request.BaseHandler

This is the base class for all registered handlers — and handles only the simple mechanics of registration.

class urllib.request.HTTPDefaultErrorHandler

A class which defines a default handler for HTTP error responses; all responses are turned into `HTTPError` exceptions.

class urllib.request.HTTPRedirectHandler

A class to handle redirections.

class urllib.request.HTTPCookieProcessor(cookiejar=None)

A class to handle HTTP Cookies.

class urllib.request.ProxyHandler(proxies=None)

Cause requests to go through a proxy. If `proxies` is given, it must be a dictionary mapping protocol names to URLs of proxies. The default is to read the list of proxies from the environment variables `<protocol>_proxy`. If no proxy environment variables are set, then in a Windows environment proxy settings are obtained from the registry's Internet Settings section, and in a macOS environment proxy information is retrieved from the System Configuration Framework. To disable autodetected proxy pass an

class urllib.request.HTTPPasswordMgr

Keep a database of `(realm, uri) -> (user, password)` mappings.

class urllib.request.HTTPPasswordMgrWithDefaultRealm

Keep a database of `(realm, uri) -> (user, password)` mappings. A realm of `None` is considered a catch-all realm, which is searched if no other realm fits.

class urllib.request.HTTPPasswordMgrWithPriorAuth

A variant of `HTTPPasswordMgrWithDefaultRealm` that also has a database of `uri -> is_authenticated` mappings. Can be used by a `BasicAuth` handler to determine when to send authentication credentials immediately instead of waiting for a `401` response first. Added in version 3.5.

classurllib.request.AbstractBasicAuthHandler(password_mgr=None)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. password_mgr, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. If password_mgr also provides is_authenticated and update_authenticated methods (see HTTPPasswordMgrWithPriorAuth Objects), then the handler will use their is_authenticated result for a given URI to determine whether or not to send au

classurllib.request.HTTPBasicAuthHandler(password_mgr=None)

Handle authentication with the remote host. password_mgr, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. HTTPBasicAuthHandler will raise a ValueError when presented with a wrong Authentication scheme.

classurllib.request.ProxyBasicAuthHandler(password_mgr=None)

Handle authentication with the proxy. password_mgr, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

classurllib.request.AbstractDigestAuthHandler(password_mgr=None)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. password_mgr, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. Changed in version 3.14: Added support for HTTP digest authentication algorithm SHA-256.

classurllib.request.HTTPDigestAuthHandler(password_mgr=None)

Handle authentication with the remote host. password_mgr, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. When both Digest Authentication Handler and Basic Authentication Handler are both added, Digest Authentication is always tried first. If the Digest Authentication returns a 40x response again, it is sent to Basic Authentication handler to Handle. This Handler method will rais

classurllib.request.ProxyDigestAuthHandler(password_mgr=None)

Handle authentication with the proxy. password_mgr, if given, should be something that is compatible with HTTPPasswordMgr; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

classurllib.request.HTTPHandler

A class to handle opening of HTTP URLs.

classurllib.request.HTTPSHandler(debuglevel=0,context=None,check_hostname=None)

A class to handle opening of HTTPS URLs. context and check_hostname have the same meaning as in http.client.HTTPSConnection. Changed in version 3.2: context and check_hostname were added.

classurllib.request.FileHandler

Open local files.

classurllib.request.DataHandler

Open data URLs. Added in version 3.4.

classurllib.request.FTPHandler

Open FTP URLs.

classurllib.request.CacheFTPHandler

Open FTP URLs, keeping a cache of open FTP connections to minimize delays.

classurllib.request.UnknownHandler

A catch-all class to handle unknown URLs.

classurllib.request.HTTPErrorProcessor

Process HTTP error responses.

Request Objects

The following methods describe `Request`'s public interface, and so all may be overridden in subclasses. It also defines several public attributes that can be used by clients to inspect the parsed request.

Request.full_url

The original URL passed to the constructor. Changed in version 3.4: `Request.full_url` is a property with setter, getter and a deleter. `Request.full_url` returns the original request URL with the fragment, if it was present.

Request.type

The URI scheme.

Request.host

The URI authority, typically a host, but may also contain a port separated by a colon.

Request.origin_req_host

The original host for the request, without port.

Request.selector

The URI path. If the request uses a proxy, then selector will be the full URL that is passed to the proxy.

Request.data

The entity body for the request, or `None` if not specified. Changed in version 3.4: Changing value of `Request.data` now deletes "Content-Length" header if it was previously set or calculated.

Request.unverifiable

boolean, indicates whether the request is unverifiable as defined by RFC 2965.

Request.method

The HTTP request method to use. By default its value is `None`, which means that `get_method()` will do its normal computation of the method to be used. Its value can be set (thus overriding the default computation in `get_method()`) either by providing a default value by setting it at the class level in a `Request` subclass, or by passing a value in to the `Request` constructor via the `method` argument. Added in version 3.3. Changed in version 3.4: A default value can now be set in subclasses; previously it could only be set on the class.

Request.get_method()

Return a string indicating the HTTP request method. If `Request.method` is not `None`, return its value, otherwise return 'GET' if `Request.data` is `None`, or 'POST' if it's not. This is only meaningful for HTTP requests. Changed in version 3.3: `get_method` now looks at the value of `Request.method`.

Request.add_header(key, val)

Add another header to the request. Headers are currently ignored by all handlers except HTTP handlers, where they are added to the list of headers sent to the server. Note that there cannot be more than one header with the same name, and later calls will overwrite previous calls in case the key collides. Currently, this is no loss

of HTTP functionality, since all headers which have meaning when used more than once have a (header-specific) way of gaining the same functionality using only one head

Request.add_unredirected_header(key,header)

Add a header that will not be added to a redirected request.

Request.has_header(header)

Return whether the instance has the named header (checks both regular and unredirected).

Request.remove_header(header)

Remove named header from the request instance (both from regular and unredirected headers).Added in version 3.4.

Request.get_full_url()

Return the URL given in the constructor.Changed in version 3.4.ReturnsRequest.full_url

Request.set_proxy(host,type)

Prepare the request by connecting to a proxy server. Thehostandtypewill replace those of the instance, and the instance's selector will be the original URL given in the constructor.

Request.get_header(header_name,default=None)

Return the value of the given header. If the header is not present, return the default value.

Request.header_items()

Return a list of tuples (header_name, header_value) of the Request headers.

Changed in version 3.4:The request methods add_data, has_data, get_data, get_type, get_host, get_selector, get_origin_req_host and is_unverifiable that were deprecated since 3.3 have been removed.

OpenerDirector Objects

OpenerDirectorinstances have the following methods:

OpenerDirector.add_handler(handler)

handlershould be an instance ofBaseHandler. The following methods are searched, and added to the possible chains (note that HTTP errors are a special case). Note that, in the following,protocolshould be replaced with the actual protocol to handle, for examplehttp_response()would be the HTTP protocol response handler. Also,typeshould be replaced with the actual HTTP code, for examplehttp_error_404()would handle HTTP 404 errors.<protocol>_open()— signal that the handler knows how to openprotocol

OpenerDirector.open(url,data=None[,timeout])

Open the givenurl(which can be a request object or a string), optionally passing the givendata. Arguments, return values and exceptions raised are the same as those ofurlopen()(which simply calls theopen()method on the currently installed globalOpenerDirector). The optionaltimeoutparameter specifies a timeout in seconds for blocking operations like the connection attempt (if not specified, the global default timeout setting will be used). The timeout feature actually works only for HTTP, HTTPS

OpenerDirector.error(proto,*args)

Handle an error of the given protocol. This will call the registered error handlers for the given protocol with the given arguments (which are protocol specific). The HTTP protocol is a special case which uses the HTTP response code to determine the specific error handler; refer to thehttp_error_<type>()methods of the handler classes.Return values and exceptions raised are the same as those ofurlopen().

OpenerDirector objects open URLs in three stages:

The order in which these methods are called within each stage is determined by sorting the handler instances.

- Every handler with a method named like<protocol>_request()has that method called to pre-process the request.
- Handlers with a method named like<protocol>_open()are called to handle the request. This stage ends when a handler either returns a non-Nonevalue (ie. a response), or raises an exception (usuallyURLError). Exceptions are allowed to propagate.In fact, the above algorithm is first tried for methods nameddefault_open(). If all such methods returnNone, the algorithm is repeated for methods named lik
- Every handler with a method named like<protocol>_response()has that method called to post-process the response.

BaseHandler Objects

BaseHandlerobjects provide a couple of methods that are directly useful, and others that are meant to be used by derived classes. These are intended for direct use:

BaseHandler.add_parent(director)

Add a director as parent.

BaseHandler.close()

Remove any parents.

The following attribute and methods should only be used by classes derived fromBaseHandler.

■ Nota: NoteThe convention has been adopted that subclasses defining<protocol>_request()or<protocol>_response()methods are named*Processor; all others are named*Handler.

BaseHandler.parent

A validOpenerDirector, which can be used to open using a different protocol, or handle errors.

BaseHandler.default_open(req)

This method isnotdefined inBaseHandler, but subclasses should define it if they want to catch all URLs.This method, if implemented, will be called by the parentOpenerDirector. It should return a file-like object as described in the return value of theopen()method ofOpenerDirector, orNone. It should raiseURLError, unless a truly exceptional thing happens (for example,MemoryErrorshould not be mapped toURLError).This method will be called before any protocol-specific open method.

BaseHandler.<protocol>_open(req)

This method isnotdefined inBaseHandler, but subclasses should define it if they want to handle URLs with the given protocol.This method, if defined, will be called by the parentOpenerDirector. Return values should be the same as fordefault_open().

BaseHandler.unknown_open(req)

This method isnotdefined inBaseHandler, but subclasses should define it if they want to catch all URLs with no specific registered handler to open it.This method, if implemented, will be called by theparentOpenerDirector. Return values should be the same as fordefault_open().

BaseHandler.http_error_default(req,fp,code,msg,hdrs)

This method isnotdefined inBaseHandler, but subclasses should override it if they intend to provide a catch-all for otherwise unhandled HTTP errors. It will be called automatically by theOpenerDirectorgetting the error, and

should not normally be called in other circumstances. `OpenerDirector` will call this method with five positional arguments: `aRequest` object, a file-like object with the HTTP error body, the three-digit code of the error, as a string, the user-visible explanation of the code, as a string.

`BaseHandler.http_error_nnn>(req,fp,code,msg,hdrs)`

`nnn` should be a three-digit HTTP error code. This method is also not defined in `BaseHandler`, but will be called, if it exists, on an instance of a subclass, when an HTTP error with code `nnn` occurs. Subclasses should override this method to handle specific HTTP errors. Arguments, return values and exceptions raised should be the same as for `http_error_default()`.

`BaseHandler.<protocol>_request(req)`

This method is not defined in `BaseHandler`, but subclasses should define it if they want to pre-process requests of the given protocol. This method, if defined, will be called by the parent `OpenerDirector`. `req` will be a `Request` object. The return value should be a `Request` object.

`BaseHandler.<protocol>_response(req,response)`

This method is not defined in `BaseHandler`, but subclasses should define it if they want to post-process responses of the given protocol. This method, if defined, will be called by the parent `OpenerDirector`. `req` will be a `Request` object. `response` will be an object implementing the same interface as the return value of `urlopen()`. The return value should implement the same interface as the return value of `urlopen()`.

HTTPRedirectHandler Objects

■ **Note:** Some HTTP redirections require action from this module's client code. If this is the case, `HTTPError` is raised. See RFC 2616 for details of the precise meanings of the various redirection codes. An `HTTPError` exception is raised as a security consideration if the `HTTPRedirectHandler` is presented with a redirected URL which is not an HTTP, HTTPS or FTP URL.

`HTTPRedirectHandler.redirect_request(req,fp,code,msg,hdrs,newurl)`

Return a `Request` or `None` in response to a redirect. This is called by the default implementations of the `http_error_30*` methods when a redirection is received from the server. If a redirection should take place, return a new `Request` to allow `http_error_30*` to perform the redirect to `newurl`. Otherwise, raise `HTTPError` if no other handler should try to handle this URL, or return `None` if you can't but another handler might. **Note:** The default implementation of this method does not strictly follow RFC 2616, which

`HTTPRedirectHandler.http_error_301(req,fp,code,msg,hdrs)`

Redirect to the `Location`: or `URI`: URL. This method is called by the parent `OpenerDirector` when getting an HTTP 'moved permanently' response.

`HTTPRedirectHandler.http_error_302(req,fp,code,msg,hdrs)`

The same as `http_error_301()`, but called for the 'found' response.

`HTTPRedirectHandler.http_error_303(req,fp,code,msg,hdrs)`

The same as `http_error_301()`, but called for the 'see other' response.

`HTTPRedirectHandler.http_error_307(req,fp,code,msg,hdrs)`

The same as `http_error_301()`, but called for the 'temporary redirect' response. It does not allow changing the request method from `POST` to `GET`.

`HTTPRedirectHandler.http_error_308(req,fp,code,msg,hdrs)`

The same as `http_error_301()`, but called for the 'permanent redirect' response. It does not allow changing the request method from `POST` to `GET`. Added in version 3.11.

HTTPCookieProcessor Objects

HTTPCookieProcessor instances have one attribute:

HTTPCookieProcessor.cookiejar

The `http.cookiejar.CookieJar` in which cookies are stored.

ProxyHandler Objects

ProxyHandler.<protocol>_open(request)

The `ProxyHandler` will have a method `<protocol>_open()` for every protocol which has a proxy in the proxies dictionary given in the constructor. The method will modify requests to go through the proxy, by calling `request.set_proxy()`, and call the next handler in the chain to actually execute the protocol.

HTTPPasswordMgr Objects

These methods are available on `HTTPPasswordMgr` and `HTTPPasswordMgrWithDefaultRealm` objects.

HTTPPasswordMgr.add_password(realm,uri,user,passwd)

`uri` can be either a single URI, or a sequence of URIs. `realm`, `user` and `passwd` must be strings. This causes `(user,passwd)` to be used as authentication tokens when authentication for `realm` and a super-URI of any of the given URIs is given.

HTTPPasswordMgr.find_user_password(realm,authuri)

Get `user/password` for given `realm` and `URI`, if any. This method will return `(None,None)` if there is no matching `user/password`. For `HTTPPasswordMgrWithDefaultRealm` objects, the `realmNone` will be searched if the given `realm` has no matching `user/password`.

HTTPPasswordMgrWithPriorAuth Objects

This password manager extends `HTTPPasswordMgrWithDefaultRealm` to support tracking URIs for which authentication credentials should always be sent.

HTTPPasswordMgrWithPriorAuth.add_password(realm,uri,user,passwd,is_authenticated=False)

`realm`, `uri`, `user`, `passwd` are as for `HTTPPasswordMgr.add_password()`. `is_authenticated` sets the initial value of the `is_authenticated` flag for the given URI or list of URIs. If `is_authenticated` is specified as `True`, `realm` is ignored.

HTTPPasswordMgrWithPriorAuth.find_user_password(realm,authuri)

Same as for `HTTPPasswordMgrWithDefaultRealm` objects

HTTPPasswordMgrWithPriorAuth.update_authenticated(self,uri,is_authenticated=False)

Update the `is_authenticated` flag for the given `uri` or list of URIs.

HTTPPasswordMgrWithPriorAuth.is_authenticated(self,authuri)

Returns the current state of the `is_authenticated` flag for the given URI.

AbstractBasicAuthHandler Objects

AbstractBasicAuthHandler.http_error_auth_reged(authreq,host,req,headers)

Handle an authentication request by getting a user/password pair, and re-trying the request. `authreq` should be the name of the header where the information about the realm is included in the request, `host` specifies the URL and path to authenticate for, `req` should be the (failed) `Request` object, and `headers` should be the error headers. `host` is either an authority (e.g. "python.org") or a URL containing an authority component (e.g. "http://python.org/"). In either case, the authority must not contain a userinfo

HTTPBasicAuthHandler Objects

HTTPBasicAuthHandler.http_error_401(req,fp,code,msg,hdrs)

Retry the request with authentication information, if available.

ProxyBasicAuthHandler Objects

ProxyBasicAuthHandler.http_error_407(req,fp,code,msg,hdrs)

Retry the request with authentication information, if available.

AbstractDigestAuthHandler Objects

AbstractDigestAuthHandler.http_error_auth_reged(authreq,host,req,headers)

`authreq` should be the name of the header where the information about the realm is included in the request, `host` should be the host to authenticate to, `req` should be the (failed) `Request` object, and `headers` should be the error headers.

HTTPDigestAuthHandler Objects

HTTPDigestAuthHandler.http_error_401(req,fp,code,msg,hdrs)

Retry the request with authentication information, if available.

ProxyDigestAuthHandler Objects

ProxyDigestAuthHandler.http_error_407(req,fp,code,msg,hdrs)

Retry the request with authentication information, if available.

HTTPHandler Objects

HTTPHandler.http_open(req)

Send an HTTP request, which can be either GET or POST, depending on `req.data`.

HTTPSHandler Objects

HTTPSHandler.https_open(req)

Send an HTTPS request, which can be either GET or POST, depending on `req.data`.

FileHandler Objects

FileHandler.file_open(req)

Open the file locally, if there is no host name, or the host name is 'localhost'. Changed in version 3.2: This method is applicable only for local hostnames. When a remote hostname is given, a `URLError` is raised.

DataHandler Objects

DataHandler.data_open(req)

Read a data URL. This kind of URL contains the content encoded in the URL itself. The data URL syntax is specified in RFC 2397. This implementation ignores white spaces in base64 encoded data URLs so the URL may be wrapped in whatever source file it comes from. But even though some browsers don't mind about a missing padding at the end of a base64 encoded data URL, this implementation will raise a `ValueError` in that case.

FTPHandler Objects

FTPHandler.ftp_open(req)

Open the FTP file indicated by req. The login is always done with empty username and password.

CacheFTPHandler Objects

CacheFTPHandler objects are FTPHandler objects with the following additional methods:

CacheFTPHandler.setTimeout(t)

Set timeout of connections to t seconds.

CacheFTPHandler.setMaxConns(m)

Set maximum number of cached connections to m.

UnknownHandler Objects

UnknownHandler.unknown_open()

Raise a `URLError` exception.

HTTPErrorProcessor Objects

HTTPErrorProcessor.http_response(request,response)

Process HTTP error responses. For 200 error codes, the response object is returned immediately. For non-200 error codes, this simply passes the job on to the `http_error_<type>()` handler methods, via `OpenerDirector.error()`. Eventually, `HTTPDefaultErrorHandler` will raise an `HTTPError` if no other handler handles the error.

HTTPErrorProcessor.https_response(request,response)

Process HTTPS error responses. The behavior is same as `http_response()`.

Examples

In addition to the examples below, more examples are given in [HOWTO Fetch Internet Resources Using The urllib Package](#).

This example gets the python.org main page and displays the first 300 bytes of it:

```
>>> import urllib.request
>>> with urllib.request.urlopen('http://www.python.org/') as f:
...     print(f.read(300))
...
b'<!doctype html>\n<!--[if lt IE 7]>    <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">    <![endif]-->\n<!--[if IE
```

Note that `urlopen` returns a bytes object. This is because there is no way for `urlopen` to automatically determine the encoding of the byte stream it receives from the HTTP server. In general, a program will decode the returned bytes object to string once it determines or guesses the appropriate encoding.

The following HTML spec document, <https://html.spec.whatwg.org/#charset>, lists the various ways in which an HTML or an XML document could have specified its encoding information.

For additional information, see the W3C document: <https://www.w3.org/International/questions/qa-html-encoding-declarations>.

As the python.org website uses `utf-8` encoding as specified in its meta tag, we will use the same for decoding the bytes object:

```
>>> with urllib.request.urlopen('http://www.python.org/') as f:
...     print(f.read(100).decode('utf-8'))
...
<!doctype html>
<!--[if lt IE 7]>    <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">    <![endif]-->
<!--
```

It is also possible to achieve the same result without using the `context manager` approach:

```
>>> import urllib.request
>>> f = urllib.request.urlopen('http://www.python.org/')
>>> try:
...     print(f.read(100).decode('utf-8'))
... finally:
...     f.close()
...
<!doctype html>
<!--[if lt IE 7]>    <html class="no-js ie6 lt-ie7 lt-ie8 lt-ie9">    <![endif]-->
<!--
```

In the following example, we are sending a data-stream to the `stdin` of a CGI and reading the data it returns to us. Note that this example will only work when the Python installation supports SSL.

```
>>> import urllib.request
>>> req = urllib.request.Request(url='https://localhost/cgi-bin/test.cgi',
...                             data=b'This data is passed to stdin of the CGI')
>>> with urllib.request.urlopen(req) as f:
...     print(f.read().decode('utf-8'))
...
Got Data: "This data is passed to stdin of the CGI"
```

The code for the sample CGI used in the above example is:

```
#!/usr/bin/env python
import sys
data = sys.stdin.read()
print('Content-type: text/plain\n\nGot Data: "%s"' % data)
```

Here is an example of doing a `PUT` request using `Request`:

```
import urllib.request
DATA = b'some data'
req = urllib.request.Request(url='http://localhost:8080', data=DATA, method='PUT')
with urllib.request.urlopen(req) as f:
    pass
print(f.status)
print(f.reason)
```

Use of Basic HTTP Authentication:

```
import urllib.request
# Create an OpenerDirector with support for Basic HTTP Authentication...
auth_handler = urllib.request.HTTPBasicAuthHandler()
auth_handler.add_password(realm='PDQ Application',
                        uri='https://mahler:8092/site-updates.py',
                        user='klem',
                        passwd='kadidd!ehopper')
opener = urllib.request.build_opener(auth_handler)
# ...and install it globally so it can be used with urlopen.
urllib.request.install_opener(opener)
with urllib.request.urlopen('http://www.example.com/login.html') as f:
    print(f.read().decode('utf-8'))
```

`build_opener()` provides many handlers by default, including a `ProxyHandler`. By default, `ProxyHandler` uses the environment variables named `<scheme>_proxy`, where `<scheme>` is the URL scheme involved. For example, the `http_proxy` environment variable is read to obtain the HTTP proxy's URL.

This example replaces the default `ProxyHandler` with one that uses programmatically supplied proxy URLs, and adds proxy authorization support with `ProxyBasicAuthHandler`.

```
proxy_handler = urllib.request.ProxyHandler({'http': 'http://www.example.com:3128/'})
proxy_auth_handler = urllib.request.ProxyBasicAuthHandler()
proxy_auth_handler.add_password('realm', 'host', 'username', 'password')

opener = urllib.request.build_opener(proxy_handler, proxy_auth_handler)
# This time, rather than install the OpenerDirector, we use it directly:
with opener.open('http://www.example.com/login.html') as f:
    print(f.read().decode('utf-8'))
```

Adding HTTP headers:

Use the `headers` argument to the `Request` constructor, or:

```
import urllib.request
req = urllib.request.Request('http://www.example.com/')
req.add_header('Referer', 'http://www.python.org/')
# Customize the default User-Agent header value:
req.add_header('User-Agent', 'urllib-example/0.1 (Contact: . . .)')
with urllib.request.urlopen(req) as f:
    print(f.read().decode('utf-8'))
```

`OpenerDirector` automatically adds a `User-Agent` header to every `Request`. To change this:

```
import urllib.request
opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
with opener.open('http://www.example.com/') as f:
    print(f.read().decode('utf-8'))
```

Also, remember that a few standard headers (`Content-Length`, `Content-Type` and `Host`) are added when the `Request` is passed to `urlopen()` (or `OpenerDirector.open()`).

Here is an example session that uses the `GET` method to retrieve a URL containing parameters:

```
>>> import urllib.request
>>> import urllib.parse
```

```
>>> params = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacon': 0})
>>> url = "http://www.musi-cal.com/cgi-bin/query?%s" % params
>>> with urllib.request.urlopen(url) as f:
...     print(f.read().decode('utf-8'))
...
```

The following example uses the `POST` method instead. Note that params output from `urlencode` is encoded to bytes before it is sent to `urlopen` as data:

```
>>> import urllib.request
>>> import urllib.parse
>>> data = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacon': 0})
>>> data = data.encode('ascii')
>>> with urllib.request.urlopen("http://requestb.in/xrbl82xr", data) as f:
...     print(f.read().decode('utf-8'))
...
```

The following example uses an explicitly specified HTTP proxy, overriding environment settings:

```
>>> import urllib.request
>>> proxies = {'http': 'http://proxy.example.com:8080/'}
>>> opener = urllib.request.build_opener(urllib.request.ProxyHandler(proxies))
>>> with opener.open("http://www.python.org") as f:
...     f.read().decode('utf-8')
...
```

The following example uses no proxies at all, overriding environment settings:

```
>>> import urllib.request
>>> opener = urllib.request.build_opener(urllib.request.ProxyHandler({}))
>>> with opener.open("http://www.python.org") as f:
...     f.read().decode('utf-8')
...
```

Legacy interface

The following functions and classes are ported from the Python 2 module `urllib` (as opposed to `urllib2`). They might become deprecated at some point in the future.

`urllib.request.urlretrieve(url,filename=None,reporthook=None,data=None)`

Copy a network object denoted by a URL to a local file. If the URL points to a local file, the object will not be copied unless `filename` is supplied. Return a tuple `(filename,headers)` where `filename` is the local file name under which the object can be found, and `headers` is whatever the `info()` method of the object returned by `urlopen()` returned (for a remote object). Exceptions are the same as for `urlopen()`. The second argument, if present, specifies the file location to copy to (if absent, the location will

`urllib.request.urlcleanup()`

Cleans up temporary files that may have been left behind by previous calls to `urlretrieve()`.

`urllib.request` Restrictions

- Currently, only the following protocols are supported: HTTP (versions 0.9 and 1.0), FTP, local files, and data URLs. Changed in version 3.4: Added support for data URLs.
- The caching feature of `urlretrieve()` has been disabled until someone finds the time to hack proper processing of Expiration time headers.
- There should be a function to query whether a particular URL is in the cache.

- For backward compatibility, if a URL appears to point to a local file but the file can't be opened, the URL is re-interpreted using the FTP protocol. This can sometimes cause confusing error messages.
- The `urlopen()` and `urlretrieve()` functions can cause arbitrarily long delays while waiting for a network connection to be set up. This means that it is difficult to build an interactive web client using these functions without using threads.
- The data returned by `urlopen()` or `urlretrieve()` is the raw data returned by the server. This may be binary data (such as an image), plain text or (for example) HTML. The HTTP protocol provides type information in the reply header, which can be inspected by looking at the `Content-Type` header. If the returned data is HTML, you can use the module `html.parser` to parse it.
- The code handling the FTP protocol cannot differentiate between a file and a directory. This can lead to unexpected behavior when attempting to read a URL that points to a file that is not accessible. If the URL ends in `a/`, it is assumed to refer to a directory and will be handled accordingly. But if an attempt to read a file leads to a 550 error (meaning the URL cannot be found or is not acces

urllib.response— Response classes used by urllib

The `urllib.response` module defines functions and classes which define a minimal file-like interface, including `read()` and `readline()`. Functions defined by this module are used internally by the `urllib.request` module. The typical response object is `urllib.response.addinfourl` instance:

class `urllib.response.addinfourl`

`url`

`headers`

`status`

`geturl()`

`info()`

`code`

`getcode()`

`url` URL of the resource retrieved, commonly used to determine if a redirect was followed.
`headers` Returns the headers of the response in the form of an `EmailMessage` instance.
`status` Added in version 3.9. Status code returned by server.
`geturl()` Deprecated since version 3.9: Deprecated in favor of `url.info()`
`info()` Deprecated since version 3.9: Deprecated in favor of `headers.code`
`code` Deprecated since version 3.9: Deprecated in favor of `status`.
`getcode()` Deprecated since version 3.9: Deprecated in favor of `status`.

URL of the resource retrieved, commonly used to determine if a redirect was followed.

Returns the headers of the response in the form of an `EmailMessage` instance.

Added in version 3.9. Status code returned by server.

Deprecated since version 3.9: Deprecated in favor of `url`.

Deprecated since version 3.9: Deprecated in favor of `headers`.

Deprecated since version 3.9: Deprecated in favor of `status`.

Deprecated since version 3.9: Deprecated in favor of `status`.

urllib.parse— Parse URLs into components

Source code: `Lib/urllib/parse.py`

This module defines a standard interface to break Uniform Resource Locator (URL) strings up in components (addressing scheme, network location, path etc.), to combine the components back into a URL string, and to convert a “relative URL” to an absolute URL given a “base URL.”

The module has been designed to match the internet RFC on Relative Uniform Resource Locators. It supports the following URL schemes: `file,ftp,gopher,hdl,http,https,imap,itms-services,mailto,mms,news,nntp,prospero,rsync,rtsp,rtsp,rtspu,sftp,shhttp,sip,sips,snews,svn,svn+ssh,telnet,wais,ws,wss`.

CPython implementation detail: The inclusion of the `itms-services` URL scheme can prevent an app from passing Apple’s App Store review process for the macOS and iOS App Stores. Handling for the `itms-services` scheme is always removed on iOS; on macOS, it may be removed if CPython has been built with the `--with-app-store-compliance` option.

The `urllib.parse` module defines functions that fall into two broad categories: URL parsing and URL quoting. These are covered in detail in the following sections.

This module’s functions use the deprecated term `netloc` (or `net_loc`), which was introduced in RFC 1808. However, this term has been obsoleted by RFC 3986, which introduced the term `authority` as its replacement. The use of `netloc` is continued for backward compatibility.

URL Parsing

The URL parsing functions focus on splitting a URL string into its components, or on combining URL components into a URL string.

`urllib.parse.urlsplit(urlstring,scheme=None,allow_fragments=True)`

Parse a URL into five components, returning a 5-item named tuple `SplitResult` or `SplitResultBytes`. This corresponds to the general structure of a URL: `scheme://netloc/path?query#fragment`. Each tuple item is a string, possibly empty. The components are not broken up into smaller parts (for example, the network location is a single string), and `%` escapes are not expanded. The delimiters as shown above are not part of the result, except for a leading slash in the `path` component, which is retained if present.

`urllib.parse.parse_qs(qs,keep_blank_values=False,strict_parsing=False,encoding='utf-8',errors='replace',max_num_fields=None,separator='&')`

Parse a query string given as a string argument (data of type `application/x-www-form-urlencoded`). Data are returned as a dictionary. The dictionary keys are the unique query variable names and the values are lists of values for each name. The optional argument `keep_blank_values` is a flag indicating whether blank values in percent-encoded queries should be treated as blank strings. A true value indicates that blanks should be retained as blank strings. The default false value indicates that blank

`urllib.parse.parse_qsl(qs,keep_blank_values=False,strict_parsing=False,encoding='utf-8',errors='replace',max_num_fields=None,separator='&')`

Parse a query string given as a string argument (data of type `application/x-www-form-urlencoded`). Data are returned as a list of name, value pairs. The optional argument `keep_blank_values` is a flag indicating whether blank values in percent-encoded queries should be treated as blank strings. A true value indicates that blanks should be retained as blank strings. The default false value indicates that blank values are to be ignored and treated as if they were not included. The optional argument `stri`

`urllib.parse.urlunsplit(parts)`

Construct a URL from a tuple as returned by `urlsplit()`. The `parts` argument can be any five-item iterable. This may result in a slightly different, but equivalent URL, if the URL that was parsed originally had unnecessary delimiters (for example, a `?` with an empty query; the RFC states that these are equivalent).

`urllib.parse.urlparse(urlstring,scheme=None,allow_fragments=True)`

This is similar to `urlsplit()`, but additionally splits the `path` component on `path` and `params`. This function returns a 6-item named tuple `ParseResult` or `ParseResultBytes`. Its items are the same as for the `urlsplit()` result, except that `params` is inserted at index 3, between `path` and `query`. This function is based on obsoleted RFC 1738 and RFC 1808, which listed `params` as the main URL component. The more recent URL syntax allows parameters to be applied to each segment of the `path` portion of the URL (see RFC 3986). `urlsplit()`

`urllib.parse.urlunparse(parts)`

Combine the elements of a tuple as returned by `urlparse()` into a complete URL as a string. The `parts` argument can be any six-item iterable. This may result in a slightly different, but equivalent URL, if the URL that was parsed originally had unnecessary delimiters (for example, a `?` with an empty query; the RFC states that these are equivalent).

`urllib.parse.urljoin(base,url,allow_fragments=True)`

Construct a full (“absolute”) URL by combining a “base URL” (`base`) with another URL (`url`). Informally, this uses components of the base URL, in particular the addressing scheme, the network location and (part of) the path, to provide missing components in the relative URL. For example:

```
>>>from urllib.parse import urljoin>>>urljoin('http://www.cwi.nl/%7Eguido/Python.html','FAQ.html')'http://www.cwi.nl/%7Eguido/FAQ.html'
```

 The `allow_fragments` argument has the same meaning and default as for `urlsplit()`. Not

`urllib.parse.urldefrag(url)`

If `url` contains a fragment identifier, return a modified version of `url` with no fragment identifier, and the fragment identifier as a separate string. If there is no fragment identifier in `url`, return `url` unmodified and an empty string. The return value is a named tuple, its items can be accessed by index or as named attributes: `AttributeIndexValueValue` if not present `url` `URL` with no fragment `empty string` `fragment` `1` `Fragment` `identifier` `empty string` See section `Structured Parse Results` for more information on the

`urllib.parse.unwrap(url)`

Extract the `url` from a wrapped URL (that is, a string formatted as `<URL:scheme://host/path>`, `<scheme://host/path>`, `URL:scheme://host/path` or `scheme://host/path`). If `url` is not a wrapped URL, it is returned without changes.

URL parsing security

The `urlsplit()` and `urlparse()` APIs do not perform validation of inputs. They may not raise errors on inputs that other applications consider invalid. They may also succeed on some inputs that might not be considered URLs elsewhere. Their purpose is for practical functionality rather than purity.

Instead of raising an exception on unusual input, they may instead return some component parts as empty strings. Or components may contain more than perhaps they should.

We recommend that users of these APIs where the values may be used anywhere with security implications code defensively. Do some verification within your code before trusting a returned component part. Does that scheme make sense? Is that a sensible path? Is there anything strange about that host name? etc.

What constitutes a URL is not universally well defined. Different applications have different needs and desired constraints. For instance the living WHATWG spec describes what user facing web clients such as a web browser require. While RFC 3986 is more general. These functions incorporate some aspects of both, but cannot be claimed compliant with either. The APIs and existing user code with expectations on specific behaviors

predate both standards leading us to be very cautious about making API behavior changes.

Parsing ASCII Encoded Bytes

The URL parsing functions were originally designed to operate on character strings only. In practice, it is useful to be able to manipulate properly quoted and encoded URLs as sequences of ASCII bytes. Accordingly, the URL parsing functions in this module all operate on `bytes` and `bytearray` objects in addition to `str` objects.

If `strdata` is passed in, the result will also contain `onlystrdata`. If `bytes` or `bytearray` data is passed in, the result will contain `onlybytesdata`.

Attempting to mix `strdata` with `bytes` or `bytearray` in a single function call will result in a `TypeError` being raised, while attempting to pass in non-ASCII byte values will trigger `UnicodeDecodeError`.

To support easier conversion of result objects between `str` and `bytes`, all return values from URL parsing functions provide either an `encode()` method (when the result contains `strdata`) or a `decode()` method (when the result contains `bytesdata`). The signatures of these methods match those of the corresponding `str` and `bytes` methods (except that the default encoding is `'ascii'` rather than `'utf-8'`). Each produces a value of a corresponding type that contains either `bytesdata` (for `encode()` methods) or `strdata` (for `decode()` methods).

Applications that need to operate on potentially improperly quoted URLs that may contain non-ASCII data will need to do their own decoding from bytes to characters before invoking the URL parsing methods.

The behaviour described in this section applies only to the URL parsing functions. The URL quoting functions use their own rules when producing or consuming byte sequences as detailed in the documentation of the individual URL quoting functions.

Changed in version 3.2: URL parsing functions now accept ASCII encoded byte sequences

Structured Parse Results

The result objects from `urlsplit()`, `urlparse()` and `urldefrag()` functions are subclasses of the `tuple` type. These subclasses add the attributes listed in the documentation for those functions, the encoding and decoding support described in the previous section, as well as an additional method:

`urllib.parse.SplitResult.geturl()`

Return the re-combined version of the original URL as a string. This may differ from the original URL in that the scheme may be normalized to lower case and empty components may be dropped. Specifically, empty parameters, queries, and fragment identifiers will be removed. For `urldefrag()` results, only empty fragment identifiers will be removed. For `urlsplit()` and `urlparse()` results, all noted changes will be made to the URL returned by this method. The result of this method remains unchanged if passed `b`

The following classes provide the implementations of the structured parse results when operating on `str` objects:

`class urllib.parse.DefragResult(url, fragment)`

Concrete class for `urldefrag()` results containing `strdata`. The `encode()` method returns a `DefragResultBytes` instance. Added in version 3.2.

`class urllib.parse.ParseResult(scheme, netloc, path, params, query, fragment)`

Concrete class for `urlparse()` results containing `strdata`. The `encode()` method returns a `ParseResultBytes` instance.

`class urllib.parse.SplitResult(scheme, netloc, path, query, fragment)`

Concrete class for `urlsplit()` results containing `strdata`. The `encode()` method returns a `SplitResultBytes` instance.

The following classes provide the implementations of the parse results when operating on bytes or bytearray objects:

class urllib.parse.DefragResultBytes(url, fragment)

Concrete class for `urldefrag()` results containing bytes data. The `decode()` method returns a `DefragResult` instance. Added in version 3.2.

class urllib.parse.ParseResultBytes(scheme, netloc, path, params, query, fragment)

Concrete class for `urlparse()` results containing bytes data. The `decode()` method returns a `ParseResult` instance. Added in version 3.2.

class urllib.parse.SplitResultBytes(scheme, netloc, path, query, fragment)

Concrete class for `urlsplit()` results containing bytes data. The `decode()` method returns a `SplitResult` instance. Added in version 3.2.

URL Quoting

The URL quoting functions focus on taking program data and making it safe for use as URL components by quoting special characters and appropriately encoding non-ASCII text. They also support reversing these operations to recreate the original data from the contents of a URL component if that task isn't already covered by the URL parsing functions above.

urllib.parse.quote(string, safe='/', encoding=None, errors=None)

Replace special characters in string using the `%xxescape`. Letters, digits, and the characters `'_.~'` are never quoted. By default, this function is intended for quoting the path section of a URL. The optional `safe` parameter specifies additional ASCII characters that should not be quoted — its default value is `'/'`. `string` may be either a `str` or a `bytes` object. Changed in version 3.7: Moved from RFC 2396 to RFC 3986 for quoting URL strings. “~” is now included in the set of unreserved characters. The optional `encoding` and

urllib.parse.quote_plus(string, safe='', encoding=None, errors=None)

Like `quote()`, but also replace spaces with plus signs, as required for quoting HTML form values when building up a query string to go into a URL. Plus signs in the original string are escaped unless they are included in `safe`. It also does not have a `safe` default to `'/'`. Example: `quote_plus('/El Niño/')` yields `'%2FEI+Ni%C3%B1o%2F'`.

urllib.parse.quote_from_bytes(bytes, safe='/')

Like `quote()`, but accepts a `bytes` object rather than a `str`, and does not perform string-to-bytes encoding. Example: `quote_from_bytes(b'a&\xef')` yields `'a%26%EF'`.

urllib.parse.unquote(string, encoding='utf-8', errors='replace')

Replace `%xxescape`s with their single-character equivalent. The optional `encoding` and `errors` parameters specify how to decode percent-encoded sequences into Unicode characters, as accepted by the `bytes.decode()` method. `string` may be either a `str` or a `bytes` object. `encoding` defaults to `'utf-8'`. `errors` defaults to `'replace'`, meaning invalid sequences are replaced by a placeholder character. Example: `unquote('/El%20Ni%C3%B1o/')` yields `'El Niño/'`. Changed in version 3.9: `string` parameter supports bytes and str objects (previo

urllib.parse.unquote_plus(string, encoding='utf-8', errors='replace')

Like `unquote()`, but also replace plus signs with spaces, as required for unquoting HTML form values. `string` must be a `str`. Example: `unquote_plus('/El+Ni%C3%B1o/')` yields `'El Niño/'`.

urllib.parse.unquote_to_bytes(string)

Replace `%xx` escapes with their single-octet equivalent, and return a `bytesobject`. `string` may be either a `str` or a `bytesobject`. If it is a `str`, unescaped non-ASCII characters in `string` are encoded into UTF-8 bytes. Example: `unquote_to_bytes('a%26%EF')` yields `b'a&\xef'`.

`urllib.parse.urlencode(query, doseq=False, safe='', encoding=None, errors=None, quote_via=quote_plus)`

Convert a mapping object or a sequence of two-element tuples, which may contain `str` or `bytes` objects, to a percent-encoded ASCII text string. If the resultant string is to be used as a data for POST operation with the `urlopen()` function, then it should be encoded to bytes, otherwise it would result in a `TypeError`. The resulting string is a series of `key=value` pairs separated by `'&'` characters, where both `key` and `value` are quoted using the `quote_via` function. By default, `quote_plus()` is used to quote the values, which

See also

WHATWG- URL Living standard

RFC 3986- Uniform Resource Identifiers

RFC 2732- Format for Literal IPv6 Addresses in URL's.

RFC 2396- Uniform Resource Identifiers (URI): Generic Syntax

RFC 2368- The mailto URL scheme.

RFC 1808- Relative Uniform Resource Locators

RFC 1738- Uniform Resource Locators (URL)

Working Group for the URL Standard that defines URLs, domains, IP addresses, the `application/x-www-form-urlencoded` format, and their API.

This is the current standard (STD66). Any changes to `urllib.parse` module should conform to this. Certain deviations could be observed, which are mostly for backward compatibility purposes and for certain de-facto parsing requirements as commonly observed in major browsers.

This specifies the parsing requirements of IPv6 URLs.

Document describing the generic syntactic requirements for both Uniform Resource Names (URNs) and Uniform Resource Locators (URLs).

Parsing requirements for mailto URL schemes.

This Request For Comments includes the rules for joining an absolute and a relative URL, including a fair number of "Abnormal Examples" which govern the treatment of border cases.

This specifies the formal syntax and semantics of absolute URLs.

urllib.error— Exception classes raised by urllib.request

Source code: `Lib/urllib/error.py`

The `urllib.error` module defines the exception classes for exceptions raised by `urllib.request`. The base exception class is `URLError`.

The following exceptions are raised by `urllib.error` as appropriate:

`exception urllib.error.URLError`

reason

The handlers raise this exception (or derived exceptions) when they run into a problem. It is a subclass of `OSError`. **reason** The reason for this error. It can be a message string or another exception instance. Changed in version 3.3: `URLError` used to be a subtype of `IOError`, which is now an alias of `OSError`.

The reason for this error. It can be a message string or another exception instance.

`exception urllib.error.HTTPError(url,code,msg,hdrs,fp)`

url

code

reason

headers

fp

Though being an exception (a subclass of `URLError`), an `HTTPError` can also function as a non-exceptional file-like return value (the same thing that `urlopen()` returns). This is useful when handling exotic HTTP errors, such as requests for authentication. **url** Contains the request URL. An alias for `filename` attribute. **code** An HTTP status code as defined in RFC 2616. This numeric value corresponds to a value found in the dictionary of codes as found in `http.server.BaseHTTPRequestHandler.responses`. **reason** This is

Contains the request URL. An alias for `filename` attribute.

An HTTP status code as defined in RFC 2616. This numeric value corresponds to a value found in the dictionary of codes as found in `http.server.BaseHTTPRequestHandler.responses`.

This is usually a string explaining the reason for this error. An alias for `msg` attribute.

The HTTP response headers for the HTTP request that caused the `HTTPError`. An alias for `hdrs` attribute. Added in version 3.4.

A file-like object where the HTTP error body can be read from.

`exception urllib.error.ContentTooShortError(msg,content)`

content

This exception is raised when the `urlretrieve()` function detects that the amount of the downloaded data is less than the expected amount (given by the `Content-Length` header). **content** The downloaded (and supposedly truncated) data.

The downloaded (and supposedly truncated) data.

urllib.robotparser— Parser for robots.txt

Source code: `Lib/urllib/robotparser.py`

This module provides a single class, `RobotFileParser`, which answers questions about whether or not a particular user agent can fetch a URL on the website that published the robots.txt file. For more details on the structure of robots.txt files, see <http://www.robotstxt.org/orig.html>.

`class urllib.robotparser.RobotFileParser(url="")`

`set_url(url)`

`read()`

`parse(lines)`

`can_fetch(useragent,url)`

`mtime()`

`modified()`

`crawl_delay(useragent)`

`request_rate(useragent)`

`site_maps()`

This class provides methods to read, parse and answer questions about the robots.txt file at `url`. `set_url(url)` Sets the URL referring to the robots.txt file. `read()` Reads the robots.txt URL and feeds it to the parser. `parse(lines)` Parses the lines argument. `can_fetch(useragent,url)` Returns `True` if the user agent is allowed to fetch the URL according to the rules contained in the parsed robots.txt file. `mtime()` Returns the time the robots.txt file was last fetched. This is useful for long-running web spiders that need to check

Sets the URL referring to the robots.txt file.

Reads the robots.txt URL and feeds it to the parser.

Parses the lines argument.

Returns `True` if the user agent is allowed to fetch the URL according to the rules contained in the parsed robots.txt file.

Returns the time the robots.txt file was last fetched. This is useful for long-running web spiders that need to check for new robots.txt files periodically.

Sets the time the robots.txt file was last fetched to the current time.

Returns the value of the Crawl-delay parameter from the robots.txt for the user agent in question. If there is no such parameter or it doesn't apply to the user agent specified or the robots.txt entry for this parameter has invalid syntax, return `None`. Added in version 3.6.

Returns the contents of the Request-rate parameter from the robots.txt as a named tuple `RequestRate(requests,seconds)`. If there is no such parameter or it doesn't apply to the user agent specified or the robots.txt entry for this parameter has invalid syntax, return `None`. Added in version 3.6.

Returns the contents of the Sitemap parameter from the robots.txt in the form of a list(). If there is no such parameter or the robots.txt entry for this parameter has invalid syntax, return `None`. Added in version 3.8.

The following example demonstrates basic use of the `RobotFileParser` class:

```
>>> import urllib.robotparser
>>> rp = urllib.robotparser.RobotFileParser()
```

```
>>> rp.set_url("http://www.pythontest.net/robots.txt")
>>> rp.read()
>>> rrate = rp.request_rate("")
>>> rrate.requests
1
>>> rrate.seconds
1
>>> rp.crawl_delay("")
6
>>> rp.can_fetch("", "http://www.pythontest.net/")
True
>>> rp.can_fetch("", "http://www.pythontest.net/no-robots-here/")
False
```

http— HTTP modules

Source code: `Lib/http/__init__.py`

`http` is a package that collects several modules for working with the HyperText Transfer Protocol:

- `http.client` is a low-level HTTP protocol client; for high-level URL opening use `urllib.request`
- `http.server` contains basic HTTP server classes based on `socketserver`
- `http.cookies` has utilities for implementing state management with cookies
- `http.cookiejar` provides persistence of cookies

The `http` module also defines the following enums that help you work with `http` related code:

`class http.HTTPStatus`

Added in version 3.5. A subclass of `enum.IntEnum` that defines a set of HTTP status codes, reason phrases and long descriptions written in English. Usage:

```
>>> from http import HTTPStatus
>>> HTTPStatus.OK
HTTPStatus.OK
>>> HTTPStatus.OK == 200
True
>>> HTTPStatus.OK.value
200
>>> HTTPStatus.OK.phrase
'OK'
>>> HTTPStatus.OK.description
'Request fulfilled, document follows'
>>> list(HTTPStatus)
[HTTPStatus.CONTINUE, HTTPStatus.SWITCHING_PROTOCOLS, ...]
```

HTTP status codes

Supported, IANA-registered status codes available in `http.HTTPStatus` are:

Code	Enum Name	Details
100	CONTINUE	HTTP Semantics RFC 9110 , Section 15.2.1
101	SWITCHING_PROTOCOLS	HTTP Semantics RFC 9110 , Section 15.2.2
102	PROCESSING	WebDAV RFC 2518 , Section 10.1
103	EARLY_HINTS	An HTTP Status Code for Indicating Hints RFC 8297
200	OK	HTTP Semantics RFC 9110 , Section 15.3.1
201	CREATED	HTTP Semantics RFC 9110 , Section 15.3.2
202	ACCEPTED	HTTP Semantics RFC 9110 , Section 15.3.3
203	NON_AUTHORITATIVE_INFORMATION	HTTP Semantics RFC 9110 , Section 15.3.4
204	NO_CONTENT	HTTP Semantics RFC 9110 , Section 15.3.5
205	RESET_CONTENT	HTTP Semantics RFC 9110 , Section 15.3.6
206	PARTIAL_CONTENT	HTTP Semantics RFC 9110 , Section 15.3.7
207	MULTI_STATUS	WebDAV

In order to preserve backwards compatibility, enum values are also present in the `http.client` module in the form of constants. The enum name is equal to the constant name (i.e. `http.HTTPStatus.OK` is also available as `http.client.OK`).

Changed in version 3.7: Added `421 MISDIRECTED_REQUEST` status code.

Added in version 3.8: Added `451 UNAVAILABLE_FOR_LEGAL_REASONS` status code.

Added in version 3.9: Added `103 EARLY_HINTS`, `418 IM_A_TEAPOT` and `425 TOO_EARLY` status codes.

Changed in version 3.13: Implemented RFC 9110 naming for status constants. Old constant names are preserved for backwards compatibility: `413 REQUEST_ENTITY_TOO_LARGE`, `414 REQUEST_URI_TOO_LONG`, `416 REQUESTED_RANGE_NOT_SATISFIABLE` and `422 UNPROCESSABLE_ENTITY`.

HTTP status category

Added in version 3.12.

The enum values have several properties to indicate the HTTP status category:

Property	Indicates that	Details
<code>is_informational</code>	<code>100 <= status <= 199</code>	HTTP Semantics RFC 9110 , Section 15
<code>is_success</code>	<code>200 <= status <= 299</code>	HTTP Semantics

```
| RFC 9110 | , Section 15 | is_redirection | 300 | <= | status | <= | 399 | HTTP Semantics | RFC 9110 | , Section 15 | is_client_error | 400 | <= | status | <= | 499 | HTTP Semantics | RFC 9110 | , Section 15 | is_server_error | 500 | <= | status | <= | 599 | HTTP Semantics | RFC 9110 | , Section 15
```

Usage:

```
>>> from http import HTTPStatus
>>> HTTPStatus.OK.is_success
True
>>> HTTPStatus.OK.is_client_error
False
```

classhttp.HTTPMethod

Added in version 3.11. A subclass of `enum.StrEnum` that defines a set of HTTP methods and descriptions written in English. Usage:

```
>>> from http import HTTPMethod
>>>>> HTTPMethod.GET
<HTTPMethod.GET>
>>>>> HTTPMethod.GET == 'GET'
True
>>>>> HTTPMethod.GET.value
'GET'
>>>>> HTTPMethod.GET.description
'Retrieve the target.'
>>> list(HTTPMethod)
[<HTTPMethod.CONNECT>, <HTTPMethod.DELETE>, <HTTPMethod.GET>, <HTTPMethod.HEAD>, <HTTPMethod.OPTIONS>, <HTTPMethod.PATCH>, <HTTPMethod.POST>, <HTTPMethod.PUT>, <HTTPMethod.TRACE>]
```

HTTP methods

Supported, IANA-registered methods available in `http.HTTPMethod` are:

```
Method | Enum Name | Details | GET | GET | HTTP Semantics | RFC 9110 | , Section 9.3.1 | HEAD | HEAD | HTTP Semantics | RFC 9110 | , Section 9.3.2 | POST | POST | HTTP Semantics | RFC 9110 | , Section 9.3.3 | PUT | PUT | HTTP Semantics | RFC 9110 | , Section 9.3.4 | DELETE | DELETE | HTTP Semantics | RFC 9110 | , Section 9.3.5 | CONNECT | CONNECT | HTTP Semantics | RFC 9110 | , Section 9.3.6 | OPTIONS | OPTIONS | HTTP Semantics | RFC 9110 | , Section 9.3.7 | TRACE | TRACE | HTTP Semantics | RFC 9110 | , Section 9.3.8 | PATCH | PATCH | HTTP/1.1 | RFC 5789
```

http.client— HTTP protocol client

Source code: `Lib/http/client.py`

This module defines classes that implement the client side of the HTTP and HTTPS protocols. It is normally not used directly — the module `urllib.request` uses it to handle URLs that use HTTP and HTTPS.

See also

The `Requests` package is recommended for a higher-level HTTP client interface.

■ **Nota:** Note HTTPS support is only available if Python was compiled with SSL support (through the `ssl` module).

Availability: not WASI.

This module does not work or is not available on WebAssembly. See WebAssembly platforms for more information.

The module provides the following classes:

class `http.client.HTTPConnection`(`host`, `port=None`, [`timeout`], `source_address=None`, `blocksize=8192`)

An `HTTPConnection` instance represents one transaction with an HTTP server. It should be instantiated by passing it a host and optional port number. If no port number is passed, the port is extracted from the host string if it has the form `host:port`, else the default HTTP port (80) is used. If the optional `timeout` parameter is given, blocking operations (like connection attempts) will timeout after that many seconds (if it is not given, the global default timeout setting is used). The optional `source`

class `http.client.HTTPSConnection`(`host`, `port=None`, *, [`timeout`], `source_address=None`, `context=None`, `blocksize=8192`)

A subclass of `HTTPConnection` that uses SSL for communication with secure servers. Default port is 443. If `context` is specified, it must be an `ssl.SSLContext` instance describing the various SSL options. Please read [Security considerations](#) for more information on best practices. Changed in version 3.2: `source_address`, `context` and `check_hostname` were added. Changed in version 3.2: This class now supports HTTPS virtual hosts if possible (that is, if `ssl.HAS_SNI` is true). Changed in version 3.4: The `strict` parameter was re

class `http.client.HTTPResponse`(`sock`, `debuglevel=0`, `method=None`, `url=None`)

Class whose instances are returned upon successful connection. Not instantiated directly by user. Changed in version 3.4: The `strict` parameter was removed. HTTP 0.9 style “Simple Responses” are no longer supported.

This module provides the following function:

`http.client.parse_headers(fp)`

Parse the headers from a file pointer `fp` representing a HTTP request/response. The file has to be a `BufferedReader` (i.e. not text) and must provide a valid RFC 5322 style header. This function returns an instance of `http.client.HTTPMessage` that holds the header fields, but no payload (the same as `HTTPResponse.msg` and `http.server.BaseHTTPRequestHandler.headers`). After returning, the file pointer `fp` is ready to read the HTTP body. Note `parse_headers()` does not parse the start-line of a HTTP message; it only

The following exceptions are raised as appropriate:

exception `http.client.HTTPException`

The base class of the other exceptions in this module. It is a subclass of `Exception`.

exception `http.client.NotConnected`

A subclass of `HTTPException`.

exceptionhttp.client.InvalidURL

A subclass ofHTTPException, raised if a port is given and is either non-numeric or empty.

exceptionhttp.client.UnknownProtocol

A subclass ofHTTPException.

exceptionhttp.client.UnknownTransferEncoding

A subclass ofHTTPException.

exceptionhttp.client.UnimplementedFileMode

A subclass ofHTTPException.

exceptionhttp.client.IncompleteRead

A subclass ofHTTPException.

exceptionhttp.client.ImproperConnectionState

A subclass ofHTTPException.

exceptionhttp.client.CannotSendRequest

A subclass ofImproperConnectionState.

exceptionhttp.client.CannotSendHeader

A subclass ofImproperConnectionState.

exceptionhttp.client.ResponseNotReady

A subclass ofImproperConnectionState.

exceptionhttp.client.BadStatusLine

A subclass ofHTTPException. Raised if a server responds with a HTTP status code that we don't understand.

exceptionhttp.client.LineTooLong

A subclass ofHTTPException. Raised if an excessively long line is received in the HTTP protocol from the server.

exceptionhttp.client.RemoteDisconnected

A subclass ofConnectionResetErrorandBadStatusLine. Raised byHTTPConnection.getresponse()when the attempt to read the response results in no data read from the connection, indicating that the remote end has closed the connection.Added in version 3.5:Previously,BadStatusLine("")was raised.

The constants defined in this module are:

http.client.HTTP_PORT

The default port for the HTTP protocol (always80).

http.client.HTTPS_PORT

The default port for the HTTPS protocol (always443).

http.client.responses

This dictionary maps the HTTP 1.1 status codes to the W3C names.Example:http.client.responses[http.client.NOT_FOUND]is'NotFound'.

SeeHTTP status codesfor a list of HTTP status codes that are available in this module as constants.

HTTPConnection Objects

HTTPConnection instances have the following methods:

HTTPConnection.request(method,url,body=None,headers={},*,encode_chunked=False)

This will send a request to the server using the HTTP request method `method` and the request URI `url`. The provided `url` must be an absolute path to conform with RFC 2616 §5.1.2 (unless connecting to an HTTP proxy server or using the `OPTIONS` or `CONNECT` methods). If `body` is specified, the specified data is sent after the headers are finished. It may be a str, a bytes-like object, an open file object, or an iterable of bytes. If `body` is a string, it is encoded as ISO-8859-1, the default for HTTP. If it is a bytes-like

HTTPConnection.getresponse()

Should be called after a request is sent to get the response from the server. Returns an `HTTPResponse` instance. Changed in version 3.5: If a `ConnectionError` or subclass is raised, the `HTTPConnection` object will be ready to reconnect when a new request is sent. Note that this does not apply to `OSError`s raised by the underlying socket. Instead the caller is responsible to call `close()` on the existing connection.

HTTPConnection.set_debuglevel(level)

Set the debugging level. The default debug level is 0, meaning no debugging output is printed. Any value greater than 0 will cause all currently defined debug output to be printed to stdout. The `debuglevel` is passed to any new `HTTPResponse` objects that are created. Added in version 3.1.

HTTPConnection.set_tunnel(host,port=None,headers=None)

Set the host and the port for HTTP Connect Tunnelling. This allows running the connection through a proxy server. The `host` and `port` arguments specify the endpoint of the tunneled connection (i.e. the address included in the `CONNECT` request, not the address of the proxy server). The `headers` argument should be a mapping of extra HTTP headers to send with the `CONNECT` request. As HTTP/1.1 is used for HTTP CONNECT tunnelling request, as per the RFC, a `HTTPHost` header must be provided, matching the authority-form

HTTPConnection.get_proxy_response_headers()

Returns a dictionary with the headers of the response received from the proxy server to the `CONNECT` request. If the `CONNECT` request was not sent, the method returns `None`. Added in version 3.12.

HTTPConnection.connect()

Connect to the server specified when the object was created. By default, this is called automatically when making a request if the client does not already have a connection. Raises an `auditing` event `http.client.connect` with arguments `self, host, port`.

HTTPConnection.close()

Close the connection to the server.

HTTPConnection.blocksize

Buffer size in bytes for sending a file-like message body. Added in version 3.7.

As an alternative to using the `request()` method described above, you can also send your request step by step, by using the four functions below.

HTTPConnection.putrequest(method,url,skip_host=False,skip_accept_encoding=False)

This should be the first call after the connection to the server has been made. It sends a line to the server consisting of the `method` string, the `url` string, and the HTTP version (HTTP/1.1). To disable automatic sending of `Host`: or `Accept-Encoding`: headers (for example to accept additional content encodings), specify `skip_host` or `skip_accept_encoding` with non-False values.

HTTPConnection.putheader(header,argument[,...])

Send anRFC 822-style header to the server. It sends a line to the server consisting of the header, a colon and a space, and the first argument. If more arguments are given, continuation lines are sent, each consisting of a tab and an argument.

HTTPConnection.endheaders(message_body=None,*,encode_chunked=False)

Send a blank line to the server, signalling the end of the headers. The optionalmessage_bodyargument can be used to pass a message body associated with the request.Ifencode_chunkedisTrue, the result of each iteration ofmessage_bodywill be chunk-encoded as specified inRFC 7230, Section 3.3.1. How the data is encoded is dependent on the type ofmessage_body. Ifmessage_bodyimplements thebuffer interface the encoding will result in a single chunk. Ifmessage_bodyis acollections.abc.Iterable, each ite

HTTPConnection.send(data)

Send data to the server. This should be used directly only after theendheaders()method has been called and beforegetresponse()is called.Raises anauditing eventhttp.client.sendwith argumentsself,data.

HTTPResponse Objects

AnHTTPResponseinstance wraps the HTTP response from the server. It provides access to the request headers and the entity body. The response is an iterable object and can be used in a with statement.

Changed in version 3.5:Theio.BufferedIOBaseinterface is now implemented and all of its reader operations are supported.

HTTPResponse.read([amt])

Reads and returns the response body, or up to the nextamtbytes.

HTTPResponse.readinto(b)

Reads up to the next len(b) bytes of the response body into the bufferb. Returns the number of bytes read.Added in version 3.3.

HTTPResponse.getheader(name,default=None)

Return the value of the headername, ordefaultif there is no header matchingname. If there is more than one header with the namename, return all of the values joined by ' '. Ifdefaultis any iterable other than a single string, its elements are similarly returned joined by commas.

HTTPResponse.getheaders()

Return a list of (header, value) tuples.

HTTPResponse.fileno()

Return thefilenoof the underlying socket.

HTTPResponse.msg

Ahttp.client.HTTPMessageinstance containing the response headers.http.client.HTTPMessageis a subclass ofemail.message.Message.

HTTPResponse.version

HTTP protocol version used by server. 10 for HTTP/1.0, 11 for HTTP/1.1.

HTTPResponse.url

URL of the resource retrieved, commonly used to determine if a redirect was followed.

HTTPResponse.headers

Headers of the response in the form of an email.message.EmailMessage instance.

HTTPResponse.status

Status code returned by server.

HTTPResponse.reason

Reason phrase returned by server.

HTTPResponse.debuglevel

A debugging hook. If debuglevel is greater than zero, messages will be printed to stdout as the response is read and parsed.

HTTPResponse.closed

Is True if the stream is closed.

HTTPResponse.geturl()

Deprecated since version 3.9: Deprecated in favor of url.

HTTPResponse.info()

Deprecated since version 3.9: Deprecated in favor of headers.

HTTPResponse.getcode()

Deprecated since version 3.9: Deprecated in favor of status.

Examples

Here is an example session that uses the GET method:

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("www.python.org")
>>> conn.request("GET", "/")
>>> r1 = conn.getresponse()
>>> print(r1.status, r1.reason)
200 OK
>>> data1 = r1.read() # This will return entire content.
>>> # The following example demonstrates reading data in chunks.
>>> conn.request("GET", "/")
>>> r1 = conn.getresponse()
>>> while chunk := r1.read(200):
...     print(repr(chunk))
b'<!doctype html>\n<!--[if"...
...
>>> # Example of an invalid request
>>> conn = http.client.HTTPSConnection("docs.python.org")
>>> conn.request("GET", "/parrot.spam")
>>> r2 = conn.getresponse()
>>> print(r2.status, r2.reason)
404 Not Found
>>> data2 = r2.read()
>>> conn.close()
```

Here is an example session that uses the HEAD method. Note that the HEAD method never returns any data.

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("www.python.org")
>>> conn.request("HEAD", "/")
>>> res = conn.getresponse()
```

```
>>> print(res.status, res.reason)
200 OK
>>> data = res.read()
>>> print(len(data))
0
>>> data == b''
True
```

Here is an example session that uses the `POST` method:

```
>>> import http.client, urllib.parse
>>> params = urllib.parse.urlencode({'@number': 12524, '@type': 'issue', '@action': 'show'})
>>> headers = {"Content-type": "application/x-www-form-urlencoded",
...           "Accept": "text/plain"}
>>> conn = http.client.HTTPConnection("bugs.python.org")
>>> conn.request("POST", "", params, headers)
>>> response = conn.getresponse()
>>> print(response.status, response.reason)
302 Found
>>> data = response.read()
>>> data
b'Redirecting to <a href="https://bugs.python.org/issue12524">https://bugs.python.org/issue12524</a>'
>>> conn.close()
```

Client side `HTTPPUT` requests are very similar to `POST` requests. The difference lies only on the server side where HTTP servers will allow resources to be created via `PUT` requests. It should be noted that custom HTTP methods are also handled in `urllib.request.Request` by setting the appropriate method attribute. Here is an example session that uses the `PUT` method:

```
>>> # This creates an HTTP request
>>> # with the content of BODY as the enclosed representation
>>> # for the resource http://localhost:8080/file
...
>>> import http.client
>>> BODY = """filecontents"""
>>> conn = http.client.HTTPConnection("localhost", 8080)
>>> conn.request("PUT", "/file", BODY)
>>> response = conn.getresponse()
>>> print(response.status, response.reason)
200, OK
```

HTTPMessage Objects

`class http.client.HTTPMessage(email.message.Message)`

An `http.client.HTTPMessage` instance holds the headers from an HTTP response. It is implemented using the `email.message.Message` class.

ftplib— FTP protocol client

Source code: `Lib/ftplib.py`

This module defines the class `FTP` and a few related items. The `FTP` class implements the client side of the FTP protocol. You can use this to write Python programs that perform a variety of automated FTP jobs, such as mirroring other FTP servers. It is also used by the module `urllib.request` to handle URLs that use FTP. For more information on FTP (File Transfer Protocol), see [internetRFC 959](#).

The default encoding is UTF-8, following [RFC 2640](#).

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Here's a sample session using the `ftplib` module:

```
>>> from ftplib import FTP
>>> ftp = FTP('ftp.us.debian.org') # connect to host, default port
>>> ftp.login()                    # user anonymous, passwd anonymous@
'230 Login successful.'
>>> ftp.cwd('debian')              # change into "debian" directory
'250 Directory successfully changed.'
>>> ftp.retrlines('LIST')          # list directory contents
-rw-rw-r--  1 1176      1176      1063 Jun 15 10:18 README
...
drwxr-sr-x   5 1176      1176      4096 Dec 19  2000 pool
drwxr-sr-x   4 1176      1176      4096 Nov 17  2008 project
drwxr-xr-x   3 1176      1176      4096 Oct 10  2012 tools
'226 Directory send OK.'
>>> with open('README', 'wb') as fp:
>>>     ftp.retrbinary('RETR README', fp.write)
'226 Transfer complete.'
>>> ftp.quit()
'221 Goodbye.'
```

Reference

FTP objects

`class ftplib.FTP(host="", user="", passwd="", acct="", timeout=None, source_address=None, *, encoding='utf-8')`

Parameters:

`set_debuglevel(level)`

`connect(host="", port=0, timeout=None, source_address=None)`

Parameters:

`getwelcome()`

`login(user='anonymous', passwd="", acct="")`

Parameters:

`abort()`

`sendcmd(cmd)`

`voidcmd(cmd)`

retrbinary(cmd,callback,blocksize=8192,rest=None)

Parameters:

retrlines(cmd,callback=None)

set_pasv(val)

storbinary(cmd,fp,blocksize=8192,callback=None,rest=None)

Parameters:

storlines(cmd,fp,callback=None)

transfercmd(cmd,rest=None)

ntransfercmd(cmd,rest=None)

mlsd(path="",facts=[])

nlst(argument[,...])

dir(argument[,...])

rename(fromname,toname)

delete(filename)

cwd(pathname)

mkd(pathname)

pwd()

rmd(dirname)

size(filename)

quit()

close()

Return a new instance of theFTPclass. Parameters: host(str) – The hostname to connect to. If given, connect(host) is implicitly called by the constructor. user(str) – The username to log in with (default: 'anonymous'). If given, login(host, passwd, acct) is implicitly called by the constructor. passwd(str) – The password to use when logging in. If not given, and if passwd is the empty string or "-", a password will be automatically generated. acct(str) – Account information to be used for the ACCTFTP command. F

host(str) – The hostname to connect to. If given, connect(host) is implicitly called by the constructor. user(str) – The username to log in with (default: 'anonymous'). If given, login(host, passwd, acct) is implicitly called by the constructor. passwd(str) – The password to use when logging in. If not given, and if passwd is the empty string or "-", a password will be automatically generated. acct(str) – Account information to be used for the ACCTFTP command. Few systems implement this. See RFC-959 for more de

Set the instance's debugging level as an int. This controls the amount of debugging output printed. The debug levels are: 0 (default): No debug output. 1: Produce a moderate amount of debug output, generally a single line per request. 2 or higher: Produce the maximum amount of debugging output, logging each line sent and received on the control connection.

Connect to the given host and port. This function should be called only once for each instance; it should not be called if a host argument was given when the FTP instance was created. All other FTP methods can only be called after a connection has successfully been made. Parameters: host(str) – The host to connect to. port(int) – The TCP port to connect to (default: 21, as specified by the FTP protocol specification). It is rarely needed to specify a different port number. timeout(float|None) – A timeout i

host(str) – The host to connect to.port(int) – The TCP port to connect to (default:21, as specified by the FTP protocol specification). It is rarely needed to specify a different port number.timeout(float|None) – A timeout in seconds for the connection attempt (default: the global default timeout setting).source_address(tuple|None) – A 2-tuple(host,port)for the socket to bind to as its source address before connecting.

Return the welcome message sent by the server in reply to the initial connection. (This message sometimes contains disclaimers or help information that may be relevant to the user.)

Log on to the connected FTP server. This function should be called only once for each instance, after a connection has been established; it should not be called if thehostanduserarguments were given when theFTPinstance was created. Most FTP commands are only allowed after the client has logged in.Parameters:user(str) – The username to log in with (default:'anonymous').passwd(str) – The password to use when logging in. If not given, and ifpasswdis the empty string or"-", a password will be automa

user(str) – The username to log in with (default:'anonymous').passwd(str) – The password to use when logging in. If not given, and ifpasswdis the empty string or"-", a password will be automatically generated.acct(str) – Account information to be used for theACCTFTP command. Few systems implement this. SeeRFC-959for more details.

Abort a file transfer that is in progress. Using this does not always work, but it's worth a try.

Send a simple command string to the server and return the response string.Raises anauditing eventftplib.sendcmdwith argumentssself,cmd.

Send a simple command string to the server and handle the response. Return the response string if the response code corresponds to success (codes in the range 200–299). Raiseerror_replyotherwise.Raises anauditing eventftplib.sendcmdwith argumentssself,cmd.

Retrieve a file in binary transfer mode.Parameters:cmd(str) – An appropriateRETRcommand:"RETRfilename".callback(callable) – A single parameter callable that is called for each block of data received, with its single argument being the data asbytes.blocksize(int) – The maximum chunk size to read on the low-levelsocketobject created to do the actual transfer. This also corresponds to the largest size of data that will be passed tocallback. Defaults to8192.rest(int) – ARESTcommand to be sent to the

cmd(str) – An appropriateRETRcommand:"RETRfilename".callback(callable) – A single parameter callable that is called for each block of data received, with its single argument being the data asbytes.blocksize(int) – The maximum chunk size to read on the low-levelsocketobject created to do the actual transfer. This also corresponds to the largest size of data that will be passed tocallback. Defaults to8192.rest(int) – ARESTcommand to be sent to the server. See the documentation for therestparameter

Retrieve a file or directory listing in the encoding specified by theencodingparameter at initialization.cmdshould be an appropriateRETRcommand (seeretrbinary()) or a command such asLISTorNLST(usually just the string'LIST').LISTretrieves a list of files and information about those files.NLSTretrieves a list of file names. Thecallbackfunction is called for each line with a string argument containing the line with the trailing CRLF stripped. The defaultcallbackprints the line tosys.stdout.

Enable "passive" mode ifvalis true, otherwise disable passive mode. Passive mode is on by default.

Store a file in binary transfer mode.Parameters:cmd(str) – An appropriateSTORcommand:"STORfilename".fp(file object) – A file object (opened in binary mode) which is read until EOF, using itsread()method in blocks of sizeblocksize to provide the data to be stored.blocksize(int) – The read block size. Defaults to8192.callback(callable) – A single parameter callable that is called for each block of data sent, with its single argument being the data asbytes.rest(int) – ARESTcommand to be sent to the

cmd(str) – An appropriateSTORcommand:"STORfilename".fp(file object) – A file object (opened in binary mode) which is read until EOF, using itsread()method in blocks of sizeblocksize to provide the data to be stored.blocksize(int) – The read block size. Defaults to8192.callback(callable) – A single parameter callable that is called for each block of data sent, with its single argument being the data asbytes.rest(int) – ARESTcommand

to be sent to the server. See the documentation for `transfercmd`

Store a file in line mode. `cmd` should be an appropriate `STOR` command (see `storbinary()`). Lines are read until EOF from the file object `fp` (opened in binary mode) using its `readline()` method to provide the data to be stored. `callback` is an optional single parameter callable that is called on each line after it is sent.

Initiate a transfer over the data connection. If the transfer is active, send an `EPRT` or `PORT` command and the transfer command specified by `cmd`, and accept the connection. If the server is passive, send an `EPSV` or `PASV` command, connect to it, and start the transfer command. Either way, return the socket for the connection. If optional `rest` is given, a `REST` command is sent to the server, passing `rest` as an argument. `rest` is usually a byte offset into the requested file, telling the server to restart sending the

Like `transfercmd()`, but returns a tuple of the data connection and the expected size of the data. If the expected size could not be computed, `None` will be returned as the expected size. `cmd` and `rest` means the same thing as in `transfercmd()`.

List a directory in a standardized format by using `MLSD` command (RFC 3659). If `path` is omitted the current directory is assumed. `facts` is a list of strings representing the type of information desired (e.g. `["type", "size", "perm"]`). Return a generator object yielding a tuple of two elements for every file found in `path`. First element is the file name, the second one is a dictionary containing facts about the file name. Content of this dictionary might be limited by the `facts` argument but server is not

Return a list of file names as returned by the `NLST` command. The optional argument is a directory to list (default is the current server directory). Multiple arguments can be used to pass non-standard options to the `NLST` command. Note if your server supports the command, `mlsd()` offers a better API.

Produce a directory listing as returned by the `LIST` command, printing it to standard output. The optional argument is a directory to list (default is the current server directory). Multiple arguments can be used to pass non-standard options to the `LIST` command. If the last argument is a function, it is used as a callback function as for `retrlines()`; the default prints to `sys.stdout`. This method returns `None`. Note if your server supports the command, `mlsd()` offers a better API.

Rename file `fromname` on the server to `to name`.

Remove the file named `filename` from the server. If successful, returns the text of the response, otherwise raises `error_perm` on permission errors or `error_reply` on other errors.

Set the current directory on the server.

Create a new directory on the server.

Return the pathname of the current directory on the server.

Remove the directory named `dirname` on the server.

Request the size of the file named `filename` on the server. On success, the size of the file is returned as an integer, otherwise `None` is returned. Note that the `SIZE` command is not standardized, but is supported by many common server implementations.

Send a `QUIT` command to the server and close the connection. This is the “polite” way to close a connection, but it may raise an exception if the server responds with an error to the `QUIT` command. This implies a call to the `close()` method which renders the `FTP` instance useless for subsequent calls (see below).

Close the connection unilaterally. This should not be applied to an already closed connection such as after a successful call to `quit()`. After this call the `FTP` instance should not be used any more (after a call to `close()` or `quit()` you cannot reopen the connection by issuing another `login()` method).

FTP_TLS objects

classftplib.FTP_TLS(host="",user="",passwd="",acct="*,context=None,timeout=None,source_address=None,encoding='utf-8')

Parameters:

ssl_version

auth()

ccc()

prot_p()

prot_c()

AnFTP subclass which adds TLS support to FTP as described inRFC 4217. Connect to port 21 implicitly securing the FTP control connection before authenticating. NoteThe user must explicitly secure the data connection by calling theprot_p()method. Parameters: host(str) – The hostname to connect to. If given, connect(host)is implicitly called by the constructor. user(str) – The username to log in with (default:'anonymous'). If given, login(host,passwd,acct)is implicitly called by the constructor. passwd(str)

host(str) – The hostname to connect to. If given, connect(host)is implicitly called by the constructor. user(str) – The username to log in with (default:'anonymous'). If given, login(host,passwd,acct)is implicitly called by the constructor. passwd(str) – The password to use when logging in. If not given, and ifpasswdis the empty string or"-", a password will be automatically generated. acct(str) – Account information to be used for theACCTFTP command. Few systems implement this. SeeRFC-959for more de

The SSL version to use (defaults tossl.PROTOCOL_SSLv23).

Set up a secure control connection by using TLS or SSL, depending on what is specified in thessl_versionattribute. Changed in version 3.4: The method now supports hostname check withssl.SSLContext.check_hostnameandServer Name Indication(seessl.HAS_SNI).

Revert control channel back to plaintext. This can be useful to take advantage of firewalls that know how to handle NAT with non-secure FTP without opening fixed ports. Added in version 3.3.

Set up secure data connection.

Set up clear text data connection.

Module variables

exceptionftplib.error_reply

Exception raised when an unexpected reply is received from the server.

exceptionftplib.error_temp

Exception raised when an error code signifying a temporary error (response codes in the range 400–499) is received.

exceptionftplib.error_perm

Exception raised when an error code signifying a permanent error (response codes in the range 500–599) is received.

exceptionftplib.error_proto

Exception raised when a reply is received from the server that does not fit the response specifications of the File Transfer Protocol, i.e. begin with a digit in the range 1–5.

ftplib.all_errors

The set of all exceptions (as a tuple) that methods of `FTP` instances may raise as a result of problems with the FTP connection (as opposed to programming errors made by the caller). This set includes the four exceptions listed above as well as `OSError` and `EOFError`.

See also

Module `netrc`

Parser for the `.netrc` file format. The file `.netrc` is typically used by FTP clients to load user authentication information before prompting the user.

poplib— POP3 protocol client

Source code: `Lib/poplib.py`

This module defines a class, `POP3`, which encapsulates a connection to a POP3 server and implements the protocol as defined in RFC 1939. The `POP3` class supports both the minimal and optional command sets from RFC 1939. The `POP3` class also supports the `STLS` command introduced in RFC 2595 to enable encrypted communication on an already established connection.

Additionally, this module provides a class `POP3_SSL`, which provides support for connecting to POP3 servers that use SSL as an underlying protocol layer.

Note that POP3, though widely supported, is obsolescent. The implementation quality of POP3 servers varies widely, and too many are quite poor. If your mailserver supports IMAP, you would be better off using `imaplib.IMAP4` class, as IMAP servers tend to be better implemented.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

The `poplib` module provides two classes:

`class poplib.POP3(host, port=POP3_PORT[, timeout])`

This class implements the actual POP3 protocol. The connection is created when the instance is initialized. If `port` is omitted, the standard POP3 port (110) is used. The optional `timeout` parameter specifies a timeout in seconds for the connection attempt (if not specified, the global default timeout setting will be used). Raises an `auditing` event `poplib.connect` with arguments `self, host, port`. All commands will raise an `auditing` event `poplib.putline` with arguments `self` and `line`, where `line` is the bytes about to be

`class poplib.POP3_SSL(host, port=POP3_SSL_PORT, *, timeout=None, context=None)`

This is a subclass of `POP3` that connects to the server over an SSL encrypted socket. If `port` is not specified, 995, the standard POP3-over-SSL port is used. `timeout` works as in the `POP3` constructor. `context` is an optional `ssl.SSLContext` object which allows bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure. Please read [Security considerations](#) for best practices. Raises an `auditing` event `poplib.connect` with arguments `self, host, port`. All commands will

One exception is defined as an attribute of the `poplib` module:

`exception poplib.error_proto`

Exception raised on any errors from this module (errors from `socket` module are not caught). The reason for the exception is passed to the constructor as a string.

See also

Module `imaplib`

Frequently Asked Questions About Fetchmail

The standard Python IMAP module.

The FAQ for the `fetchmail` POP/IMAP client collects information on POP3 server variations and RFC noncompliance that may be useful if you need to write an application based on the POP protocol.

POP3 Objects

All POP3 commands are represented by methods of the same name, in lowercase; most return the response text sent by the server.

A POP3 instance has the following methods:

POP3.set_debuglevel(level)

Set the instance's debugging level. This controls the amount of debugging output printed. The default, 0, produces no debugging output. A value of 1 produces a moderate amount of debugging output, generally a single line per request. A value of 2 or higher produces the maximum amount of debugging output, logging each line sent and received on the control connection.

POP3.getwelcome()

Returns the greeting string sent by the POP3 server.

POP3.capability()

Query the server's capabilities as specified in RFC 2449. Returns a dictionary in the form {'name': ['param'...]}. Added in version 3.4.

POP3.user(username)

Send user command, response should indicate that a password is required.

POP3.pass_(password)

Send password, response includes message count and mailbox size. Note: the mailbox on the server is locked until `quit()` is called.

POP3.apop(user,secret)

Use the more secure APOP authentication to log into the POP3 server.

POP3.rpop(user)

Use RPOP authentication (similar to UNIX r-commands) to log into POP3 server.

POP3.stat()

Get mailbox status. The result is a tuple of 2 integers: (messagecount, mailboxsize).

POP3.list([which])

Request message list, result is in the form (response, ['msg_numoctets',...], octets). If `which` is set, it is the message to list.

POP3.retr(which)

Retrieve whole message number `which`, and set its seen flag. Result is in form (response, ['line',...], octets).

POP3.dele(which)

Flag message number `which` for deletion. On most servers deletions are not actually performed until QUIT (the major exception is Eudora QPOP, which deliberately violates the RFCs by doing pending deletes on any disconnect).

POP3.rset()

Remove any deletion marks for the mailbox.

POP3.noop()

Do nothing. Might be used as a keep-alive.

POP3.quit()

Signoff: commit changes, unlock mailbox, drop connection.

POP3.top(which,howmuch)

Retrieves the message header plus how much lines of the message after the header of message number which. Result is in form (response, ['line', ...], octets). The POP3 TOP command this method uses, unlike the RETR command, doesn't set the message's seen flag; unfortunately, TOP is poorly specified in the RFCs and is frequently broken in off-brand servers. Test this method by hand against the POP3 servers you will use before trusting it.

POP3.uidl(which=None)

Return message digest (unique id) list. If which is specified, result contains the unique id for that message in the form (response, ['mesgnumuid', ...], octets). Otherwise result is list(response, ['mesgnumuid', ...], octets).

POP3.utf8()

Try to switch to UTF-8 mode. Returns the server response if successful, raises error_proto if not. Specified in RFC 6856. Added in version 3.5.

POP3.stls(context=None)

Start a TLS session on the active connection as specified in RFC 2595. This is only allowed before user authentication context parameter is ssl.SSLContext object which allows bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure. Please read Security considerations for best practices. This method supports hostname checking via ssl.SSLContext.check_hostname and Server Name Indication (see ssl.HAS_SNI). Added in version 3.4.

Instances of POP3_SSL have no additional methods. The interface of this subclass is identical to its parent.

POP3 Example

Here is a minimal example (without error checking) that opens a mailbox and retrieves and prints all messages:

```
import getpass, poplib

M = poplib.POP3('localhost')
M.user(getpass.getuser())
M.pass_(getpass.getpass())
numMessages = len(M.list()[1])
for i in range(numMessages):
    for j in M.retr(i+1)[1]:
        print(j)
```

At the end of the module, there is a test section that contains a more extensive example of usage.

imaplib— IMAP4 protocol client

Source code: `Lib/imaplib.py`

This module defines three classes, `IMAP4`, `IMAP4_SSL` and `IMAP4_stream`, which encapsulate a connection to an IMAP4 server and implement a large subset of the IMAP4rev1 client protocol as defined in RFC 2060. It is backward compatible with IMAP4 (RFC 1730) servers, but note that the `STATUS` command is not supported in IMAP4.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

Three classes are provided by the `imaplib` module, `IMAP4` is the base class:

`class imaplib.IMAP4(host="", port=IMAP4_PORT, timeout=None)`

This class implements the actual IMAP4 protocol. The connection is created and protocol version (IMAP4 or IMAP4rev1) is determined when the instance is initialized. If `host` is not specified, "(the local host) is used. If `port` is omitted, the standard IMAP4 port (143) is used. The optional `timeout` parameter specifies a timeout in seconds for the connection attempt. If `timeout` is not given or is `None`, the global default socket timeout is used. The `IMAP4` class supports the `with` statement. When used like this

Three exceptions are defined as attributes of the `IMAP4` class:

`exception IMAP4.error`

Exception raised on any errors. The reason for the exception is passed to the constructor as a string.

`exception IMAP4.abort`

IMAP4 server errors cause this exception to be raised. This is a sub-class of `IMAP4.error`. Note that closing the instance and instantiating a new one will usually allow recovery from this exception.

`exception IMAP4.readonly`

This exception is raised when a writable mailbox has its status changed by the server. This is a sub-class of `IMAP4.error`. Some other client now has write permission, and the mailbox will need to be re-opened to re-obtain write permission.

There's also a subclass for secure connections:

`class imaplib.IMAP4_SSL(host="", port=IMAP4_SSL_PORT, *, ssl_context=None, timeout=None)`

This is a subclass derived from `IMAP4` that connects over an SSL encrypted socket (to use this class you need a socket module that was compiled with SSL support). If `host` is not specified, "(the local host) is used. If `port` is omitted, the standard IMAP4-over-SSL port (993) is used. `ssl_context` is an `ssl.SSLContext` object which allows bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure. Please read [Security considerations](#) for best practices. The

The second subclass allows for connections created by a child process:

`class imaplib.IMAP4_stream(command)`

This is a subclass derived from `IMAP4` that connects to the `stdin/stdout` file descriptors created by passing `command` to `subprocess.Popen()`.

The following utility functions are defined:

`imaplib.Internaldate2tuple(datestr)`

Parse an IMAP4INTERNALDATE string and return corresponding local time. The return value is `time.struct_timetuple` or `None` if the string has wrong format.

imaplib.Int2AP(num)

Converts an integer into a bytes representation using characters from the set [A..P].

imaplib.ParseFlags(flagstr)

Converts an IMAP4FLAGS response to a tuple of individual flags.

imaplib.Time2Internaldate(date_time)

Convert `date_time` to an IMAP4INTERNALDATE representation. The return value is a string in the form: "DD-Mmm-YYYYHH:MM:SS+HHMM" (including double-quotes). The `date_time` argument can be a number (int or float) representing seconds since epoch (as returned by `time.time()`), a 9-tuple representing local time an instance of `time.struct_time` (as returned by `time.localtime()`), an aware instance of `datetime.datetime`, or a double-quoted string. In the last case, it is assumed to already be in the correct format.

Note that IMAP4 message numbers change as the mailbox changes; in particular, after an EXPUNGE command performs deletions the remaining messages are renumbered. So it is highly advisable to use UIDs instead, with the UID command.

At the end of the module, there is a test section that contains a more extensive example of usage.

See also

Documents describing the protocol, sources for servers implementing it, by the University of Washington's IMAP Information Center can all be found at (Source Code) <https://github.com/uw-imap/imap> (Not Maintained).

IMAP4 Objects

All IMAP4rev1 commands are represented by methods of the same name, either uppercase or lowercase.

All arguments to commands are converted to strings, except for AUTHENTICATE, and the last argument to APPEND which is passed as an IMAP4 literal. If necessary (the string contains IMAP4 protocol-sensitive characters and isn't enclosed with either parentheses or double quotes) each string is quoted. However, the password argument to the LOGIN command is always quoted. If you want to avoid having an argument string quoted (eg: the flags argument to STORE) then enclose the string in parentheses (eg: `r'(\Deleted)'`).

Most commands return a tuple: (type, [data, ...]) where type is usually 'OK' or 'NO', and data is either the text from the command response, or mandated results from the command. Each data is either a bytes, or a tuple. If a tuple, then the first part is the header of the response, and the second part contains the data (ie: 'literal' value).

The `message_set` options to commands below is a string specifying one or more messages to be acted upon. It may be a simple message number ('1'), a range of message numbers ('2:4'), or a group of non-contiguous ranges separated by commas ('1:3,6:9'). A range can contain an asterisk to indicate an infinite upper bound ('3:*').

An IMAP4 instance has the following methods:

IMAP4.append(mailbox, flags, date_time, message)

Append message to named mailbox.

IMAP4.authenticate(mechanism, authobject)

Authenticate command — requires response processing. `mechanism` specifies which authentication mechanism is to be used - it should appear in the instance variable `capabilities` in the form `AUTH=mechanism`. `authobject` must be a callable object: `data=authobject(response)` It will be called to

process server continuation responses; the response argument it is passed will be bytes. It should return bytes data that will be base64 encoded and sent to the server. It should return None if the client aborts response*should

IMAP4.check()

Checkpoint mailbox on server.

IMAP4.close()

Close currently selected mailbox. Deleted messages are removed from writable mailbox. This is the recommended command before LOGOUT.

IMAP4.copy(message_set, new_mailbox)

Copy message_set messages onto end of new_mailbox.

IMAP4.create(mailbox)

Create new mailbox named mailbox.

IMAP4.delete(mailbox)

Delete old mailbox named mailbox.

IMAP4.deleteacl(mailbox, who)

Delete the ACLs (remove any rights) set for who on mailbox.

IMAP4.enable(capability)

Enable capability (see RFC 5161). Most capabilities do not need to be enabled. Currently only the UTF8=ACCEPT capability is supported (see RFC 6855). Added in version 3.5: The enable() method itself, and RFC 6855 support.

IMAP4.expunge()

Permanently remove deleted items from selected mailbox. Generates an EXPUNGE response for each deleted message. Returned data contains a list of EXPUNGE message numbers in order received.

IMAP4.fetch(message_set, message_parts)

Fetch (parts of) messages. message_parts should be a string of message part names enclosed within parentheses, eg: "(UID BODY [TEXT])". Returned data are tuples of message part envelope and data.

IMAP4.getacl(mailbox)

Get the ACLs for mailbox. The method is non-standard, but is supported by the Cyrus server.

IMAP4.getannotation(mailbox, entry, attribute)

Retrieve the specified ANNOTATIONS for mailbox. The method is non-standard, but is supported by the Cyrus server.

IMAP4.getquota(root)

Get the quota root's resource usage and limits. This method is part of the IMAP4 QUOTA extension defined in RFC 2087.

IMAP4.getquotaroot(mailbox)

Get the list of quota roots for the named mailbox. This method is part of the IMAP4 QUOTA extension defined in RFC 2087.

IMAP4.idle(duration=None)

Idler.burst(interval=0.1)

Return an `idler`: an iterable context manager implementing the `IMAP4IDLE` command as defined in RFC 2177. The returned object sends the `IDLE` command when activated by the `with` statement, produces IMAP untagged responses via the iterator protocol, and sends `DONE` upon context exit. All untagged responses that arrive after sending the `IDLE` command (including any that arrive before the server acknowledges the command) will be available via iteration. Any leftover responses (those not iterated in the `with` context) can

Yield a burst of responses no more than `interval` seconds apart (expressed as an `int` or `float`). This generator is an alternative to iterating one response at a time, intended to aid in efficient batch processing. It retrieves the next response along with any immediately available subsequent responses. (For example, a rapid series of `EXPUNGE` responses after a bulk delete.) Requires a socket connection; does not work on `IMAP4_stream` connections. >>> `M.idle()` as `idler: ... #` get a response and any others following

IMAP4.list([directory[, pattern]])

List mailbox names in `directory` matching `pattern`. `directory` defaults to the top-level mail folder, and `pattern` defaults to match anything. Returned data contains a list of `LIST` responses.

IMAP4.login(user, password)

Identify the client using a plaintext password. The password will be quoted.

IMAP4.login_cram_md5(user, password)

Force use of CRAM-MD5 authentication when identifying the client to protect the password. Will only work if the server `CAPABILITY` response includes the phrase `AUTH=CRAM-MD5`. Changed in version 3.14: An `IMAP4.error` is raised if MD5 support is not available.

IMAP4.logout()

Shutdown connection to server. Returns server `BYE` response. Changed in version 3.8: The method no longer ignores silently arbitrary exceptions.

IMAP4.lsub(directory="'", pattern="*")

List subscribed mailbox names in `directory` matching `pattern`. `directory` defaults to the top level directory and `pattern` defaults to match any mailbox. Returned data are tuples of message part envelope and data.

IMAP4.myrights(mailbox)

Show my ACLs for a mailbox (i.e. the rights that I have on mailbox).

IMAP4.namespace()

Returns IMAP namespaces as defined in RFC 2342.

IMAP4.noop()

Send `NOOP` to server.

IMAP4.open(host, port, timeout=None)

Opens socket to `port` at `host`. The optional `timeout` parameter specifies a timeout in seconds for the connection attempt. If `timeout` is not given or is `None`, the global default socket timeout is used. Also note that if the `timeout` parameter is set to be zero, it will raise a `ValueError` to reject creating a non-blocking socket. This method is implicitly called by the `IMAP4` constructor. The connection objects established by this method will be used in the `IMAP4.read()`, `IMAP4.readline()`, `IMAP4.send()`, and `IMAP4.shut`

IMAP4.partial(message_num, message_part, start, length)

Fetch truncated part of a message. Returned data is a tuple of message part envelope and data.

IMAP4.proxyauth(user)

Assume authentication as `user`. Allows an authorised administrator to proxy into any user's mailbox.

IMAP4.read(size)

Readsizebytes from the remote server. You may override this method.

IMAP4.readline()

Reads one line from the remote server. You may override this method.

IMAP4.recent()

Prompt server for an update. Returned data isNoneif no new messages, else value ofRECENTresponse.

IMAP4.rename(oldmailbox,newmailbox)

Rename mailbox namedoldmailboxtonewmailbox.

IMAP4.response(code)

Return data for responsecodeif received, orNone. Returns the given code, instead of the usual type.

IMAP4.search(charset,criterion[,...])

Search mailbox for matching messages.charsetmay beNone, in which case noCHARSETwill be specified in the request to the server. The IMAP protocol requires that at least one criterion be specified; an exception will be raised when the server returns an error.charsetmust beNoneif theUTF8=ACCEPTcapability was enabled using theenable()command.Example:# M is a connected IMAP4 instance...typ,msgnums=M.search(None,'FROM','"LDJ"')# or:typ,msgnums=M.search(None,'(FROM "LDJ")')

IMAP4.select(mailbox='INBOX',readonly=False)

Select a mailbox. Returned data is the count of messages inmailbox(EXISTSresponse). The defaultmailboxis'INBOX'. If thereadonlyflag is set, modifications to the mailbox are not allowed.

IMAP4.send(data)

Sendsdatato the remote server. You may override this method.Raises anauditing eventimaplib.sendwith argumentsself,data.

IMAP4.setacl(mailbox,who,what)

Set anACLformailbox. The method is non-standard, but is supported by theCyrusserver.

IMAP4.setannotation(mailbox,entry,attribute[,...])

SetANNOTATIONS formailbox. The method is non-standard, but is supported by theCyrusserver.

IMAP4.setquota(root,limits)

Set thequotaroot's resourcelimits. This method is part of the IMAP4 QUOTA extension defined in rfc2087.

IMAP4.shutdown()

Close connection established inopen. This method is implicitly called byIMAP4.logout(). You may override this method.

IMAP4.socket()

Returns socket instance used to connect to server.

IMAP4.sort(sort_criteria,charset,search_criterion[,...])

The sortcommand is a variant ofsearchwith sorting semantics for the results. Returned data contains a space separated list of matching message numbers.Sort has two arguments before thesearch_criterionargument(s); a parenthesized list ofsort_criteria, and the searchingcharset. Note that unlike search, the searchingcharsetargument is mandatory. There is also auidsortcommand which corresponds tosortthe way thatuidsearchcorresponds tosearch. The sortcommand first searches the mailbox for messages t

IMAP4.starttls(ssl_context=None)

Send a STARTTLS command. The `ssl_context` argument is optional and should be an `ssl.SSLContext` object. This will enable encryption on the IMAP connection. Please read [Security considerations](#) for best practices. Added in version 3.2. Changed in version 3.4: The method now supports hostname check with `ssl.SSLContext.check_hostname` and `Server Name Indication` (see `ssl.HAS_SNI`).

IMAP4.status(mailbox, names)

Request named status conditions for mailbox.

IMAP4.store(message_set, command, flag_list)

Alters flag dispositions for messages in mailbox. `command` is specified by section 6.4.6 of RFC 2060 as being one of "FLAGS", "+FLAGS", or "-FLAGS", optionally with a suffix of ".SILENT". For example, to set the delete flag on all messages: `typ, data=M.search(None, 'ALL').for num in data[0].split(): M.store(num, '+FLAGS', '\\Deleted') M.expunge()` Note: Creating flags containing ']' (for example: "[test]") violates RFC 3501 (the IMAP protocol). However, `imaplib` has historically allowed creation of such tags, and `popu`

IMAP4.subscribe(mailbox)

Subscribe to new mailbox.

IMAP4.thread(threading_algorithm, charset, search_criterion[,...])

The `thread` command is a variant of `search` with threading semantics for the results. Returned data contains a space separated list of thread members. Thread members consist of zero or more messages numbers, delimited by spaces, indicating successive parent and child. Thread has two arguments before the `search_criterion` argument(s); `threading_algorithm`, and the `searching_charset`. Note that unlike `search`, the `searching_charset` argument is mandatory. There is also `uidthread` command which corresponds to `thread`

IMAP4.uid(command, arg[,...])

Execute command args with messages identified by UID, rather than message number. Returns response appropriate to command. At least one argument must be supplied; if none are provided, the server will return an error and an exception will be raised.

IMAP4.unsubscribe(mailbox)

Unsubscribe from old mailbox.

IMAP4.unselect()

`imaplib.IMAP4.unselect()` frees server's resources associated with the selected mailbox and returns the server to the authenticated state. This command performs the same actions as `imaplib.IMAP4.close()`, except that no messages are permanently removed from the currently selected mailbox. Added in version 3.9.

IMAP4.xatom(name[,...])

Allow simple extension commands notified by server in `CAPABILITY` response.

The following attributes are defined on instances of `IMAP4`:

IMAP4.PROTOCOL_VERSION

The most recent supported protocol in the `CAPABILITY` response from the server.

IMAP4.debug

Integer value to control debugging output. The initialize value is taken from the module variable `Debug`. Values greater than three trace each command.

IMAP4.utf8_enabled

Boolean value that is normally `False`, but is set to `True` if an `enable()` command is successfully issued for the `UTF8=ACCEPT` capability. Added in version 3.5.

IMAP4 Example

Here is a minimal example (without error checking) that opens a mailbox and retrieves and prints all messages:

```
import getpass, imaplib

M = imaplib.IMAP4(host='example.org')
M.login(getpass.getuser(), getpass.getpass())
M.select()
typ, data = M.search(None, 'ALL')
for num in data[0].split():
    typ, data = M.fetch(num, '(RFC822)')
    print('Message %s\n%s\n' % (num, data[0][1]))
M.close()
M.logout()
```

smtplib— SMTP protocol client

Source code: `Lib/smtplib.py`

The `smtplib` module defines an SMTP client session object that can be used to send mail to any internet machine with an SMTP or ESMTP listener daemon. For details of SMTP and ESMTP operation, consult RFC 821 (Simple Mail Transfer Protocol) and RFC 1869 (SMTP Service Extensions).

Availability: not WASI.

This module does not work or is not available on WebAssembly. See WebAssembly platforms for more information.

`smtplib.SMTP(host="", port=0, local_hostname=None, [timeout,] source_address=None)`

An `SMTP` instance encapsulates an SMTP connection. It has methods that support a full repertoire of SMTP and ESMTP operations. If the optional `host` and `port` parameters are given, the `SMTP.connect()` method is called with those parameters during initialization. If specified, `local_hostname` is used as the FQDN of the local host in the HELO/EHLO command. Otherwise, the local hostname is found using `socket.getfqdn()`. If the `connect()` call returns anything other than a success code, an `SMTPConnectError` is raised.

`smtplib.SMTP_SSL(host="", port=0, local_hostname=None, *, [timeout,] context=None, source_address=None)`

An `SMTP_SSL` instance behaves exactly the same as instances of `SMTP`. `SMTP_SSL` should be used for situations where SSL is required from the beginning of the connection and using `starttls()` is not appropriate. If `host` is not specified, the local host is used. If `port` is zero, the standard SMTP-over-SSL port (465) is used. The optional arguments `local_hostname`, `timeout` and `source_address` have the same meaning as they do in the `SMTP` class. `context`, also optional, can contain a `SSLContext` and allows configuring various a

`smtplib.LMTP(host="", port=LMTP_PORT, local_hostname=None, source_address=None, [timeout])`

The LMTP protocol, which is very similar to ESMTP, is heavily based on the standard SMTP client. It's common to use Unix sockets for LMTP, so our `connect()` method must support that as well as a regular `host:port` server. The optional arguments `local_hostname` and `source_address` have the same meaning as they do in the `SMTP` class. To specify a Unix socket, you must use an absolute path for `host`, starting with a `'/'`. Authentication is supported, using the regular SMTP mechanism. When using a Unix socket, LMTP

A nice selection of exceptions is defined as well:

`smtplib.SMTPException`

Subclass of `OSError` that is the base exception class for all the other exceptions provided by this module. Changed in version 3.4: `SMTPException` became subclass of `OSError`

`smtplib.SMTPServerDisconnected`

This exception is raised when the server unexpectedly disconnects, or when an attempt is made to use the `SMTP` instance before connecting it to a server.

`smtplib.SMTPResponseException`

`smtp_code`

`smtp_error`

Base class for all exceptions that include an SMTP error code. These exceptions are generated in some instances when the SMTP server returns an error code. `smtp_code` The error code. `smtp_error` The error message.

The error code.

The error message.

exceptionsmtplib.SMTPSenderRefused

Sender address refused. In addition to the attributes set by on allSMTPResponseExceptionexceptions, this sets 'sender' to the string that the SMTP server refused.

exceptionsmtplib.SMTPRecipientsRefused

recipients

All recipient addresses refused.recipientsA dictionary of exactly the same sort as returned bySMTP.sendmail()containing the errors for each recipient.

A dictionary of exactly the same sort as returned bySMTP.sendmail()containing the errors for each recipient.

exceptionsmtplib.SMTPDataError

The SMTP server refused to accept the message data.

exceptionsmtplib.SMTPConnectError

Error occurred during establishment of a connection with the server.

exceptionsmtplib.SMTPHeloError

The server refused ourHELOmessage.

exceptionsmtplib.SMTPNotSupportedError

The command or option attempted is not supported by the server.Added in version 3.5.

exceptionsmtplib.SMTPAuthenticationError

SMTP authentication went wrong. Most probably the server didn't accept the username/password combination provided.

See also

RFC 821- Simple Mail Transfer Protocol

RFC 1869- SMTP Service Extensions

Protocol definition for SMTP. This document covers the model, operating procedure, and protocol details for SMTP.

Definition of the ESMTP extensions for SMTP. This describes a framework for extending SMTP with new commands, supporting dynamic discovery of the commands provided by the server, and defines a few additional commands.

SMTP Objects

AnSMTPinstance has the following methods:

SMTP.set_debuglevel(level)

Set the debug output level. A value of 1 orTrueforlevelresults in debug messages for connection and for all messages sent to and received from the server. A value of 2 forlevelresults in these messages being timestamped.Changed in version 3.5:Added debuglevel 2.

SMTP.docmd(cmd,args="")

Send a commandcmdto the server. The optional argumentargsis simply concatenated to the command, separated by a space.This returns a 2-tuple composed of a numeric response code and the actual response line

(multiline responses are joined into one long line.) In normal operation it should not be necessary to call this method explicitly. It is used to implement other methods and may be useful for testing private extensions. If the connection to the server is lost while waiting for the reply, SMTPServ

SMTP.connect(host='localhost',port=0)

Connect to a host on a given port. The defaults are to connect to the local host at the standard SMTP port (25). If the hostname ends with a colon (':') followed by a number, that suffix will be stripped off and the number interpreted as the port number to use. This method is automatically invoked by the constructor if a host is specified during instantiation. Returns a 2-tuple of the response code and message sent by the server in its connection response. Raises an auditing eventsmtplib.connect

SMTP.helo(name="")

Identify yourself to the SMTP server using HELO. The hostname argument defaults to the fully qualified domain name of the local host. The message returned by the server is stored as the `helo_resp` attribute of the object. In normal operation it should not be necessary to call this method explicitly. It will be implicitly called by `sendmail()` when necessary.

SMTP.ehlo(name="")

Identify yourself to an ESMTP server using EHLO. The hostname argument defaults to the fully qualified domain name of the local host. Examine the response for ESMTP option and store them for use by `has_extn()`. Also sets several informational attributes: the message returned by the server is stored as the `ehlo_resp` attribute, `does_esmtp` is set to `True` or `False` depending on whether the server supports ESMTP, and `esmtp_features` will be a dictionary containing the names of the SMTP service extensions this server

SMTP.ehlo_or_helo_if_needed()

SMTPHeloError

This method calls `ehlo()` and/or `helo()` if there has been no previous EHLO or HELO command this session. It tries ESMTP EHLO first. SMTPHeloError The server didn't reply properly to the HELO greeting.

The server didn't reply properly to the HELO greeting.

SMTP.has_extn(name)

Return `True` if `name` is in the set of SMTP service extensions returned by the server, `False` otherwise. Case is ignored.

SMTP.verify(address)

Check the validity of an address on this server using SMTPVRFY. Returns a tuple consisting of code 250 and a full RFC 822 address (including human name) if the user address is valid. Otherwise returns an SMTP error code of 400 or greater and an error string. Note Many sites disable SMTPVRFY in order to foil spammers.

SMTP.login(user,password,*,initial_response_ok=True)

SMTPHeloError

SMTPAuthenticationError

SMTPNotSupportedError

SMTPException

Log in on an SMTP server that requires authentication. The arguments are the username and the password to authenticate with. If there has been no previous EHLO or HELO command this session, this method tries ESMTP EHLO first. This method will return normally if the authentication was successful, or may raise the following exceptions: SMTPHeloError The server didn't reply properly to the HELO greeting. SMTPAuthenticationError The server didn't accept the username/password

combination.SMTPNotSupportedErrorThe

The server didn't reply properly to theHELOgreeting.

The server didn't accept the username/password combination.

TheAUTHcommand is not supported by the server.

No suitable authentication method was found.

SMTP.auth(mechanism,authobject,*,initial_response_ok=True)

Issue anSMTPAUTHcommand for the specified authenticationmechanism, and handle the challenge response viaauthobject.mechanismSpecifies which authentication mechanism is to be used as argument to theAUTHcommand; the valid values are those listed in theauthobject.features.authobjectmust be a callable object taking an optional single argument:data=authobject(challenge=None)If optional keyword argumentinitial_response_okis true,authobject()will be called first with no argument. It can return

SMTP.starttls(*,context=None)

SMTPHeloError

SMTPNotSupportedError

RuntimeError

Put the SMTP connection in TLS (Transport Layer Security) mode. All SMTP commands that follow will be encrypted. You should then callstarttls()again.Ifkeyfileandcertfileare provided, they are used to create anssl.SSLContext.Optionalcontextparameter is anssl.SSLContextobject; This is an alternative to using a keyfile and a certfile and if specified bothkeyfileandcertfileshould beNone.If there has been no previousEHLOorHELOcommand this session, this method tries ESMTPEHLOfirst.Changed in version 3.1

The server didn't reply properly to theHELOgreeting.

The server does not support the STARTTLS extension.

SSL/TLS support is not available to your Python interpreter.

SMTP.sendmail(from_addr,to_addrs,msg,mail_options=(),rcpt_options=())

SMTPRecipientsRefused

SMTPHeloError

SMTPSenderRefused

SMTPDataError

SMTPNotSupportedError

Send mail. The required arguments are anRFC 822from-address string, a list ofRFC 822to-address strings (a bare string will be treated as a list with 1 address), and a message string. The caller may pass a list of ESMTPOptions (such as"8bitmime") to be used inMAILFROMcommands asmail_options. ESMTPOptions (such asDSNcommands) that should be used with allRCPTcommands can be passed asrcpt_options. Each option should be passed as a string containing the full text of the option, including any parameters

All recipients were refused. Nobody got the mail.

The server didn't reply properly to theHELOgreeting.

The server didn't accept thefrom_addr.

The server replied with an unexpected error code (other than a refusal of a recipient).

SMTPUTF8was given in themail_optionsbut is not supported by the server.

SMTP.send_message(msg,from_addr=None,to_addrs=None,mail_options={},rcpt_options=())

This is a convenience method for calling `sendmail()` with the message represented by an `email.message.Message` object. The arguments have the same meaning as for `sendmail()`, except that `msg` is a `Message` object. If `from_addr` is `None` or `to_addrs` is `None`, `send_message` fills those arguments with addresses extracted from the headers of `msg` as specified in RFC 5322: `from_addr` is set to the `Sender` field if it is present, and otherwise to the `From` field. `to_addrs` combines the values (if any) of the `To`, `Cc`, and `Bcc` fields from `msg`. If exa

SMTP.quit()

Terminate the SMTP session and close the connection. Return the result of the `SMTPQUIT` command.

Low-level methods corresponding to the standard SMTP/ESMTP commands `HELP`, `RSET`, `NOOP`, `MAIL`, `RCPT`, and `DATA` are also supported. Normally these do not need to be called directly, so they are not documented here. For details, consult the module code.

Additionally, an SMTP instance has the following attributes:

SMTP.helo_resp

The response to the `HELO` command, see `helo()`.

SMTP.ehlo_resp

The response to the `EHLO` command, see `ehlo()`.

SMTP.does_esmtp

A boolean value indicating whether the server supports ESMTP, see `ehlo()`.

SMTP.esmtp_features

A dictionary of the names of SMTP service extensions supported by the server, see `ehlo()`.

SMTP Example

This example prompts the user for addresses needed in the message envelope ('To' and 'From' addresses), and the message to be delivered. Note that the headers to be included with the message must be included in the message as entered; this example doesn't do any processing of the RFC 822 headers. In particular, the 'To' and 'From' addresses must be included in the message headers explicitly:

```
import smtplib

def prompt(title):
    return input(title).strip()

from_addr = prompt("From: ")
to_addrs = prompt("To: ").split()
print("Enter message, end with ^D (Unix) or ^Z (Windows):")

# Add the From: and To: headers at the start!
lines = [f"From: {from_addr}", f"To: {' '.join(to_addrs)}", ""]
while True:
    try:
        line = input()
    except EOFError:
        break
    else:
        lines.append(line)

msg = "\r\n".join(lines)
print("Message length is", len(msg))
```

```
server = smtplib.SMTP("localhost")
server.set_debuglevel(1)
server.sendmail(from_addr, to_addrs, msg)
server.quit()
```

■ Nota: NoteIn general, you will want to use theemailpackage's features to construct an email message, which you can then send via `send_message()`; seeemail: Examples.

uuid— UUID objects according toRFC 9562

Source code:Lib/uuid.py

This module provides immutableUUID objects (theUUID class) and functions for generating UUIDs corresponding to a specific UUID version as specified inRFC 9562(which supersedesRFC 4122), for example,uuid1()for UUID version 1,uuid3()for UUID version 3, and so on. Note that UUID version 2 is deliberately omitted as it is outside the scope of the RFC.

If all you want is a unique ID, you should probably calluuid1()oruuid4(). Note thatuuid1()may compromise privacy since it creates a UUID containing the computer’s network address.uuid4()creates a random UUID.

Depending on support from the underlying platform,uuid1()may or may not return a “safe” UUID. A safe UUID is one which is generated using synchronization methods that ensure no two processes can obtain the same UUID. All instances ofUUID have anis_safeattribute which relays any information about the UUID’s safety, using this enumeration:

class uuid.SafeUUID

safe

unsafe

unknown

Added in version 3.7.**safe**The UUID was generated by the platform in a multiprocessing-safe way.**unsafe**The UUID was not generated in a multiprocessing-safe way.**unknown**The platform does not provide information on whether the UUID was generated safely or not.

The UUID was generated by the platform in a multiprocessing-safe way.

The UUID was not generated in a multiprocessing-safe way.

The platform does not provide information on whether the UUID was generated safely or not.

class uuid.UUID(hex=None, bytes=None, bytes_le=None, fields=None, int=None, version=None, *, is_safe=SafeUUID.unknown)

Create a UUID from either a string of 32 hexadecimal digits, a string of 16 bytes in big-endian order as thebytesargument, a string of 16 bytes in little-endian order as thebytes_leargument, a tuple of six integers (32-bittime_low, 16-bittime_mid, 16-bittime_hi_version, 8-bitclock_seq_hi_variant, 8-bitclock_seq_low, 48-bitnode) as thefieldsargument, or a single 128-bit integer as theintargument. When a string of hex digits is given, curly braces, hyphens, and a URN prefix are all optional. For

UUID instances have these read-only attributes:

UUID.bytes

The UUID as a 16-byte string (containing the six integer fields in big-endian byte order).

UUID.bytes_le

The UUID as a 16-byte string (withtime_low,time_mid, andtime_hi_versionin little-endian byte order).

UUID.fields

A tuple of the six integer fields of the UUID, which are also available as six individual attributes and two derived attributes:

Field	Meaning	UUID.	time_low	The first 32 bits of the UUID. Only relevant to version 1.
		UUID.	time_mid	The next 16 bits of the UUID. Only relevant to version 1.
		UUID.	time_hi_version	The next 16 bits of the UUID. Only relevant to version 1.
		UUID.	clock_seq_hi_variant	The next 8 bits of the UUID. Only relevant to versions 1 and 6.

clock_seq_low | The next 8 bits of the UUID. Only relevant to versions 1 and 6. | UUID. | node | The last 48 bits of the UUID. Only relevant to version 1. | UUID. | time | The 60-bit timestamp as a count of 100-nanosecond intervals since Gregorian epoch (1582-10-15 00:00:00) for versions 1 and 6, or the 48-bit timestamp in milliseconds since Unix epoch (1970-01-01 00:00:00) for version 7. | UUID. | clock_seq | The 14-bit sequence

UUID.hex

The UUID as a 32-character lowercase hexadecimal string.

UUID.int

The UUID as a 128-bit integer.

UUID.urn

The UUID as a URN as specified in RFC 9562.

UUID.variant

The UUID variant, which determines the internal layout of the UUID. This will be one of the constants `RESERVED_NCS`, `RFC_4122`, `RESERVED_MICROSOFT`, or `RESERVED_FUTURE`.

UUID.version

The UUID version number (1 through 8, meaningful only when the variant is `RFC_4122`). Changed in version 3.14: Added UUID versions 6, 7 and 8.

UUID.is_safe

An enumeration of `SafeUUID` which indicates whether the platform generated the UUID in a multiprocessing-safe way. Added in version 3.7.

The `uuid` module defines the following functions:

uuid.getnode()

Get the hardware address as a 48-bit positive integer. The first time this runs, it may launch a separate program, which could be quite slow. If all attempts to obtain the hardware address fail, we choose a random 48-bit number with the multicast bit (least significant bit of the first octet) set to 1 as recommended in RFC 4122. "Hardware address" means the MAC address of a network interface. On a machine with multiple network interfaces, universally administered MAC addresses (i.e. where the

uuid.uuid1(node=None, clock_seq=None)

Generate a UUID from a host ID, sequence number, and the current time according to RFC 9562, §5.1. When `node` is not specified, `getnode()` is used to obtain the hardware address as a 48-bit positive integer. When a sequence number `clock_seq` is not specified, a pseudo-random 14-bit positive integer is generated. If `node` or `clock_seq` exceed their expected bit count, only their least significant bits are kept.

uuid.uuid3(namespace, name)

Generate a UUID based on the MD5 hash of a namespace identifier (which is a UUID) and a name (which is a bytes object or a string that will be encoded using UTF-8) according to RFC 9562, §5.3.

uuid.uuid4()

Generate a random UUID in a cryptographically-secure method according to RFC 9562, §5.4.

uuid.uuid5(namespace, name)

Generate a UUID based on the SHA-1 hash of a namespace identifier (which is a UUID) and a name (which is a bytes object or a string that will be encoded using UTF-8) according to RFC 9562, §5.5.

uuid.uuid6(node=None, clock_seq=None)

Generate a UUID from a sequence number and the current time according to RFC 9562, §5.6. This is an alternative to `uuid1()` to improve database locality. When `node` is not specified, `getnode()` is used to obtain the hardware address as a 48-bit positive integer. When a sequence number `clock_seq` is not specified, a pseudo-random 14-bit positive integer is generated. If `node` or `clock_seq` exceed their expected bit count, only their least significant bits are kept. Added in version 3.14.

uuid.uuid7()

Generate a time-based UUID according to RFC 9562, §5.7. For portability across platforms lacking sub-millisecond precision, UUIDs produced by this function embed a 48-bit timestamp and use a 42-bit counter to guarantee monotonicity within a millisecond. Added in version 3.14.

uuid.uuid8(a=None, b=None, c=None)

Generate a pseudo-random UUID according to RFC 9562, §5.8. When specified, the parameters `a`, `b` and `c` are expected to be positive integers of 48, 12 and 62 bits respectively. If they exceed their expected bit count, only their least significant bits are kept; non-specified arguments are substituted for a pseudo-random integer of appropriate size. By default, `a`, `b` and `c` are not generated by a cryptographically secure pseudo-random number generator (CSPRNG). Use `uuid4()` when a UUID needs to be used in a security-

The `uuid` module defines the following namespace identifiers for use with `uuid3()` or `uuid5()`.

uuid.NAMESPACE_DNS

When this namespace is specified, then `namestring` is a fully qualified domain name.

uuid.NAMESPACE_URL

When this namespace is specified, then `namestring` is a URL.

uuid.NAMESPACE_OID

When this namespace is specified, then `namestring` is an ISO OID.

uuid.NAMESPACE_X500

When this namespace is specified, then `namestring` is an X.500 DN in DER or a text output format.

The `uuid` module defines the following constants for the possible values of the `variant` attribute:

uuid.RESERVED_NCS

Reserved for NCS compatibility.

uuid.RFC_4122

Specifies the UUID layout given in RFC 4122. This constant is kept for backward compatibility even though RFC 4122 has been superseded by RFC 9562.

uuid.RESERVED_MICROSOFT

Reserved for Microsoft compatibility.

uuid.RESERVED_FUTURE

Reserved for future definition.

The `uuid` module defines the special Nil and Max UUID values:

uuid.NIL

A special form of UUID that is specified to have all 128 bits set to zero according to RFC 9562, §5.9. Added in version 3.14.

uuid.MAX

A special form of UUID that is specified to have all 128 bits set to one according to RFC 9562, §5.10. Added in version 3.14.

See also

RFC 9562- A Universally Unique Identifier (UUID) URN Namespace

This specification defines a Uniform Resource Name namespace for UUIDs, the internal format of UUIDs, and methods of generating UUIDs.

Command-Line Usage

Added in version 3.12.

The `uuid` module can be executed as a script from the command line.

```
python -m uuid [-h] [-u {uuid1,uuid3,uuid4,uuid5,uuid6,uuid7,uuid8}] [-n NAMESPACE] [-N NAME]
```

The following options are accepted:

-h,--help

Show the help message and exit.

-u<uuid>

--uuid<uuid>

Specify the function name to use to generate the uuid. By default `uuid4()` is used. Changed in version 3.14: Allow generating UUID versions 6, 7 and 8.

-n<namespace>

--namespace<namespace>

The namespace is a UUID, or `@nswhere` is a well-known predefined UUID addressed by namespace name. Such as `@dns`, `@url`, `@oid`, and `@x500`. Only required for `uuid3()`/`uuid5()` functions.

-N<name>

--name<name>

The name used as part of generating the uuid. Only required for `uuid3()`/`uuid5()` functions.

-C<num>

--count<num>

Generate `num` fresh UUIDs. Added in version 3.14.

Example

Here are some examples of typical usage of the `uuid` module:

```
>>> import uuid

>>> # make a UUID based on the host ID and current time
>>> uuid.uuid1()
UUID('a8098c1a-f86e-11da-bd1a-00112444be1e')

>>> # make a UUID using an MD5 hash of a namespace UUID and a name
>>> uuid.uuid3(uuid.NAMESPACE_DNS, 'python.org')
UUID('6fa459ea-ee8a-3ca4-894e-db77e160355e')
```

```

>>> # make a random UUID
>>> uuid.uuid4()
UUID('16fd2706-8baf-433b-82eb-8c7fada847da')

>>> # make a UUID using a SHA-1 hash of a namespace UUID and a name
>>> uuid.uuid5(uuid.NAMESPACE_DNS, 'python.org')
UUID('886313e1-3b8a-5372-9b90-0c9aee199e5d')

>>> # make a UUID from a string of hex digits (braces and hyphens ignored)
>>> x = uuid.UUID('{00010203-0405-0607-0809-0a0b0c0d0e0f}')

>>> # convert a UUID to a string of hex digits in standard form
>>> str(x)
'00010203-0405-0607-0809-0a0b0c0d0e0f'

>>> # get the raw 16 bytes of the UUID
>>> x.bytes
b'\x00\x01\x02\x03\x04\x05\x06\x07\x08\t\n\x0b\x0c\r\x0e\x0f'

>>> # make a UUID from a 16-byte string
>>> uuid.UUID(bytes=x.bytes)
UUID('00010203-0405-0607-0809-0a0b0c0d0e0f')

>>> # get the Nil UUID
>>> uuid.NIL
UUID('00000000-0000-0000-0000-000000000000')

>>> # get the Max UUID
>>> uuid.MAX
UUID('ffffffff-ffff-ffff-ffff-fffffffffffffff')

>>> # same as UUIDv1 but with fields reordered to improve DB locality
>>> uuid.uuid6()
UUID('1f0799c0-98b9-62db-92c6-a0d365b91053')

>>> # get UUIDv7 creation (local) time as a timestamp in milliseconds
>>> u = uuid.uuid7()
>>> u.time
1743936859822

>>> # get UUIDv7 creation (local) time as a datetime object
>>> import datetime as dt
>>> dt.datetime.fromtimestamp(u.time / 1000)
datetime.datetime(...)

>>> # make a UUID with custom blocks
>>> uuid.uuid8(0x12345678, 0x9abcdef0, 0x11223344)
UUID('00001234-5678-8ef0-8000-000011223344')

```

Command-Line Example

Here are some examples of typical usage of the `uuid` command-line interface:

```

# generate a random UUID - by default uuid4() is used
$ python -m uuid

# generate a UUID using uuid1()
$ python -m uuid -u uuid1

# generate a UUID using uuid5
$ python -m uuid -u uuid5 -n @url -N example.com

# generate 42 random UUIDs

```



```
$ python -m uuid -C 42
```

socketserver— A framework for network servers

Source code: `Lib/socketserver.py`

The `socketserver` module simplifies the task of writing network servers.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

There are four basic concrete server classes:

`class socketserver.TCPServer(server_address, RequestHandlerClass, bind_and_activate=True)`

This uses the internet TCP protocol, which provides for continuous streams of data between the client and server. If `bind_and_activate` is true, the constructor automatically attempts to invoke `server_bind()` and `server_activate()`. The other parameters are passed to the `BaseServer` base class.

`class socketserver.UDPServer(server_address, RequestHandlerClass, bind_and_activate=True)`

This uses datagrams, which are discrete packets of information that may arrive out of order or be lost while in transit. The parameters are the same as for `TCPServer`.

`class socketserver.UnixStreamServer(server_address, RequestHandlerClass, bind_and_activate=True)`

`class socketserver.UnixDatagramServer(server_address, RequestHandlerClass, bind_and_activate=True)`

These more infrequently used classes are similar to the TCP and UDP classes, but use Unix domain sockets; they're not available on non-Unix platforms. The parameters are the same as for `TCPServer`.

These four classes process requests synchronously; each request must be completed before the next request can be started. This isn't suitable if each request takes a long time to complete, because it requires a lot of computation, or because it returns a lot of data which the client is slow to process. The solution is to create a separate process or thread to handle each request; the `ForkingMixIn` and `ThreadingMixIn` classes can be used to support asynchronous behaviour.

Creating a server requires several steps. First, you must create a request handler class by subclassing the `BaseRequestHandler` class and overriding its `handle()` method; this method will process incoming requests. Second, you must instantiate one of the server classes, passing it the server's address and the request handler class. It is recommended to use the server in a `with` statement. Then call the `handle_request()` or `serve_forever()` method of the server object to process one or many requests. Finally, call `server_close()` to close the socket (unless you used a `with` statement).

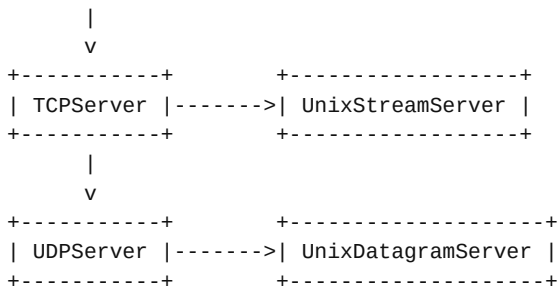
When inheriting from `ThreadingMixIn` for threaded connection behavior, you should explicitly declare how you want your threads to behave on an abrupt shutdown. The `ThreadingMixIn` class defines an attribute `daemon_threads`, which indicates whether or not the server should wait for thread termination. You should set the flag explicitly if you would like threads to behave autonomously; the default is `False`, meaning that Python will not exit until all threads created by `ThreadingMixIn` have exited.

Server classes have the same external methods and attributes, no matter what network protocol they use.

Server Creation Notes

There are five classes in an inheritance diagram, four of which represent synchronous servers of four types:

```
+-----+
| BaseServer |
+-----+
```



Note that `UnixDatagramServer` derives from `UDPServer`, not from `UnixStreamServer`— the only difference between an IP and a Unix server is the address family.

`class socketserver.ForkingMixIn`

`class socketserver.ThreadingMixIn`

`block_on_close`

`max_children`

`daemon_threads`

Forking and threading versions of each type of server can be created using these mix-in classes. For instance, `ThreadingUDPServer` is created as follows: `class ThreadingUDPServer(ThreadingMixIn, UDPServer):` pass The mix-in class comes first, since it overrides a method defined in `UDPServer`. Setting the various attributes also changes the behavior of the underlying server mechanism. `ForkingMixIn` and the Forking classes mentioned below are only available on POSIX platforms that support `fork()`. `block_on_close`

`ForkingMixIn.server_close` waits until all child processes complete, except if `block_on_close` attribute is `False`. `ThreadingMixIn.server_close` waits until all non-daemon threads complete, except if `block_on_close` attribute is `False`.

Specify how many child processes will exist to handle requests at a time for `ForkingMixIn`. If the limit is reached, new requests will wait until one child process has finished.

For `ThreadingMixIn` use daemon threads by setting `ThreadingMixIn.daemon_threads` to `True` to not wait until threads complete.

`class socketserver.ForkingTCPServer`

`class socketserver.ForkingUDPServer`

`class socketserver.ThreadingTCPServer`

`class socketserver.ThreadingUDPServer`

`class socketserver.ForkingUnixStreamServer`

`class socketserver.ForkingUnixDatagramServer`

`class socketserver.ThreadingUnixStreamServer`

`class socketserver.ThreadingUnixDatagramServer`

These classes are pre-defined using the mix-in classes.

Added in version 3.12: The `ForkingUnixStreamServer` and `ForkingUnixDatagramServer` classes were added.

To implement a service, you must derive a class from `BaseRequestHandler` and redefine its `handle()` method. You can then run various versions of the service by combining one of the server classes with your request handler class. The request handler class must be different for datagram or stream services. This can be hidden by using the handler subclasses `StreamRequestHandler` or `DatagramRequestHandler`.

Of course, you still have to use your head! For instance, it makes no sense to use a forking server if the service contains state in memory that can be modified by different requests, since the modifications in the child process would never reach the initial state kept in the parent process and passed to each child. In this case, you can use a threading server, but you will probably have to use locks to protect the integrity of the shared data.

On the other hand, if you are building an HTTP server where all data is stored externally (for instance, in the file system), a synchronous class will essentially render the service “deaf” while one request is being handled – which may be for a very long time if a client is slow to receive all the data it has requested. Here a threading or forking server is appropriate.

In some cases, it may be appropriate to process part of a request synchronously, but to finish processing in a forked child depending on the request data. This can be implemented by using a synchronous server and doing an explicit fork in the request handler classhandle()method.

Another approach to handling multiple simultaneous requests in an environment that supports neither threads norfork()(or where these are too expensive or inappropriate for the service) is to maintain an explicit table of partially finished requests and to useselectors to decide which request to work on next (or whether to handle a new incoming request). This is particularly important for stream services where each client can potentially be connected for a long time (if threads or subprocesses cannot be used).

Server Objects

classsocketserver.BaseServer(server_address,RequestHandlerClass)

fileno()

handle_request()

serve_forever(poll_interval=0.5)

service_actions()

shutdown()

server_close()

address_family

RequestHandlerClass

server_address

socket

allow_reuse_address

request_queue_size

socket_type

timeout

finish_request(request,client_address)

get_request()

handle_error(request,client_address)

handle_timeout()

process_request(request,client_address)

server_activate()

server_bind()

verify_request(request,client_address)

This is the superclass of all Server objects in the module. It defines the interface, given below, but does not implement most of the methods, which is done in subclasses. The two parameters are stored in the `server_address` and `RequestHandlerClass` attributes.
`fileno()` Return an integer file descriptor for the socket on which the server is listening. This function is most commonly passed to `selectors`, to allow monitoring multiple servers in the same process.
`handle_request()` Process a single

Return an integer file descriptor for the socket on which the server is listening. This function is most commonly passed to `selectors`, to allow monitoring multiple servers in the same process.

Process a single request. This function calls the following methods in order: `get_request()`, `verify_request()`, and `process_request()`. If the user-provided `handle()` method of the handler class raises an exception, the server's `handle_error()` method will be called. If no request is received within `timeoutseconds`, `handle_timeout()` will be called and `handle_request()` will return.

Handle requests until an explicit `shutdown()` request. Poll for shutdown every `poll_intervalseconds`. Ignores the `timeout` attribute. It also calls `service_actions()`, which may be used by a subclass or mixin to provide actions specific to a given service. For example, the `ForkingMixIn` class uses `service_actions()` to clean up zombie child processes. Changed in version 3.3: Added `service_actions` call to the `serve_forever` method.

This is called in the `serve_forever()` loop. This method can be overridden by subclasses or mixin classes to perform actions specific to a given service, such as cleanup actions. Added in version 3.3.

Tell the `serve_forever()` loop to stop and wait until it does. `shutdown()` must be called while `serve_forever()` is running in a different thread otherwise it will deadlock.

Clean up the server. May be overridden.

The family of protocols to which the server's socket belongs. Common examples are `socket.AF_INET`, `socket.AF_INET6`, and `socket.AF_UNIX`. Subclass the TCP or UDP server classes in this module with class attribute `address_family=AF_INET6` set if you want IPv6 server classes.

The user-provided request handler class; an instance of this class is created for each request.

The address on which the server is listening. The format of addresses varies depending on the protocol family; see the documentation for the `socket` module for details. For internet protocols, this is a tuple containing a string giving the address, and an integer port number: `('127.0.0.1',80)`, for example.

The socket object on which the server will listen for incoming requests.

Whether the server will allow the reuse of an address. This defaults to `False`, and can be set in subclasses to change the policy.

The size of the request queue. If it takes a long time to process a single request, any requests that arrive while the server is busy are placed into a queue, up to `request_queue_size` requests. Once the queue is full, further requests from clients will get a "Connection denied" error. The default value is usually 5, but this can be overridden by subclasses.

The type of socket used by the server; `socket.SOCK_STREAM` and `socket.SOCK_DGRAM` are two common values.

Timeout duration, measured in seconds, or `None` if no timeout is desired. If `handle_request()` receives no incoming requests within the timeout period, the `handle_timeout()` method is called.

Actually processes the request by instantiating `RequestHandlerClass` and calling its `handle()` method.

Must accept a request from the socket, and return a 2-tuple containing the new socket object to be used to communicate with the client, and the client's address.

This function is called if the `handle()` method of a `RequestHandlerClass` instance raises an exception. The default action is to print the traceback to standard error and continue handling further requests. Changed in version 3.6: Now only called for exceptions derived from the `Exception` class.

This function is called when the `timeout` attribute has been set to a value other than `None` and the timeout period has passed with no requests being received. The default action for forking servers is to collect the status of any child processes that have exited, while in threading servers this method does nothing.

Calls `finish_request()` to create an instance of the `RequestHandlerClass`. If desired, this function can create a new process or thread to handle the request; the `ForkingMixIn` and `ThreadingMixIn` classes do this.

Called by the server's constructor to activate the server. The default behavior for a TCP server just invokes `listen()` on the server's socket. May be overridden.

Called by the server's constructor to bind the socket to the desired address. May be overridden.

Must return a Boolean value; if the value is `True`, the request will be processed, and if it's `False`, the request will be denied. This function can be overridden to implement access controls for a server. The default implementation always returns `True`.

Request Handler Objects

`class socketserver.BaseRequestHandler`

`setup()`

`handle()`

`finish()`

`request`

`client_address`

`server`

This is the superclass of all request handler objects. It defines the interface, given below. A concrete request handler subclass must define a new `handle()` method, and can override any of the other methods. A new instance of the subclass is created for each request. **`setup()`** Called before the `handle()` method to perform any initialization actions required. The default implementation does nothing. **`handle()`** This function must do all the work required to service a request. The default implementation do

Called before the `handle()` method to perform any initialization actions required. The default implementation does nothing.

This function must do all the work required to service a request. The default implementation does nothing. Several instance attributes are available to it; the request is available as `request`; the client address as `client_address`; and the server instance as `server`, in case it needs access to per-server information. The type of `request` is different for datagram or stream services. For stream services, `request` is a socket object; for datagram services, `request` is a pair of string and socket.

Called after the `handle()` method to perform any clean-up actions required. The default implementation does nothing. If `setup()` raises an exception, this function will not be called.

The new socket object to be used to communicate with the client.

Client address returned by `BaseServer.get_request()`.

`BaseServer` object used for handling the request.

`class socketserver.StreamRequestHandler`

class socketserver.DatagramRequestHandler

rfile

wfile

These BaseRequestHandler subclasses override the setup() and finish() methods, and provide rfile and wfile attributes. rfile A file object from which the request is read. Support the io.BufferedIOBase readable interface. wfile A file object to which the reply is written. Support the io.BufferedIOBase writable interface. Changed in version 3.6: wfile also supports the io.BufferedIOBase writable interface.

A file object from which the request is read. Support the io.BufferedIOBase readable interface.

A file object to which the reply is written. Support the io.BufferedIOBase writable interface

Examples

socketserver.TCPServerExample

This is the server side:

```
import socketserver

class MyTCPHandler(socketserver.BaseRequestHandler):
    """
    The request handler class for our server.

    It is instantiated once per connection to the server, and must
    override the handle() method to implement communication to the
    client.
    """

    def handle(self):
        # self.request is the TCP socket connected to the client
        pieces = [b'']
        total = 0
        while b'\n' not in pieces[-1] and total < 10_000:
            pieces.append(self.request.recv(2000))
            total += len(pieces[-1])
        self.data = b''.join(pieces)
        print(f"Received from {self.client_address[0]}:")
        print(self.data.decode("utf-8"))
        # just send back the same data, but upper-cased
        self.request.sendall(self.data.upper())
        # after we return, the socket will be closed.

if __name__ == "__main__":
    HOST, PORT = "localhost", 9999

    # Create the server, binding to localhost on port 9999
    with socketserver.TCPServer((HOST, PORT), MyTCPHandler) as server:
        # Activate the server; this will keep running until you
        # interrupt the program with Ctrl-C
        server.serve_forever()
```

An alternative request handler class that makes use of streams (file-like objects that simplify communication by providing the standard file interface):

```
class MyTCPHandler(socketserver.StreamRequestHandler):

    def handle(self):
        # self.rfile is a file-like object created by the handler.
```

```

# We can now use e.g. readline() instead of raw recv() calls.
# We limit ourselves to 10000 bytes to avoid abuse by the sender.
self.data = self.rfile.readline(10000).rstrip()
print(f"{self.client_address[0]} wrote:")
print(self.data.decode("utf-8"))
# Likewise, self.wfile is a file-like object used to write back
# to the client
self.wfile.write(self.data.upper())

```

The difference is that `readline()` call in the second handler will call `recv()` multiple times until it encounters a newline character, while the first handler had to use `arecv()` loop to accumulate data until a newline itself. If it had just used a single `recv()` without the loop it would just have returned what has been received so far from the client. TCP is stream based: data arrives in the order it was sent, but there is no correlation between `clientsend()` or `sendall()` calls and the number of `recv()` calls on the server required to receive it.

This is the client side:

```

import socket
import sys

HOST, PORT = "localhost", 9999
data = " ".join(sys.argv[1:])

# Create a socket (SOCK_STREAM means a TCP socket)
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
    # Connect to server and send data
    sock.connect((HOST, PORT))
    sock.sendall(bytes(data, "utf-8"))
    sock.sendall(b"\n")

    # Receive data from the server and shut down
    received = str(sock.recv(1024), "utf-8")

print("Sent: ", data)
print("Received:", received)

```

The output of the example should look something like this:

Server:

```

$ python TCPServer.py
127.0.0.1 wrote:
b'hello world with TCP'
127.0.0.1 wrote:
b'python is nice'

```

Client:

```

$ python TCPClient.py hello world with TCP
Sent:      hello world with TCP
Received:  HELLO WORLD WITH TCP
$ python TCPClient.py python is nice
Sent:      python is nice
Received:  PYTHON IS NICE

```

socketserver.UDPServerExample

This is the server side:

```

import socketserver

class MyUDPHandler(socketserver.BaseRequestHandler):
    """
    This class works similar to the TCP handler class, except that
    self.request consists of a pair of data and client socket, and since
    there is no connection the client address must be given explicitly
    """

```



```

when sending data back via sendto().
"""

def handle(self):
    data = self.request[0].strip()
    socket = self.request[1]
    print(f"{self.client_address[0]} wrote:")
    print(data)
    socket.sendto(data.upper(), self.client_address)

if __name__ == "__main__":
    HOST, PORT = "localhost", 9999
    with socketserver.UDPServer((HOST, PORT), MyUDPHandler) as server:
        server.serve_forever()

```

This is the client side:

```

import socket
import sys

HOST, PORT = "localhost", 9999
data = " ".join(sys.argv[1:])

# SOCK_DGRAM is the socket type to use for UDP sockets
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# As you can see, there is no connect() call; UDP has no connections.
# Instead, data is directly sent to the recipient via sendto().
sock.sendto(bytes(data + "\n", "utf-8"), (HOST, PORT))
received = str(sock.recv(1024), "utf-8")

print("Sent: ", data)
print("Received:", received)

```

The output of the example should look exactly like for the TCP server example.

Asynchronous Mixins

To build asynchronous handlers, use the `ThreadingMixIn` and `ForkingMixIn` classes.

An example for the `ThreadingMixIn` class:

```

import socket
import threading
import socketserver

class ThreadedTCPRequestHandler(socketserver.BaseRequestHandler):

    def handle(self):
        data = str(self.request.recv(1024), 'ascii')
        cur_thread = threading.current_thread()
        response = bytes("{}: {}".format(cur_thread.name, data), 'ascii')
        self.request.sendall(response)

class ThreadedTCPServer(socketserver.ThreadingMixIn, socketserver.TCPServer):
    pass

def client(ip, port, message):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.connect((ip, port))
        sock.sendall(bytes(message, 'ascii'))
        response = str(sock.recv(1024), 'ascii')
        print("Received: {}".format(response))

if __name__ == "__main__":
    # Port 0 means to select an arbitrary unused port

```

```

HOST, PORT = "localhost", 0

server = ThreadedTCPServer((HOST, PORT), ThreadedTCPRequestHandler)
with server:
    ip, port = server.server_address

    # Start a thread with the server -- that thread will then start one
    # more thread for each request
    server_thread = threading.Thread(target=server.serve_forever)
    # Exit the server thread when the main thread terminates
    server_thread.daemon = True
    server_thread.start()
    print("Server loop running in thread:", server_thread.name)

    client(ip, port, "Hello World 1")
    client(ip, port, "Hello World 2")
    client(ip, port, "Hello World 3")

server.shutdown()

```

The output of the example should look something like this:

```

$ python ThreadedTCPServer.py
Server loop running in thread: Thread-1
Received: Thread-2: Hello World 1
Received: Thread-3: Hello World 2
Received: Thread-4: Hello World 3

```

The `ForkingMixIn` class is used in the same way, except that the server will spawn a new process for each request. Available only on POSIX platforms that support `fork()`.

http.server— HTTP servers

Source code: `Lib/http/server.py`

This module defines classes for implementing HTTP servers.

■■ Advertencia: Warning `http.server` is not recommended for production. It only implements basic security checks.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

One class, `HTTPServer`, is a `socketserver.TCPServer` subclass. It creates and listens at the HTTP socket, dispatching the requests to a handler. Code to create and run the server looks like this:

```
def run(server_class=HTTPServer, handler_class=BaseHTTPRequestHandler):
    server_address = ('', 8000)
    httpd = server_class(server_address, handler_class)
    httpd.serve_forever()
```

class `http.server.HTTPServer(server_address, RequestHandlerClass)`

This class builds on the `TCPServer` class by storing the server address as instance variables named `server_name` and `server_port`. The server is accessible by the handler, typically through the handler's `server` instance variable.

class `http.server.ThreadingHTTPServer(server_address, RequestHandlerClass)`

This class is identical to `HTTPServer` but uses threads to handle requests by using the `ThreadingMixIn`. This is useful to handle web browsers pre-opening sockets, on which `HTTPServer` would wait indefinitely. Added in version 3.7.

class `http.server.HTTPSServer(server_address, RequestHandlerClass, bind_and_activate=True, *, certfile, keyfile=None, password=None, alpn_protocols=None)`

Subclass of `HTTPServer` with a wrapped socket using the `ssl` module. If the `ssl` module is not available, instantiating a `HTTPSServer` object fails with a `RuntimeError`. The `certfile` argument is the path to the SSL certificate chain file, and the `keyfile` is the path to file containing the private key. A `password` can be specified for files protected and wrapped with PKCS#8, but beware that this could possibly expose hardcoded passwords in clear. See also `ssl.SSLContext.load_cert_chain()` for additional information on `t`

class `http.server.ThreadingHTTPSServer(server_address, RequestHandlerClass, bind_and_activate=True, *, certfile, keyfile=None, password=None, alpn_protocols=None)`

This class is identical to `HTTPSServer` but uses threads to handle requests by inheriting from `ThreadingMixIn`. This is analogous to `ThreadingHTTPServer` only using `HTTPSServer`. Added in version 3.14.

The `HTTPServer`, `ThreadingHTTPServer`, `HTTPSServer` and `ThreadingHTTPSServer` must be given a `RequestHandlerClass` on instantiation, of which this module provides three different variants:

class `http.server.BaseHTTPRequestHandler(request, client_address, server)`

client_address

server

close_connection

requestline

command

path
request_version
headers
rfile
wfile
server_version
sys_version
error_message_format
error_content_type
protocol_version
MessageClass
responses
handle()
handle_one_request()
handle_expect_100()
send_error(code,message=None,explain=None)
send_response(code,message=None)
send_header(keyword,value)
send_response_only(code,message=None)
end_headers()
flush_headers()
log_request(code='-',size='-')
log_error(...)
log_message(format,...)
version_string()
date_time_string(timestamp=None)
log_date_time_string()
address_string()

This class is used to handle the HTTP requests that arrive at the server. By itself, it cannot respond to any actual HTTP requests; it must be subclassed to handle each request method (e.g. GET or POST). `BaseHTTPRequestHandler` provides a number of class and instance variables, and methods for use by subclasses. The handler will parse the request and the headers, then call a method specific to the request type. The method name is constructed from the request. For example, for the request method `SPAM`

Contains a tuple of the form (host,port) referring to the client's address.

Contains the server instance.

Boolean that should be set before `handle_one_request()` returns, indicating if another request may be expected, or if the connection should be shut down.

Contains the string representation of the HTTP request line. The terminating CRLF is stripped. This attribute should be set by `handle_one_request()`. If no valid request line was processed, it should be set to the empty string.

Contains the command (request type). For example, 'GET'.

Contains the request path. If query component of the URL is present, then path includes the query. Using the terminology of RFC 3986, path here includes hier-part and the query.

Contains the version string from the request. For example, 'HTTP/1.0'.

Holds an instance of the class specified by the `MessageClass` class variable. This instance parses and manages the headers in the HTTP request. The `parse_headers()` function from `http.client` is used to parse the headers and it requires that the HTTP request provide a valid RFC 5322 style header.

Anio.BufferedIOBase input stream, ready to read from the start of the optional input data.

Contains the output stream for writing a response back to the client. Proper adherence to the HTTP protocol must be used when writing to this stream in order to achieve successful interoperability with HTTP clients. Changed in version 3.6: This is anio.BufferedIOBase stream.

Specifies the server software version. You may want to override this. The format is multiple whitespace-separated strings, where each string is of the form `name[/version]`. For example, 'BaseHTTP/0.2'.

Contains the Python system version, in a form usable by the `version_string` method and the `server_version` class variable. For example, 'Python/1.4'.

Specifies a format string that should be used by `send_error()` method for building an error response to the client. The string is filled by default with variables from responses based on the status code that passed to `send_error()`.

Specifies the Content-Type HTTP header of error responses sent to the client. The default value is 'text/html'.

Specifies the HTTP version to which the server is conformant. It is sent in responses to let the client know the server's communication capabilities for future requests. If set to 'HTTP/1.1', the server will permit HTTP persistent connections; however, your server must then include an accurate Content-Length header (using `send_header()`) in all of its responses to clients. For backwards compatibility, the setting defaults to 'HTTP/1.0'.

Specifies an `email.message.Message`-like class to parse HTTP headers. Typically, this is not overridden, and it defaults to `http.client.HTTPMessage`.

This attribute contains a mapping of error code integers to two-element tuples containing a short and long message. For example, `{code:(shortmessage,longmessage)}`. The `shortmessage` is usually used as the `message` key in an error response, and `longmessage` as the `explain` key. It is used by `send_response_only()` and `send_error()` methods.

Call `handle_one_request()` once (or, if persistent connections are enabled, multiple times) to handle incoming HTTP requests. You should never need to override it; instead, implement appropriate `do_*` methods.

This method will parse and dispatch the request to the appropriate `do_*` method. You should never need to override it.

When an HTTP/1.1 conformant server receives an `Expect:100-continue` request header it responds back with a `100Continue` followed by `200OK` headers. This method can be overridden to raise an error if the server does not want the client to continue. For e.g. server can choose to send `417ExpectationFailed` as a response header and return `False`. Added in version 3.2.

Sends and logs a complete error reply to the client. The `numericcode` specifies the HTTP error code, with `message` as an optional, short, human readable description of the error. The `explain` argument can be used to

provide more detailed information about the error; it will be formatted using the `error_message_format` attribute and emitted, after a complete set of headers, as the response body. The `responses` attribute holds the default values for `message` and `explain` that will be used if no value is provided; f

Adds a response header to the headers buffer and logs the accepted request. The HTTP response line is written to the internal buffer, followed by `Server` and `Date` headers. The values for these two headers are picked up from the `version_string()` and `date_time_string()` methods, respectively. If the server does not intend to send any other headers using the `send_header()` method, then `send_response()` should be followed by `anend_headers()` call. Changed in version 3.3: Headers are stored to an internal buffer and `end_`

Adds the HTTP header to an internal buffer which will be written to the output stream when either `end_headers()` or `flush_headers()` is invoked. `keyword` should specify the header keyword, with `value` specifying its value. Note that, after the `send_header` calls are done, `end_headers()` MUST BE called in order to complete the operation. Changed in version 3.2: Headers are stored in an internal buffer.

Sends the response header only, used for the purposes when 100 Continue response is sent by the server to the client. The headers not buffered and sent directly to the output stream. If the `message` is not specified, the HTTP message corresponding to the `response_code` is sent. Added in version 3.2.

Adds a blank line (indicating the end of the HTTP headers in the response) to the headers buffer and calls `flush_headers()`. Changed in version 3.2: The buffered headers are written to the output stream.

Finally send the headers to the output stream and flush the internal headers buffer. Added in version 3.3.

Logs an accepted (successful) request. `code` should specify the numeric HTTP code associated with the response. If a size of the response is available, then it should be passed as the `size` parameter.

Logs an error when a request cannot be fulfilled. By default, it passes the message to `log_message()`, so it takes the same arguments (format and additional values).

Logs an arbitrary message to `sys.stderr`. This is typically overridden to create custom error logging mechanisms. The `format` argument is a standard printf-style format string, where the additional arguments to `log_message()` are applied as inputs to the formatting. The client ip address and current date and time are prefixed to every message logged.

Returns the server software's version string. This is a combination of the `server_version` and `sys_version` attributes.

Returns the date and time given by `timestamp` (which must be `None` or in the format returned by `time.time()`), formatted for a message header. If `timestamp` is omitted, it uses the current date and time. The result looks like 'Sun,06Nov199408:49:37GMT'.

Returns the current date and time, formatted for logging.

Returns the client address. Changed in version 3.3: Previously, a name lookup was performed. To avoid name resolution delays, it now always returns the IP address.

`class http.server.SimpleHTTPRequestHandler(request, client_address, server, directory=None)`

`server_version`

`extensions_map`

`do_HEAD()`

`do_GET()`

This class serves files from the directory `directory` and below, or the current directory if `directory` is not provided, directly mapping the directory structure to HTTP requests. Changed in version 3.7: Added the `directory` parameter. Changed in version 3.9: The `directory` parameter accepts a path-like object. A lot of the work, such as parsing the request, is done by the base class `BaseHTTPRequestHandler`. This class implements

thedo_GET()anddo_HEAD()functions.The following are defined as class-level attributes

This will be"SimpleHTTP/"+"__version__", where__version__is defined at the module level.

A dictionary mapping suffixes into MIME types, contains custom overrides for the default system mappings. The mapping is used case-insensitively, and so should contain only lower-cased keys.Changed in version 3.9:This dictionary is no longer filled with the default system mappings, but only contains overrides.

This method serves the'HEAD'request type: it sends the headers it would send for the equivalentGETrequest. See thedo_GET()method for a more complete explanation of the possible headers.

The request is mapped to a local file by interpreting the request as a path relative to the current working directory.If the request was mapped to a directory, the directory is checked for a file namedindex.htmlorindex.htm(in that order). If found, the file's contents are returned; otherwise a directory listing is generated by calling thelist_directory()method. This method usesos.listdir()to scan the directory, and returns a404error response if thelistdir()fails.If the request was mapped to a fi

TheSimpleHTTPRequestHandlerclass can be used in the following manner in order to create a very basic webserver serving files relative to the current directory:

```
import http.server
import socketserver

PORT = 8000

Handler = http.server.SimpleHTTPRequestHandler

with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print("serving at port", PORT)
    httpd.serve_forever()
```

SimpleHTTPRequestHandlercan also be subclassed to enhance behavior, such as using different index file names by overriding the class attributeindex_pages.

classhttp.server.CGIHTTPRequestHandler(request,client_address,server)

cgi_directories

do_POST()

This class is used to serve either files or output of CGI scripts from the current directory and below. Note that mapping HTTP hierarchic structure to local directory structure is exactly as inSimpleHTTPRequestHandler.NoteCGI scripts run by theCGIHTTPRequestHandlerclass cannot execute redirects (HTTP code 302), because code 200 (script output follows) is sent prior to execution of the CGI script. This pre-empts the status code.The class will however, run the CGI script, instead of serving it as

This defaults to['/cgi-bin','/htbin']and describes directories to treat as containing CGI scripts.

This method serves the'POST'request type, only allowed for CGI scripts. Error 501, "Can only POST to CGI scripts", is output when trying to POST to a non-CGI url.

Command-line interface

http.servercan also be invoked directly using the-mswitch of the interpreter. The following example illustrates how to serve files relative to the current directory:

```
python -m http.server [OPTIONS] [port]
```

The following options are accepted:

port

The server listens to port 8000 by default. The default can be overridden by passing the desired port number as an argument: `python-mhttp.server9000`

-b,--bind<address>

Specifies a specific address to which it should bind. Both IPv4 and IPv6 addresses are supported. By default, the server binds itself to all interfaces. For example, the following command causes the server to bind to localhost only: `python-mhttp.server--bind127.0.0.1` Added in version 3.4. Changed in version 3.8: Support IPv6 in the `--bind` option.

-d,--directory<dir>

Specifies a directory to which it should serve the files. By default, the server uses the current directory. For example, the following command uses a specific directory: `python-mhttp.server--directory/tmp/` Added in version 3.7.

-p,--protocol<version>

Specifies the HTTP version to which the server is conformant. By default, the server is conformant to HTTP/1.0. For example, the following command runs an HTTP/1.1 conformant server: `python-mhttp.server--protocolHTTP/1.1` Added in version 3.11.

--cgi

CGIHTTPRequestHandler can be enabled in the command line by passing the `--cgi` option: `python-mhttp.server--cgi` Deprecated since version 3.13, will be removed in version 3.15: `http.server` command line `--cgi` support is being removed because CGIHTTPRequestHandler is being removed.

■■ Advertencia: Warning CGIHTTPRequestHandler and the `--cgi` command-line option are not intended for use by untrusted clients and may be vulnerable to exploitation. Always use within a secure environment.

--tls-cert

Specifies a TLS certificate chain for HTTPS connections: `python-mhttp.server--tls-certfullchain.pem` Added in version 3.14.

--tls-key

Specifies a private key file for HTTPS connections. This option requires `--tls-cert` to be specified. Added in version 3.14.

--tls-password-file

Specifies the password file for password-protected private keys: `python-mhttp.server--tls-certcert.pem--tls-keykey.pem--tls-password-filepassword.txt` This option requires `--tls-cert` to be specified. Added in version 3.14.

Security considerations

SimpleHTTPRequestHandler will follow symbolic links when handling requests, this makes it possible for files outside of the specified directory to be served.

Earlier versions of Python did not scrub control characters from the log messages emitted to stderr from `python-mhttp.server` or the default `BaseHTTPRequestHandler.log_message` implementation. This could allow remote clients connecting to your server to send nefarious control codes to your terminal.

Changed in version 3.12: Control characters are scrubbed in stderr logs.

http.cookies— HTTP state management

Source code: `Lib/http/cookies.py`

The `http.cookies` module defines classes for abstracting the concept of cookies, an HTTP state management mechanism. It supports both simple string-only cookies, and provides an abstraction for having any serializable data-type as cookie value.

The module formerly strictly applied the parsing rules described in the RFC 2109 and RFC 2068 specifications. It has since been discovered that MSIE 3.0x didn't follow the character rules outlined in those specs; many current-day browsers and servers have also relaxed parsing rules when it comes to cookie handling. As a result, this module now uses parsing rules that are a bit less strict than they once were.

The character set, `string.ascii_letters`, `string.digits` and `!#$%&'*+-.^_`|~` denote the set of valid characters allowed by this module in a cookie name (as key).

Changed in version 3.3: Allowed `'` as a valid cookie name character.

■ Nota: Note On encountering an invalid cookie, `CookieError` is raised, so if your cookie data comes from a browser you should always prepare for invalid data and catch `CookieError` on parsing.

exception `http.cookies.CookieError`

Exception failing because of RFC 2109 invalidity: incorrect attributes, incorrect `Set-Cookie` header, etc.

class `http.cookies.BaseCookie([input])`

This class is a dictionary-like object whose keys are strings and whose values are `Morsel` instances. Note that upon setting a key to a value, the value is first converted to a `Morsel` containing the key and the value. If `input` is given, it is passed to the `load()` method.

class `http.cookies.SimpleCookie([input])`

This class derives from `BaseCookie` and overrides `value_decode()` and `value_encode()`. `SimpleCookie` supports strings as cookie values. When setting the value, `SimpleCookie` calls the builtin `str()` to convert the value to a string. Values received from HTTP are kept as strings.

See also

Module `http.cookiejar`

RFC 2109- HTTP State Management Mechanism

HTTP cookie handling for webclients. The `http.cookiejar` and `http.cookies` modules do not depend on each other.

This is the state management specification implemented by this module.

Cookie Objects

`BaseCookie.value_decode(val)`

Return a tuple (`real_value`, `coded_value`) from a string representation. `real_value` can be any type. This method does no decoding in `BaseCookie`— it exists so it can be overridden.

`BaseCookie.value_encode(val)`

Return a tuple (`real_value`, `coded_value`). `val` can be any type, but `coded_value` will always be converted to a string. This method does no encoding in `BaseCookie`— it exists so it can be overridden. In general, it should be the case that `value_encode()` and `value_decode()` are inverses on the range of `value_decode`.

`BaseCookie.output(attrs=None, header='Set-Cookie:', sep='\r\n')`

Return a string representation suitable to be sent as HTTP headers. `attrs` and `header` are sent to each `Morsel.output()` method. `sep` is used to join the headers together, and is by default the combination `"\r\n"` (CRLF).

BaseCookie.js_output(attrs=None)

Return an embeddable JavaScript snippet, which, if run on a browser which supports JavaScript, will act the same as if the HTTP headers was sent. The meaning for `attrs` is the same as in `output()`.

BaseCookie.load(rawdata)

If `rawdata` is a string, parse it as an HTTP_COOKIE and add the values found there as `Morsels`. If it is a dictionary, it is equivalent to: `for k, v in rawdata.items(): cookie[k]=v`

Morsel Objects

class http.cookies.Morsel

expires

path

comment

domain

max-age

secure

version

httponly

samesite

partitioned

Abstract a key/value pair, which has some RFC 2109 attributes. `Morsels` are dictionary-like objects, whose set of keys is constant — the valid RFC 2109 attributes, which are: `expires`, `path`, `comment`, `domain`, `max-age`, `secure`, `version`, `httponly`, `samesite`, `partitioned`. The attribute `httponly` specifies that the cookie is only transferred in HTTP requests, and is not accessible through JavaScript. This is intended to mitigate some forms of cross-site scripting. The attribute `samesite` controls when the browser sends the cookie with c

Morsel.value

The value of the cookie.

Morsel.coded_value

The encoded value of the cookie — this is what should be sent.

Morsel.key

The name of the cookie.

Morsel.set(key, value, coded_value)

Set the `key`, `value` and `coded_value` attributes.

Morsel.isReservedKey(K)

Whether `K` is a member of the set of keys of a `Morsel`.

Morsel.output(attrs=None,header='Set-Cookie:')

Return a string representation of the Morsel, suitable to be sent as an HTTP header. By default, all the attributes are included, unless `attrs` is given, in which case it should be a list of attributes to use. `header` is by default "Set-Cookie:".

Morsel.js_output(attrs=None)

Return an embeddable JavaScript snippet, which, if run on a browser which supports JavaScript, will act the same as if the HTTP header was sent. The meaning for `attrs` is the same as in `output()`.

Morsel.OutputString(attrs=None)

Return a string representing the Morsel, without any surrounding HTTP or JavaScript. The meaning for `attrs` is the same as in `output()`.

Morsel.update(values)

Update the values in the Morsel dictionary with the values in the dictionary `values`. Raise an error if any of the keys in the `values` dict is not a valid RFC 2109 attribute. Changed in version 3.5: an error is raised for invalid keys.

Morsel.copy(value)

Return a shallow copy of the Morsel object. Changed in version 3.5: return a Morsel object instead of a dict.

Morsel.setdefault(key,value=None)

Raise an error if key is not a valid RFC 2109 attribute, otherwise behave the same as `dict.setdefault()`.

Example

The following example demonstrates how to use the `http.cookies` module.

```
>>> from http import cookies
>>> C = cookies.SimpleCookie()
>>> C["fig"] = "newton"
>>> C["sugar"] = "wafer"
>>> print(C) # generate HTTP headers
Set-Cookie: fig=newton
Set-Cookie: sugar=wafer
>>> print(C.output()) # same thing
Set-Cookie: fig=newton
Set-Cookie: sugar=wafer
>>> C = cookies.SimpleCookie()
>>> C["rocky"] = "road"
>>> C["rocky"]["path"] = "/cookie"
>>> print(C.output(header="Cookie:"))
Cookie: rocky=road; Path=/cookie
>>> print(C.output(attrs=[], header="Cookie:"))
Cookie: rocky=road
>>> C = cookies.SimpleCookie()
>>> C.load("chips=ahoy; vienna=finger") # load from a string (HTTP header)
>>> print(C)
Set-Cookie: chips=ahoy
Set-Cookie: vienna=finger
>>> C = cookies.SimpleCookie()
>>> C.load('keebler="E=everybody; L=\\"Loves\\"; fudge=;";')
>>> print(C)
Set-Cookie: keebler="E=everybody; L=\\"Loves\\"; fudge=;"
>>> C = cookies.SimpleCookie()
>>> C["oreo"] = "doublestuff"
>>> C["oreo"]["path"] = "/"
>>> print(C)
Set-Cookie: oreo=doublestuff; Path=/"
```

```
>>> C = cookies.SimpleCookie()
>>> C["twix"] = "none for you"
>>> C["twix"].value
'none for you'
>>> C = cookies.SimpleCookie()
>>> C["number"] = 7 # equivalent to C["number"] = str(7)
>>> C["string"] = "seven"
>>> C["number"].value
'7'
>>> C["string"].value
'seven'
>>> print(C)
Set-Cookie: number=7
Set-Cookie: string=seven
```

http.cookiejar— Cookie handling for HTTP clients

Source code: Lib/http/cookiejar.py

The `http.cookiejar` module defines classes for automatic handling of HTTP cookies. It is useful for accessing websites that require small pieces of data –cookies– to be set on the client machine by an HTTP response from a web server, and then returned to the server in later HTTP requests.

Both the regular Netscape cookie protocol and the protocol defined by RFC 2965 are handled. RFC 2965 handling is switched off by default. RFC 2109 cookies are parsed as Netscape cookies and subsequently treated either as Netscape or RFC 2965 cookies according to the 'policy' in effect. Note that the great majority of cookies on the internet are Netscape cookies. `http.cookiejar` attempts to follow the de-facto Netscape cookie protocol (which differs substantially from that set out in the original Netscape specification), including taking note of the `max-age` and `port` cookie attributes introduced with RFC 2965.

■ Nota: Note The various named parameters found in `Set-Cookie` and `Set-Cookie2` headers (eg. `domain` and `expires`) are conventionally referred to as attributes. To distinguish them from Python attributes, the documentation for this module uses the term `cookie-attribute` instead.

The module defines the following exception:

exception `http.cookiejar.LoadError`

Instances of `FileCookieJar` raise this exception on failure to load cookies from a file. `LoadError` is a subclass of `OSError`. Changed in version 3.3: `LoadError` used to be a subtype of `IOError`, which is now an alias of `OSError`.

The following classes are provided:

class `http.cookiejar.CookieJar(policy=None)`

`policy` is an object implementing the `CookiePolicy` interface. The `CookieJar` class stores HTTP cookies. It extracts cookies from HTTP requests, and returns them in HTTP responses. `CookieJar` instances automatically expire contained cookies when necessary. Subclasses are also responsible for storing and retrieving cookies from a file or database.

class `http.cookiejar.FileCookieJar(filename=None, delayload=None, policy=None)`

`policy` is an object implementing the `CookiePolicy` interface. For the other arguments, see the documentation for the corresponding attributes. A `CookieJar` which can load cookies from, and perhaps save cookies to, a file on disk. Cookies are NOT loaded from the named file until either the `load()` or `revert()` method is called. Subclasses of this class are documented in section `FileCookieJar` subclasses and co-operation with web browsers. This should not be initialized directly – use its subclasses below instead

class `http.cookiejar.CookiePolicy`

This class is responsible for deciding whether each cookie should be accepted from / returned to the server.

class `http.cookiejar.DefaultCookiePolicy(blocked_domains=None, allowed_domains=None, netscape=True, rfc2965=False, rfc2109_as_netscape=None, hide_cookie2=False, strict_domain=False, strict_rfc2965_unverifiable=True, strict_ns_unverifiable=False, strict_ns_domain=DefaultCookiePolicy.DomainLiberal, strict_ns_set`

Constructor arguments should be passed as keyword arguments only. `blocked_domains` is a sequence of domain names that we never accept cookies from, nor return cookies to. `allowed_domains` if not `None`, this is a sequence of the only domains for which we accept and return cookies. `secure_protocols` is a sequence of protocols for which secure cookies can be added to. By default `https` and `wss` (secure websocket) are considered secure protocols. For all other arguments, see the documentation for `CookiePolicy` and `DefaultCookiePolicy`.

class `http.cookiejar.Cookie`

This class represents Netscape, RFC 2109 and RFC 2965 cookies. It is not expected that users of `http.cookiejar` construct their own `Cookie` instances. Instead, if necessary, call `make_cookies()` on a `CookieJar` instance.

See also

Module `urllib.request`

Module `http.cookies`

https://curl.se/rfc/cookie_spec.html

RFC 2109- HTTP State Management Mechanism

RFC 2965- HTTP State Management Mechanism

<https://kristol.org/cookie/errata.html>

URL opening with automatic cookie handling.

HTTP cookie classes, principally useful for server-side code. The `http.cookiejar` and `http.cookies` modules do not depend on each other.

The specification of the original Netscape cookie protocol. Though this is still the dominant protocol, the 'Netscape cookie protocol' implemented by all the major browsers (and `http.cookiejar`) only bears a passing resemblance to the one sketched out in `cookie_spec.html`.

Obsoleted by RFC 2965. Uses `Set-Cookie` with `version=1`.

The Netscape protocol with the bugs fixed. Uses `Set-Cookie2` in place of `Set-Cookie`. Not widely used.

Unfinished errata to RFC 2965.

RFC 2964- Use of HTTP State Management

CookieJar and FileCookieJar Objects

`CookieJar` objects support the iterator protocol for iterating over contained `Cookie` objects.

`CookieJar` has the following methods:

`CookieJar.add_cookie_header(request)`

Add correct `Cookie` header to `request`. If policy allows (ie. the `rfc2965` and `hide_cookie2` attributes of the `CookieJar`'s `CookiePolicy` instance are true and false respectively), the `Cookie2` header is also added when appropriate. The `request` object (usually `urllib.request.Request` instance) must support the methods `get_full_url()`, `has_header()`, `get_header()`, `header_items()`, `add_unredirected_header()` and the attributes `host`, `type`, `unverifiable` and `origin_req_host` as documented by `urllib.request`. Changed in version 3.3: `request` object

`CookieJar.extract_cookies(response, request)`

Extract cookies from `HTTPResponse` and store them in the `CookieJar`, where allowed by policy. The `CookieJar` will look for allowable `Set-Cookie` and `Set-Cookie2` headers in the `response` argument, and store cookies as appropriate (subject to the `CookiePolicy.set_ok()` method's approval). The `response` object (usually the result of a call to `urllib.request.urlopen()`, or similar) should support an `info()` method, which returns an `email.message.Message` instance. The `request` object (usually `urllib.request.Request` instance) must sup

`CookieJar.set_policy(policy)`

Set the `CookiePolicy` instance to be used.

`CookieJar.make_cookies(response, request)`

Return sequence of `Cookie` objects extracted from `response` object. See the documentation for `extract_cookies()` for the interfaces required of the `response` and `request` arguments.

`CookieJar.set_cookie_if_ok(cookie,request)`

Set a `Cookie` if policy says it's OK to do so.

`CookieJar.set_cookie(cookie)`

Set a `Cookie`, without checking with policy to see whether or not it should be set.

`CookieJar.clear([domain[,path[,name]]])`

Clear some cookies. If invoked without arguments, clear all cookies. If given a single argument, only cookies belonging to that domain will be removed. If given two arguments, cookies belonging to the specified domain and URL path are removed. If given three arguments, then the cookie with the specified domain, path and name is removed. Raises `KeyError` if no matching cookie exists.

`CookieJar.clear_session_cookies()`

Discard all session cookies. Discards all contained cookies that have a `true` `discard` attribute (usually because they had either `no-max-age` or `expires=cookie-attribute`, or an explicit `discard=cookie-attribute`). For interactive browsers, the end of a session usually corresponds to closing the browser window. Note that the `save()` method won't save session cookies anyway, unless you ask otherwise by passing a `true` `ignore_discard` argument.

`FileCookieJar` implements the following additional methods:

`FileCookieJar.save(filename=None,ignore_discard=False,ignore_expires=False)`

Save cookies to a file. This base class raises `NotImplementedError`. Subclasses may leave this method unimplemented. `filename` is the name of file in which to save cookies. If `filename` is not specified, `self.filename` is used (whose default is the value passed to the constructor, if any); if `self.filename` is `None`, `ValueError` is raised. `ignore_discard`: save even cookies set to be discarded. `ignore_expires`: save even cookies that have expired. The file is overwritten if it already exists, thus wiping all the cookie

`FileCookieJar.load(filename=None,ignore_discard=False,ignore_expires=False)`

Load cookies from a file. Old cookies are kept unless overwritten by newly loaded ones. Arguments are as for `save()`. The named file must be in the format understood by the class, or `LoadError` will be raised. Also, `OSErr` may be raised, for example if the file does not exist. Changed in version 3.3: `IOError` used to be raised, it is now an alias of `OSErr`.

`FileCookieJar.revert(filename=None,ignore_discard=False,ignore_expires=False)`

Clear all cookies and reload cookies from a saved file. `revert()` can raise the same exceptions as `load()`. If there is a failure, the object's state will not be altered.

`FileCookieJar` instances have the following public attributes:

`FileCookieJar.filename`

Filename of default file in which to keep cookies. This attribute may be assigned to.

`FileCookieJar.delayload`

If `true`, load cookies lazily from disk. This attribute should not be assigned to. This is only a hint, since this only affects performance, not behaviour (unless the cookies on disk are changing). A `CookieJar` object may ignore it. None of the `FileCookieJar` classes included in the standard library lazily loads cookies.

FileCookieJar subclasses and co-operation with web browsers

The following `CookieJar` subclasses are provided for reading and writing.

classhttp.cookiejar.MozillaCookieJar(filename=None,delayload=None,policy=None)

AFileCookieJarthat can load from and save cookies to disk in the Mozillacookies.txtfile format (which is also used by curl and the Lynx and Netscape browsers).NoteThis loses information aboutRFC 2965cookies, and also about newer or non-standard cookie-attributes such asport.WarningBack up your cookies before saving if you have cookies whose loss / corruption would be inconvenient (there are some subtleties which may lead to slight changes in the file over a load / save round-trip).Also note that

classhttp.cookiejar.LWPCookieJar(filename=None,delayload=None,policy=None)

AFileCookieJarthat can load from and save cookies to disk in format compatible with the libwww-perl library'sSet-Cookie3file format. This is convenient if you want to store cookies in a human-readable file.Changed in version 3.8:The filename parameter supports a path-like object.

CookiePolicy Objects

Objects implementing theCookiePolicyinterface have the following methods:

CookiePolicy.set_ok(cookie,request)

Return boolean value indicating whether cookie should be accepted from server.cookieis aCookieinstance.requestis an object implementing the interface defined by the documentation forCookieJar.extract_cookies().

CookiePolicy.return_ok(cookie,request)

Return boolean value indicating whether cookie should be returned to server.cookieis aCookieinstance.requestis an object implementing the interface defined by the documentation forCookieJar.add_cookie_header().

CookiePolicy.domain_return_ok(domain,request)

ReturnFalseif cookies should not be returned, given cookie domain.This method is an optimization. It removes the need for checking every cookie with a particular domain (which might involve reading many files). Returning true fromdomain_return_ok()andpath_return_ok()leaves all the work toreturn_ok().Ifdomain_return_ok()returns true for the cookie domain,path_return_ok()is called for the cookie path. Otherwise,path_return_ok()andreturn_ok()are never called for that cookie domain. Ifpath_retur

CookiePolicy.path_return_ok(path,request)

ReturnFalseif cookies should not be returned, given cookie path.See the documentation fordomain_return_ok().

In addition to implementing the methods above, implementations of theCookiePolicyinterface must also supply the following attributes, indicating which protocols should be used, and how. All of these attributes may be assigned to.

CookiePolicy.netscape

Implement Netscape protocol.

CookiePolicy.rfc2965

ImplementRFC 2965protocol.

CookiePolicy.hide_cookie2

Don't addCookie2header to requests (the presence of this header indicates to the server that we understandRFC 2965cookies).

The most useful way to define a `CookiePolicy` class is by subclassing from `DefaultCookiePolicy` and overriding some or all of the methods above. `CookiePolicy` itself may be used as a 'null policy' to allow setting and receiving any and all cookies (this is unlikely to be useful).

DefaultCookiePolicy Objects

Implements the standard rules for accepting and returning cookies.

Both RFC 2965 and Netscape cookies are covered. RFC 2965 handling is switched off by default.

The easiest way to provide your own policy is to override this class and call its methods in your overridden implementations before adding your own additional checks:

```
import http.cookiejar
class MyCookiePolicy(http.cookiejar.DefaultCookiePolicy):
    def set_ok(self, cookie, request):
        if not http.cookiejar.DefaultCookiePolicy.set_ok(self, cookie, request):
            return False
        if i_dont_want_to_store_this_cookie(cookie):
            return False
        return True
```

In addition to the features required to implement the `CookiePolicy` interface, this class allows you to block and allow domains from setting and receiving cookies. There are also some strictness switches that allow you to tighten up the rather loose Netscape protocol rules a little bit (at the cost of blocking some benign cookies).

A domain blocklist and allowlist is provided (both off by default). Only domains not in the blocklist and present in the allowlist (if the allowlist is active) participate in cookie setting and returning. Use the `blocked_domains` constructor argument, and `blocked_domains()` and `set_blocked_domains()` methods (and the corresponding argument and methods for `allowed_domains`). If you set an allowlist, you can turn it off again by setting it to `None`.

Domains in block or allow lists that do not start with a dot must equal the cookie domain to be matched. For example, "example.com" matches a blocklist entry of "example.com", but "www.example.com" does not. Domains that do start with a dot are matched by more specific domains too. For example, both "www.example.com" and "www.coyote.example.com" match ".example.com" (but "example.com" itself does not). IP addresses are an exception, and must match exactly. For example, if `blocked_domains` contains "192.168.1.2" and ".168.1.2", 192.168.1.2 is blocked, but 193.168.1.2 is not.

`DefaultCookiePolicy` implements the following additional methods:

DefaultCookiePolicy.blocked_domains()

Return the sequence of blocked domains (as a tuple).

DefaultCookiePolicy.set_blocked_domains(blocked_domains)

Set the sequence of blocked domains.

DefaultCookiePolicy.is_blocked(domain)

Return `True` if domain is on the blocklist for setting or receiving cookies.

DefaultCookiePolicy.allowed_domains()

Return `None`, or the sequence of allowed domains (as a tuple).

DefaultCookiePolicy.set_allowed_domains(allowed_domains)

Set the sequence of allowed domains, or `None`.

DefaultCookiePolicy.is_not_allowed(domain)

ReturnTrueifdomainis not on the allowlist for setting or receiving cookies.

DefaultCookiePolicyinstances have the following attributes, which are all initialised from the constructor arguments of the same name, and which may all be assigned to.

DefaultCookiePolicy.rfc2109_as_netscape

If true, request that theCookieJarinstance downgradeRFC 2109cookies (ie. cookies received in aSet-Cookieheader with a version cookie-attribute of 1) to Netscape cookies by setting the version attribute of theCookieinstance to 0. The default value isNone, in which case RFC 2109 cookies are downgraded if and only ifRFC 2965handling is turned off. Therefore, RFC 2109 cookies are downgraded by default.

General strictness switches:

DefaultCookiePolicy.strict_domain

Don't allow sites to set two-component domains with country-code top-level domains like.co.uk,.gov.uk,.co.nz.etc. This is far from perfect and isn't guaranteed to work!

RFC 2965protocol strictness switches:

DefaultCookiePolicy.strict_rfc2965_unverifiable

FollowRFC 2965rules on unverifiable transactions (usually, an unverifiable transaction is one resulting from a redirect or a request for an image hosted on another site). If this is false, cookies areneverblocked on the basis of verifiability

Netscape protocol strictness switches:

DefaultCookiePolicy.strict_ns_unverifiable

ApplyRFC 2965rules on unverifiable transactions even to Netscape cookies.

DefaultCookiePolicy.strict_ns_domain

Flags indicating how strict to be with domain-matching rules for Netscape cookies. See below for acceptable values.

DefaultCookiePolicy.strict_ns_set_initial_dollar

Ignore cookies in Set-Cookie: headers that have names starting with'\$'.

DefaultCookiePolicy.strict_ns_set_path

Don't allow setting cookies whose path doesn't path-match request URI.

strict_ns_domainis a collection of flags. Its value is constructed by or-ing together (for example,DomainStrictNoDots|DomainStrictNonDomainmeans both flags are set).

DefaultCookiePolicy.DomainStrictNoDots

When setting cookies, the 'host prefix' must not contain a dot (eg.www.foo.bar.comcan't set a cookie for.bar.com, becausewww.foocontains a dot).

DefaultCookiePolicy.DomainStrictNonDomain

Cookies that did not explicitly specify adomaincookie-attribute can only be returned to a domain equal to the domain that set the cookie (eg.spam.example.comwon't be returned cookies fromexample.comthat had nodomaincookie-attribute).

DefaultCookiePolicy.DomainRFC2965Match

When setting cookies, require a fullRFC 2965domain-match.

The following attributes are provided for convenience, and are the most useful combinations of the above flags:

DefaultCookiePolicy.DomainLiberal

Equivalent to 0 (ie. all of the above Netscape domain strictness flags switched off).

DefaultCookiePolicy.DomainStrict

Equivalent to DomainStrictNoDots|DomainStrictNonDomain.

Cookie Objects

Cookie instances have Python attributes roughly corresponding to the standard cookie-attributes specified in the various cookie standards. The correspondence is not one-to-one, because there are complicated rules for assigning default values, because the max-age and expires cookie-attributes contain equivalent information, and because RFC 2109 cookies may be 'downgraded' by http.cookiejar from version 1 to version 0 (Netscape) cookies.

Assignment to these attributes should not be necessary other than in rare circumstances in a CookiePolicy method. The class does not enforce internal consistency, so you should know what you're doing if you do that.

Cookie.version

Integer or None. Netscape cookies have version 0. RFC 2965 and RFC 2109 cookies have a version cookie-attribute of 1. However, note that http.cookiejar may 'downgrade' RFC 2109 cookies to Netscape cookies, in which case version is 0.

Cookie.name

Cookie name (a string).

Cookie.value

Cookie value (a string), or None.

Cookie.port

String representing a port or a set of ports (eg. '80', or '80,8080'), or None.

Cookie.domain

Cookie domain (a string).

Cookie.path

Cookie path (a string, eg. '/acme/rocket_launchers').

Cookie.secure

True if cookie should only be returned over a secure connection.

Cookie.expires

Integer expiry date in seconds since epoch, or None. See also the is_expired() method.

Cookie.discard

True if this is a session cookie.

Cookie.comment

String comment from the server explaining the function of this cookie, or None.

Cookie.comment_url

URL linking to a comment from the server explaining the function of this cookie, or None.

Cookie.rfc2109

True if this cookie was received as an RFC 2109 cookie (ie. the cookie arrived in a Set-Cookie header, and the value of the Version cookie-attribute in that header was 1). This attribute is provided because http.cookiejar may 'downgrade' RFC 2109 cookies to Netscape cookies, in which case version is 0.

Cookie.port_specified

True if a port or set of ports was explicitly specified by the server (in the Set-Cookie/Set-Cookie2 header).

Cookie.domain_specified

True if a domain was explicitly specified by the server.

Cookie.domain_initial_dot

True if the domain explicitly specified by the server began with a dot ('.').

Cookies may have additional non-standard cookie-attributes. These may be accessed using the following methods:

Cookie.has_nonstandard_attr(name)

Return True if cookie has the named cookie-attribute.

Cookie.get_nonstandard_attr(name, default=None)

If cookie has the named cookie-attribute, return its value. Otherwise, return default.

Cookie.set_nonstandard_attr(name, value)

Set the value of the named cookie-attribute.

The Cookie class also defines the following method:

Cookie.is_expired(now=None)

True if cookie has passed the time at which the server requested it should expire. If now is given (in seconds since the epoch), return whether the cookie has expired at the specified time.

Examples

The first example shows the most common usage of http.cookiejar:

```
import http.cookiejar, urllib.request
cj = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
r = opener.open("http://example.com/")
```

This example illustrates how to open a URL using your Netscape, Mozilla, or Lynx cookies (assumes Unix/Netscape convention for location of the cookies file):

```
import os, http.cookiejar, urllib.request
cj = http.cookiejar.MozillaCookieJar()
cj.load(os.path.join(os.path.expanduser("~"), ".netscape", "cookies.txt"))
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
r = opener.open("http://example.com/")
```

The next example illustrates the use of DefaultCookiePolicy. Turn on RFC 2965 cookies, be more strict about domains when setting and returning Netscape cookies, and block some domains from setting cookies or having them returned:

```
import urllib.request
from http.cookiejar import CookieJar, DefaultCookiePolicy
policy = DefaultCookiePolicy(
```

```
        rfc2965=True, strict_ns_domain=Policy.DomainStrict,
        blocked_domains=["ads.net", ".ads.net"])
cj = CookieJar(policy)
opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(cj))
r = opener.open("http://example.com/")
```

xmlrpc— XMLRPC server and client modules

XML-RPC is a Remote Procedure Call method that uses XML passed via HTTP as a transport. With it, a client can call methods with parameters on a remote server (the server is named by a URI) and get back structured data.

xmlrpc is a package that collects server and client modules implementing XML-RPC. The modules are:

- `xmlrpc.client`
- `xmlrpc.server`

xmlrpc.client— XML-RPC client access

Source code: `Lib/xmlrpc/client.py`

XML-RPC is a Remote Procedure Call method that uses XML passed via HTTP(S) as a transport. With it, a client can call methods with parameters on a remote server (the server is named by a URI) and get back structured data. This module supports writing XML-RPC client code; it handles all the details of translating between conformable Python objects and XML on the wire.

■■ Advertencia: WarningThe `xmlrpc.client` module is not secure against maliciously constructed data. If you need to parse untrusted or unauthenticated data, see [XML security](#).

Changed in version 3.5: For HTTPS URIs, `xmlrpc.client` now performs all the necessary certificate and hostname checks by default.

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

`class xmlrpc.client.ServerProxy(uri, transport=None, encoding=None, verbose=False, allow_none=False, use_datetime=False, use_builtin_types=False, *, headers=(), context=None)`

A `ServerProxy` instance is an object that manages communication with a remote XML-RPC server. The required first argument is a URI (Uniform Resource Indicator), and will normally be the URL of the server. The optional second argument is a transport factory instance; by default it is an internal `SafeTransport` instance for https: URLs and an internal `HTTPTransport` instance otherwise. The optional third argument is an encoding, by default UTF-8. The optional fourth argument is a debugging flag. The fol

See also

XML-RPC HOWTO

XML-RPC Introspection

XML-RPC Specification

A good description of XML-RPC operation and client software in several languages. Contains pretty much everything an XML-RPC client developer needs to know.

Describes the XML-RPC protocol extension for introspection.

The official specification.

ServerProxy Objects

A `ServerProxy` instance has a method corresponding to each remote procedure call accepted by the XML-RPC server. Calling the method performs an RPC, dispatched by both name and argument signature (e.g. the same method name can be overloaded with multiple argument signatures). The RPC finishes by returning a value, which may be either returned data in a conformant type or a `Fault` or `ProtocolError` object indicating an error.

Servers that support the XML introspection API support some common methods grouped under the `reservedsystem` attribute:

`ServerProxy.system.listMethods()`

This method returns a list of strings, one for each (non-system) method supported by the XML-RPC server.

`ServerProxy.system.methodSignature(name)`

This method takes one parameter, the name of a method implemented by the XML-RPC server. It returns an array of possible signatures for this method. A signature is an array of types. The first of these types is the return type of the method, the rest are parameters. Because multiple signatures (ie. overloading) is permitted, this method returns a list of signatures rather than a singleton. Signatures themselves are restricted to the top level parameters expected by a method. For instance if a meth

ServerProxy.system.methodHelp(name)

This method takes one parameter, the name of a method implemented by the XML-RPC server. It returns a documentation string describing the use of that method. If no such string is available, an empty string is returned. The documentation string may contain HTML markup.

Changed in version 3.5: Instances of `ServerProxy` support the context manager protocol for closing the underlying transport.

A working example follows. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

def is_even(n):
    return n % 2 == 0

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(is_even, "is_even")
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

with xmlrpc.client.ServerProxy("http://localhost:8000/") as proxy:
    print("3 is even: %s" % str(proxy.is_even(3)))
    print("100 is even: %s" % str(proxy.is_even(100)))
```

DateTime Objects

class xmlrpc.client.DateTime

decode(string)

encode(out)

This class may be initialized with seconds since the epoch, a time tuple, an ISO 8601 time/date string, or a `datetime.datetime` instance. It has the following methods, supported mainly for internal use by the marshalling/unmarshalling code:

- `decode(string)` Accept a string as the instance's new time value.
- `encode(out)` Write the XML-RPC encoding of this `DateTime` item to the `outstream` object.

It also supports certain of Python's built-in operators through `richcomparison` and `__repr__()` methods.

Accept a string as the instance's new time value.

Write the XML-RPC encoding of this `DateTime` item to the `outstream` object.

A working example follows. The server code:

```
import datetime as dt
from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client

def today():
    today = dt.datetime.today()
    return xmlrpc.client.DateTime(today)
```



```

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(today, "today")
server.serve_forever()

```

The client code for the preceding server:

```

import xmlrpc.client
import datetime as dt

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")

today = proxy.today()
# convert the ISO 8601 string to a datetime object
converted = dt.datetime.strptime(today.value, "%Y%m%dT%H:%M:%S")
print(f"Today: {converted.strftime('%d.%m.%Y, %H:%M')}")

```

Binary Objects

class xmlrpc.client.Binary

data

decode(bytes)

encode(out)

This class may be initialized from bytes data (which may include NULs). The primary access to the content of a `Binary` object is provided by an attribute: `data`. The binary data encapsulated by the `Binary` instance. The data is provided as a `bytes` object. `Binary` objects have the following methods, supported mainly for internal use by the marshalling/unmarshalling code:

- `decode(bytes)`: Accept a base64 `bytes` object and decode it as the instance's new data.
- `encode(out)`: Write the XML-RPC base 64 encoding of this binary

The binary data encapsulated by the `Binary` instance. The data is provided as a `bytes` object.

Accept a base64 `bytes` object and decode it as the instance's new data.

Write the XML-RPC base 64 encoding of this binary item to the `outstream` object. The encoded data will have newlines every 76 characters as per RFC 2045 section 6.8, which was the de facto standard base64 specification when the XML-RPC spec was written.

Example usage of the binary objects. We're going to transfer an image over XMLRPC:

```

from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client

def python_logo():
    with open("python_logo.jpg", "rb") as handle:
        return xmlrpc.client.Binary(handle.read())

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(python_logo, 'python_logo')

server.serve_forever()

```

The client gets the image and saves it to a file:

```

import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
with open("fetched_python_logo.jpg", "wb") as handle:
    handle.write(proxy.python_logo().data)

```

Fault Objects

classxmlrpc.client.Fault

faultCode

faultString

A `Fault` object encapsulates the content of an XML-RPC fault tag. Fault objects have the following attributes: `faultCode` An int indicating the fault type. `faultString` A string containing a diagnostic message associated with the fault.

An int indicating the fault type.

A string containing a diagnostic message associated with the fault.

In the following example we're going to intentionally cause a `Fault` by returning a complex type object. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

# A marshalling error is going to occur because we're returning a
# complex number
def add(x, y):
    return x+y+0j

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(add, 'add')

server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
try:
    proxy.add(2, 5)
except xmlrpc.client.Fault as err:
    print("A fault occurred")
    print("Fault code: %d" % err.faultCode)
    print("Fault string: %s" % err.faultString)
```

ProtocolError Objects

classxmlrpc.client.ProtocolError

url

errcode

errmsg

headers

A `ProtocolError` object describes a protocol error in the underlying transport layer (such as a 404 'not found' error if the server named by the URI does not exist). It has the following attributes: `url` The URI or URL that triggered the error. `errcode` The error code. `errmsg` The error message or diagnostic string. `headers` A dict containing the headers of the HTTP/HTTPS request that triggered the error.

The URI or URL that triggered the error.

The error code.

The error message or diagnostic string.

A dict containing the headers of the HTTP/HTTPS request that triggered the error.

In the following example we're going to intentionally cause a `ProtocolError` by providing an invalid URI:

```
import xmlrpc.client

# create a ServerProxy with a URI that doesn't respond to XMLRPC requests
proxy = xmlrpc.client.ServerProxy("http://google.com/")

try:
    proxy.some_method()
except xmlrpc.client.ProtocolError as err:
    print("A protocol error occurred")
    print("URL: %s" % err.url)
    print("HTTP/HTTPS headers: %s" % err.headers)
    print("Error code: %d" % err.errcode)
    print("Error message: %s" % err.errmsg)
```

MultiCall Objects

The `MultiCall` object provides a way to encapsulate multiple calls to a remote server into a single request[1].

`xmlrpc.client.MultiCall(server)`

Create an object used to boxcar method calls. `server` is the eventual target of the call. Calls can be made to the result object, but they will immediately return `None`, and only store the call name and parameters in the `MultiCall` object. Calling the object itself causes all stored calls to be transmitted as a single `system.multicall` request. The result of this call is a generator; iterating over this generator yields the individual results.

A usage example of this class follows. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    return x // y

# A simple server with simple arithmetic functions
server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_multicall_functions()
server.register_function(add, 'add')
server.register_function(subtract, 'subtract')
server.register_function(multiply, 'multiply')
server.register_function(divide, 'divide')
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client
```

```

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
multicall = xmlrpc.client.MultiCall(proxy)
multicall.add(7, 3)
multicall.subtract(7, 3)
multicall.multiply(7, 3)
multicall.divide(7, 3)
result = multicall()

print("7+3=%d, 7-3=%d, 7*3=%d, 7//3=%d" % tuple(result))

```

Convenience Functions

xmlrpc.client.dumps(params,methodname=None,methodresponse=None,encoding=None,allow_none=False)

Convert `params` into an XML-RPC request, or into a response if `methodresponse` is true. `params` can be either a tuple of arguments or an instance of the `FaultException` class. If `methodresponse` is true, only a single value can be returned, meaning that `params` must be of length 1. `encoding`, if supplied, is the encoding to use in the generated XML; the default is UTF-8. Python's `None` value cannot be used in standard XML-RPC; to allow using it via an extension, provide a true value for `allow_none`.

xmlrpc.client.loads(data,use_datetime=False,use_builtin_types=False)

Convert an XML-RPC request or response into Python objects, `a(params,methodname)`. `params` is a tuple of argument; `methodname` is a string, or `None` if no method name is present in the packet. If the XML-RPC packet represents a fault condition, this function will raise a `FaultException`. The `use_builtin_types` flag can be used to cause date/time values to be presented as `datetime.datetime` objects and binary data to be presented as `bytes` objects; this flag is false by default. The obsolete `use_datetime` flag is similar

Example of Client Usage

```

# simple test program (from the XML-RPC specification)
from xmlrpc.client import ServerProxy, Error

# server = ServerProxy("http://localhost:8000") # local server
with ServerProxy("http://betty.userland.com") as proxy:

    print(proxy)

    try:
        print(proxy.examples.getStateName(41))
    except Error as v:
        print("ERROR", v)

```

To access an XML-RPC server through a HTTP proxy, you need to define a custom transport. The following example shows how:

```

import http.client
import xmlrpc.client

class ProxiedTransport(xmlrpc.client.Transport):

    def set_proxy(self, host, port=None, headers=None):
        self.proxy = host, port
        self.proxy_headers = headers

    def make_connection(self, host):
        connection = http.client.HTTPConnection(*self.proxy)
        connection.set_tunnel(host, headers=self.proxy_headers)

```

```
        self._connection = host, connection
    return connection

transport = ProxiedTransport()
transport.set_proxy('proxy-server', 8080)
server = xmlrpc.client.ServerProxy('http://betty.userland.com', transport=transport)
print(server.examples.getStateName(41))
```

Example of Client and Server Usage

SeeSimpleXMLRPCServer example.

Footnotes

This approach has been first presented in a discussion on xmlrpc.com.

xmlrpc.server— Basic XML-RPC servers

Source code: `Lib/xmlrpc/server.py`

The `xmlrpc.server` module provides a basic server framework for XML-RPC servers written in Python. Servers can either be free standing, using `SimpleXMLRPCServer`, or embedded in a CGI environment, using `CGIXMLRPCRequestHandler`.

■■ Advertencia: WarningThe `xmlrpc.server` module is not secure against maliciously constructed data. If you need to parse untrusted or unauthenticated data, see [XML security](#).

Availability: not WASI.

This module does not work or is not available on WebAssembly. See [WebAssembly platforms](#) for more information.

`class xmlrpc.server.SimpleXMLRPCServer(addr, requestHandler=SimpleXMLRPCRequestHandler, logRequests=True, allow_none=False, encoding=None, bind_and_activate=True, use_builtin_types=False)`

Create a new server instance. This class provides methods for registration of functions that can be called by the XML-RPC protocol. The `requestHandler` parameter should be a factory for request handler instances; it defaults to `SimpleXMLRPCRequestHandler`. The `addr` and `requestHandler` parameters are passed to the `socketserver.TCPServer` constructor. If `logRequests` is true (the default), requests will be logged; setting this parameter to false will turn off logging. The `allow_none` and `encoding` parameters are

`class xmlrpc.server.CGIXMLRPCRequestHandler(allow_none=False, encoding=None, use_builtin_types=False)`

Create a new instance to handle XML-RPC requests in a CGI environment. The `allow_none` and `encoding` parameters are passed on to `xmlrpc.client` and control the XML-RPC responses that will be returned from the server. The `use_builtin_types` parameter is passed to the `loads()` function and controls which types are processed when date/times values or binary data are received; it defaults to false. Changed in version 3.3: The `use_builtin_types` flag was added.

`class xmlrpc.server.SimpleXMLRPCRequestHandler`

Create a new request handler instance. This request handler supports POST requests and modifies logging so that the `logRequests` parameter to the `SimpleXMLRPCServer` constructor parameter is honored.

SimpleXMLRPCServer objects

The `SimpleXMLRPCServer` class is based on `socketserver.TCPServer` and provides a means of creating simple, stand alone XML-RPC servers.

`SimpleXMLRPCServer.register_function(function=None, name=None)`

Register a function that can respond to XML-RPC requests. If `name` is given, it will be the method name associated with `function`, otherwise `function.__name__` will be used. `name` is a string, and may contain characters not legal in Python identifiers, including the period character. This method can also be used as a decorator. When used as a decorator, `name` can only be given as a keyword argument to `register_function` under `name`. If `name` is given, `function.__name__` will be used. Changed in version 3.7: `register_`

`SimpleXMLRPCServer.register_instance(instance, allow_dotted_names=False)`

Register an object which is used to expose method names which have not been registered using `register_function()`. If `instance` contains a `_dispatch()` method, it is called with the requested method name and the parameters from the request. Its API is `def _dispatch(self, method, params)` (note that `params` does not

represent a variable argument list). If it calls an underlying function to perform its task, that function is called `asfunc(*params)`, expanding the parameter list. The return value from `_dispatch()`

SimpleXMLRPCServer.register_introspection_functions()

Registers the XML-RPC introspection
functionssystem.listMethods,system.methodHelpandsystem.methodSignature.

SimpleXMLRPCServer.register_multicall_functions()

Registers the XML-RPC multicall function system.multicall.

SimpleXMLRPCRequestHandler.rpc_paths

An attribute value that must be a tuple listing valid path portions of the URL for receiving XML-RPC requests. Requests posted to other paths will result in a 404 “no such page” HTTP error. If this tuple is empty, all paths will be considered valid. The default value is('/', '/RPC2').

SimpleXMLRPCServer example

Server code:

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

# Restrict to a particular path.
class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)

# Create server
with SimpleXMLRPCServer(('localhost', 8000),
                        requestHandler=RequestHandler) as server:
    server.register_introspection_functions()

    # Register pow() function; this will use the value of
    # pow.__name__ as the name, which is just 'pow'.
    server.register_function(pow)

    # Register a function under a different name
    def adder_function(x, y):
        return x + y
    server.register_function(adder_function, 'add')

    # Register an instance; all the methods of the instance are
    # published as XML-RPC methods (in this case, just 'mul').
    class MyFuncs:
        def mul(self, x, y):
            return x * y

    server.register_instance(MyFuncs())

    # Run the server's main loop
    server.serve_forever()
```

The following client code will call the methods made available by the preceding server:

```
import xmlrpc.client

s = xmlrpc.client.ServerProxy('http://localhost:8000')
print(s.pow(2,3)) # Returns 2**3 = 8
print(s.add(2,3)) # Returns 5
print(s.mul(5,2)) # Returns 5*2 = 10

# Print list of available methods
print(s.system.listMethods())
```

register_function() can also be used as a decorator. The previous server example can register functions in a decorator way:

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)

with SimpleXMLRPCServer(('localhost', 8000),
                        requestHandler=RequestHandler) as server:
    server.register_introspection_functions()

    # Register pow() function; this will use the value of
    # pow.__name__ as the name, which is just 'pow'.
    server.register_function(pow)

    # Register a function under a different name, using
    # register_function as a decorator. *name* can only be given
    # as a keyword argument.
    @server.register_function(name='add')
    def adder_function(x, y):
        return x + y

    # Register a function under function.__name__.
    @server.register_function
    def mul(x, y):
        return x * y

    server.serve_forever()
```

The following example included in the `Lib/xmlrpc/server.py` module shows a server allowing dotted names and registering a multicall function.

■■ Advertencia: WarningEnabling the `allow_dotted_names` option allows intruders to access your module's global variables and may allow intruders to execute arbitrary code on your machine. Only use this example within a secure, closed network.

```
import datetime as dt

class ExampleService:
    def getData(self):
        return '42'

    class currentTime:
        @staticmethod
        def getCurrentTime():
            return dt.datetime.now()

with SimpleXMLRPCServer(("localhost", 8000)) as server:
    server.register_function(pow)
    server.register_function(lambda x,y: x+y, 'add')
    server.register_instance(ExampleService(), allow_dotted_names=True)
    server.register_multicall_functions()
    print('Serving XML-RPC on localhost port 8000')
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nKeyboard interrupt received, exiting.")
        sys.exit(0)
```

This ExampleService demo can be invoked from the command line:

```
python -m xmlrpc.server
```


The client that interacts with the above server is included in `Lib/xmlrpc/client.py`:

```
server = ServerProxy("http://localhost:8000")

try:
    print(server.currentTime.getCurrentTime())
except Error as v:
    print("ERROR", v)

multi = MultiCall(server)
multi.getData()
multi.pow(2,9)
multi.add(1,2)
try:
    for response in multi():
        print(response)
except Error as v:
    print("ERROR", v)
```

This client which interacts with the demo XMLRPC server can be invoked as:

```
python -m xmlrpc.client
```

CGIXMLRPCRequestHandler

The `CGIXMLRPCRequestHandler` class can be used to handle XML-RPC requests sent to Python CGI scripts.

`CGIXMLRPCRequestHandler.register_function(function=None,name=None)`

Register a function that can respond to XML-RPC requests. If `name` is given, it will be the method name associated with `function`, otherwise `function.__name__` will be used. `name` is a string, and may contain characters not legal in Python identifiers, including the period character. This method can also be used as a decorator. When used as a decorator, `name` can only be given as a keyword argument to `register_function` under `name`. If `name` is given, `function.__name__` will be used. Changed in version 3.7: `register_`

`CGIXMLRPCRequestHandler.register_instance(instance)`

Register an object which is used to expose method names which have not been registered using `register_function()`. If `instance` contains a `a_dispatch()` method, it is called with the requested method name and the parameters from the request; the return value is returned to the client as the result. If `instance` does not have a `a_dispatch()` method, it is searched for an attribute matching the name of the requested method; if the requested method name contains periods, each component of the method name

`CGIXMLRPCRequestHandler.register_introspection_functions()`

Register the XML-RPC introspection `function` `system.listMethods`, `system.methodHelp` and `system.methodSignature`.

`CGIXMLRPCRequestHandler.register_multicall_functions()`

Register the XML-RPC multicall `function` `system.multicall`.

`CGIXMLRPCRequestHandler.handle_request(request_text=None)`

Handle an XML-RPC request. If `request_text` is given, it should be the POST data provided by the HTTP server, otherwise the contents of `stdin` will be used.

Example:

```
class MyFuncs:
    def mul(self, x, y):
        return x * y
```

```

handler = CGIXMLRPCRequestHandler()
handler.register_function(pow)
handler.register_function(lambda x,y: x+y, 'add')
handler.register_introspection_functions()
handler.register_instance(MyFuncs())
handler.handle_request()

```

Documenting XMLRPC server

These classes extend the above classes to serve HTML documentation in response to HTTP GET requests. Servers can either be free standing, using `DocXMLRPCServer`, or embedded in a CGI environment, using `DocCGIXMLRPCRequestHandler`.

`classxmlrpc.server.DocXMLRPCServer(addr,requestHandler=DocXMLRPCRequestHandler,logRequests=True,allow_none=False,encoding=None,bind_and_activate=True,use_builtin_types=True)`

Create a new server instance. All parameters have the same meaning as for `SimpleXMLRPCServer`; `requestHandler` defaults to `DocXMLRPCRequestHandler`. Changed in version 3.3: The `use_builtin_types` flag was added.

`classxmlrpc.server.DocCGIXMLRPCRequestHandler`

Create a new instance to handle XML-RPC requests in a CGI environment.

`classxmlrpc.server.DocXMLRPCRequestHandler`

Create a new request handler instance. This request handler supports XML-RPC POST requests, documentation GET requests, and modifies logging so that the `logRequests` parameter to the `DocXMLRPCServer` constructor parameter is honored.

DocXMLRPCServer objects

The `DocXMLRPCServer` class is derived from `SimpleXMLRPCServer` and provides a means of creating self-documenting, stand alone XML-RPC servers. HTTP POST requests are handled as XML-RPC method calls. HTTP GET requests are handled by generating pydoc-style HTML documentation. This allows a server to provide its own web-based documentation.

`DocXMLRPCServer.set_server_title(server_title)`

Set the title used in the generated HTML documentation. This title will be used inside the HTML “title” element.

`DocXMLRPCServer.set_server_name(server_name)`

Set the name used in the generated HTML documentation. This name will appear at the top of the generated documentation inside a “h1” element.

`DocXMLRPCServer.set_server_documentation(server_documentation)`

Set the description used in the generated HTML documentation. This description will appear as a paragraph, below the server name, in the documentation.

DocCGIXMLRPCRequestHandler

The `DocCGIXMLRPCRequestHandler` class is derived from `CGIXMLRPCRequestHandler` and provides a means of creating self-documenting, XML-RPC CGI scripts. HTTP POST requests are handled as XML-RPC method calls. HTTP GET requests are handled by generating pydoc-style HTML documentation. This allows a server to provide its own web-based documentation.

DocCGIXMLRPCRequestHandler.set_server_title(server_title)

Set the title used in the generated HTML documentation. This title will be used inside the HTML “title” element.

DocCGIXMLRPCRequestHandler.set_server_name(server_name)

Set the name used in the generated HTML documentation. This name will appear at the top of the generated documentation inside a “h1” element.

DocCGIXMLRPCRequestHandler.set_server_documentation(server_documentation)

Set the description used in the generated HTML documentation. This description will appear as a paragraph, below the server name, in the documentation.

ipaddress— IPv4/IPv6 manipulation library

Source code: `Lib/ipaddress.py`

`ipaddress` provides the capabilities to create, manipulate and operate on IPv4 and IPv6 addresses and networks.

The functions and classes in this module make it straightforward to handle various tasks related to IP addresses, including checking whether or not two hosts are on the same subnet, iterating over all hosts in a particular subnet, checking whether or not a string represents a valid IP address or network definition, and so on.

This is the full module API reference—for an overview and introduction, see [An introduction to the ipaddress module](#).

Added in version 3.3.

Convenience factory functions

The `ipaddress` module provides factory functions to conveniently create IP addresses, networks and interfaces:

`ipaddress.ip_address(address)`

Return an `IPv4Address` or `IPv6Address` object depending on the IP address passed as argument. Either IPv4 or IPv6 addresses may be supplied; integers less than 2^{32} will be considered to be IPv4 by default. A `ValueError` is raised if `address` does not represent a valid IPv4 or IPv6 address. >>> `ipaddress.ip_address('192.168.0.1')` `IPv4Address('192.168.0.1')` >>> `ipaddress.ip_address('2001:db8::')` `IPv6Address('2001:db8::')`

`ipaddress.ip_network(address,strict=True)`

Return an `IPv4Network` or `IPv6Network` object depending on the IP address passed as argument. `address` is a string or integer representing the IP network. Either IPv4 or IPv6 networks may be supplied; integers less than 2^{32} will be considered to be IPv4 by default. `strict` is passed to `IPv4Network` or `IPv6Network` constructor. A `ValueError` is raised if `address` does not represent a valid IPv4 or IPv6 address, or if the network has host bits set. >>> `ipaddress.ip_network('192.168.0.0/28')` `IPv4Network('192.168.0.0/28')`

`ipaddress.ip_interface(address)`

Return an `IPv4Interface` or `IPv6Interface` object depending on the IP address passed as argument. `address` is a string or integer representing the IP address. Either IPv4 or IPv6 addresses may be supplied; integers less than 2^{32} will be considered to be IPv4 by default. A `ValueError` is raised if `address` does not represent a valid IPv4 or IPv6 address.

One downside of these convenience functions is that the need to handle both IPv4 and IPv6 formats means that error messages provide minimal information on the precise error, as the functions don't know whether the IPv4 or IPv6 format was intended. More detailed error reporting can be obtained by calling the appropriate version specific class constructors directly.

IP Addresses

Address objects

The `IPv4Address` and `IPv6Address` objects share a lot of common attributes. Some attributes that are only meaningful for IPv6 addresses are also implemented by `IPv4Address` objects, in order to make it easier to write code that handles both IP versions correctly. Address objects are hashable, so they can be used as keys in dictionaries.

- version
- max_prefixlen
- compressed
- exploded
- packed
- reverse_pointer
- is_multicast
- is_private
- is_global
- is_unspecified
- is_reserved
- is_loopback
- is_link_local
- ipv6_mapped

The appropriate version number:4for IPv4,6for IPv6.Changed in version 3.14:Made available on the class.

The string representation in dotted decimal notation. Leading zeroes are never included in the representation. As IPv4 does not define a shorthand notation for addresses with octets set to zero, these two attributes are always the same as `str(addr)` for IPv4 addresses. Exposing these attributes makes it easier to write display code that can handle both IPv4 and IPv6 addresses.

The name of the reverse DNS PTR record for the IP address, e.g.:>>>ipaddress.ip_address("127.0.0.1").reverse_pointer'1.0.0.127.in-addr.arpa>>>ipaddress.ip_address("2001:db8::1").reverse_pointer'1.0.b.d.0.1.0.0.2.ip6.arpa

This is the name that could be used for performing a PTR lookup, not the resolved hostname itself.

Added in version 3.5.

True if the address is defined as not globally reachable by `iana-ipv4-special-registry` (for IPv4) `iana-ipv6-special-registry` (for IPv6) with the following exceptions: `is_private` is `False` for the shared address space (100.64.0.0/10). For IPv4-mapped IPv6-addresses the `is_private` value is determined by the semantics of the underlying IPv4 addresses and the following condition holds (see `IPv6Address.ipv4_mapped`): `address.is_private == address.ipv4_mapped.is_private` is `True` has value opposite to `is_global`, except

True if the address is defined as globally reachable byiana-ipv4-special-registry(for IPv4) oriana-ipv6-special-registry(for IPv6) with the following exception:For IPv4-mapped IPv6-addresses theiris_privatevalue is determined by the semantics of the underlying IPv4 addresses and the following condition holds (seeIPv6Address.ipv4_mapped):address.is_global==address.ipv4_mapped.is_globalis_globalhas value opposite tois_private, except for the shared address space (100.64.0.0/10range) where they are bo

True if the address is unspecified. SeeRFC 5735(for IPv4) orRFC 2373(for IPv6).

True if the address is noted as reserved by the IETF. For IPv4, this is only240.0.0.0/4, theReservedaddress block. For IPv6, this is all addressesallocatedasReservedbyIETFfor future use.NoteFor IPv4,is_reservedis not related to the address block value of theReserved-by-Protocolcolumn iniana-ipv4-special-registry.CautionFor IPv6,fec0::/10a former Site-Local scoped address prefix is currently excluded from that list (seeis_site_local&RFC 3879).

True if this is a loopback address. SeeRFC 3330(for IPv4) orRFC 2373(for IPv6).

True if the address is reserved for link-local usage. SeeRFC 3927.

IPv4Addressobject representing the IPv4-mapped IPv6 address. SeeRFC 4291.Added in version 3.13.

IPv4Address.__format__(fmt)

Returns a string representation of the IP address, controlled by an explicit format string.fmtcan be one of the following:'s', the default option, equivalent tostr(),'b'for a zero-padded binary string,'X'or'x'for an uppercase or lowercase hexadecimal representation, or'n', which is equivalent to'b'for IPv4 addresses and'x'for IPv6. For binary and hexadecimal representations, the form specifier'#'and the grouping option'_'are available.__format__is used byformat,str.formatand f-strings.>>>format(

classipaddress.IPv6Address(address)

compressed

exploded

packed

reverse_pointer

version

max_prefixlen

is_multicast

is_private

is_global

is_unspecified

is_reserved

is_loopback

is_link_local

is_site_local

ipv4_mapped

scope_id

sixtofour

teredo

Construct an IPv6 address. An `AddressValueError` is raised if `address` is not a valid IPv6 address. The following constitutes a valid IPv6 address: A string consisting of eight groups of four hexadecimal digits, each group representing 16 bits. The groups are separated by colons. This describes an exploded (longhand) notation. The string can also be compressed (shorthand notation) by various means. See RFC 4291 for details. For example, "0000:0000:0000:0000:0000:0abc:0007:0def" can be compressed to "::abc:7".

Added in version 3.4.

True if the address is reserved for site-local usage. Note that the site-local address space has been deprecated by RFC 3879. Use `is_private` to test if this address is in the space of unique local addresses as defined by RFC 4193.

For addresses that appear to be IPv4 mapped addresses (starting with ::FFFF/96), this property will report the embedded IPv4 address. For any other address, this property will be `None`.

For scoped addresses as defined by RFC 4007, this property identifies the particular zone of the address's scope that the address belongs to, as a string. When no scope zone is specified, this property will be `None`.

For addresses that appear to be 6to4 addresses (starting with 2002::/16) as defined by RFC 3056, this property will report the embedded IPv4 address. For any other address, this property will be `None`.

For addresses that appear to be Teredo addresses (starting with 2001::/32) as defined by RFC 4380, this property will report the embedded (server, client) IP address pair. For any other address, this property will be `None`.

IPv6Address.__format__(fmt)

Refer to the corresponding method documentation in `IPv4Address`. Added in version 3.9.

Conversion to Strings and Integers

To interoperate with networking interfaces such as the `socket` module, addresses must be converted to strings or integers. This is handled using the `str()` and `int()` builtin functions:

```
>>> str(ipaddress.IPv4Address('192.168.0.1'))
'192.168.0.1'
>>> int(ipaddress.IPv4Address('192.168.0.1'))
3232235521
>>> str(ipaddress.IPv6Address('::1'))
 '::1'
>>> int(ipaddress.IPv6Address('::1'))
1
```

Note that IPv6 scoped addresses are converted to integers without scope zone ID.

Operators

Address objects support some operators. Unless stated otherwise, operators can only be applied between compatible objects (i.e. IPv4 with IPv4, IPv6 with IPv6).

Comparison operators

Address objects can be compared with the usual set of comparison operators. Same IPv6 addresses with different scope zone IDs are not equal. Some examples:

```
>>> IPv4Address('127.0.0.2') > IPv4Address('127.0.0.1')
True
>>> IPv4Address('127.0.0.2') == IPv4Address('127.0.0.1')
False
>>> IPv4Address('127.0.0.2') != IPv4Address('127.0.0.1')
True
>>> IPv6Address('fe80::1234') == IPv6Address('fe80::1234%1')
False
```

```
>>> IPv6Address('fe80::1234%1') != IPv6Address('fe80::1234%2')
True
```

Arithmetic operators

Integers can be added to or subtracted from address objects. Some examples:

```
>>> IPv4Address('127.0.0.2') + 3
IPv4Address('127.0.0.5')
>>> IPv4Address('127.0.0.2') - 3
IPv4Address('126.255.255.255')
>>> IPv4Address('255.255.255.255') + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ipaddress.AddressValueError: 4294967296 (>= 2**32) is not permitted as an IPv4 address
```

IP Network definitions

The `IPv4Network` and `IPv6Network` objects provide a mechanism for defining and inspecting IP network definitions. A network definition consists of a mask and a network address, and as such defines a range of IP addresses that equal the network address when masked (binary AND) with the mask. For example, a network definition with the mask `255.255.255.0` and the network address `192.168.1.0` consists of IP addresses in the inclusive range `192.168.1.0` to `192.168.1.255`.

Prefix, net mask and host mask

There are several equivalent ways to specify IP network masks. A `prefix/<nbits>` is a notation that denotes how many high-order bits are set in the network mask. A `net mask` is an IP address with some number of high-order bits set. Thus the `prefix/24` is equivalent to the net mask `255.255.255.0` in IPv4, or `ffff:ff00::` in IPv6. In addition, a `host mask` is the logical inverse of a net mask, and is sometimes used (for example in Cisco access control lists) to denote a network mask. The host mask equivalent to `/24` in IPv4 is `0.0.0.255`.

Network objects

All attributes implemented by address objects are implemented by network objects as well. In addition, network objects implement additional attributes. All of these are common between `IPv4Network` and `IPv6Network`, so to avoid duplication they are only documented for `IPv4Network`. Network objects are hashable, so they can be used as keys in dictionaries.

`class ipaddress.IPv4Network(address, strict=True)`

version

max_prefixlen

is_multicast

is_private

is_unspecified

is_reserved

is_loopback

is_link_local

network_address

broadcast_address

hostmask

netmask

with_prefixlen

compressed

exploded

with_netmask

with_hostmask

num_addresses

prefixlen

hosts()

overlaps(other)

address_exclude(network)

subnets(prefixlen_diff=1,new_prefix=None)

supernet(prefixlen_diff=1,new_prefix=None)

subnet_of(other)

supernet_of(other)

compare_networks(other)

Construct an IPv4 network definition.`address` can be one of the following: A string consisting of an IP address and an optional mask, separated by a slash (/). The IP address is the network address, and the mask can be either a single number, which means it's a prefix, or a string representation of an IPv4 address. If it's the latter, the mask is interpreted as a net mask if it starts with a non-zero field, or as a host mask if it starts with a zero field, with the single exception of an all-zero mask

Refer to the corresponding attribute documentation in `IPv4Address`.

These attributes are true for the network as a whole if they are true for both the network address and the broadcast address.

The network address for the network. The network address and the prefix length together uniquely define a network.

The broadcast address for the network. Packets sent to the broadcast address should be received by every host on the network.

The host mask, as an `IPv4Address` object.

The net mask, as an `IPv4Address` object.

A string representation of the network, with the mask in prefix notation. `with_prefixlen` and `compressed` are always the same as `str(network).exploded` uses the exploded form of the network address.

A string representation of the network, with the mask in net mask notation.

A string representation of the network, with the mask in host mask notation.

The total number of addresses in the network.

Length of the network prefix, in bits.

Returns an iterator over the usable hosts in the network. The usable hosts are all the IP addresses that belong to the network, except the network address itself and the network broadcast address. For networks with a mask length of 31, the network address and network broadcast address are also included in the result. Networks with

a mask of 32 will return a list containing the single host

address.>>>list(ip_network('192.0.2.0/29').hosts())[IPv4Address('192.0.2.1'), IPv4Address('192.0.2.2'), IPv4Address('192.0.2.3'), IPv4Address('192.0.2.4')]

True if this network is partly or wholly contained in another or other is wholly contained in this network.

Computes the network definitions resulting from removing the given network from this one. Returns an iterator of network objects. Raises ValueError if network is not completely contained in this network.>>>n1=ip_network('192.0.2.0/28')>>>n2=ip_network('192.0.2.1/32')>>>list(n1.address_exclude(n2))[IPv4Network('192.0.2.8/29'), IPv4Network('192.0.2.4/30'), IPv4Network('192.0.2.2/31'), IPv4Network('192.0.2.0/32')]

The subnets that join to make the current network definition, depending on the argument values.prefixlen_diff is the amount our prefix length should be increased by.new_prefix is the desired new prefix of the subnets; it must be larger than our prefix. One and only one of prefixlen_diff and new_prefix must be set. Returns an iterator of network objects.>>>list(ip_network('192.0.2.0/24').subnets())[IPv4Network('192.0.2.0/25'), IPv4Network('192.0.2.128/25')]>>>list(ip_network('192.0.2.0/24').subnets(prefixlen_diff=2))

The supernet containing this network definition, depending on the argument values.prefixlen_diff is the amount our prefix length should be decreased by.new_prefix is the desired new prefix of the supernet; it must be smaller than our prefix. One and only one of prefixlen_diff and new_prefix must be set. Returns a single network object.>>>ip_network('192.0.2.0/24').supernet(IPv4Network('192.0.2.0/23'))>>>ip_network('192.0.2.0/24').supernet(prefixlen_diff=2)IPv4Network('192.0.0.0/22')>>>ip_network('192.0.0.0/22').supernet(prefixlen_diff=2)

Return True if this network is a subnet of other.>>>a=ip_network('192.168.1.0/24')>>>b=ip_network('192.168.1.128/30')>>>b.subnet_of(a)TrueAdded in version 3.7.

Return True if this network is a supernet of other.>>>a=ip_network('192.168.1.0/24')>>>b=ip_network('192.168.1.128/30')>>>a.supernet_of(b)TrueAdded in version 3.7.

Compare this network to other. In this comparison only the network addresses are considered; host bits aren't. Returns either -1, 0 or 1.>>>ip_network('192.0.2.1/32').compare_networks(ip_network('192.0.2.2/32'))-1>>>ip_network('192.0.2.1/32').compare_networks(ip_network('192.0.2.0/32'))1>>>ip_network('192.0.2.1/32').compare_networks(ip_network('192.0.2.1/32'))0Deprecated since version 3.7: It uses the same ordering and comparison algorithm as "<", "=", and ">"

class ipaddress.IPv6Network(address, strict=True)

version

max_prefixlen

is_multicast

is_private

is_unspecified

is_reserved

is_loopback

is_link_local

network_address

broadcast_address

hostmask

netmask

with_prefixlen

compressed
exploded
with_netmask
with_hostmask
num_addresses
prefixlen
hosts()
overlaps(other)
address_exclude(network)
subnets(prefixlen_diff=1,new_prefix=None)
supernet(prefixlen_diff=1,new_prefix=None)
subnet_of(other)
supernet_of(other)
compare_networks(other)
is_site_local

Construct an IPv6 network definition. **address** can be one of the following: A string consisting of an IP address and an optional prefix length, separated by a slash (/). The IP address is the network address, and the prefix length must be a single number, the **prefix**. If no prefix length is provided, it's considered to be /128. Note that currently expanded netmasks are not supported. That means 2001:db00::0/24 is a valid argument while 2001:db00::0/ffff:ff00:: is not. An integer that fits into 128 bits.

Returns an iterator over the usable hosts in the network. The usable hosts are all the IP addresses that belong to the network, except the Subnet-Router anycast address. For networks with a mask length of 127, the Subnet-Router anycast address is also included in the result. Networks with a mask of 128 will return a list containing the single host address.

Refer to the corresponding attribute documentation in `IPv4Network`.

This attribute is true for the network as a whole if it is true for both the network address and the broadcast address.

Operators

Network objects support some operators. Unless stated otherwise, operators can only be applied between compatible objects (i.e. IPv4 with IPv4, IPv6 with IPv6).

Logical operators

Network objects can be compared with the usual set of logical operators. Network objects are ordered first by network address, then by net mask.

Iteration

Network objects can be iterated to list all the addresses belonging to the network. For iteration, **allhosts** are returned, including unusable hosts (for usable hosts, use the **hosts()** method). An example:

```
>>> for addr in IPv4Network('192.0.2.0/28'):
...     addr
...
IPv4Address('192.0.2.0')
```

```

IPv4Address('192.0.2.1')
IPv4Address('192.0.2.2')
IPv4Address('192.0.2.3')
IPv4Address('192.0.2.4')
IPv4Address('192.0.2.5')
IPv4Address('192.0.2.6')
IPv4Address('192.0.2.7')
IPv4Address('192.0.2.8')
IPv4Address('192.0.2.9')
IPv4Address('192.0.2.10')
IPv4Address('192.0.2.11')
IPv4Address('192.0.2.12')
IPv4Address('192.0.2.13')
IPv4Address('192.0.2.14')
IPv4Address('192.0.2.15')

```

Networks as containers of addresses

Network objects can act as containers of addresses. Some examples:

```

>>> IPv4Network('192.0.2.0/28')[0]
IPv4Address('192.0.2.0')
>>> IPv4Network('192.0.2.0/28')[15]
IPv4Address('192.0.2.15')
>>> IPv4Address('192.0.2.6') in IPv4Network('192.0.2.0/28')
True
>>> IPv4Address('192.0.3.6') in IPv4Network('192.0.2.0/28')
False

```

Interface objects

Interface objects are hashable, so they can be used as keys in dictionaries.

class ipaddress.IPv4Interface(address)

ip

network

with_prefixlen

with_netmask

with_hostmask

Construct an IPv4 interface. The meaning of `address` is as in the constructor of `IPv4Network`, except that arbitrary host addresses are always accepted. `IPv4Interface` is a subclass of `IPv4Address`, so it inherits all the attributes from that class. In addition, the following attributes are available:

- `ip`: The address (`IPv4Address`) without network

```

information.>>> interface = IPv4Interface('192.0.2.5/24')>>> interface.ip
IPv4Address('192.0.2.5')
network The network (IPv4Network) this interface belongs to.>>> interface.network
IPv4Network('192.0.2.0/24')

```

The address (`IPv4Address`) without network

```

information.>>> interface = IPv4Interface('192.0.2.5/24')>>> interface.ip
IPv4Address('192.0.2.5')

```

The network (`IPv4Network`) this interface belongs to.

```

to.>>> interface = IPv4Interface('192.0.2.5/24')>>> interface.network
IPv4Network('192.0.2.0/24')

```

A string representation of the interface with the mask in prefix notation.

```

notation.>>> interface = IPv4Interface('192.0.2.5/24')>>> interface.with_prefixlen
'192.0.2.5/24'

```

A string representation of the interface with the network as a netmask.

```

mask.>>> interface = IPv4Interface('192.0.2.5/24')>>> interface.with_netmask
'192.0.2.5/255.255.255.0'

```

A string representation of the interface with the network as a host mask.>>>interface=IPv4Interface('192.0.2.5/24')>>>interface.with_hostmask('192.0.2.5/0.0.0.255')

class ipaddress.IPv6Interface(address)

ip

network

with_prefixlen

with_netmask

with_hostmask

Construct an IPv6 interface. The meaning of `address` is as in the constructor of `IPv6Network`, except that arbitrary host addresses are always accepted. `IPv6Interface` is a subclass of `IPv6Address`, so it inherits all the attributes from that class. In addition, the following attributes are available: `ipnetwork`, `with_prefixlen`, `with_netmask`, `with_hostmask`. Refer to the corresponding attribute documentation in `IPv4Interface`.

Refer to the corresponding attribute documentation in `IPv4Interface`.

Operators

Interface objects support some operators. Unless stated otherwise, operators can only be applied between compatible objects (i.e. IPv4 with IPv4, IPv6 with IPv6).

Logical operators

Interface objects can be compared with the usual set of logical operators.

For equality comparison (`==` and `!=`), both the IP address and network must be the same for the objects to be equal. An interface will not compare equal to any address or network object.

For ordering (`<`, `>`, etc) the rules are different. Interface and address objects with the same IP version can be compared, and the address objects will always sort before the interface objects. Two interface objects are first compared by their networks and, if those are the same, then by their IP addresses.

Other Module Level Functions

The module also provides the following module level functions:

ipaddress.v4_int_to_packed(address)

Represent an address as 4 packed bytes in network (big-endian) order. `address` is an integer representation of an IPv4 IP address. A `ValueError` is raised if the integer is negative or too large to be an IPv4 IP address.>>>ipaddress.ip_address(3221225985)IPv4Address('192.0.2.1')>>>ipaddress.v4_int_to_packed(3221225985)b'\xc0\x00\x02\x01'

ipaddress.v6_int_to_packed(address)

Represent an address as 16 packed bytes in network (big-endian) order. `address` is an integer representation of an IPv6 IP address. A `ValueError` is raised if the integer is negative or too large to be an IPv6 IP address.

ipaddress.summarize_address_range(first,last)

Return an iterator of the summarized network range given the first and last IP addresses. `first` is the first `IPv4Address` or `IPv6Address` in the range and `last` is the last `IPv4Address` or `IPv6Address` in the range. A `TypeError` is raised if `first` or `last` are not IP addresses or are not of the same version. A `ValueError` is raised if `last` is not greater than `first` or if `first` address version is not 4 or 6.>>>[ipaddr for ipaddr in ipaddress.summarize_address_ra

nge(...ipaddress.IPv4Address('192.0.2.0'),...ipaddress.IPv4Address('192

ipaddress.collapse_addresses(addresses)

Return an iterator of the collapsedIPv4NetworkorIPv6Networkobjects.addressesis
an iterable of IPv4NetworkorIPv6Networkobjects. ATypeErroris raised ifaddressescontains mixed version objects.
>>>[ipaddrforipaddrin...ipaddress.collapse_addresses([ipaddress.IPv4Network('192.0.2.0/25'),...ipaddress.IPv4
Network('192.0.2.128/25')])][IPv4Network('192.0.2.0/24')]

ipaddress.get_mixed_type_key(obj)

Return a key suitable for sorting between networks and addresses. Address and Network objects are not
sortable by default; they're fundamentally different, so the
expression:IPv4Address('192.0.2.0')<=IPv4Network('192.0.2.0/24')doesn't make sense. There are some times
however, where you may wish to haveipaddresssort these anyway. If you need to do this, you can use this
function as thekeyargument tosorted().objis either a network or address object.

Custom Exceptions

To support more specific error reporting from class constructors, the module defines the following exceptions:

exceptionipaddress.AddressValueError(ValueError)

Any value error related to the address.

exceptionipaddress.NetmaskValueError(ValueError)

Any value error related to the net mask.

Multimedia Services

The modules described in this chapter implement various algorithms or interfaces that are mainly useful for multimedia applications. They are available at the discretion of the installation. Here's an overview:

- wave— Read and write WAV filesWave_read ObjectsWave_write Objects
- colorsys— Conversions between color systems

wave— Read and write WAV files

Source code: `Lib/wave.py`

The `wave` module provides a convenient interface to the Waveform Audio “WAVE” (or “WAV”) file format. Only uncompressed PCM encoded wave files are supported.

Changed in version 3.12: Support for `WAVE_FORMAT_EXTENSIBLE` headers was added, provided that the extended format is `KSDATAFORMAT_SUBTYPE_PCM`.

The `wave` module defines the following function and exception:

`wave.open(file, mode=None)`

`'rb'`

`'wb'`

If `file` is a string, open the file by that name, otherwise treat it as a file-like object. `mode` can be: `'rb'` Read only mode. `'wb'` Write only mode. Note that it does not allow read/write WAV files. A `mode` of `'rb'` returns a `Wave_read` object, while a `mode` of `'wb'` returns a `Wave_write` object. If `mode` is omitted and a file-like object is passed as `file`, `file.mode` is used as the default value for `mode`. If you pass in a file-like object, the wave object will not close it when its `close()` method is called; it is the caller's respons

Read only mode.

Write only mode.

exception `wave.Error`

An error raised when something is impossible because it violates the WAV specification or hits an implementation deficiency.

Wave_read Objects

`class wave.Wave_read`

`close()`

`getnchannels()`

`getsampwidth()`

`getframerate()`

`getnframes()`

`getcomptype()`

`getcompname()`

`getparams()`

`readframes(n)`

`rewind()`

`getmarkers()`

`getmark(id)`

`setpos(pos)`

tell()

Read a WAV file. `Wave_read` objects, as returned by `open()`, have the following methods: `close()` Close the stream if it was opened by `wave`, and make the instance unusable. This is called automatically on object collection. `getnchannels()` Returns number of audio channels (1 for mono, 2 for stereo). `getsampwidth()` Returns sample width in bytes. `getframerate()` Returns sampling frequency. `getnframes()` Returns number of audio frames. `getcomptype()` Returns compression type ('NONE' is the only supported type). `getcompname()`

Close the stream if it was opened by `wave`, and make the instance unusable. This is called automatically on object collection.

Returns number of audio channels (1 for mono, 2 for stereo).

Returns sample width in bytes.

Returns sampling frequency.

Returns number of audio frames.

Returns compression type ('NONE' is the only supported type).

Human-readable version of `getcomptype()`. Usually 'not compressed' parallels 'NONE'.

Returns an `namedtuple()` (`nchannels`, `sampwidth`, `framerate`, `nframes`, `comptype`, `compname`), equivalent to output of the `get*()` methods.

Reads and returns at most `nframes` of audio, as a `bytesobject`.

Rewind the file pointer to the beginning of the audio stream.

Returns `None`. Deprecated since version 3.13, will be removed in version 3.15: The method only existed for compatibility with the `aifc` module which has been removed in Python 3.13.

Raise an error. Deprecated since version 3.13, will be removed in version 3.15: The method only existed for compatibility with the `aifc` module which has been removed in Python 3.13.

Set the file pointer to the specified position.

Return current file pointer position.

Wave_write Objects

`class wave.Wave_write`

`close()`

`setnchannels(n)`

`getnchannels()`

`setsampwidth(n)`

`getsampwidth()`

`setframerate(n)`

`getframerate()`

`setnframes(n)`

`getnframes()`

`setcomptype(type, name)`

`getcomptype()`

getcompname()

setparams(tuple)

getparams()

tell()

writeframesraw(data)

writeframes(data)

Write a WAV file. `Wave_write` objects, as returned by `open()`. For seekable output streams, the `waveheader` will automatically be updated to reflect the number of frames actually written. For unseekable streams, then `frames` value must be accurate when the first frame data is written. An accurate `frames` value can be achieved either by calling `setnframes()` or `setparams()` with the number of frames that will be written before `close()` is called and then using `writeframesraw()` to write the frame data, or by calling `wr`

Make sure `frames` is correct, and close the file if it was opened by `wave`. This method is called upon object collection. It will raise an exception if the output stream is not seekable and `nframes` does not match the number of frames actually written.

Set the number of channels.

Return the number of channels.

Set the sample width to `nbytes`.

Return the sample width in bytes.

Set the frame rate to `n`. Changed in version 3.2: A non-integral input to this method is rounded to the nearest integer.

Return the frame rate.

Set the number of frames to `n`. This will be changed later if the number of frames actually written is different (this update attempt will raise an error if the output stream is not seekable).

Return the number of audio frames written so far.

Set the compression type and description. At the moment, only compression type `NONE` is supported, meaning no compression.

Return the compression type ('`NONE`').

Return the human-readable compression type name.

The tuple should be `(nchannels, sampwidth, framerate, nframes, comptype, compname)`, with values valid for the `set*()` methods. Sets all parameters.

Return an `namedtuple()` `(nchannels, sampwidth, framerate, nframes, comptype, compname)` containing the current output parameters.

Return current position in the file, with the same disclaimer for the `Wave_read.tell()` and `Wave_read.setpos()` methods.

Write audio frames, without correcting `nframes`. Changed in version 3.4: Any bytes-like object is now accepted.

Write audio frames and make sure `frames` is correct. It will raise an error if the output stream is not seekable and the total number of frames that have been written after `data` has been written does not match the previously set value for `nframes`. Changed in version 3.4: Any bytes-like object is now accepted. Note that it is invalid to set any parameters after calling `writeframes()` or `writeframesraw()`, and any attempt to do so will raise `wave.Error`.

colorsys— Conversions between color systems

Source code: `Lib/colorsys.py`

The `colorsys` module defines bidirectional conversions of color values between colors expressed in the RGB (Red Green Blue) color space used in computer monitors and three other coordinate systems: YIQ, HLS (Hue Lightness Saturation) and HSV (Hue Saturation Value). Coordinates in all of these color spaces are floating-point values. In the YIQ space, the Y coordinate is between 0 and 1, but the I and Q coordinates can be positive or negative. In all other spaces, the coordinates are all between 0 and 1.

See also

More information about color spaces can be found

at <https://poynton.ca/ColorFAQ.html> and <https://www.cambridgeincolour.com/tutorials/color-spaces.htm>.

The `colorsys` module defines the following functions:

`colorsys.rgb_to_yiq(r,g,b)`

Convert the color from RGB coordinates to YIQ coordinates.

`colorsys.yiq_to_rgb(y,i,q)`

Convert the color from YIQ coordinates to RGB coordinates.

`colorsys.rgb_to_hls(r,g,b)`

Convert the color from RGB coordinates to HLS coordinates.

`colorsys.hls_to_rgb(h,l,s)`

Convert the color from HLS coordinates to RGB coordinates.

`colorsys.rgb_to_hsv(r,g,b)`

Convert the color from RGB coordinates to HSV coordinates.

`colorsys.hsv_to_rgb(h,s,v)`

Convert the color from HSV coordinates to RGB coordinates.

Example:

```
>>> import colorsys
>>> colorsys.rgb_to_hsv(0.2, 0.4, 0.4)
(0.5, 0.5, 0.4)
>>> colorsys.hsv_to_rgb(0.5, 0.5, 0.4)
(0.2, 0.4, 0.4)
```

Internationalization

The modules described in this chapter help you write software that is independent of language and locale by providing mechanisms for selecting a language to be used in program messages or by tailoring output to match local conventions.

The list of modules described in this chapter is:

- gettext— Multilingual internationalization servicesGNUgettextAPIClass-based APITheNullTranslationsclassTheGNUTranslationsclassSolaris message catalog supportThe Catalog constructorInternationalizing your programs and modulesLocalizing your moduleLocalizing your applicationChanging languages on the flyDeferred translationsAcknowledgements
- locale— Internationalization servicesBackground, details, hints, tips and caveatsLocale namesFor extension writers and programs that embed PythonAccess to message catalogs

gettext— Multilingual internationalization services

Source code: `Lib/gettext.py`

The `gettext` module provides internationalization (I18N) and localization (L10N) services for your Python modules and applications. It supports both the GNU `gettext` message catalog API and a higher level, class-based API that may be more appropriate for Python files. The interface described below allows you to write your module and application messages in one natural language, and provide a catalog of translated messages for running under different natural languages.

Some hints on localizing your Python modules and applications are also given.

GNU `gettext` API

The `gettext` module defines the following API, which is very similar to the GNU `gettext` API. If you use this API you will affect the translation of your entire application globally. Often this is what you want if your application is monolingual, with the choice of language dependent on the locale of your user. If you are localizing a Python module, or if your application needs to switch languages on the fly, you probably want to use the class-based API instead.

`gettext.bindtextdomain(domain,localedir=None)`

Bind the domain to the locale directory `localedir`. More concretely, `gettext` will look for binary `.mo` files for the given domain using the path (on Unix): `localedir/language/LC_MESSAGES/domain.mo`, where `language` is searched for in the environment variables `LANGUAGE`, `LC_ALL`, `LC_MESSAGES`, and `LANG` respectively. If `localedir` is omitted or `None`, then the current binding for domain is returned.[1]

`gettext.textdomain(domain=None)`

Change or query the current global domain. If `domain` is `None`, then the current global domain is returned, otherwise the global domain is set to `domain`, which is returned.

`gettext.gettext(message)`

Return the localized translation of `message`, based on the current global domain, language, and locale directory. This function is usually aliased as `_()` in the local namespace (see examples below).

`gettext.dgettext(domain,message)`

Like `gettext()`, but look the message up in the specified domain.

`gettext.ngettext(singular,plural,n)`

Like `gettext()`, but consider plural forms. If a translation is found, apply the plural formula to `n`, and return the resulting message (some languages have more than two plural forms). If no translation is found, return `singular` if `n` is 1; return `plural` otherwise. The Plural formula is taken from the catalog header. It is a C or Python expression that has a free variable `n`; the expression evaluates to the index of the plural in the catalog. See the GNU `gettext` documentation for the precise syntax to be used.

`gettext.dngettext(domain,singular,plural,n)`

Like `ngettext()`, but look the message up in the specified domain.

`gettext.pgettext(context,message)`

`gettext.dpgettext(domain,context,message)`

`gettext.npgettext(context,singular,plural,n)`

gettext.dngettext(domain,context,singular,plural,n)

Similar to the corresponding functions without the prefix (that is, `gettext()`, `dgettext()`, `ngettext()`, `dngettext()`), but the translation is restricted to the given message context. Added in version 3.8.

Note that GNU gettext also defines `adcggettext()` method, but this was deemed not useful and so it is currently unimplemented.

Here's an example of typical usage for this API:

```
import gettext
gettext.bindtextdomain('myapplication', '/path/to/my/language/directory')
gettext.textdomain('myapplication')
_ = gettext.gettext
# ...
print(_('This is a translatable string.'))
```

Class-based API

The class-based API of the `gettext` module gives you more flexibility and greater convenience than the GNU gettext API. It is the recommended way of localizing your Python applications and modules. `gettext` defines a `GNUTranslations` class which implements the parsing of GNU .mo format files, and has methods for returning strings. Instances of this class can also install themselves in the built-in namespace as the `_()` function.

gettext.find(domain,localedir=None,languages=None,all=False)

This function implements the standard .mo file search algorithm. It takes a `domain`, identical to what `textdomain()` takes. Optional `localedir` is as in `bindtextdomain()`. Optional `languages` is a list of strings, where each string is a language code. If `localedir` is not given, then the default system locale directory is used.^[2] If `languages` is not given, then the following environment variables are searched: `LANGUAGE`, `LC_ALL`, `LC_MESSAGES`, and `LANG`. The first one returning a non-empty value is used for the `languages` sv

gettext.translation(domain,localedir=None,languages=None,class_=None,fallback=False)

Return a `*Translations` instance based on the `domain`, `localedir`, and `languages`, which are first passed to `find()` to get a list of the associated .mo file paths. Instances with identical .mo file names are cached. The actual class instantiated is `class_` if provided, otherwise `GNUTranslations`. The class's constructor must take a single file object argument. If multiple files are found, later files are used as fallbacks for earlier ones. To allow setting the fallback, `copy.copy()` is used to clone each translation o

gettext.install(domain,localedir=None,*,names=None)

This installs the `_()` function in Python's builtins namespace, based on `domain` and `localedir` which are passed to the function `translation()`. For the `names` parameter, please see the description of the translation object's `install()` method. As seen below, you usually mark the strings in your application that are candidates for translation, by wrapping them in a call to the `_()` function, like this: `print(_('This string will be translated.'))` For convenience, you want the `_()` function to be installed in Python's built

The NullTranslations class

Translation classes are what actually implement the translation of original source file message strings to translated message strings. The base class used by all translation classes is `NullTranslations`; this provides the basic interface you can use to write your own specialized translation classes. Here are the methods of `NullTranslations`:

class gettext.NullTranslations(fp=None)

`_parse(fp)`

`add_fallback(fallback)`

`gettext(message)`

`ngettext(singular,plural,n)`

`pgettext(context,message)`

`npgettext(context,singular,plural,n)`

`info()`

`charset()`

`install(names=None)`

Takes an optional file object `fp`, which is ignored by the base class. Initializes “protected” instance variables `_info` and `_charset` which are set by derived classes, as well as `_fallback`, which is set through `add_fallback()`. It then calls `self._parse(fp)` if `fp` is not `None`. `_parse(fp)` No-op in the base class, this method takes file object `fp`, and reads the data from the file, initializing its message catalog. If you have an unsupported message catalog file format, you should override this method to parse your

No-op in the base class, this method takes file object `fp`, and reads the data from the file, initializing its message catalog. If you have an unsupported message catalog file format, you should override this method to parse your format.

Add `fallback` as the fallback object for the current translation object. A translation object should consult the fallback if it cannot provide a translation for a given message.

If a fallback has been set, forward `gettext()` to the fallback. Otherwise, return `message`. Overridden in derived classes.

If a fallback has been set, forward `ngettext()` to the fallback. Otherwise, return `singular` if `n` is 1; return `plural` otherwise. Overridden in derived classes.

If a fallback has been set, forward `pgettext()` to the fallback. Otherwise, return the translated message. Overridden in derived classes. Added in version 3.8.

If a fallback has been set, forward `npgettext()` to the fallback. Otherwise, return the translated message. Overridden in derived classes. Added in version 3.8.

Return a dictionary containing the metadata found in the message catalog file.

Return the encoding of the message catalog file.

This method installs `gettext()` into the built-in namespace, binding it to `_`. If the `names` parameter is given, it must be a sequence containing the names of functions you want to install in the built-in namespace in addition to `_`. Supported names are `'gettext'`, `'ngettext'`, `'pgettext'`, and `'npgettext'`. Note that this is only one way, albeit the most convenient way, to make the `_()` function available to your application. Because it affects the entire application globally, and specifically the built-in namespace

The GNUTranslations class

The `gettext` module provides one additional class derived from `NullTranslations`: `GNUTranslations`. This class overrides `_parse()` to enable reading `GNUgettextformat.mof` files in both big-endian and little-endian format.

`GNUTranslations` parses optional metadata out of the translation catalog. It is convention with `GNUgettext` to include metadata as the translation for the empty string. This metadata is in RFC 822-style `key:value` pairs, and should contain the `Project-Id-Version` key. If the `keyContent-Type` is found, then the `charset` property is used to initialize the “protected” `_charset` instance variable, defaulting to `None` if not found. If the charset encoding is

specified, then all message ids and message strings read from the catalog are converted to Unicode using this encoding, else ASCII is assumed.

Since message ids are read as Unicode strings too, all `*gettext()` methods will assume message ids as Unicode strings, not byte strings.

The entire set of key/value pairs are placed into a dictionary and set as the “protected” `_info` instance variable.

If the `.mo` file’s magic number is invalid, the major version number is unexpected, or if other problems occur while reading the file, instantiating a `GNUTranslations` class can raise `OSError`.

class `gettext.GNUTranslations`

`gettext(message)`

`ngettext(singular,plural,n)`

`pgettext(context,message)`

`npgettext(context,singular,plural,n)`

The following methods are overridden from the base class implementation:

- `gettext(message)` Look up the `messageid` in the catalog and return the corresponding message string, as a Unicode string. If there is no entry in the catalog for the `messageid`, and a fallback has been set, the look up is forwarded to the fallback’s `sgettext()` method. Otherwise, the `messageid` is returned.
- `ngettext(singular,plural,n)` Do a plural-forms lookup of a message id. `singular` is used as the message id for purposes of lookup in the catalog, while `n` is used to determine which plural form to use. The returned message string is a Unicode string. If the message id is not found in the catalog, and a fallback is specified, the request is forwarded to the fallback’s `sgettext()` method. Otherwise, when `n` is 1 `singular` is returned, and `plural` is returned in all other cases. Here is an example:
`n=len(os.listdir('.'))cat=GNUTranslations(somefile)`

Look up the `messageid` in the catalog and return the corresponding message string, as a Unicode string. If there is no entry in the catalog for the `messageid`, and a fallback has been set, the look up is forwarded to the fallback’s `sgettext()` method. Otherwise, the `messageid` is returned.

Do a plural-forms lookup of a message id. `singular` is used as the message id for purposes of lookup in the catalog, while `n` is used to determine which plural form to use. The returned message string is a Unicode string. If the message id is not found in the catalog, and a fallback is specified, the request is forwarded to the fallback’s `sgettext()` method. Otherwise, when `n` is 1 `singular` is returned, and `plural` is returned in all other cases. Here is an example:
`n=len(os.listdir('.'))cat=GNUTranslations(somefile)`

Look up the `context` and `messageid` in the catalog and return the corresponding message string, as a Unicode string. If there is no entry in the catalog for the `messageid` and `context`, and a fallback has been set, the look up is forwarded to the fallback’s `spgettext()` method. Otherwise, the `messageid` is returned. Added in version 3.8.

Do a plural-forms lookup of a message id. `singular` is used as the message id for purposes of lookup in the catalog, while `n` is used to determine which plural form to use. If the message id for `context` is not found in the catalog, and a fallback is specified, the request is forwarded to the fallback’s `sngettext()` method. Otherwise, when `n` is 1 `singular` is returned, and `plural` is returned in all other cases. Added in version 3.8.

Solaris message catalog support

The Solaris operating system defines its own binary `.mo` file format, but since no documentation can be found on this format, it is not supported at this time.

The Catalog constructor

GNOME uses a version of the `gettext` module by James Henstridge, but this version has a slightly different API. Its documented usage was:

```
import gettext
cat = gettext.Catalog(domain, localedir)
_ = cat.gettext
print(_('hello world'))
```


For compatibility with this older module, the function `Catalog()` is an alias for the `translation()` function described above.

One difference between this module and Henstridge's: his catalog objects supported access through a mapping API, but this appears to be unused and so is not currently supported.

Internationalizing your programs and modules

Internationalization (I18N) refers to the operation by which a program is made aware of multiple languages. Localization (L10N) refers to the adaptation of your program, once internationalized, to the local language and cultural habits. In order to provide multilingual messages for your Python programs, you need to take the following steps:

- prepare your program or module by specially marking translatable strings
- run a suite of tools over your marked files to generate raw messages catalogs
- create language-specific translations of the message catalogs
- use the `gettext` module so that message strings are properly translated

In order to prepare your code for I18N, you need to look at all the strings in your files. Any string that needs to be translated should be marked by wrapping it in `_('...')`—that is, a call to the function `_`. For example:

```
filename = 'mylog.txt'
message = _('writing a log message')
with open(filename, 'w') as fp:
    fp.write(message)
```

In this example, the string `'writing a log message'` is marked as a candidate for translation, while the strings `'mylog.txt'` and `'w'` are not.

There are a few tools to extract the strings meant for translation. The original GNU `gettext` only supported C or C++ source code but its extended version `xgettext` scans code written in a number of languages, including Python, to find strings marked as translatable. `Babel` is a Python internationalization library that includes `pybabel` script to extract and compile message catalogs. François Pinard's program called `xpot` does a similar job and is available as part of his `po-utils` package.

(Python also includes pure-Python versions of these programs, called `pygettext.py` and `msgfmt.py`; some Python distributions will install them for you. `pygettext.py` is similar to `xgettext`, but only understands Python source code and cannot handle other programming languages such as C or C++. `pygettext.py` supports a command-line interface similar to `xgettext`; for details on its use, run `pygettext.py --help`. `msgfmt.py` is binary compatible with GNU `msgfmt`. With these two programs, you may not need the GNU `gettext` package to internationalize your Python applications.)

`xgettext`, `pygettext`, and similar tools generate `.po` files that are message catalogs. They are structured human-readable files that contain every marked string in the source code, along with a placeholder for the translated versions of these strings.

Copies of these `.po` files are then handed over to the individual human translators who write translations for every supported natural language. They send back the completed language-specific versions as a `<language-name>.po` file that's compiled into a machine-readable `.mo` binary catalog file using the `msgfmt` program. The `.mo` files are used by the `gettext` module for the actual translation processing at run-time.

How you use the `gettext` module in your code depends on whether you are internationalizing a single module or your entire application. The next two sections will discuss each case.

Localizing your module

If you are localizing your module, you must take care not to make global changes, e.g. to the built-in namespace. You should not use the GNUgettextAPI but instead the class-based API.

Let's say your module is called "spam" and the module's various natural language translation.mo files reside in /usr/share/locale in GNUgettext format. Here's what you would put at the top of your module:

```
import gettext
t = gettext.translation('spam', '/usr/share/locale')
_ = t.gettext
```

Localizing your application

If you are localizing your application, you can install the `_()` function globally into the built-in namespace, usually in the main driver file of your application. This will let all your application-specific files just use `_('...')` without having to explicitly install it in each file.

In the simple case then, you need only add the following bit of code to the main driver file of your application:

```
import gettext
gettext.install('myapplication')
```

If you need to set the locale directory, you can pass it into the `install()` function:

```
import gettext
gettext.install('myapplication', '/usr/share/locale')
```

Changing languages on the fly

If your program needs to support many languages at the same time, you may want to create multiple translation instances and then switch between them explicitly, like so:

```
import gettext

lang1 = gettext.translation('myapplication', languages=['en'])
lang2 = gettext.translation('myapplication', languages=['fr'])
lang3 = gettext.translation('myapplication', languages=['de'])

# start by using language1
lang1.install()

# ... time goes by, user selects language 2
lang2.install()

# ... more time goes by, user selects language 3
lang3.install()
```

Deferred translations

In most coding situations, strings are translated where they are coded. Occasionally however, you need to mark strings for translation, but defer actual translation until later. A classic example is:

```
animals = ['mollusk',
           'albatross',
           'rat',
           'penguin',
           'python', ]

# ...
for a in animals:
    print(a)
```

Here, you want to mark the strings in the `animals` list as being translatable, but you don't actually want to translate them until they are printed.

Here is one way you can handle this situation:

```
def _(message): return message
```

```
animals = [_('mollusk'),
            _('albatross'),
            _('rat'),
            _('penguin'),
            _('python'), ]
```

```
del _
```

```
# ...
for a in animals:
    print(_(a))
```

This works because the dummy definition of `_()` simply returns the string unchanged. And this dummy definition will temporarily override any definition of `_()` in the built-in namespace (until the `del` command). Take care, though if you have a previous definition of `_()` in the local namespace.

Note that the second use of `_()` will not identify “a” as being translatable to the `gettext` program, because the parameter is not a string literal.

Another way to handle this is with the following example:

```
def N_(message): return message
```

```
animals = [N_('mollusk'),
            N_('albatross'),
            N_('rat'),
            N_('penguin'),
            N_('python'), ]
```

```
# ...
for a in animals:
    print(_(a))
```

In this case, you are marking translatable strings with the function `N_()`, which won’t conflict with any definition of `_()`. However, you will need to teach your message extraction program to look for translatable strings marked with `N_()`. `xgettext`, `pygettext`, `pybabel extract`, and `xpotall` support this through the use of the `-k` command-line switch. The choice of `N_()` here is totally arbitrary; it could have just as easily been `MarkThisStringForTranslation()`.

Acknowledgements

The following people contributed code, feedback, design suggestions, previous implementations, and valuable experience to the creation of this module:

- Peter Funk
- James Henstridge
- Juan David Ibáñez Palomar
- Marc-André Lemburg
- Martin von Löwis
- François Pinard
- Barry Warsaw
- Gustavo Niemeyer

Footnotes

The default locale directory is system dependent; for example, on Red Hat Linux it is `/usr/share/locale`, but on Solaris it is `/usr/lib/locale`. The `gettext` module does not try to support these system dependent defaults; instead its default is `sys.base_prefix/share/locale` (see `sys.base_prefix`). For this reason, it is always best to call `bindtextdomain()` with an explicit absolute path at the start of your application.

See the footnote for `bindtextdomain()` above.

locale— Internationalization services

Source code: `Lib/locale.py`

The `locale` module opens access to the POSIX locale database and functionality. The POSIX locale mechanism allows programmers to deal with certain cultural issues in an application, without requiring the programmer to know all the specifics of each country where the software is executed.

The `locale` module is implemented on top of the `_locale` module, which in turn uses an ANSI C locale implementation if available.

The `locale` module defines the following exception and functions:

exception `locale.Error`

Exception raised when the locale passed to `setlocale()` is not recognized.

`locale.setlocale(category, locale=None)`

If `locale` is given and not `None`, `setlocale()` modifies the locale setting for the `category`. The available categories are listed in the data description below. `locale` may be a string, or a pair, language code and encoding. An empty string specifies the user's default settings. If the modification of the locale fails, the exception `Error` is raised. If successful, the new locale setting is returned. If `locale` is a pair, it is converted to a locale name using the locale aliasing engine. The language code has the s

`locale.localeconv()`

Returns the database of the local conventions as a dictionary. This dictionary has the following strings as keys: `Category` `Key` `Meaning` `LC_NUMERIC` `'decimal_point'` `'Decimal point character.'` `'grouping'` `'Sequence of numbers specifying which relative positions the` `'thousands_sep'` `'is expected. If the sequence is terminated with` `CHAR_MAX`, no further grouping is performed. If the sequence terminates with a 0, the last group size is repeatedly used. `'thousands_sep'` `'Character used between groups.` `LC_MONETARY` `'int_curr_sy`

`locale.nl_langinfo(option)`

`locale.CODESET`

`locale.D_T_FMT`

`locale.D_FMT`

`locale.T_FMT`

`locale.T_FMT_AMPM`

`locale.DAY_1`

`locale.DAY_2`

`locale.DAY_3`

`locale.DAY_4`

`locale.DAY_5`

`locale.DAY_6`

`locale.DAY_7`

`locale.ABDAY_1`

`locale.ABDAY_2`

`locale.ABDAY_3`

locale.ABDAY_4
locale.ABDAY_5
locale.ABDAY_6
locale.ABDAY_7
locale.MON_1
locale.MON_2
locale.MON_3
locale.MON_4
locale.MON_5
locale.MON_6
locale.MON_7
locale.MON_8
locale.MON_9
locale.MON_10
locale.MON_11
locale.MON_12
locale.ABMON_1
locale.ABMON_2
locale.ABMON_3
locale.ABMON_4
locale.ABMON_5
locale.ABMON_6
locale.ABMON_7
locale.ABMON_8
locale.ABMON_9
locale.ABMON_10
locale.ABMON_11
locale.ABMON_12
locale.RADIXCHAR
locale.THOUSEP
locale.YESEXP
locale.NOEXPR
locale.CRNCYSTR
locale.ERA
locale.ERA_D_T_FMT
locale.ERA_D_FMT

locale.ERA_T_FMT

locale.ALT_DIGITS

Return some locale-specific information as a string. This function is not available on all systems, and the set of possible options might also vary across platforms. The possible argument values are numbers, for which symbolic constants are available in the locale module. Then `l_langinfo()` function accepts one of the following keys. Most descriptions are taken from the corresponding description in the GNU C library. `locale.CODESET` Get a string with the name of the character encoding used in the se

Get a string with the name of the character encoding used in the selected locale.

Get a string that can be used as a format string for `time.strftime()` to represent date and time in a locale-specific way.

Get a string that can be used as a format string for `time.strftime()` to represent a date in a locale-specific way.

Get a string that can be used as a format string for `time.strftime()` to represent a time in a locale-specific way.

Get a format string for `time.strftime()` to represent time in the am/pm format.

Get the name of the n-th day of the week. Note This follows the US convention of `DAY_1` being Sunday, not the international convention (ISO 8601) that Monday is the first day of the week.

Get the abbreviated name of the n-th day of the week.

Get the name of the n-th month.

Get the abbreviated name of the n-th month.

Get the radix character (decimal dot, decimal comma, etc.).

Get the separator character for thousands (groups of three digits).

Get a regular expression that can be used with the `regex` function to recognize a positive response to a yes/no question.

Get a regular expression that can be used with the `regex(3)` function to recognize a negative response to a yes/no question. Note The regular expressions for `YESEXPR` and `NOEXPR` use syntax suitable for the `regex` function from the C library, which might differ from the syntax used in re.

Get the currency symbol, preceded by “-” if the symbol should appear before the value, “+” if the symbol should appear after the value, or “.” if the symbol should replace the radix character.

Get a string which describes how years are counted and displayed for each era in a locale. Most locales do not define this value. An example of a locale which does define this value is the Japanese one. In Japan, the traditional representation of dates includes the name of the era corresponding to the then-emperor's reign. Normally it should not be necessary to use this value directly. Specifying the `E` modifier in their format strings causes the `time.strftime()` function to use this information. The

Get a format string for `time.strftime()` to represent date and time in a locale-specific era-based way.

Get a format string for `time.strftime()` to represent a date in a locale-specific era-based way.

Get a format string for `time.strftime()` to represent a time in a locale-specific era-based way.

Get a string consisting of up to 100 semicolon-separated symbols used to represent the values 0 to 99 in a locale-specific way. In most locales this is an empty string.

locale.getdefaultlocale([envvars])

Tries to determine the default locale settings and returns them as a tuple of the form (languagecode, encoding). According to POSIX, a program which has not called `setlocale(LC_ALL, "")` runs using the portable 'C' locale. Calling `setlocale(LC_ALL, "")` lets it use the default locale as defined by

the `LANG` variable. Since we do not want to interfere with the current locale setting we thus emulate the behavior in the way described above. To maintain compatibility with other platforms, not only the `LANG` variable

locale.getlocale(category=LC_CTYPE)

Returns the current setting for the given locale category as a tuple containing the language code and encoding. `category` may be one of the `LC_*` values except `LC_ALL`. It defaults to `LC_CTYPE`. The language code has the same format as a locale name, but without encoding and `@`-modifier. The language code and encoding may be `None` if their values cannot be determined. The "C" locale is represented as `(None, None)`.

locale.getpreferredencoding(do_setlocale=True)

Return the locale encoding used for text data, according to user preferences. User preferences are expressed differently on different systems, and might not be available programmatically on some systems, so this function only returns a guess. On some systems, it is necessary to invoke `setlocale()` to obtain the user preferences, so this function is not thread-safe. If invoking `setlocale` is not necessary or desired, `do_setlocale` should be set to `False`. On Android or if the Python UTF-8 Mode is enabled, always

locale.getencoding()

Get the current locale encoding: On Android and VxWorks, return "utf-8". On Unix, return the encoding of the current `LC_CTYPE` locale. Return "utf-8" if `nl_langinfo(CODESET)` returns an empty string: for example, if the current `LC_CTYPE` locale is not supported. On Windows, return the ANSI code page. The Python preinitialization configures the `LC_CTYPE` locale. See also the filesystem encoding and error handler. This function is similar to `getpreferredencoding(False)` except this function ignores the Python UTF-8 Mode.

locale.normalize(localename)

Returns a normalized locale code for the given locale name. The returned locale code is formatted for use with `setlocale()`. If normalization fails, the original name is returned unchanged. If the given encoding is not known, the function defaults to the default encoding for the locale code just like `setlocale()`.

locale.strcoll(string1, string2)

Compares two strings according to the current `LC_COLLATE` setting. As any other compare function, returns a negative, or a positive value, or 0, depending on whether `string1` collates before or after `string2` or is equal to it.

locale.strxfrm(string)

Transforms a string to one that can be used in locale-aware comparisons. For example, `strxfrm(s1) < strxfrm(s2)` is equivalent to `strcoll(s1, s2) < 0`. This function can be used when the same string is compared repeatedly, e.g. when collating a sequence of strings.

locale.format_string(format, val, grouping=False, monetary=False)

Formats a number `val` according to the current `LC_NUMERIC` setting. The format follows the conventions of the `%` operator. For floating-point values, the decimal point is modified if appropriate. If `grouping` is `True`, also takes the grouping into account. If `monetary` is `True`, the conversion uses monetary thousands separator and grouping strings. Processes formatting specifiers as in `format % val`, but takes the current locale settings into account. Changed in version 3.7: The `monetary` keyword parameter was added.

locale.currency(val, symbol=True, grouping=False, international=False)

Formats a number `val` according to the current `LC_MONETARY` settings. The returned string includes the currency symbol if `symbol` is `True`, which is the default. If `grouping` is `True` (which is not the default), grouping is done with the value. If `international` is `True` (which is not the default), the international currency symbol is used. Note This function will not work with the 'C' locale, so you have to set a locale via `setlocale()` first.

locale.str(float)

Formats a floating-point number using the same format as the built-in function `str(float)`, but takes the decimal point into account.

locale.delocalize(string)

Converts a string into a normalized number string, following the `LC_NUMERIC` settings. Added in version 3.5.

locale.localize(string, grouping=False, monetary=False)

Converts a normalized number string into a formatted string following the `LC_NUMERIC` settings. Added in version 3.10.

locale.atof(string, func=float)

Converts a string to a number, following the `LC_NUMERIC` settings, by calling `func` on the result of calling `delocalize()` on `string`.

locale.atoi(string)

Converts a string to an integer, following the `LC_NUMERIC` conventions.

locale.LC_CTYPE

Locale category for the character type functions. Most importantly, this category defines the text encoding, i.e. how bytes are interpreted as Unicode codepoints. See [PEP 538](#) and [PEP 540](#) for how this variable might be automatically coerced to C.UTF-8 to avoid issues created by invalid settings in containers or incompatible settings passed over remote SSH connections. Python doesn't internally use locale-dependent character transformation functions from `ctype.h`. Instead, `pyctype.h` provides locale-indepen

locale.LC_COLLATE

Locale category for sorting strings. The functions `strcoll()` and `strxfrm()` of the `locale` module are affected.

locale.LC_TIME

Locale category for the formatting of time. The function `time.strftime()` follows these conventions.

locale.LC_MONETARY

Locale category for formatting of monetary values. The available options are available from the `localeconv()` function.

locale.LC_MESSAGES

Locale category for message display. Python currently does not support application specific locale-aware messages. Messages displayed by the operating system, like those returned by `os.strerror()` might be affected by this category. This value may not be available on operating systems not conforming to the POSIX standard, most notably Windows.

locale.LC_NUMERIC

Locale category for formatting numbers. The functions `format_string()`, `atoi()`, `atof()` and `str()` of the `locale` module are affected by that category. All other numeric formatting operations are not affected.

locale.LC_ALL

Combination of all locale settings. If this flag is used when the locale is changed, setting the locale for all categories is attempted. If that fails for any category, no category is changed at all. When the locale is retrieved using this flag, a string indicating the setting for all categories is returned. This string can be later used to restore the settings.

locale.CHAR_MAX

This is a symbolic constant used for different values returned by `localeconv()`.

Background, details, hints, tips and caveats

The C standard defines the locale as a program-wide property that may be relatively expensive to change. On top of that, some implementations are broken in such a way that frequent locale changes may cause core dumps. This makes the locale somewhat painful to use correctly.

Initially, when a program is started, the locale is the `C` locale, no matter what the user's preferred locale is. There is one exception: the `LC_CTYPE` category is changed at startup to set the current locale encoding to the user's preferred locale encoding. The program must explicitly say that it wants the user's preferred locale settings for other categories by calling `setlocale(LC_ALL, "")`.

It is generally a bad idea to call `setlocale()` in some library routine, since as a side effect it affects the entire program. Saving and restoring it is almost as bad: it is expensive and affects other threads that happen to run before the settings have been restored.

If, when coding a module for general use, you need a locale independent version of an operation that is affected by the locale (such as certain formats used with `time.strftime()`), you will have to find a way to do it without using the standard library routine. Even better is convincing yourself that using locale settings is okay. Only as a last resort should you document that your module is not compatible with non-`C` locale settings.

The only way to perform numeric operations according to the locale is to use the special functions defined by this module: `atof()`, `atoi()`, `format_string()`, `str()`.

There is no way to perform case conversions and character classifications according to the locale. For (Unicode) text strings these are done according to the character value only, while for byte strings, the conversions and classifications are done according to the ASCII value of the byte, and bytes whose high bit is set (i.e., non-ASCII bytes) are never converted or considered part of a character class such as letter or whitespace.

Locale names

The format of the locale name is platform dependent, and the set of supported locales can depend on the system configuration.

On Posix platforms, it usually has the format[1]:

```
language ["_" territory] [ "." charset] [ "@" modifier]
```

where `language` is a two- or three-letter language code from ISO 639, `territory` is a two-letter country or region code from ISO 3166, `charset` is a locale encoding, and `modifier` is a script name, a language subtag, a sort order identifier, or other locale modifier (for example, "latin", "valencia", "stroke" and "euro").

On Windows, several formats are supported.[2][3] A subset of IETF BCP 47 tags:

```
language ["-" script] ["-" territory] [ "." charset]
language ["-" script] "-" territory "-" modifier
```

where `language` and `territory` have the same meaning as in Posix, `script` is a four-letter script code from ISO 15924, and `modifier` is a language subtag, a sort order identifier or custom modifier (for example, "valencia", "stroke" or "x-python"). Both hyphen ('-') and underscore ('_') separators are supported. Only UTF-8 encoding is allowed for BCP 47 tags.

Windows also supports locale names in the format:

```
language ["_" territory] [ "." charset]
```

where `language` and `territory` are full names, such as “English” and “United States”, and `charset` is either a code page number (for example, “1252”) or UTF-8. Only the underscore separator is supported in this format.

The “C” locale is supported on all platforms.

IEEE Std 1003.1-2024; 8.2 Internationalization Variables

UCRT Locale names, Languages, and Country/Region strings

Locale Names

For extension writers and programs that embed Python

Extension modules should never call `setlocale()`, except to find out what the current locale is. But since the return value can only be used portably to restore it, that is not very useful (except perhaps to find out whether or not the locale is C).

When Python code uses the `locale` module to change the locale, this also affects the embedding application. If the embedding application doesn't want this to happen, it should remove the `_localeextension` module (which does all the work) from the table of built-in modules in `theconfig.cfile`, and make sure that the `_locale` module is not accessible as a shared library.

Access to message catalogs

`locale.gettext(msg)`

`locale.dgettext(domain,msg)`

`locale.dcgettext(domain,msg,category)`

`locale.textdomain(domain)`

`locale.bindtextdomain(domain,dir)`

`locale.bind_textdomain_codeset(domain,codeset)`

The `locale` module exposes the C library's `gettext` interface on systems that provide this interface. It consists of the functions `gettext()`, `dgettext()`, `dcgettext()`, `textdomain()`, `bindtextdomain()`, and `bind_textdomain_codeset()`. These are similar to the same functions in the `gettext` module, but use the C library's binary format for message catalogs, and the C library's search algorithms for locating message catalogs.

Python applications should normally find no need to invoke these functions, and should use `gettext` instead. A known exception to this rule are applications that link with additional C libraries which internally invoke C functions `gettext` or `dcgettext`. For these applications, it may be necessary to bind the text domain, so that the libraries can properly locate their message catalogs.

Graphical user interfaces with Tk

Tk/Tcl has long been an integral part of Python. It provides a robust and platform independent windowing toolkit, that is available to Python programmers using the `tkinter` package, and its extension, the `tkinter.ttk` module.

The `tkinter` package is a thin object-oriented layer on top of Tcl/Tk. To use `tkinter`, you don't need to write Tcl code, but you will need to consult the Tk documentation, and occasionally the Tcl documentation. `tkinter` is a set of wrappers that implement the Tk widgets as Python classes.

`tkinter`'s chief virtues are that it is fast, and that it usually comes bundled with Python. Although its standard documentation is weak, good material is available, which includes: references, tutorials, a book and others. `tkinter` is also famous for having an outdated look and feel, which has been vastly improved in Tk 8.5. Nevertheless, there are many other GUI libraries that you could be interested in. The Python wiki lists several alternative GUI frameworks and tools.

- `tkinter`— Python interface to Tcl/Tk
Architecture
Tkinter Modules
Tkinter Life Preserver
A Hello World Program
Important Tk Concepts
Understanding How Tkinter Wraps Tcl/Tk
How do I...? What option does...?
Navigating the Tcl/Tk Reference Manual
Threading model
Handy Reference
Setting Options
The Packer
Packer Options
Coupling Widget Variables
The Window Manager
Tk Option Data Types
Bindings and Events
The index
Parameters
- `tkinter.colorchooser`— Color choosing dialog
- `tkinter.font`— Tkinter font wrapper
- Tkinter Dialogs
`tkinter.simpledialog`— Standard Tkinter input dialog
`tkinter.filedialog`— File selection dialogs
Native Load/Save Dialog
`tkinter.commondialog`— Dialog window templates
- `tkinter.messagebox`— Tkinter message prompts
- `tkinter.scrolledtext`— Scrolled Text Widget
- `tkinter.dnd`— Drag and drop support
- `tkinter.ttk`— Tk themed widgets
Using Tk
Ttk Widgets
Widget Standard Options
Scrollable Widget Options
Label Options
Compatibility Options
Widget State
`ttk.Widget`
`Combobox`
Options
Virtual events
`ttk.Combobox`
`Spinbox`
Options
Virtual events
`ttk.Spinbox`
`Notebook`
Options
Tab Options
Tab Identifiers
Virtual Events
`ttk.Notebook`
`Progressbar`
Options
`ttk.Progressbar`
`Separator`
Options
Sizegrip
Platform-specific notes
Bugs
Treeview
Options
- IDLE — Python editor and shell
Menus
File menu (Shell and Editor)
Edit menu (Shell and Editor)
Format menu (Editor window only)
Run menu (Editor window only)
Shell menu (Shell window only)
Debug menu (Shell window only)
Options menu (Shell and Editor)
Window menu (Shell and Editor)
Help menu (Shell and Editor)
Context menus
Editing and Navigation
Editor windows
Key bindings
Automatic indentation
Search and Replace
- `turtle`— Turtle graphics
Introduction
Get started
Tutorial
Starting a turtle environment
Basic drawing
Pen control
The turtle's position
Making algorithmic patterns
How to...
Get started as quickly as possible
Automatically begin and end filling
Use the `turtle` module namespace
Use turtle graphics in a script
Use object-oriented turtle graphics
Turtle graphics reference
Turtle methods
Methods of Turtle
Screen/ScreenMethods

tkinter— Python interface to Tcl/Tk

Source code: `Lib/tkinter/__init__.py`

The `tkinter` package (“Tk interface”) is the standard Python interface to the Tcl/Tk GUI toolkit. Both Tk and `tkinter` are available on most Unix platforms, including macOS, as well as on Windows systems.

Running `python -mtkinter` from the command line should open a window demonstrating a simple Tk interface, letting you know that `tkinter` is properly installed on your system, and also showing what version of Tcl/Tk is installed, so you can read the Tcl/Tk documentation specific to that version.

Tkinter supports a range of Tcl/Tk versions, built either with or without thread support. The official Python binary release bundles Tcl/Tk 8.6 threaded. See the source code for the `_tkinter` module for more information about supported versions.

Tkinter is not a thin wrapper, but adds a fair amount of its own logic to make the experience more pythonic. This documentation will concentrate on these additions and changes, and refer to the official Tcl/Tk documentation for details that are unchanged.

■ **Note:** Note Tcl/Tk 8.5 (2007) introduced a modern set of themed user interface components along with a new API to use them. Both old and new APIs are still available. Most documentation you will find online still uses the old API and can be woefully outdated.

This is an optional module. If it is missing from your copy of CPython, look for documentation from your distributor (that is, whoever provided Python to you). If you are the distributor, see [Requirements for optional modules](#).

See also

- [TkDocs](#) Extensive tutorial on creating user interfaces with Tkinter. Explains key concepts, and illustrates recommended approaches using the modern API.
- [Tkinter 8.5 reference: a GUI for Python](#) Reference documentation for Tkinter 8.5 detailing available classes, methods, and options.

Tcl/Tk Resources:

- [Tk commands](#) Comprehensive reference to each of the underlying Tcl/Tk commands used by Tkinter.
- [Tcl/Tk Home Page](#) Additional documentation, and links to Tcl/Tk core development.

Books:

- [Modern Tkinter for Busy Python Developers](#) By Mark Roseman. (ISBN 978-1999149567)
- [Python GUI programming with Tkinter](#) By Alan D. Moore. (ISBN 978-1788835886)
- [Programming Python](#) By Mark Lutz; has excellent coverage of Tkinter. (ISBN 978-0596158101)
- [Tcl and the Tk Toolkit \(2nd edition\)](#) By John Ousterhout, inventor of Tcl/Tk, and Ken Jones; does not cover Tkinter. (ISBN 978-0321336330)

Architecture

Tcl/Tk is not a single library but rather consists of a few distinct modules, each with separate functionality and its own official documentation. Python’s binary releases also ship an add-on module together with it.

Tcl

Tk

Ttk

Tcl is a dynamic interpreted programming language, just like Python. Though it can be used on its own as a general-purpose programming language, it is most commonly embedded into C applications as a scripting engine or an interface to the Tk toolkit. The Tcl library has a C interface to create and manage one or more instances of a Tcl interpreter, run Tcl commands and scripts in those instances, and add custom commands implemented in either Tcl or C. Each interpreter has an event queue, and ther

Tk is aTcl packageimplemented in C that adds custom commands to create and manipulate GUI widgets. EachTkobject embeds its own Tcl interpreter instance with Tk loaded into it. Tk's widgets are very customizable, though at the cost of a dated appearance. Tk uses Tcl's event queue to generate and process GUI events.

Themed Tk (Ttk) is a newer family of Tk widgets that provide a much better appearance on different platforms than many of the classic Tk widgets. Ttk is distributed as part of Tk, starting with Tk version 8.5. Python bindings are provided in a separate module,tkinter.ttk.

Internally, Tk and Ttk use facilities of the underlying operating system, i.e., Xlib on Unix/X11, Cocoa on macOS, GDI on Windows.

When your Python application uses a class in Tkinter, e.g., to create a widget, thetkintermodule first assembles a Tcl/Tk command string. It passes that Tcl command string to an internal_tkinterbinary module, which then calls the Tcl interpreter to evaluate it. The Tcl interpreter will then call into the Tk and/or Ttk packages, which will in turn make calls to Xlib, Cocoa, or GDI.

Tkinter Modules

Support for Tkinter is spread across several modules. Most applications will need the maintkintermodule, as well as thetkinter.ttkmodule, which provides the modern themed widget set and API:

```
from tkinter import *
from tkinter import ttk
```

```
classTkinter.Tk(screenName=None,baseName=None,className='Tk',useTk=True,sync=False,use=None
)
```

screenName

baseName

className

useTk

sync

use

tk

master

children

Construct a toplevel Tk widget, which is usually the main window of an application, and initialize a Tcl interpreter for this widget. Each instance has its own associated Tcl interpreter.TheTkclass is typically instantiated using all default values. However, the following keyword arguments are currently recognized:screenNameWhen given (as a string), sets theDISPLAYenvironment variable. (X11 only)baseNameName of the profile file. By default,baseNameis derived from the program name (sys.argv[0])

When given (as a string), sets theDISPLAYenvironment variable. (X11 only)

Name of the profile file. By default, `baseName` is derived from the program name (`sys.argv[0]`).

Name of the widget class. Used as a profile file and also as the name with which Tcl is invoked (`argv[0]` in `interp`).

If `True`, initialize the Tk subsystem. The `tkinter.Tcl()` function sets this to `False`.

If `True`, execute all X server commands synchronously, so that errors are reported immediately. Can be used for debugging. (X11 only)

Specifies the id of the window in which to embed the application, instead of it being created as an independent toplevel window. `id` must be specified in the same way as the value for the `-use` option for toplevel widgets (that is, it has a form like that returned by `winfo_id()`). Note that on some platforms this will only work correctly if `id` refers to a Tk frame or toplevel that has its `-container` option enabled.

The Tk application object created by instantiating `Tk`. This provides access to the Tcl interpreter. Each widget that is attached the same instance of `Tk` has the same value for its `tk` attribute.

The widget object that contains this widget. For `Tk`, the master is `None` because it is the main window. The terms `master` and `parent` are similar and sometimes used interchangeably as argument names; however, calling `winfo_parent()` returns a string of the widget name whereas `master` returns the object. `parent/child` reflects the tree-like relationship while `master(or container)/content` reflects the container structure.

The immediate descendants of this widget as a dict with the child widget names as the keys and the child instance objects as the values.

`tkinter.Tcl(screenName=None, baseName=None, className='Tk', useTk=False)`

The `Tcl()` function is a factory function which creates an object much like that created by the `Tk` class, except that it does not initialize the Tk subsystem. This is most often useful when driving the Tcl interpreter in an environment where one doesn't want to create extraneous toplevel windows, or where one cannot (such as Unix/Linux systems without an X server). An object created by the `Tcl()` object can have a Toplevel window created (and the Tk subsystem initialized) by calling its `loadtk()` method.

The modules that provide Tk support include:

`tkinter`

`tkinter.colorchooser`

`tkinter.commondialog`

`tkinter.filedialog`

`tkinter.font`

`tkinter.messagebox`

`tkinter.scrolledtext`

`tkinter.simpledialog`

`tkinter.ttk`

Main Tkinter module.

Dialog to let the user choose a color.

Base class for the dialogs defined in the other modules listed here.

Common dialogs to allow the user to specify a file to open or save.

Utilities to help work with fonts.

Access to standard Tk dialog boxes.

Text widget with a vertical scroll bar built in.

Basic dialogs and convenience functions.

Themed widget set introduced in Tk 8.5, providing modern alternatives for many of the classic widgets in the `maintkinter` module.

Additional modules:

`_tkinter`

`idlelib`

`tkinter.constants`

`tkinter.dnd`

`turtle`

A binary module that contains the low-level interface to Tcl/Tk. It is automatically imported by the `maintkinter` module, and should never be used directly by application programmers. It is usually a shared library (or DLL), but might in some cases be statically linked with the Python interpreter.

Python's Integrated Development and Learning Environment (IDLE). Based on `ontkinter`.

Symbolic constants that can be used in place of strings when passing various parameters to Tkinter calls. Automatically imported by the `maintkinter` module.

(experimental) Drag-and-drop support for `tkinter`. This will become deprecated when it is replaced with the Tk DND.

Turtle graphics in a Tk window.

Tkinter Life Preserver

This section is not designed to be an exhaustive tutorial on either Tk or Tkinter. For that, refer to one of the external resources noted earlier. Instead, this section provides a very quick orientation to what a Tkinter application looks like, identifies foundational Tk concepts, and explains how the Tkinter wrapper is structured.

The remainder of this section will help you to identify the classes, methods, and options you'll need in your Tkinter application, and where to find more detailed documentation on them, including in the official Tcl/Tk reference manual.

A Hello World Program

We'll start by walking through a "Hello World" application in Tkinter. This isn't the smallest one we could write, but has enough to illustrate some key concepts you'll need to know.

```
from tkinter import *
from tkinter import ttk
root = Tk()
frm = ttk.Frame(root, padding=10)
frm.grid()
ttk.Label(frm, text="Hello World!").grid(column=0, row=0)
ttk.Button(frm, text="Quit", command=root.destroy).grid(column=1, row=0)
root.mainloop()
```

After the imports, the next line creates an instance of the `Tk` class, which initializes Tk and creates its associated Tcl interpreter. It also creates a toplevel window, known as the root window, which serves as the main window of the application.

The following line creates a frame widget, which in this case will contain a label and a button we'll create next. The frame is fit inside the root window.

The next line creates a label widget holding a static text string. The `grid()` method is used to specify the relative layout (position) of the label within its containing frame widget, similar to how tables in HTML work.

A button widget is then created, and placed to the right of the label. When pressed, it will call the `destroy()` method of the root window.

Finally, the `mainloop()` method puts everything on the display, and responds to user input until the program terminates.

Important Tk Concepts

Even this simple program illustrates the following key Tk concepts:

widgets

widget hierarchy

configuration options

geometry management

event loop

A Tkinter user interface is made up of individual widgets. Each widget is represented as a Python object, instantiated from classes like `tk.Frame`, `tk.Label`, and `tk.Button`.

Widgets are arranged in a hierarchy. The label and button were contained within a frame, which in turn was contained within the root window. When creating each child widget, its parent widget is passed as the first argument to the widget constructor.

Widgets have configuration options, which modify their appearance and behavior, such as the text to display in a label or button. Different classes of widgets will have different sets of options.

Widgets aren't automatically added to the user interface when they are created. A geometry manager like `grid` controls where in the user interface they are placed.

Tkinter reacts to user input, changes from your program, and even refreshes the display only when actively running an event loop. If your program isn't running the event loop, your user interface won't update.

Understanding How Tkinter Wraps Tcl/Tk

When your application uses Tkinter's classes and methods, internally Tkinter is assembling strings representing Tcl/Tk commands, and executing those commands in the Tcl interpreter attached to your application's `Tk` instance.

Whether it's trying to navigate reference documentation, trying to find the right method or option, adapting some existing code, or debugging your Tkinter application, there are times that it will be useful to understand what those underlying Tcl/Tk commands look like.

To illustrate, here is the Tcl/Tk equivalent of the main part of the Tkinter script above.

```
ttk::frame .frm -padding 10
grid .frm
grid [ttk::label .frm.lbl -text "Hello World!"] -column 0 -row 0
grid [ttk::button .frm.btn -text "Quit" -command "destroy ."] -column 1 -row 0
```

Tcl's syntax is similar to many shell languages, where the first word is the command to be executed, with arguments to that command following it, separated by spaces. Without getting into too many details, notice the following:

- The commands used to create widgets (like `ttk::frame`) correspond to widget classes in Tkinter.
- Tcl widget options (like `-text`) correspond to keyword arguments in Tkinter.

- Widgets are referred to by a pathname in Tcl (like `.frm.btn`), whereas Tkinter doesn't use names but object references.
- A widget's place in the widget hierarchy is encoded in its (hierarchical) pathname, which uses a `.` (dot) as a path separator. The pathname for the root window is just `.` (dot). In Tkinter, the hierarchy is defined not by pathname but by specifying the parent widget when creating each child widget.
- Operations which are implemented as separate commands in Tcl (like `grid` or `destroy`) are represented as methods on Tkinter widget objects. As you'll see shortly, at other times Tcl uses what appear to be method calls on widget objects, which more closely mirror what is used in Tkinter.

How do I...? What option does...?

If you're not sure how to do something in Tkinter, and you can't immediately find it in the tutorial or reference documentation you're using, there are a few strategies that can be helpful.

First, remember that the details of how individual widgets work may vary across different versions of both Tkinter and Tcl/Tk. If you're searching documentation, make sure it corresponds to the Python and Tcl/Tk versions installed on your system.

When searching for how to use an API, it helps to know the exact name of the class, option, or method that you're using. Introspection, either in an interactive Python shell or with `print()`, can help you identify what you need.

To find out what configuration options are available on any widget, call its `configure()` method, which returns a dictionary containing a variety of information about each object, including its default and current values. Use `keys()` to get just the names of each option.

```
btn = ttk.Button(frm, ...)
print(btn.configure().keys())
```

As most widgets have many configuration options in common, it can be useful to find out which are specific to a particular widget class. Comparing the list of options to that of a simpler widget, like a frame, is one way to do that.

```
print(set(btn.configure().keys()) - set(frm.configure().keys()))
```

Similarly, you can find the available methods for a widget object using the standard `dir()` function. If you try it, you'll see there are over 200 common widget methods, so again identifying those specific to a widget class is helpful.

```
print(dir(btn))
print(set(dir(btn)) - set(dir(frm)))
```

Navigating the Tcl/Tk Reference Manual

As noted, the official Tk commands reference manual (man pages) is often the most accurate description of what specific operations on widgets do. Even when you know the name of the option or method that you need, you may still have a few places to look.

While all operations in Tkinter are implemented as method calls on widget objects, you've seen that many Tcl/Tk operations appear as commands that take a widget pathname as its first parameter, followed by optional parameters, e.g.

```
destroy .
grid .frm.btn -column 0 -row 0
```

Others, however, look more like methods called on a widget object (in fact, when you create a widget in Tcl/Tk, it creates a Tcl command with the name of the widget pathname, with the first parameter to that command being the name of a method to call).

```
.frm.btn invoke
```

```
.frm.lbl configure -text "Goodbye"
```

In the official Tcl/Tk reference documentation, you'll find most operations that look like method calls on the man page for a specific widget (e.g., you'll find the `invoke()` method on the `ttk::button` man page), while functions that take a widget as a parameter often have their own man page (e.g., `grid`).

You'll find many common options and methods in the `options.ttk::widget` man pages, while others are found in the man page for a specific widget class.

You'll also find that many Tkinter methods have compound names, e.g., `wininfo_x()`, `wininfo_height()`, `wininfo_viewable()`. You'd find documentation for all of these in the `wininfo` man page.

■ **Nota:** Somewhat confusingly, there are also methods on all Tkinter widgets that don't actually operate on the widget, but operate at a global scope, independent of any widget. Examples are methods for accessing the clipboard or the system bell. (They happen to be implemented as methods in the `BaseWidget` class that all Tkinter widgets inherit from).

Threading model

Python and Tcl/Tk have very different threading models, which Tkinter tries to bridge. If you use threads, you may need to be aware of this.

A Python interpreter may have many threads associated with it. In Tcl, multiple threads can be created, but each thread has a separate Tcl interpreter instance associated with it. Threads can also create more than one interpreter instance, though each interpreter instance can be used only by the one thread that created it.

Each Tk object created by Tkinter contains a Tcl interpreter. It also keeps track of which thread created that interpreter. Calls to Tkinter can be made from any Python thread. Internally, if a call comes from a thread other than the one that created the Tk object, an event is posted to the interpreter's event queue, and when executed, the result is returned to the calling Python thread.

Tcl/Tk applications are normally event-driven, meaning that after initialization, the interpreter runs an event loop (i.e. `Tk.mainloop()`) and responds to events. Because it is single-threaded, event handlers must respond quickly, otherwise they will block other events from being processed. To avoid this, any long-running computations should not run in an event handler, but are either broken into smaller pieces using timers, or run in another thread. This is different from many GUI toolkits where the GUI runs in a completely separate thread from all application code including event handlers.

If the Tcl interpreter is not running the event loop and processing events, any Tkinter calls made from threads other than the one running the Tcl interpreter will fail.

A number of special cases exist:

- Tcl/Tk libraries can be built so they are not thread-aware. In this case, Tkinter calls the library from the originating Python thread, even if this is different than the thread that created the Tcl interpreter. A global lock ensures only one call occurs at a time.
- While Tkinter allows you to create more than one instance of a Tk object (with its own interpreter), all interpreters that are part of the same thread share a common event queue, which gets ugly fast. In practice, don't create more than one instance of Tk at a time. Otherwise, it's best to create them in separate threads and ensure you're running a thread-aware Tcl/Tk build.
- Blocking event handlers are not the only way to prevent the Tcl interpreter from reentering the event loop. It is even possible to run multiple nested event loops or abandon the event loop entirely. If you're doing anything tricky when it comes to events or threads, be aware of these possibilities.
- There are a few select Tkinter functions that presently work only when called from the thread that created the Tcl interpreter.

Handy Reference

Setting Options

Options control things like the color and border width of a widget. Options can be set in three ways:

At object creation time, using keyword arguments

After object creation, treating the option name like a dictionary index

Use the `config()` method to update multiple attrs subsequent to object creation

```
fred=Button(self,fg="red",bg="blue")

fred["fg"]="red"fred["bg"]="blue"

fred.config(fg="red",bg="blue")
```

For a complete explanation of a given option and its behavior, see the Tk man pages for the widget in question.

Note that the man pages list “STANDARD OPTIONS” and “WIDGET SPECIFIC OPTIONS” for each widget. The former is a list of options that are common to many widgets, the latter are the options that are idiosyncratic to that particular widget. The Standard Options are documented on the `options(3)` man page.

No distinction between standard and widget-specific options is made in this document. Some options don't apply to some kinds of widgets. Whether a given widget responds to a particular option depends on the class of the widget; buttons have a `command` option, labels do not.

The options supported by a given widget are listed in that widget's man page, or can be queried at runtime by calling the `config()` method without arguments, or by calling the `keys()` method on that widget. The return value of these calls is a dictionary whose key is the name of the option as a string (for example, 'relief') and whose values are 5-tuples.

Some options, like `bg` are synonyms for common options with long names (`bg` is shorthand for “background”). Passing the `config()` method the name of a shorthand option will return a 2-tuple, not 5-tuple. The 2-tuple passed back will contain the name of the synonym and the “real” option (such as ('bg','background')).

Index	Meaning	Example	0	option name	'relief'	1	option name for database lookup	'relief'	2	option class for database lookup	'Relief'	3	default value	'raised'	4	current value	'groove'
-------	---------	---------	---	-------------	----------	---	---------------------------------	----------	---	----------------------------------	----------	---	---------------	----------	---	---------------	----------

Example:

```
>>> print(fred.config())
{'relief': ('relief', 'relief', 'Relief', 'raised', 'groove')}
```

Of course, the dictionary printed will include all the options available and their values. This is meant only as an example.

The Packer

The packer is one of Tk's geometry-management mechanisms. Geometry managers are used to specify the relative positioning of widgets within their container. In contrast to the more cumbersome `placer` (which is used less commonly, and we do not cover here), the packer takes qualitative relationship specification -above, to the left of, filling, etc - and works everything out to determine the exact placement coordinates for you.

The size of any container widget is determined by the size of the “content widgets” inside. The packer is used to control where content widgets appear inside the container into which they are packed. You can pack widgets into frames, and frames into other frames, in order to achieve the kind of layout you desire. Additionally, the arrangement is dynamically adjusted to accommodate incremental changes to the configuration, once it is packed.

Note that widgets do not appear until they have had their geometry specified with a geometry manager. It's a common early mistake to leave out the geometry specification, and then be surprised when the widget is created but nothing appears. A widget will appear only after it has had, for example, the packer's `pack()` method applied to it.

The `pack()` method can be called with keyword-option/value pairs that control where the widget is to appear within its container, and how it is to behave when the main application window is resized. Here are some examples:

```
fred.pack()                # defaults to side = "top"
fred.pack(side="left")
fred.pack(expand=1)
```

Packer Options

For more extensive information on the packer and the options that it can take, see the man pages and page 183 of John Ousterhout's book.

anchor

expand

fill

ipadx and ipady

padx and pady

side

Anchor type. Denotes where the packer is to place each content in its parcel.

Boolean, 0 or 1.

Legal values: 'x', 'y', 'both', 'none'.

A distance - designating internal padding on each side of the content.

A distance - designating external padding on each side of the content.

Legal values are: 'left', 'right', 'top', 'bottom'.

Coupling Widget Variables

The current-value setting of some widgets (like text entry widgets) can be connected directly to application variables by using special options. These options are `variable`, `textvariable`, `onvalue`, `offvalue`, and `value`. This connection works both ways: if the variable changes for any reason, the widget it's connected to will be updated to reflect the new value.

Unfortunately, in the current implementation of Tkinter it is not possible to hand over an arbitrary Python variable to a widget through a `variable` or `textvariable` option. The only kinds of variables for which this works are variables that are subclassed from a class called `Variable`, defined in `tkinter`.

There are many useful subclasses of `Variable` already defined: `StringVar`, `IntVar`, `DoubleVar`, and `BooleanVar`. To read the current value of such a variable, call the `get()` method on it, and to change its value you call the `set()` method. If you follow this protocol, the widget will always track the value of the variable, with no further intervention on your part.

For example:

```
import tkinter as tk

class App(tk.Frame):
    def __init__(self, master):
```

```

super().__init__(master)
self.pack()

self.entrythingy = tk.Entry()
self.entrythingy.pack()

# Create the application variable.
self.contents = tk.StringVar()
# Set it to some value.
self.contents.set("this is a variable")
# Tell the entry widget to watch this variable.
self.entrythingy["textvariable"] = self.contents

# Define a callback for when the user hits return.
# It prints the current value of the variable.
self.entrythingy.bind('<Key-Return>',
                      self.print_contents)

def print_contents(self, event):
    print("Hi. The current entry content is:",
          self.contents.get())

root = tk.Tk()
myapp = App(root)
myapp.mainloop()

```

The Window Manager

In Tk, there is a utility command, `wm`, for interacting with the window manager. Options to the `wm` command allow you to control things like titles, placement, icon bitmaps, and the like. In tkinter, these commands have been implemented as methods on the `Wmclass`. Toplevel widgets are subclassed from the `Wmclass`, and so can call the `Wm` methods directly.

To get at the toplevel window that contains a given widget, you can often just refer to the widget's `master`. Of course if the widget has been packed inside of a frame, the `master` won't represent a toplevel window. To get at the toplevel window that contains an arbitrary widget, you can call the `_root()` method. This method begins with an underscore to denote the fact that this function is part of the implementation, and not an interface to Tk functionality.

Here are some examples of typical usage:

```

import tkinter as tk

class App(tk.Frame):
    def __init__(self, master=None):
        super().__init__(master)
        self.pack()

# create the application
myapp = App()

#
# here are method calls to the window manager class
#
myapp.master.title("My Do-Nothing Application")
myapp.master.maxsize(1000, 400)

# start the program
myapp.mainloop()

```

Tk Option Data Types

anchor

bitmap

boolean

callback

color

cursor

distance

font

geometry

justify

region

relief

scrollcommand

wrap

Legal values are points of the compass:"n","ne","e","se","s","sw","w","nw", and also"center".

There are eight built-in, named

bitmaps:'error','gray25','gray50','hourglass','info','questhead','question','warning'. To specify an X bitmap filename, give the full path to the file, preceded with an@, as in"@/usr/contrib/bitmap/gumby.bit".

You can pass integers 0 or 1 or the strings"yes"or"no".

This is any Python function that takes no arguments. For example: `def print_it(): print("hi there")` `fred["command"] = print_it`

Colors can be given as the names of X colors in the rgb.txt file, or as strings representing RGB values in 4 bit:"#RGB", 8 bit:"#RRGGBB", 12 bit:"#RRRGGBBB", or 16 bit:"#RRRRGGGGBBBB" ranges, where R,G,B here represent any legal hex digit. See page 160 of Ousterhout's book for details.

The standard X cursor names from cursorfont.h can be used, without the XC_ prefix. For example to get a hand cursor (XC_hand2), use the string"hand2". You can also specify a bitmap and mask file of your own. See page 179 of Ousterhout's book.

Screen distances can be specified in either pixels or absolute distances. Pixels are given as numbers and absolute distances as strings, with the trailing character denoting units:cfor centimetres,i for inches,mfor millimetres,pfor printer's points. For example, 3.5 inches is expressed as"3.5i".

Tk uses a list font name format, such as{courier10bold}. Font sizes with positive numbers are measured in points; sizes with negative numbers are measured in pixels.

This is a string of the formwidthxheight, where width and height are measured in pixels for most widgets (in characters for widgets displaying text). For example: `fred["geometry"] = "200x100"`.

Legal values are the strings:"left", "center", and "right".

This is a string with four space-delimited elements, each of which is a legal distance (see above). For example:"2345"and"3i2i4.5i2i"and"3c2c4c10.43c"are all legal regions.

Determines what the border style of a widget will be. Legal values are:"raised", "sunken", "flat", "groove", and "ridge".

This is almost always the set() method of some scrollbar widget, but can be any widget method that takes a single argument.

Must be one of:"none","char", or"word".

Bindings and Events

The bind method from the widget command allows you to watch for certain events and to have a callback function trigger when that event type occurs. The form of the bind method is:

```
def bind(self, sequence, func, add='')
```

where:

sequence

func

add

is a string that denotes the target kind of event. (See thebind(3tk)man page, and page 201 of John Ousterhout's book,Tcl and the Tk Toolkit (2nd edition), for details).

is a Python function, taking one argument, to be invoked when the event occurs. An Event instance will be passed as the argument. (Functions deployed this way are commonly known ascallbacks.)

is optional, either"or"+"'. Passing an empty string denotes that this binding is to replace any other bindings that this event is associated with. Passing a'+'means that this function is to be added to the list of functions bound to this event type.

For example:

```
def turn_red(self, event):
    event.widget["activeforeground"] = "red"
```

```
self.button.bind("<Enter>", self.turn_red)
```

Notice how the widget field of the event is being accessed in theturn_red()callback. This field contains the widget that caught the X event. The following table lists the other event fields you can access, and how they are denoted in Tk, which can be useful when referring to the Tk man pages.

Tk		Tkinter	Event	Field		Tk		Tkinter	Event	Field		%f		focus		%A		char		%h		height		%E	
	send_event		%k		keycode		%K		keysym		%s		state		%N		keysym_num		%t		time		%T		type
	%w		width		%W		widget		%x		x		%X		x_root		%y		y		%Y		y_root		

The index Parameter

A number of widgets require "index" parameters to be passed. These are used to point at a specific place in a Text widget, or to particular characters in an Entry widget, or to particular menu items in a Menu widget.

Entry widget indexes (index, view index, etc.)

Text widget indexes

Menu indexes (menu.invoke(), menu.entryconfig(), etc.)

Entry widgets have options that refer to character positions in the text being displayed. You can use thesetkinterfunctions to access these special points in text widgets:

The index notation for Text widgets is very rich and is best described in the Tk man pages.

Some options and methods for menus manipulate specific menu entries. Anytime a menu index is needed for an option or a parameter, you may pass in:an integer which refers to the numeric position of the entry in the widget, counted from the top, starting with 0;the string"active", which refers to the menu position that is currently under the cursor;the string"last"which refers to the last menu item;An integer preceded by@, as in@6, where the integer is interpreted as a y pixel coordinate in the me

Images

Images of different formats can be created through the corresponding subclass of `tkinter.Image`:

- `BitmapImage` for images in XBM format.
- `PhotoImage` for images in PGM, PPM, GIF and PNG formats. The latter is supported starting with Tk 8.6.

Either type of image is created through either the `file` or the `data` option (other options are available as well).

Changed in version 3.13: Added the `PhotoImage` method `copy_replace()` to copy a region from one image to other image, possibly with pixel zooming and/or subsampling. Add `from_coord` parameter to `PhotoImage` methods `copy()`, `zoom()` and `subsample()`. Add `zoom` and `subsample` parameters to `PhotoImage` method `copy()`.

The image object can then be used wherever an image option is supported by some widget (e.g. labels, buttons, menus). In these cases, Tk will not keep a reference to the image. When the last Python reference to the image object is deleted, the image data is deleted as well, and Tk will display an empty box wherever the image was used.

See also

The `Pillow` package adds support for formats such as BMP, JPEG, TIFF, and WebP, among others.

File Handlers

Tk allows you to register and unregister a callback function which will be called from the Tk mainloop when I/O is possible on a file descriptor. Only one handler may be registered per file descriptor. Example code:

```
import tkinter
widget = tkinter.Tk()
mask = tkinter.READABLE | tkinter.WRITABLE
widget.tk.createfilehandler(file, mask, callback)
...
widget.tk.deletefilehandler(file)
```

This feature is not available on Windows.

Since you don't know how many bytes are available for reading, you may not want to use the `BufferedIOBase` or `TextIOBase` `read()` or `readline()` methods, since these will insist on reading a predefined number of bytes. For sockets, the `recv()` or `recvfrom()` methods will work fine; for other files, use raw reads `os.read(file.fileno(), maxbytecount)`.

Widget.tk.createfilehandler(file,mask,func)

Registers the file handler callback function `func`. The `file` argument may either be an object with a `fileno()` method (such as a file or socket object), or an integer file descriptor. The `mask` argument is an ORed combination of any of the three constants below. The callback is called as follows: `callback(file,mask)`

Widget.tk.deletefilehandler(file)

Unregisters a file handler.

`_tkinter.READABLE`

`_tkinter.WRITABLE`

`_tkinter.EXCEPTION`

Constants used in the `mask` arguments.

tkinter.colorchooser— Color choosing dialog

Source code: `Lib/tkinter/colorchooser.py`

The `tkinter.colorchooser` module provides the `Chooser` class as an interface to the native color picker dialog. `Chooser` implements a modal color choosing dialog window. The `Chooser` class inherits from the `Dialog` class.

`class tkinter.colorchooser.Chooser(master=None, **options)`

`tkinter.colorchooser.askcolor(color=None, **options)`

Create a color choosing dialog. A call to this method will show the window, wait for the user to make a selection, and return the selected color (or `None`) to the caller.

See also

Module `tkinter.commondialog`

Tkinter standard dialog module

tkinter.font— Tkinter font wrapper

Source code: `Lib/tkinter/font.py`

The `tkinter.font` module provides the `Font` class for creating and using named fonts.

The different font weights and slants are:

`tkinter.font.NORMAL`

`tkinter.font.BOLD`

`tkinter.font.ITALIC`

`tkinter.font.ROMAN`

`class tkinter.font.Font(root=None, font=None, name=None, exists=False, **options)`

`actual(option=None, displayof=None)`

`cget(option)`

`config(**options)`

`copy()`

`measure(text, displayof=None)`

`metrics(*options, **kw)`

ascent- distance between baseline and highest point that a

descent- distance between baseline and lowest point that a

linespace- minimum vertical separation necessary between any two

The `Font` class represents a named font. `Font` instances are given unique names and can be specified by their family, size, and style configuration. Named fonts are Tk's method of creating and identifying fonts as a single object, rather than specifying a font by its attributes with each occurrence. arguments: `font`- font specifier tuple (family, size, options) `name`- unique font name `exists`- self points to existing named font if true additional keyword options (ignored if font is specified): `family`- font famil

Return the attributes of the font.

Retrieve an attribute of the font.

Modify attributes of the font.

Return new instance of the current font.

Return amount of space the text would occupy on the specified display when formatted in the current font. If no display is specified then the main application window is assumed.

Return font-specific data. Options include: `ascent`- distance between baseline and highest point that a character of the font can occupy `descent`- distance between baseline and lowest point that a character of the font can occupy `linespace`- minimum vertical separation necessary between any two characters of the font that ensures no vertical overlap between lines. `fixed`- 1 if font is fixed-width else 0

character of the font can occupy

character of the font can occupy

characters of the font that ensures no vertical overlap between lines.

`tkinter.font.families(root=None, displayof=None)`

Return the different font families.

tkinter.font.names(root=None)

Return the names of defined fonts.

tkinter.font.nametofont(name,root=None)

Return aFontrepresentation of a tk named font.Changed in version 3.10:The rootparameter was added.

Tkinter Dialogs

tkinter.simpledialog— Standard Tkinter input dialogs

Source code: `Lib/tkinter/simpledialog.py`

The `tkinter.simpledialog` module contains convenience classes and functions for creating simple modal dialogs to get a value from the user.

`tkinter.simpledialog.askfloat(title, prompt, **kw)`

`tkinter.simpledialog.askinteger(title, prompt, **kw)`

`tkinter.simpledialog.askstring(title, prompt, **kw)`

The above three functions provide dialogs that prompt the user to enter a value of the desired type.

`class tkinter.simpledialog.Dialog(parent, title=None)`

`body(master)`

`buttonbox()`

The base class for custom dialogs. `body(master)` Override to construct the dialog's interface and return the widget that should have initial focus. `buttonbox()` Default behaviour adds OK and Cancel buttons. Override for custom button layouts.

Override to construct the dialog's interface and return the widget that should have initial focus.

Default behaviour adds OK and Cancel buttons. Override for custom button layouts.

tkinter.filedialog— File selection dialogs

Source code: `Lib/tkinter/filedialog.py`

The `tkinter.filedialog` module provides classes and factory functions for creating file/directory selection windows.

Native Load/Save Dialogs

The following classes and functions provide file dialog windows that combine a native look-and-feel with configuration options to customize behaviour. The following keyword arguments are applicable to the classes and functions listed below:

Static factory functions

The below functions when called create a modal, native look-and-feel dialog, wait for the user's selection, then return the selected value(s) or `None` to the caller.

`tkinter.filedialog.askopenfile(mode='r', **options)`

`tkinter.filedialog.askopenfiles(mode='r', **options)`

The above two functions create an `OpenFileDialog` and return the opened file object(s) in read-only mode.

`tkinter.filedialog.asksaveasfile(mode='w', **options)`

Create a `SaveAsDialog` and return a file object opened in write-only mode.

`tkinter.filedialog.askopenfilename(**options)`

`tkinter.filedialog.askopenfilenames(**options)`

The above two functions create an `OpenDialog` and return the selected filename(s) that correspond to existing file(s).

`tkinter.filedialog.asksaveasfilename(**options)`

Create a `SaveAsDialog` and return the selected filename.

`tkinter.filedialog.askdirectory(**options)`

Prompt user to select a directory. Additional keyword option: `mustexist` - determines if selection must be an existing directory.

`tkinter.filedialog.Open(master=None, **options)`

`tkinter.filedialog.SaveAs(master=None, **options)`

The above two classes provide native dialog windows for saving and loading files.

Convenience classes

The below classes are used for creating file/directory windows from scratch. These do not emulate the native look-and-feel of the platform.

`tkinter.filedialog.Directory(master=None, **options)`

Create a dialog prompting the user to select a directory.

■ Nota: `NoteFileDialog` class should be subclassed for custom event handling and behaviour.

`tkinter.filedialog.FileDialog(master, title=None)`

`cancel_command(event=None)`

`dirs_double_event(event)`

`dirs_select_event(event)`

`files_double_event(event)`

`files_select_event(event)`

`filter_command(event=None)`

`get_filter()`

`get_selection()`

`go(dir_or_file=os.curdir, pattern="*", default="", key=None)`

`ok_event(event)`

`quit(how=None)`

`set_filter(dir, pat)`

`set_selection(file)`

Create a basic file selection dialog. `cancel_command(event=None)` Trigger the termination of the dialog window. `dirs_double_event(event)` Event handler for double-click event on

directory. `dirs_select_event(event)` Event handler for click event on directory. `files_double_event(event)` Event handler for double-click event on file. `files_select_event(event)` Event handler for single-click event on file. `filter_command(event=None)` Filter the files by directory. `get_filter()` Retrieve the file filter currently in use. `g`

Trigger the termination of the dialog window.

Event handler for double-click event on directory.

Event handler for click event on directory.

Event handler for double-click event on file.

Event handler for single-click event on file.

Filter the files by directory.

Retrieve the file filter currently in use.

Retrieve the currently selected item.

Render dialog and start event loop.

Exit dialog returning current selection.

Exit dialog returning filename, if any.

Set the file filter.

Update the current file selection to file.

tkinter.filedialog.LoadFileDialog(master,title=None)

ok_command()

A subclass of `FileDialog` that creates a dialog window for selecting an existing file.
`ok_command()` Test that a file is provided and that the selection indicates an already existing file.

Test that a file is provided and that the selection indicates an already existing file.

tkinter.filedialog.SaveFileDialog(master,title=None)

ok_command()

A subclass of `FileDialog` that creates a dialog window for selecting a destination file.
`ok_command()` Test whether or not the selection points to a valid file that is not a directory. Confirmation is required if an already existing file is selected.

Test whether or not the selection points to a valid file that is not a directory. Confirmation is required if an already existing file is selected.

tkinter.commondialog— Dialog window templates

Source code: `Lib/tkinter/commondialog.py`

The `tkinter.commondialog` module provides the `Dialog` class that is the base class for dialogs defined in other supporting modules.

tkinter.commondialog.Dialog(master=None,options)**

show(options)**

`show(**options)` Render the Dialog window.

Render the Dialog window.

See also

Modules `tkinter.messagebox`, `Reading and Writing Files`

tkinter.messagebox— Tkinter message prompts

Source code: `Lib/tkinter/messagebox.py`

The `tkinter.messagebox` module provides a template base class as well as a variety of convenience methods for commonly used configurations. The message boxes are modal and will return a subset of (`True`, `False`, `None`, `OK`, `CANCEL`, `YES`, `NO`) based on the user's selection. Common message box styles and layouts include but are not limited to:

`class tkinter.messagebox.Message(master=None, **options)`

`command`

`default`

`detail`

`icon`

`message`

`parent`

`title`

`type`

`show(**options)`

Create a message window with an application-specified message, an icon and a set of buttons. Each of the buttons in the message window is identified by a unique symbolic name (see the `typeoptions`). The following options are supported: `command` Specifies the function to invoke when the user closes the dialog. The name of the button clicked by the user to close the dialog is passed as argument. This is only available on macOS. `default` Gives the symbolic name of the default button for this message window (O

Specifies the function to invoke when the user closes the dialog. The name of the button clicked by the user to close the dialog is passed as argument. This is only available on macOS.

Gives the symbolic name of the default button for this message window (`OK`, `CANCEL`, and so on). If this option is not specified, the first button in the dialog will be made the default.

Specifies an auxiliary message to the main message given by the `message` option. The message detail will be presented beneath the main message and, where supported by the OS, in a less emphasized font than the main message.

Specifies an icon to display. If this option is not specified, then the `INFO` icon will be displayed.

Specifies the message to display in this message box. The default value is an empty string.

Makes the specified window the logical parent of the message box. The message box is displayed on top of its parent window.

Specifies a string to display as the title of the message box. This option is ignored on macOS, where platform guidelines forbid the use of a title on this kind of dialog.

Arranges for a predefined set of buttons to be displayed.

Display a message window and wait for the user to select one of the buttons. Then return the symbolic name of the selected button. Keyword arguments can override options specified in the constructor.

Information message box

`tkinter.messagebox.showinfo(title=None, message=None, **options)`

Creates and displays an information message box with the specified title and message.

Warning message boxes

tkinter.messagebox.showwarning(title=None,message=None,options)**

Creates and displays a warning message box with the specified title and message.

tkinter.messagebox.showerror(title=None,message=None,options)**

Creates and displays an error message box with the specified title and message.

Question message boxes

tkinter.messagebox.askquestion(title=None,message=None,*,type=YESNO,options)**

Ask a question. By default shows buttonsYESandNO. Returns the symbolic name of the selected button.

tkinter.messagebox.askokcancel(title=None,message=None,options)**

Ask if operation should proceed. Shows buttonsOKandCANCEL. ReturnsTrueif the answer is ok andFalseotherwise.

tkinter.messagebox.askretrycancel(title=None,message=None,options)**

Ask if operation should be retried. Shows buttonsRETRYandCANCEL. ReturnTrueif the answer is yes andFalseotherwise.

tkinter.messagebox.askyesno(title=None,message=None,options)**

Ask a question. Shows buttonsYESandNO. ReturnsTrueif the answer is yes andFalseotherwise.

tkinter.messagebox.askyesnocancel(title=None,message=None,options)**

Ask a question. Shows buttonsYES,NOandCANCEL. ReturnTrueif the answer is yes,Noneif cancelled, andFalseotherwise.

Symbolic names of buttons:

tkinter.messagebox.ABORT='abort'

tkinter.messagebox.RETRY='retry'

tkinter.messagebox.IGNORE='ignore'

tkinter.messagebox.OK='ok'

tkinter.messagebox.CANCEL='cancel'

tkinter.messagebox.YES='yes'

tkinter.messagebox.NO='no'

Predefined sets of buttons:

tkinter.messagebox.ABORTRETRYIGNORE='abortretryignore'

Displays three buttons whose symbolic names areABORT,RETRYandIGNORE.

tkinter.messagebox.OK='ok'

Displays one button whose symbolic name isOK.

tkinter.messagebox.OKCANCEL='okcancel'

Displays two buttons whose symbolic names areOKandCANCEL.

tkinter.messagebox.RETRYCANCEL='retrycancel'

Displays two buttons whose symbolic names areRETRYandCANCEL.

tkinter.messagebox.YESNO='yesno'

Displays two buttons whose symbolic names areYESandNO.

tkinter.messagebox.YESNOCANCEL='yesnocancel'

Displays three buttons whose symbolic names areYES,NOandCANCEL.

Icon images:

tkinter.messagebox.ERROR='error'

tkinter.messagebox.INFO='info'

tkinter.messagebox.QUESTION='question'

tkinter.messagebox.WARNING='warning'

tkinter.scrolledtext— Scrolled Text Widget

Source code: `Lib/tkinter/scrolledtext.py`

The `tkinter.scrolledtext` module provides a class of the same name which implements a basic text widget which has a vertical scroll bar configured to do the “right thing.” Using the `ScrolledText` class is a lot easier than setting up a text widget and scroll bar directly.

The text widget and scrollbar are packed together in a `Frame`, and the methods of the `Grid` and `Pack` geometry managers are acquired from the `Frame` object. This allows the `ScrolledText` widget to be used directly to achieve most normal geometry management behavior.

Should more specific control be necessary, the following attributes are available:

`class tkinter.scrolledtext.ScrolledText(master=None, **kw)`

`frame`

`vbar`

`frame` The frame which surrounds the text and scroll bar widgets. `vbar` The scroll bar widget.

The frame which surrounds the text and scroll bar widgets.

The scroll bar widget.